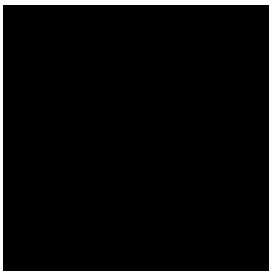
Object Assembly Reference



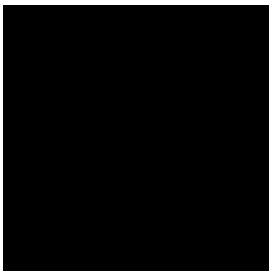
Version 2.1

GEOS Software Development Kit Library

Version 2.01



Object Assembly Reference



Geoworks
Alameda, CA

Geoworks provides this publication “as is” without warranty of any kind, either express or implied, including but not limited to the implied warranties of merchantability or fitness for a particular purpose. Geoworks may revise this publication from time to time without notice. Geoworks does not promise support of this documentation. Some states or jurisdictions do not allow disclaimer of express or implied warranties in certain transactions; therefore, this statement may not apply to you.

Geoworks® application and tools software and the GEOS® operating system software copyright © 1990-1994 Geoworks. All rights reserved. United States Patent 5,327,529. Published 1994.
Printed in the United States of America

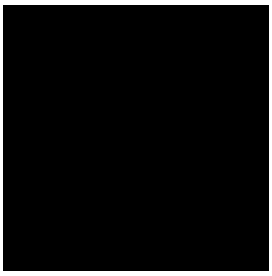
Geoworks®, Geoworks Ensemble®, Ensemble®, GEOS®, PC/GEOS®, GeoDraw®, GeoManager®, GeoPlanner®, GeoFile®, GeoDex® and GeoComm® are registered trademarks of Geoworks in the United States and Other countries.

GeoWrite, GeoBanner, GeoCalc,GeoDOS , Lights Out and the Geoworks logo are trademarks of Geoworks in the United States and other countries.

Trademarks and service marks not listed here are the property of their respective owners. Every effort has been made to treat trademarks and service marks in accordance with the United States Trademark Association's guidelines. Any omissions are unintentional and should not be regarded as affecting the validity of any trademark or service mark.



Object Assembly



1 System Classes 9
- **MetaClass** 11
- **ProcessClass** 47
- **GenProcessClass** 51

2 GenClass 63
- **GenClass** 65

3 GenApplication 125
- **GenApplicationClass** 127

4 GenDisplay 159
- **GenDisplayClass** 161
- **GenPrimaryClass** 166

5 GenTrigger 169
- **GenTriggerClass** 171

6 GenGlyph 177
- **GenGlyphClass** 179

7 GenInteraction 181
- **GenInteractionClass** 183

8 GenValue 197
- **GenValueClass** 199

9 GenView 219
- **GenViewClass** 221
- **GenViewControlClass** 241

10 Text Objects 243
GenTextClass 245
VisTextClass 254

11 The List Objects 315
GenBooleanClass 317
GenBooleanGroupClass 318
GenDynamicListClass 326
GenItemClass 330
GenItemGroupClass 331

12 Controllers 343
GenControlClass 345
GenToolControlClass 356
GenToolGroupClass 357
ColorSelectorClass 358
GenPageControlClass 367

13 Document Control 369
GenDocumentClass 371
GenDocumentGroupClass 397
GenDocumentControlClass 419

14 GenFileSelector 439
GenFileSelectorClass 441

15 Help Library 459
HelpControlClass 461

16 Impex Library 465
ImportControlClass 467

ExportControlClass 470

17 Print Controller 475
PrintControlClass 477
PageSizeControlClass 495

18 GrObj Library 499
GrObjAttributeManagerClass 501
GrObjHeadClass 501
GrObjBodyClass 503
GrObjClass 510
GroupClass 525
PointerClass 526
GrObjVisGuardianClass 526

19 Ruler Library 529
VisRulerClass 531
RulerViewClass 556
RulerShowControlClass 557

20 Spreadsheet Library 559
SpreadsheetClass 561
SpreadsheetRulerClass 587

21 Pen Library 589
InkClass 591
InkControlClass 595

22 Config Library 597
PrefClass 599
PrefValueClass 603
PrefItemGroupClass 604
PrefStringItemClass 606
PrefBooleanGroupClass 607
PrefDynamicListClass 608
TitledGlyphClass 609
PrefInteractionClass 609

Object Assembly



PrefDialogClass 610
PrefTextClass 610
PrefControlClass 611
PrefTimeDateControlClass 611
PrefTriggerClass 611
PrefTocListClass 612

23 Vis **615**
VisClass 617

24 VisComp **653**
VisCompClass 655

25 VisContent **659**
VisContentClass 661

26 System Objects **671**
GenSystemClass 673
GenFieldClass 677
GenScreenClass 682

Object Assembly

System Classes

1

MetaClass 11

ProcessClass 47

GenProcessClass 51

MetaClass

Instance Data

```
MI_base         MetaBase <>
   UIC Syntax:  NONE
```

MI_base contains no actual data; it is used solely to mark the beginning of the object's instance data.

Messages

■ MSG_META_ACK

This message acknowledges a detach message. It is sent by objects that have been notified of another object's detach. The default handler for MSG_META_DETACH simply sends MSG_META_ACK back to the object that sent the detach message.

Source: Object having received MSG_META_DETACH (default handler in **MetaClass** will reflexively respond to any MSG_META_DETACH with a MSG_META_ACK, though you can change this behavior either by using **ObjInitDetach()** or by not letting the message get to the **MetaClass** handler, and responding yourself with a MSG_META_ACK sometime later).

Destination: The optr passed in MSG_META_DETACH.

Pass:

cx	Data passed to MSG_META_ACK
dx:bp	Object which has completed detaching

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: **MetaClass** provides default handling of this message, for objects using the **ObjInitDetach()** mechanism. Objects *not* using this mechanism will want to intercept this if there is a need to know when the object asked to detach earlier has completed its detach.
MSG_META_ACK is normally inherited from **MetaClass** which calls **ObjEnableDetach()**. This routine decrements the detach count, and when

that count reaches zero, sends a MSG_DETACH_COMPLETED to the object itself.

Warning: If you are expecting a MSG_META_ACK back from anything, make sure you are using the mechanism initiated with **ObjInitDetach()** yourself, or you should handle MSG_META_ACK to prevent **MetaClass** from assuming you *are* using such a mechanism.

Library: **Objects/metaC.def**

MSG_META_ADD_VAR_DATA

This message adds a variable data type to the recipient object's instance data. If the variable data field was already present, this will change its value. This is useful for adding hints to generic objects at run-time.

Note that the object will be marked dirty even if nothing was changed.

NOTE: The *AVDP_dataType* should have VDF_SAVE_TO_STATE set as desired. VDF_EXTRA_DATA is ignored; it will be set correctly by this routine.

Source: Unrestricted.

Destination: Any object.

Pass:

ss:bp	Pointer to an **AddVarDataParams** structure.
dx	Size of **AddVarDataParams** structure.

Returns: Nothing. Object marked dirty even if data type already exists.

Destroyed: **ax, cx, dx, bp**

Interception: Generally not intercepted; default **MetaClass** handling performs the desired function.

Library: **Objects/metaC.def**

MSG_META_APP_SHUTDOWN

This message is the complement to MSG_META_APP_STARTUP. This message is sent to objects on the MGCNLT_APP_STARTUP list before an application exits but after the UI for the application is detached. Essentially, it operates in the same manner as MSG_META_DETACH except that the receiving object sends MSG_META_SHUTDOWN_ACK when its shutdown is complete.

Source: Sent by **GenProcessClass** after detaching the UI but before exiting the application; if the UI was never attached (i.e. it handled

Object Assembly Reference

MSG_META_APP_STARTUP but not MSG_META_ATTACH) the UI will obviously not be detached.

Destination: Any object that needs to be notified when the application is about to exit.

Pass: `cx` Word of data for caller's use.
`dx:bp` Optr of object to be sent MSG_META_SHUTDOWN_ACK.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: Usually intercepted by objects on the MGCNLT_APP_STARTUP list.

Library: **Objects/metaC.def**

■ MSG_META_APP_STARTUP

This message is related to MSG_META_ATTACH; the message is sent by the generic UI to the GenApplication object before it sends MSG_META_ATTACH to it. MSG_META_ATTACH is only sent when the application is becoming available to the user; if an application should be opened as a server without presenting any UI to the user, MSG_META_APP_STARTUP will be the only message sent to the application object upon start-up.

The default handler for this message will pass it on to all members of the MGCNLT_APP_STARTUP list.

Source: **GenProcessClass**; forwarded by **GenApplicationClass** to other objects. This message is sent upon application start-up before the UI for an application has been attached.

Destination: Any object that needs to be notified when the application is launched, regardless of whether the user will be interacting with the application.

Pass: `dx` Handle of an **AppLaunchBlock**.

Returns: The **AppLaunchBlock** is preserved.

Destroyed: `ax, cx, dx, bp`

Interception: Usually intercepted by any object on the MGCNLT_APP_STARTUP list.

Library: **Objects/metaC.def**

■ MSG_META_ATTACH

This message is used for two different purposes: It can be sent to any geode that has a process aspect when the geode is first loaded. It can also be sent in the object world to notify objects on an "active list" that the application has

been brought back up from a state file. As the method is used for different purposes, the data passed varies based on usage.

As sent in general by **GeodeLoad**, if the bit GA_PROCESS is set in the geode:

Source: Kernel routine **GeodeLoad**

Destination: Newly created process (other than GenProcess) object

Pass: **cx** Value passed to **GeodeLoad** in **di**
dx Value passed to **GeodeLoad** in **bp**

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: No default handling provided, so you will likely want to intercept this message to do any process initialization required.

As required by the GenProcessClass, which all application using the UI library must have their processes subclassed off of:

Source: Kernel routine **GeodeLoad**

Destination: Newly created GenProcess (or subclass thereof) object.

Pass: **dx** Block handle to block of **AppLaunchBlock**.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp, AppLaunchBlock**

Interception: Often intercepted as a convenient place to perform general application initialization that should occur whether just being launched, coming back from state, or being launched in engine mode. Note that the interface of the application will not be on screen (in fact, won't even be linked into the system wide generic tree) until the superclass is called.

As sent to the GenApplication object by GenProcessClass, upon an application starting up, either for the first time, or when being restored to the state it was in before being detached from a previous session:

Source: GenProcess object

Destination: GenApplication object

Pass: **dx** Block handle to block of **AppLaunchBlock**.
bp Extra state block from state file, or 0 if none. This is the same block as returned from

Object Assembly Reference

MSG_GEN_PROCESS_CLOSE_APPLICATION, in some previous detach.

Returns: Nothing. (**AppLaunchBlock** and state block are preserved.)

Destroyed: **ax, cx, dx, bp**

Interception: Not generally required, as the default handler broadcasts the message out to everything on the application's active list. This act causes the interface for the application to come up on screen.

As sent to objects on the Application object's active list (not done until GenApplication is set *usable* by GenProcess), or those object's active lists, etc.:

Source: GenApplication object

Destination: Any object on the application's active list

Pass:

cx	**AppAttachFlags**
dx	Handle of **AppLaunchBlock**, or 0 if none.
bp	Handle of extra state block, or 0 if none. This is the same block as returned from MSG_GEN_PROCESS_CLOSE_APPLICATION, in some previous detach.

Returns: Nothing. (**AppLaunchBlock** and state block are preserved.)

Destroyed: **ax, cx, dx, bp**

Interception: Standard UI objects defined as needing to be placed on the active list will intercept this message to do whatever it is that they needed to do when the application is first loaded. If you intercept this yourself, be sure to call the superclass, in case it expects to receive this notification itself.

Warning: If sent to an object with a MGCNLT_ACTIVE_LIST GCN list, a default handler will normally pass the message on to all items on that object's active list. Before doing so, however, it passes the message on to its superclass. If CF is returned set, it will *not* send the message on to the object's active list. This scheme has been devised to allow Specific UI objects to handle MSG_META_ATTACH as they wish. (This was added so that OLDisplayControl can delay sending MSG_META_ATTACH to GenDisplays until the GenDisplayControl gets MSG_VIS_OPEN.)

The upshot of this: If the specific UI uses this mechanism, then the GenProcessClass will have already destroyed the **AppLaunchBlock** and the extra state block by the time that MSG_META_ATTACH is sent on to the objects on its ActiveList.

As sent to a thread spawned by MSG_PROCESS_CREATE_EVENT_THREAD:

Source: Kernel.

Destination: The newly created thread.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: No default handling provided, so you will likely want to intercept this message to do any thread initialization required.

Library: **objects/metaC.def**

MSG_META_BLOCK_FREE

This message initiates a sequence which will free an entire object block when received by any object within that block. The block will be freed when its in-use count reaches zero and the message queues for the block have been cleared.

This is a fairly low-level operation, and should be performed only after the objects in the block have been removed from any tree(s) they are attached to, and are otherwise "shut down." For generic objects, this generally means first calling MSG_GEN_SET_NOT_USABLE, then MSG_GEN_REMOVE_CHILD before using this message. For Visible objects, MSG_VIS_REMOVE will both visually shut down the visible tree, and then remove it from its parent.

Source: Unrestricted.

Destination: Any object within a block that is ready to have a low-level delete performed on it (i.e. isn't on screen, isn't linked to objects in other blocks, etc.).

Pass: None.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects/metaC.def**

Object Assembly Reference

MSG_META_BRING_UP_HELP

This message finds a help context for the current object tree and sends a notification to bring up help with that context.

Source: Unrestricted, though generally an object sends this to itself.

Destination: An object within a tree.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Interception: The default handler for this message walks up the visual tree (not the generic tree) eventually finding a **GenClass** object with ATTR_GEN_HELP_CONTEXT. You may intercept at any level to change the help context for a branch.

Library: **Objects/metaC.def**

MSG_META_DEC_BLOCK_REF_COUNT

This message is a utility message to call **MemDecRefCount()** on one or two memory handles.

This message is useful for IACP, which initializes the reference count to the number of servers returned by **IACPConnect()** and records this message as the message to be returned. After each server has processed its information, the reference count will return to zero and the handles will be freed.

Source: Unrestricted.

Destination: Any object.

Pass:

cx	Handle of a block whose reference count should be decremented, or 0 if none.
dx	Handle of a block whose reference count should be decremented, or 0 if none.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Generally not intercepted.

Library: **Objects/metaC.def**

MSG_META_DELETE_VAR_DATA

This message removes a particular variable data entry from the recipient object's instance data. This is useful for removing hints from generic objects at run-time.

Source: Unrestricted.

Destination: Any object.

Interception: Generally not intercepted; default **MetaClass** handling performs the desired function.

Pass:	**dx**	Data type to delete. **VarDataFlags** ignored.
Returns:	CF	clear if data deleted, set if data was not found. (Object is marked dirty if data type was found and deleted.)

Destroyed: **ax, cx, dx, bp**

Library: **Objects/metaC.def**

MSG_META_DETACH

This message severs the links between an object and the rest of the system. The exact way this is handled depends on the object being detached. For full information on detaching objects, see "GEOS Programming," Chapter 5 of the Concepts Book.

The "state" of the object is left intact, in case an image of the object needs to be saved away in a state file for later re-creation. MSG_META_DETACH sent to an application's process will start the process by which it is detached from the system, and then exited. MSG_META_DETACH is asynchronous, in that it need not complete its job immediately upon being called. Rather, it may take as much time, invoking and waiting for the completion of subsidiary detaches (say of child objects needing to perform special actions to detach, or of threads

Object Assembly Reference

created earlier), before it responds with MSG_META_ACK to let its caller know that the detach has completed.

Source: Kernel, other objects relaying detach message.

Destination: GenProcess, GenApplication, objects on active lists.

Pass:

cx	ID of object which sent message.
dx:bp	optr of object which should be sent a MSG_META_ACK when detaching object has finished.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Intercepted as a means of finding out that the application is shutting down. Call the superclass in case it needs such notification.
If you create additional threads, or object classes which need to be notified when the application is about to be exited, you may need to extend the detach mechanism by intercepting MSG_META_DETACH in a subclass of an object already receiving that message, such as GenApplication, GenControl, GenInteraction dialogs, etc. You must make sure that all objects you've sent MSG_META_DETACH to have responded with a MSG_META_ACK before your object can reply with MSG_META_ACK. Remember that your superclass may be sending MSG_META_DETACH. The kernel provides some default behavior in **MetaClass**, and some utility routines, to make this a simpler task. The default handler for MSG_META_DETACH, for instance, at a leaf object (one which doesn't propagate the MSG_META_DETACH), performs the required response (sending a MSG_META_ACK). Thus, leaf objects can just intercept MSG_META_DETACH for notification purposes, then call the superclass, and worry no more. The utility routines **ObjInitDetach()** and **ObjEnableDetach()** work in conjunction with a default MSG_META_ACK handler in **MetaClass** to keep track of how many outstanding acknowledgments are being waited for, and call MSG_META_DETACH_COMPLETE on your object once all acknowledgments have returned (the count reaches zero). The default handler for MSG_META_DETACH_COMPLETE then generates the acknowledgment response required of your object to complete its detach. You may optionally call the superclass before sending the detach message to your children and dependents, depending on which order you want things to detach in. The call to the superclass must happen between the **ObjInitDetach()** and **ObjEnableDetach()**, however.

Library: **Objects/metaC.def**

MSG_META_DETACH_ABORT

This message causes a detach to be aborted. This can cause some very complex synchronization problems and should not be used lightly. You will find very little call to use it.

Source: Renegade object on active list, after having received MSG_META_DETACH, as an alternative to replying with MSG_META_ACK.

Destination: The optr passed in MSG_META_DETACH.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: Handled by GenField to deal with applications that refuse to die, and GenSystem for Fields that have problem applications. Other than that, any detach node wishing to provide this service will have to figure out a way to do it itself.

Library: **Objects/metaC.def**

MSG_META_DETACH_COMPLETE

This message is sent to an object being detached when all of its children and active participants have acknowledged the detach. For full information on detaching objects, see "GEOS Programming," Chapter 5 of the Concepts Book.

MSG_META_DETACH_COMPLETE is sent to the object which called **ObjInitDetach()**. This will happen when as many acknowledgments have been received as **ObjIncDetach()** was called, and **ObjEnableDetach()** was called. The **MetaClass** handler for this message sends MSG_META_ACK to the OD passed to the **ObjInitDetach()** call. This message is provided so that an object will know when all of its children have detached. Note that this message is received only if **ObjInitDetach()** has been called for this object. Note also that your superclass may call **ObjInitDetach()** without your knowing.

Source: **MetaClass** handler for MSG_META_ACK, if detach count has dropped to zero (i.e. no outstanding requests), for objects that are detach

Object Assembly Reference

nodes only (make use of **ObjInitDetach()** or **ObjEnableDetach()**).

Destination: Self.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: This is a handy message to intercept when using the **ObjInitDetach()** mechanism and need to know when all objects asked to detach have responded. Calling the superclass at this point in time will cause an MSG_META_ACK to go back to whatever object sent the MSG_META_DETACH to this object originally. There is no requirement to call the superclass at this time, and in fact this is a way to prolong the detach cycle for this object— by simply starting up another **ObjInitDetach()** sequence, for instance.

Library: **Objects/metaC.def**

MSG_META_DISPATCH_EVENT

This message causes an object to send or call a message of another object. This is useful for getting one object run by a different thread to call yet another object or to send a reply to the first object. This message can cause complex synchronization problems if not used with extreme care.

Source: Unrestricted.

Destination: Any object.

Interception: Unnecessary, as **MetaClass** does the right thing.

Pass: **cx** Event which will be sent.
dx **MessageFlags** which will determine how message is sent.

Returns: **ax, cx, dx, bp,** CF returned from handler if MF_CALL specified; otherwise, these are destroyed.
EventHandle freed.

Library: **Objects/metaC.def**

MSG_META_DUMMY

This message has no handler. You should not subclass this message to provide one. Certain object mechanisms, such as the resolution of a variant class, are activated by the object's receipt of MSG_META_DUMMY.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: Don't.

Library: **Objects/metaC.def**

MSG_META_FINISH_QUIT

This message is sent to the object that initiated MSG_META_QUIT and has received MSG_META_QUIT_ACK from each party notified. This message informs the object that it has finished sending out all MSG_META_QUIT messages and can go on with quitting (or aborting the quit if that is the case).

Source: Object that initiated MSG_META_QUIT.

Destination: Any object.

Pass: `cx` Non-zero if you want to abort the quit.

Returns: Nothing.

Destroyed: Nothing.

Library: **Objects/metaC.def**

MSG_META_GCN_LIST_ADD

This message adds the passed object to a particular notification list. It returns *true* if the object was successfully added, *false* otherwise. This message is the equivalent of **GCNListAdd()** for individual object GCN list.

Source: Unrestricted.

Destination: Object providing GCN services.

Interception: Unnecessary, as **MetaClass** does the right thing.

Pass:	**dx**	Size of **GCNListParams** structure.
	ss:bp	Pointer to **GCNListParams** structure.
Returns:	CF	Set if optr added, clear otherwise (e.g. if the optr was already on that list).

Destroyed: **ax, cx, dx, bp**

Library: **Objects/metaC.def**

MSG_META_GCN_LIST_DESTROY

This message completely destroys the GCN setup for the caller. It frees all GCN lists, cached events, and overhead data storage. This should only be used when the object is being freed. You will likely never handle or call this message.

Source: Object providing GCN services, often in handler for MSG_META_FINAL_OBJ_FREE.

Destination: Self.

Interception: Unnecessary, as **MetaClass** does the right thing.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects/metaC.def**

MSG_META_GCN_LIST_FIND_ITEM

This message checks whether an object is on a particular GCN list.

Source: Unrestricted.

Destination: Any object providing GCN services.

Pass:
dx	Size of **GCNListParams** structure.
ss:bp	Pointer to **GCNListParams** structure.

Returns:
CF	Set if the item is found on that GCN list; clear otherwise.

Interception: Unnecessary.

Destroyed: **ax, cx, dx, bp**

Library: **Objects/metaC.def**

MSG_META_GCN_LIST_REMOVE

This message removes the passed object from a particular notification list. It returns *true* if the object was successfully removed, *false* otherwise. This message is the equivalent of **GCNListRemove()** for an individual object's GCN list.

Source: Unrestricted.

Destination: Object providing GCN services.

Interception: Unnecessary, as **MetaClass** does the right thing.

Pass:
dx	Size of **GCNListParams** structure.
ss:bp	Pointer to **GCNListParams** structure.

Returns:
CF	Set if the item was found on that GCN list and removed; clear otherwise.

Destroyed: **ax, cx, dx, bp**

Library: **Objects/metaC.def**

Object Assembly Reference

MSG_META_GCN_LIST_SEND

This message sends the given event to all objects in a particular notification list. The event will be freed after being sent. This message is the equivalent of **GCNListSend()** for an individual object GCN list.

Pass: `dx` Size of **GCNListParams** structure.
`ss:bp` Pointer to **GCNListParams** structure.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects/metaC.def**

MSG_META_GET_CLASS

This message returns a pointer to the **ClassStruct** structure of the recipient object's class.

Source: Unrestricted.

Destination: Any object.

Pass: Nothing.

Returns: `cx:dx` The object's **ClassStruct**.

Destroyed: `ax, bp`

Interception: Don't.

Library: **Objects/metaC.def**

MSG_META_GET_FLAGS

This message returns the chunk flags for the object. This works just like the **ObjGetFlags()** routine, but can be used when the object queried is being run by a different thread.

Source: Unrestricted.

Destination: Any object.

Pass:	**cx**	chunk to get flags for.
Returns:	**ah**	**ObjChunkFlags**.
	ah	Set to zero.

Destroyed: **cx, dx, bp**

Interception: Unnecessary, as **MetaClass** does the right thing.

Library: **Objects/metaC.def**

MSG_META_GET_HELP_FILE

This message returns the name of the help file attached to the object sent this message. If no help file is found, the default **MetaClass** handler walks up the tree.

Source: Unrestricted.

Destination: Object in object tree containing help.

Pass:	**cx:dx**	Pointer to the buffer to store the help file name. (Buffer must be FILE_LONGNAME_BUFFER_SIZE bytes long.)
Returns:	**cx:dx**	Buffer filled in with null-terminated string (or null string if no help file was found).
	CF	Set if file was found; clear otherwise.

Destroyed: **ax**

Interception: The default handler walks up the tree; you may intercept to change the help file for a branch.

Library: **Objects/metaC.def**

Object Assembly Reference

MSG_META_GET_INI_CATEGORY

This message returns the .INI file category of the object.

Source: Unrestricted, though generally self.

Destination: Object having options.

Pass: `cx:dx` The buffer for .INI category string. This buffer size cannot store more than 64 bytes.

Returns: `cx:dx` Buffer filled in with null-terminated string (or null string if no category was found).
`CF` Set if buffer was filled; clear otherwise.

Destroyed: `ax, bp`

Interception: Default handler walks up tree, eventually finding name of application. Can be intercepted at any level to change the category for a branch.

Library: **Objects/metaC.def**

MSG_META_GET_OBJ_BLOCK_OUTPUT

This message returns the output optr of an object block that contains the object sent the message.

Source: Unrestricted.

Destination: Any object (except a process object).

Pass: Nothing.

Returns: `cx:dx` Optr of the block's output field.

Destroyed: Nothing.

Interception: Generally not intercepted.

Library: **Objects/metaC.def**

MSG_META_GET_OPTR

This message returns the object's optr. This is useful when combined with MSG_GEN_GUP_CALL_OBJECT_OF_CLASS to get the optr of an object of a given class somewhere up in a Generic Tree.

Note: MSG_GEN_GUP_CALL_OBJECT_OF_CLASS dies if an object of the class doesn't exist. Use MSG_GEN_GUP_TEST_FOR_OBJECT_OF_CLASS before



using MSG_GEN_GUP_CALL_OBJECT_OF_CLASS if there is some question as to whether an object of a given class exists.

Source: Unrestricted.

Destination: Any object.

Pass: Nothing.

Returns: `cx:dx` The object's optr.

Destroyed: `ax, bp`

Library: **Objects/metaC.def**

MSG_META_GET_TARGET_AT_TARGET_LEVEL

This message returns the **GetTargetParams** structure containing, among other things, the current target object at a given target level. The **MetaClass** handler simply returns information about the current object since it is assumed to be the current target. See "Input," Chapter 11 of the Concepts Book, for information on target.

Source: Unrestricted.

Destination: Any object.

Pass: `cx` Zero for leaf, otherwise **TargetLevel**, as defined by UI.

Returns: `cx:dx` optr of target object.
`ax:bp` **ClassStruct** of object.

Returns: Nothing returned explicitly.
retValue Filled with return values.

Interception: Must be handled by target nodes to correctly pass the request on down to the next target below current node in hierarchy.

Library: **Objects/metaC.def**

Object Assembly Reference

MSG_META_GET_VAR_DATA

This message fetches variable data of a given type from an object.

Source: Unrestricted.

Destination: Any object.

Pass: `ss:bp` Pointer to a **GetVarDataParams** structure.
`dx` Size of the **GetVarDataParams** structure.

Returns: Buffer specified by **GetVarDataParams** structure filled with data.
`ax` The size of the data returned. If the vardata entry was not found, then message will return -1.

Destroyed: `cx, dx, bp`

Interception: Generally not intercepted; default **MetaClass** handling performs the desired function.

Library: **Objects/metaC.def**

MSG_META_INITIALIZE

Every object class should provide a handler for this message which should call the superclass and then perform any initialization of the instance data required.

Note that **GenClass** and **VisClass** have a default handler that sets up the Gen and Vis parts automatically.

Source: Object system itself, often in the middle of attempting to deliver another message to an object that hasn't yet been initialized.

Destination: Object whose instance data is not yet initialized.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: Any class wishing to have default instance data values other than all zeros should intercept this message to fill in the initial values for its instance data. For classes other than master classes, standard procedure is to call the superclass first, then perform any additional instance data initialization necessary. Master classes should *not* call the superclass, as MSG_META_INITIALIZE is unique among messages in that it is sent only to classes within the particular master group that needs to be initialized. Handlers of MSG_META_INITIALIZE should limit their activities to just

stuffing instance data—specifically, object messaging is not allowed (though scanning vardata is OK). Object classes that inherit instance data (all but **MetaClass**) should call MSG_META_INITIALIZE on their superclass to initialize that portion of their instance data. In addition, they must initialize their own portion of the instance data (start by assuming it's all zeros). The order won't matter, so long as the handler doesn't depend on the inherited instance data having any particular value. When in doubt, call the superclass first.

Library: **Objects/metaC.def**

■ MSG_META_INITIALIZE_VAR_DATA

This message is sent to an object any time the **ObjVarDerefData()** routine is called and the data type is not found. It should be subclassed by any object that defines a variable type that will be used with **ObjVarDerefData()**. The object must create and initialize the data and return its offset.

Sent to an object having a variable data entry which code somewhere is attempting to access via **ObjVarDerefData()**. It is the object that defines the variable data entry type's responsibility to create the data entry and initialize it at this time, and to return a pointer to the extra data (if any), as returned by **ObjVarAddData()**.

Source: **ObjVarDerefData()** routine. Should not be used as a replacement for **ObjVarAddData()**, or MSG_ADD_VAR_DATA_ENTRY, but may be used any time code is ready to access a particular piece of variable data instance data, knows that the variable data has not yet been created, and wishes to ensure that it does exist.

Destination: Any object stored in an LMem Object block.

Pass: `cx` Variable data type.

Returns: `ax` Offset to extra data created (or, if no extra data, the start of data entry plus the size of **VarDataEntry**). Normally, this would just be the offset returned by the call to **ObjVarAddData()**.

Destroyed: `cx, dx, bp`

Interception: Required by any class which defines a variable data entry type that needs to be initialized before usage. Objects handling this message should first compare the passed data type against variable data types it understands, and pass any unknown types onto the superclass for handling.

Library: **Objects/metaC.def**

Object Assembly Reference

MSG_META_IS_OBJECT_IN_CLASS

This message determines whether the recipient object is a member of a given class (or a subclass of the given class). If the carry flag is set, the object is in the class. If not, the object is not in the class. If a variant class is encountered (when checking to see if the object is an instance of a subclass of the passed class), the object will not be grown out past that class in the search. If you want to do a complete search past variant classes, send a MSG_META_DUMMY first.

Source: Unrestricted.

Destination: Any object.

Pass: `cx:dx` Class to see if object is a member of.

Returns: CF Set if object is a member of the passed class (or a subclass); clear otherwise.

Destroyed: `ax, cx, dx, bp`

Interception: Don't.

Library: **Objects/metaC.def**

MSG_META_LOAD_OPTIONS

This message loads the object's settings from the .INI file.

Source: Unrestricted.

Destination: Any object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: Any object that should load its options should intercept this. Behavior is currently implemented for Generic UI objects.

Library: **Objects/metaC.def**

MSG_META_NOTIFY

This message notifies the recipient that some change or action has taken place. The object must have registered for the notification. The type of change that has occurred depends on the *notificationType* argument (passed in **dx**).

One word of notification data is allowed, but this should not reference a handle which must at some point be destroyed. See MSG_META_NOTIFY_WITH_DATA_BLOCK for such requirements.

Source: Unrestricted.

Destination: Any object, or any of the **GCNListSend()** routines.

Pass:

cx	Manufacturer ID associated with notification type.
dx	What sort of notification is being announced.
bp	One word of data, which will be placed in a vardata field. If more than one word is necessary, use MSG_META_NOTIFY_WITH_DATA_BLOCK, below.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: No general requirements, though particular notification types may place restrictions or requirements on such handling.

Library: **Objects/metaC.def**

■ MSG_META_NOTIFY_OBJ_BLOCK_INTERACTIBLE

This message is sent to an object block's output object when the block changes from being not in-use to being in-use. An object may handle this message to monitor changes of in-use status.

Source: Kernel.

Destination: Object which is set as the output of an object block resource either by **ObjBlockSetOutput()**, or by being pre-defined in an application resource.

Pass:

cx	Handle of object block.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: May be intercepted to learn about change in object block interactable status. No default handling is provided, though you may wish to pass the message onto the superclass in case it is interested in this data as well.

Library: **Objects/metaC.def**

Object Assembly Reference

MSG_META_OBJ_BLOCK_NOT_INTERACTIBLE

This message is sent to an object block's output object when the block changes from being not in-use to being in-use. An object may handle this message to monitor changes of in-use status.

Source: Kernel.

Destination: Object which is set as the output of an Object Block resource either by **ObjBlockSetOutput()**, or by being pre-defined in an application resource.

Pass: **cx** Handle of object block.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: May be intercepted to learn about change in object block interactable status. No default handling is provided, though you may wish to pass the message onto the superclass in case it is interested in this data as well.

Library: **Objects/metaC.def**

MSG_META_NOTIFY_WITH_DATA_BLOCK

This message acts like MSG_META_NOTIFY, but it also carries a handle of a block of data. It is absolutely imperative that if this message is subclassed, the object call its superclass in the handler.

The data block must be set up to use the Block "reference count" mechanism, i.e. be sharable and initialized with **MemInitRefCount()**. Details on the count are noted below.

Source: Unrestricted.

Destination: Any object, or any of the **GCNListSend...** routines

Pass:
cx Manufacturer ID associated with notification type.
dx What sort of notification is being announced.
bp *Sharable* data block having a "reference count" initialized via **MemInitRefCount()**.

NOTE on data block reference counts:
The reference count should hold the total number of references of this data block. This count should be incremented before sending a message holding a reference to this block (using **MemIncRefCount()**). Any messages passing such reference either must have a **MetaClass** handler which decrements

this count and frees the block if it reaches zero or must call **MemDecRefCount()** (which does exactly that). **GCNListSend()** and similar functions add in the number of optrs in any list to which a message referring to this block is sent. Thus, when creating a block which will only be sent using **GCNListSend()**, this count should be initialized to zero. If the block is to be sent to one or more objects or **GCNListSend()** calls, the calling routine should call **MemIncRefCount()** before making the calls, being sure to call **MemIncRefCount()** additionally for any objects that the message is sent to, and then call **MemDecRefCount ()** after the calls, to balance the increment call at the start.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: Message *must* eventually arrive at the **MetaClass** handler, with the handle to the data block with the reference count intact, in order for the block to be freed when no longer referenced. Failure to do so will result in garbage being left on the heap, which will kill the system with repetitive occurrences.

Warning: This message must eventually reach the default **MetaClass** handler, so that the block can be freed when no longer referenced.

Library: **Objects/metaC.def**

■ MSG_META_OBJ_FLUSH_INPUT_QUEUE

This message clears out the message queues associated with an object. This is rarely, if ever called from within an application, and there is little call to subclass it.

This queue-flushing mechanism is used in the Window, Object, and Object Block death mechanisms. Objects that implement their own “hold up input” queues must redirect this message through that queue, so that it is flushed as well.

Source: Kernel (**WinClose()**, **WinSetInfo()**, **ObjFreeObjBlock()**, MSG_META_OBJ_FREE, or MSG_META_BLOCK_FREE).

Destination: Should first be sent to the Kernel’s Input Manager (See the routine **ImInfoInputProcess()**). The message is then relayed first to the System Input Object (usually the **GenSystemClass** object), then to the Geode Input Object (usually a

Object Assembly Reference

GenApplicationClass object), and finally to the owning process, which dispatches the passed event.

Pass:
- **cx** — Event to dispatch upon conclusion of flush.
- **dx** — Block Handle that flushing is being performed for (generally the handle of the destination object in the above event). This is the block from which the "OWNING GEODE", as referenced in the **ObjFlushInputQueueNextStop** enumerated type, is determined.
- **bp** — **ObjFlushInputQueueNextStop** (Zero should be passed in call to first object, from there is sequenced by default **MetaClass** handler).

Returns: Nothing.

Destroyed: Nothing.

Interception: Default **MetaClass** handler implements relay of message from one object to the next, and dispatches the passed event. *Must* be intercepted by any input-flow controlling objects (System object, VisContent) which implement "hold-up" queues that hold up input-related messages. The handlers in such cases should pipe this method through the hold-up queue as it does with the other messages, and finish up when it comes out by sending this message, with all data intact, to the superclass for continued default processing.

Library: **Objects/metaC.def**

MSG_META_OBJ_FREE

This message initiates a sequence which will free an object. The object will be freed after its message queues have been flushed.

This is a fairly low-level operation, and should be performed only after the object has been removed from any tree it is attached to and is otherwise "shut

down." Consider using MSG_GEN_DESTROY for generic objects, MSG_VIS_DESTROY for visible ones.

Source: Unrestricted.

Destination: Any object within a block that is ready to have a low-level delete performed on it (i.e. isn't on screen, isn't linked to objects in other blocks, etc.).

Interception: Unnecessary, as **MetaClass** does the right thing.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects/metaC.def**

■ MSG_META_QUIT

This message, when sent to a GenApplication object, initiates the shutdown sequence for the application. All affected objects are notified.

GenApplication does some error checking for multiple quits or detaches and then starts this sequence by passing MSG_META_QUIT(QL_BEFORE_UI) to the process. The default process handler for MSG_META_QUIT varies depending on the **QuitLevel**, which is passed in, but only when sent to the process.

The message handler for each level of quit should then send MSG_META_QUIT_ACK with the same **QuitLevel** when it is done. The default behavior for a process' MSG_META_QUIT responses are:

QL_BEFORE_UI
: Sends MSG_META_QUIT_ACK to self via queue.

QL_UI
: Sends MSG_GEN_APPLICATION_INITIATE_UI_QUIT(0) to the GenApplication.

QL_AFTER_UI
: Sends MSG_META_QUIT_ACK to self via queue.

QL_DETACH
: Sends MSG_META_DETACH to self via queue.

QL_AFTER_DETACH
: Sends MSG_META_QUIT_ACK to self via queue.

The generic UI objects are first asked to quit via MSG_GEN_APPLICATION_INITIATE_UI_QUIT when sent to a GenApplication (active list). It will cause MSG_META_QUIT to be sent to all objects on the active list that are marked as desiring them. These objects on the active list can handle the MSG_META_QUIT any way they please. The process will be notified by a MSG_META_QUIT_ACK with the **QuitLevel** set to QL_UI.

If you need to send MSG_META_QUIT from within a handler for MSG_GEN_PROCESS_OPEN_APPLICATION, make sure to send it after calling the superclass in your OPEN_APPLICATION handler. If this is undesirable, you may encapsulate MSG_META_QUIT and send it via MSG_META_FLUSH_INPUT_QUEUE. (This is needed to avoid the system thinking that a valid IACP connection is open, aborting the quit.)

Source: Unrestricted. If you need to send MSG_META_QUIT within your MSG_GEN_PROCESS_OPEN_APPLICATION handler, please follow the guidelines illustrated above.

Destination: GenApplication object, process object, or any other object. (Arguments depend on what object the message is sent to.)

Pass: If sent to application object, no arguments passed.

If sent to process object:

dx	**QuitLevel**. If **QuitLevel** is QL_AFTER_DETACH, **si:cx** must be optr to be passed on to MSG_META_QUIT_ACK; otherwise, **cx** must be clear.

If sent to any other object:

cx:dx	optr of object to send MSG_META_QUIT_ACK to.

Returns: Nothing (all cases).

Destroyed: **ax, cx, dx, bp**

Library: **Objects/metaC.def**

MSG_META_QUIT_ACK

This message is sent to a Process object in response to a MSG_META_QUIT. The Process object handles this message by continuing the quit sequence.

Source: Any object having received MSG_META_QUIT

Destination: optr passed in MSG_META_QUIT.

Pass: `dx` **QuitLevel** acknowledging (if responding to a process).
`cx` (non-zero if you want to abort the quit).

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Warning: For processes that subclass MSG_META_QUIT, you cannot abort the quit at the QL_DETACH stage or later.

Library: **Objects/metaC.def**

MSG_META_RELOCATE

This message is sent by the object system to evaluate and resolve all of the object's relocatable instance data fields (pointers, optrs, etc.). Note that this only applies if the class' CLASSF_HAS_RELOC flag is set.

NOTE: The calling of this method is non-standard in that it does not pass through the class's method table. Rather, the handler address is placed after the method table and a direct call is issued. This means a relocation routine should not be bound to MSG_META_RELOCATE but should rather be bound to "`reloc`", which Esp understands to mean the handler is for both MSG_META_RELOCATE and MSG_META_UNRELOCATE.

Note also that relocation-by-routine happens in addition to (but before) any relocation due to the class' relocation table. To suppress relocation-by-table,

Object Assembly Reference

you should initialize the class record with the CLASSF_HAS_RELOC flag to prevent Goc from generating a table for the class.

Source: Kernel, when loading in object block, general resources, or object blocks stored in VM file format.

Destination: Individual object needing relocations beyond what the kernel can do automatically (or that simply request for this message to be sent by having their CLASSF_HAS_RELOC bit set)

Pass:
dx	Type giving some context to the relocation.
bp	Frame to pass to **ObjRelocOrUnRelocSuper()**.

Returns:
CF	Set if error.

Destroyed: **ax, cx, dx**

Interception: Intercepted by any class needing to perform special relocations on its instance data. Superclass should be called, in case a superclass also needs to perform this operation on its own instance data.

Warning: This method may not call **LMemAlloc()**, **LMemReAlloc()**, or **LMemFree()**.

Library: **Objects/metaC.def**

■ MSG_META_RESET_OPTIONS

This message resets the object's settings from the .INI file to their initial state.

Source: Unrestricted. Sent to all objects on the GAGCNLT_SELF_LOAD_OPTIONS and GAGCNLT_STARTUP_LOAD_OPTIONS lists.

Destination: Any object.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Any object that wants to reset its options should intercept this. Behavior is currently implemented for Generic UI objects.

Library: **Objects/metaC.def**

■ MSG_META_RESOLVE_VARIANT_SUPERCLASS

This message is sent by the object system when it needs to know the run-time superclass of a particular object's variant master class. The system sends this message to the object when it first attempts to deliver a message to the

superclass of a variant class. The object must examine itself and determine what its superclass for that master level should be.

Source: Object system.

Destination: Any object with a variant class in its class hierarchy.

Pass: **cx** Master offset of the level being resolved. If you know there's a variant class above your own, you will need to examine this to determine if it is your master level whose variant is being resolved, or the one above you.

Returns: **cx:dx** Superclass to use.

Destroyed: **ax, bp**

Interception: Because variant master classes tend to be strictly administrative in nature, providing useful and very generic functionality to their subclasses, all immediate children of a variant master class will need to intercept this message and return the appropriate class pointer.

Library: **Objects/metaC.def**

■ MSG_META_SAVE_OPTIONS

This message saves an object's options to the .INI file for an object. It is sent via the UI's active list mechanism.

Source: Unrestricted.

Destination: GenApplication object, which in turn broadcasts to everything on list of objects having options needing to be saved

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Objects having options to save should intercept this. Superclass should be called in case any of the superclasses needs similar notification.

Library: **Objects/metaC.def**

■ MSG_META_SEND_CLASSED_EVENT

This message is similar to several MSG_GEN_SEND_... messages defined in **GenClass**. This message sends a previously recorded classed event to a certain type of destination defined in the **TravelOption** argument in **dx**.

This message's interesting behavior is actually added by the User Interface, which defines **TravelOption** types. See the message definition in **GenClass**

Object Assembly Reference

for details. The default behavior provided here in **MetaClass** is to destroy the event if TO_NULL is passed, else to deliver the event to itself if it is capable of handling it (the object is a member of the class stored with the event). The event is always freed, whether or not it is deliverable.

MetaClass recognizes the following **TravelOption** values:

TO_NULL
TO_SELF
TO_OBJ_BLOCK_OUTPUT
TO_PROCESS

TO_OBJ_BLOCK_OUTPUT sends the event to the object block's output set in its block header.

Source: Unrestricted.

Destination: Any object.

Pass: **cx** Classed event.
dx **TravelOption** describing target of message.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**
Classed event freed.

Interception: By default, **MetaClass** handlers deal with just the most primitive of the **TravelOption** values. Object classes can add new **TravelOption** types, but must then intercept this message to implement them (calling the superclass if it doesn't recognize the **TravelOption** passed).

Library: **Objects/metaC.def**

■ MSG_META_SET_FLAGS

This message sets the chunk flags for an object. The chunk flags determine how the object is handled with regard to state saving, dirty state, etc.

Source: Unrestricted.

Destination: Any object.

Pass:

cx	Chunk to set flags for.
dl	Bits to set.
dh	Bits to clear.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Unnecessary, as **MetaClass** does the right thing.

Library: **Objects/metaC.def**

■ MSG_META_SET_HELP_FILE

This message sets the help file for the object sent this message.

Source: Unrestricted, though generally an object sends this to itself.

Destination: An object within a tree.

Pass:

cx:dx	Pointer to help file name. This buffer's size must be at least FILE_LONGNAME_BUFFER_SIZE.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects/metaC.def**

■ MSG_META_SET_OBJ_BLOCK_OUTPUT

This message sets the object block output—the block containing the object sent the message— to the passed optr.

Source: Unrestricted.

Destination: Any object (except a process object).

Pass:

cx:dx	Optr of the object to act as the block's output.

Returns: Nothing.

Destroyed: Nothing.

Library: **Objects/metaC.def**

MSG_META_SHUTDOWN_ACK

This message is sent back in response to a MSG_META_APP_SHUTDOWN. This message serves to notify the object the object has fulfilled the request.

Source: Object having received MSG_META_APP_SHUTDOWN. The default handler will dispatch MSG_META_SHUTDOWN_ACK after **MetaClass** has processed MSG_META_APP_SHUTDOWN. (You could, of course, intercept MSG_META_APP_SHUTDOWN and send MSG_META_SHUTDOWN_ACK yourself in your handler.)

Destination: Optr passed in MSG_META_APP_SHUTDOWN.

Pass:
- **cx** — Data passed in MSG_META_APP_SHUTDOWN.
- **dx:bp** — Object which has completed shutting down.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: **MetaClass** provides default handling for this message when using the **ObjInitDetach()** mechanism. Objects not using this mechanism will want to intercept this message if there is a need to know when the object has completed shutting down.

Library: **Objects/metaC.def**

MSG_META_SHUTDOWN_COMPLETE

This message is sent to the object that initiated the detach sequence after it has received MSG_META_SHUTDOWN_ACK for each **ObjIncDetach()** that was previously called. This message is only sent if **ObjInitDetach()** was previously called passing the message MSG_META_APP_SHUTDOWN.

The default handler for this message sends MSG_META_SHUTDOWN_ACK to the object passed in the original **ObjInitDetach()** call.

Source: **MetaClass** handler for MSG_META_SHUTDOWN_ACK if detach count reaches zero (i.e. no outstanding requests), for objects that are shutdown nodes only (i.e. make use of **ObjInitDetach()**.)

Destination: The object sends this message to itself.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: Intercept if you are using the **ObjInitDetach()** mechanism and need to be notified when all objects have been notified of the detach.

Library: **Objects/metaC.def**

MSG_META_SUSPEND

Suspend calculation in an object.

Source: Normally sent by a controller object but can be sent by anything.

Destination: Any object that implements the mechanism described above.

Pass: None.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: An object that wants to implement the mechanism described above.

Library: **Objects/metaC.def**

Object Assembly Reference

MSG_META_UNRELOCATE

This message causes an object to unresolve all its relocatable instance data fields, returning them to special index values.

Source: Kernel, when loading in object block, general resources, or object blocks stored in VM file format.

Destination: Individual object needing relocations beyond what the kernel can do automatically (or that simply request for this message to be sent by having their CLASSF_HAS_RELOC bit set).

Pass: **dx** Type giving some context to the relocation.
bp Frame to pass to **ObjRelocOrUnRelocSuper()**.

Returns: CF Set if error.

Destroyed: **ax, cx, dx**

Warning: This method may not call **LMemAlloc()**, **LMemReAlloc()**, or **LMemFree()**.

Interception: Intercepted by any class needing to perform special relocations on its instance data. Superclass should be called, in case a superclass also needs to perform this operation on its own instance data.

Library: **Objects/metaC.def**

MSG_META_UNSUSPEND

Unsuspend calculation in an object.

Source: Normally sent by a controller object but can be sent by anything.

Destination: Any object that implements the mechanism described above.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: An object that wants to implement the mechanism described above.

Library: **Objects/metaC.def**

MSG_META_UPDATE_WINDOW

This message is sent as part of the system's window update mechanism. Typically, this message is sent to windowed objects on the GAGNLT_WINDOWS list when the GenApplication object becomes GS_USABLE.

The message passes a bitfield of **UpdateWindowFlags**. These flags determine the type of action prompting the window update.

UWF_ATTACHING
: If set, the message is being sent because the application is attaching.

UWF_DETACHING
: If set, the message is being sent because the application is detaching.

UWF_RESTORING_FROM_STATE
: If set, the application is restoring from state. This flag will only be set if UWF_ATTACHING is also set.

UWF_FROM_WINDOWS_LIST
: If set, the message is being sent because the object is on the GAGCNLT_WINDOWS list, and not because it was later built on demand. This flag will only be set if UWF_ATTACHING is also set.

Source: Window update mechanism.

Destination: Entries on the Application's GAGCNLT_WINDOWS list.

Pass: `cx` **UpdateWindowFlags**.
`dl` **VisUpdateMode**.

Destroyed: Nothing.

Library: **Objects/metaC.def**

■ MSG_META_VM_FILE_DIRTY

This message is sent to all processes that have a VM file open when a block in the file becomes marked dirty for the first time. This is useful if many

Object Assembly Reference

processes may be sharing a VM file. The VM file must be marked VMA_NOTIFY_DIRTY in its attributes.

Source: Kernel VM code.

Destination: **ProcessClass** object.

Pass: **cx** Handle open to the VM file, from the receiving process's perspective.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: May be intercepted at process to do whatever is desired on this occurrence of this event. Default behavior in **GenProcessClass** sends notification to the current model **GenDocumentGroupClass** object.

Library: **Objects/metaC.def**

ProcessClass

Instance Data

Because of its nature, ProcessClass contains no instance data.

Messages

MSG_PROCESS_CREATE_EVENT_THREAD

This message is a utility that creates a new event-driven thread owned by the recipient Process object. Typically, a Process object will send this message to itself when it needs an additional event thread. (It cannot be used to create a non-event driven thread. Use **ThreadCreate()** for this purpose instead.) This is implemented at ProcessClass and takes care of all the details of creating a new event-driven thread. The thread will always receive a MSG_META_ATTACH as its first event.

Source: Unrestricted.

Destination: Any process. This process will own the thread.

Pass: **cx:dx** Object class for the new thread. If you don't have any special messages to handle in this thread besides those intended for

objects run by the thread, you can just specify **ProcessClass** as the object class.

bp Stack size for the new thread. 512 bytes is probably reasonable. If the thread will be running any objects that can undergo keyboard navigation (like dialog boxes and triggers and so forth), you'll probably want to make it 1K. The kernel already adds some extra space for handling interrupts (about 100 bytes).

Returns: CF Clear if thread created.
ax Handle of new thread, if thread was created; otherwise destroyed.

Destroyed: **cx, dx, bp**

Interception: Not necessary, as the default handler provides the message's utility.

Warning: Be careful of deadlock situations.

Library: **Objects\processC.def**

■ MSG_PROCESS_CREATE_UI_THREAD

This is a low-level utility message requesting that the process create the UI thread for an application. Does nothing more than MSG_PROCESS_CREATE_EVENT_THREAD, but is split out to allow for interception or to change class or stack size.

Source: First thread of application process if geode has one or more resources marked as "ui-object" blocks.

Destination: First thread of application process.

Pass: **cx:dx** Object class for the new thread. If you don't have any special messages to handle in this thread besides those intended for objects run by the thread, you can just specify **ProcessClass** as the object class.

bp Stack size for the new thread. 512 bytes is probably reasonable. If the thread will be running any objects that can undergo keyboard navigation (like dialog boxes and triggers and so forth), you'll probably want to make it 1K. The kernel

Object Assembly Reference

already adds some extra space for handling interrupts (about 100 bytes).

Returns: CF — Clear if thread created.
ax — Handle of new thread, if thread was created; otherwise destroyed.

Destroyed: cx, dx, bp

Interception: May be intercepted to change class or stack size before calling superclass, or to replace default handling completely.

Warning: Be careful of deadlock situations.

Library: **Objects\processC.def**

■ MSG_PROCESS_MEM_FULL

This message is sent to a Process object by the memory manager when the heap is getting full. A Process object receiving this message should try to free memory (or mark it discardable) if possible.

Source: Kernel's heap manager.

Destination: All processes.

Pass: **cx** — Member of the **HeapCongestion** enumerated type.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Interception: Any process which can adjust the amount of memory that it is using should respond to this message by reducing its demands on the system heap. For instance, buffers or UI object trees kept purely for performance reasons could be freed or reduced in number.

Library: **Objects\processC.def**

■ MSG_PROCESS_NOTIFY_PROCESS_EXIT

This is sent to a Process object when one of its child processes exits. Many types of processes do not need to know when a child process exists; these processes need not handle this message.

Source: Kernel

Destination: Process of creating geode of child process exiting.

Pass:

cx	Child process that exited.
dx	Exit code. May be an error code.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: This message is provided as notification only, i.e. there is no default handling of it. May be intercepted as desired.

Library: **Objects\processC.def**

■ MSG_PROCESS_NOTIFY_THREAD_EXIT

This message is sent to a Process object when a thread owned by it (via the **ThreadCreate()** routine) exits.

Source: Kernel.

Destination: Process owning thread which is exiting.

Pass:

cx	Handle of thread that exited.
dx	Exit code (may be an error code).

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: This message is provided as notification only (i.e. there is no default handling of it). May be intercepted as desired.

Library: **Objects\processC.def**

Object Assembly Reference

GenProcessClass

Instance Data

Because of its nature, GenProcessClass contains no instance data.

Messages

MSG_GEN_PROCESS_ATTACH_TO_PASSED_STATE_FILE

This message is sent by the User Interface whenever the application is being attached to a state file. This message is sent when either restoring from state or detaching. May be subclassed to provide forced state behavior.

Source: **GenProcessClass** default handler for MSG_META_ATTACH.

Destination: Self.

Pass:
- **cx** — **AppAttachMode** (matches that in **AppLaunchBlock**).
- **dx** — Block of structure **AppLaunchBlock**.

Returns:
- **ax** — Handle of extra block of state data (zero for none).

AppLaunchBlock is preserved.

Destroyed: Nothing.

Interception: May be intercepted to force use of a particular state file (by changing the name of the state file to use before calling superclass).

Library: **Objects\gProcC.def**

MSG_GEN_PROCESS_CLOSE_APPLICATION

This message is sent by the User Interface whenever the application is being shut down (during a detach) and it had been launched in application (as opposed to engine) mode.

Source: **GenProcessClass** default handler for MSG_META_DETACH only

Destination: Self.

Pass: Nothing.

Returns: **cx** Handle of block to save (or NULL for none). Process objects often save global variables to a state file in an extra block. This is the handle of that block.

Destroyed: Nothing.

Interception: Convenient place for code that needs to be executed before application exits, for non-engine mode cases. Superclass must be called.

Library: **Objects\gProcC.def**

MSG_GEN_PROCESS_CLOSE_CUSTOM

This message is sent by the User Interface whenever the application is being shut down (during a detach) and it had been launched in some custom mode (not application or engine) that **GenProcessClass** doesn't know about.

Source: Subclass of **GenProcessClass**.

Destination: Self.

Pass: Nothing.

Returns: **cx** Handle of block to save (or NULL for none).

Destroyed: Nothing.

Interception: Convenient place for code that needs to be executed before the application exits, for custom mode cases. Superclass must be called.

Library: **Objects\gProcC.def**

MSG_GEN_PROCESS_CLOSE_ENGINE

This message is sent by the User Interface whenever the application is being shut down (during a detach) and it had been launched in "engine" mode.

Source: **GenProcessClass** default handler for MSG_META_DETACH only.

Destination: Self.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Interception: Convenient place for code that needs to be executed before the application exits, for engine mode cases. Superclass must be called.

Library: **Objects\gProcC.def**

MSG_GEN_PROCESS_CREATE_NEW_STATE_FILE

This replaces MSG_GEN_PROCESS_ATTACHED_TO_PASSED_STATE_FILE if no state file had been specified in that message. This message's default handler will create a new state file and attach it normally. Can be subclassed to provide forced state file usage (e.g. use a different naming scheme).

Source: **GenProcessClass** default handler for MSG_META_DETACH, if not quitting.

Destination: Self.

Pass: **dx** Block handle to block of structure **AppInstanceReference**. Current path is set to state directory.

Returns: **ax** VM file handle (NULL if you want no state file).

Destroyed: Nothing.

Interception: May be intercepted to change name of state file to create.

Library: **Objects\gProcC.def**

MSG_GEN_PROCESS_GET_PARENT_FIELD

This message is sent by process-libraries (such as the Spool Object Library) to find out which field object is its parent. This message will return the field of the first client of the library.

Source: Unrestricted.

Destination: **GenProcessClass** object.

Pass: Nothing.

Returns: **cx:dx** Parent field.

Destroyed: Nothing.

Interception: Not necessary, as default behavior implements utility.

Library: **Objects\gProcC.def**

MSG_GEN_PROCESS_INSTALL_TOKEN

This message is sent by the desktop to a process to get that process to install its token and moniker lists into the token database.

Source: Generally whatever geode launched this application in engine mode (e.g. GeoManager).

Destination: **GenProcessClass** object of any geode launched in engine mode.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Interception: May be intercepted to install additional tokens. Default behavior installs only application icon.

Library: **Objects\gProcC.def**

MSG_GEN_PROCESS_OPEN_APPLICATION

This message is sent by the User Interface when an application is being loaded from its resource blocks. Applications will often intercept this message to set up certain things before being put on-screen. This is the handler in which, for example, you would register for certain notifications such as the quick-transfer notification.

This is sent to the process itself from MSG_META_ATTACH, whenever the application is being restored to application mode (as opposed to engine

mode), or whenever it is being invoked as in application mode. Data passed is the same as that in MSG_META_ATTACH. The default handler sets the GenApplication object GS_USABLE, and brings the UI up on screen.

This message may be intercepted to open up any data file passed, before the UI for the application is actually set GS_USABLE. Note that the blocks passed need not be freed, as this is done by the caller upon return.

If you intercept this message and need to send MSG_META_QUIT from within the handler, make sure to either send the quit after calling the superclass, or send the quit via MSG_META_FLUSH_INPUT_QUEUE. (If this is not done, the system will think that an IACP connection is open, aborting the quit.)

Source: **GenProcessClass** default handler for MSG_META_ATTACH only.

Destination: Same object.

Pass:

cx	State information about the state and data files.
dx	Handle of **AppLaunchBlock**, or zero if none. This block contains the name of any document file passed into the application on invocation.
bp	Handle of extra state block, or zero if none. This is the same block as returned from MSG_GEN_PROCESS_CLOSE_APPLICATION or MSG_GEN_PROCESS_CLOSE_ENGINE, in some previous MSG_META_DETACH. Is freed by caller—subclasses should not free the extra state block.

Returns: Nothing. (**AppLaunchBlock** and extra state block are preserved.)

Destroyed: **ax, cx, dx, bp**

Interception: Frequently intercepted by an application's own process class to find out when an application is first coming alive in the system. You *must* pass this message on to the superclass, or the application will never come up. Be aware that the entire UI tree for the application is the equivalent of not usable (~GS_USABLE) before the superclass is called, and is usable and up on screen visually after it is called. Thus, it is best to do non-UI related things, and changing of generic attributes and hints *before* calling the superclass. You must wait until *after* calling the superclass to perform any operations which require that objects be fully usable (e.g. bringing up a dialog box). Make sure that if you send MSG_META_QUIT from within your handler that you follow the guidelines mentioned above.

Library: **Objects\gProcC.def**

MSG_GEN_PROCESS_OPEN_ENGINE

This is sent to the process itself from MSG_META_ATTACH, whenever the application is being restored to engine mode, or whenever it is being invoked as in engine mode. Data passed is the same as that in MSG_META_ATTACH. There is no default handler.

This message may be intercepted to open up any data file passed, before any engine commands are delivered to the process. Note that the blocks passed need not be freed, as this is done by the caller upon return.

Source: **GenProcessClass** default handler for MSG_META_ATTACH only.

Destination: Self.

Pass:

cx	State of state and data files.
dx	Handle of **AppLaunchBlock**, or NULL if none. This block contains the name of any document file passed into the application on invocation.

Returns: Nothing. (**AppLaunchBlock** and extra state block are preserved.)

Interception: Generally unnecessary, though can be intercepted if notification of going into this mode is necessary.

Library: **Objects\gProcC.def**

MSG_GEN_PROCESS_RESTORE_FROM_STATE

This message is sent by the User Interface when an application is being loaded from a state file. This is sent to the process itself from MSG_META_ATTACH, whenever the application is being invoked as in MSG_GEN_PROCESS_RESTORE_FROM_STATE mode. Data passed is the same as that in MSG_META_ATTACH. The default handler fetches the application mode message, either MSG_GEN_PROCESS_OPEN_APPLICATION or MSG_GEN_PROCESS_OPEN_ENGINE, as saved in the application object, and sends that message to the process.

Note that the blocks passed need not be freed, as this is done by the caller upon return.

Source: Default **GenProcessClass** handler for MSG_META_ATTACH.

Destination: Self.

Pass:

cx	Flags with information about the state and data files.
dx	Handle of **AppLaunchBlock**, or zero if none. This block contains the name of any document file passed into the application on invocation.
bp	Handle of extra state block, or zero if none. This is the same block as returned from MSG_GEN_PROCESS_CLOSE_APPLICATION or MSG_GEN_PROCESS_CLOSE_ENGINE, in some previous MSG_META_DETACH. Process objects often use this extra block to save global variables to state files.

Returns: Nothing. (**AppLaunchBlock** and extra state block are preserved.)

Interception: Intercepted generally only so that application can retrieve previously saved data out of the state block passed.

Library: **Objects\gProcC.def**

■ MSG_GEN_PROCESS_SEND_TO_APP_GCN_LIST

This message sends the specified event to all the objects registered with the passed GCN list. This message should be subclassed by UI Controller objects.

The handler here merely sends the request on to GenApplication using MSG_META_GCN_LIST_SEND. Controllers should use this message, however, over direct communication with the application object, to ensure orderly updating of the list status event. One such failure case which is fixed is two target text objects, one run by the process thread in a view, the other a GenText run by the UI thread. If the GenText has the target and the user clicks quickly on view and then the GenText, the GenText may process both messages about the target being lost and gained before the process text object receives its gained and lost pair. If both objects sent MSG_META_GCN_LIST_SEND directly to the GenApplication object, the GenText's status would be wiped out by the subsequent reporting by the process text object. This problem is avoided by having both process and UI objects call here to pass status update info. This works because target

changes start out ordered in the UI thread, and that order is passed on to the process thread in either of the two cases.

Source: Any object wishing to update an application GCN list. Don't use queue-order altering message flags such as MF_PLACE_IN_FRONT when sending this message. As a convention must be established for the flag MF_FORCE_QUEUE in order to ensure orderly results, the convention in use is this: Don't pass it.
A typical call should use only the MF_STACK, MF_FIXUP_DS, and/or MF_FIXUP_ES flags, if needed.

Destination: **GenProcessClass** (application process) only.

Pass:

dx	Size of **GCNListMessageParams** structure.
ss:bp	Pointer to **GCNListMessageParams** structure.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Should not generally be intercepted, as **GenProcessClass** provides correct behavior. If intercepted and not sent onto superclass, event passed must be destroyed and data block reference count decremented, to avoid leaving obsolete data on the heap.

Library: **Objects\gProcC.def**

■ MSG_GEN_PROCESS_UNDO_ABORT_CHAIN

This message aborts the current undo chain, destroying all actions in place on the current chain, and instructs the undo mechanism to ignore any undo information until the current undo chain is ended. This latter behavior is needed because the current chain may be nested within several chains, so we must ignore undo chain actions until the outermost chain is ended.

Source: Unrestricted.

Destination: GenProcess object.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Do not intercept.

Library: **Objects\gProcC.def**

Object Assembly Reference

MSG_GEN_PROCESS_UNDO_ACCEPT_ACTIONS

This message causes a process to accept undo messages again.

Source: Edit control.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax,cx, dx, bp

Library: Objects\gProcC.def

MSG_GEN_PROCESS_UNDO_ADD_ACTION

This message adds a new undo action to the current undo chain.

Source: Any object wanting to register an action for undo.

Destination: GenProcessClass.

Pass:	ss:bp	Pointer to **AddUndoActionStruct** containing information which may be used to undo action.
Returns:	ax:bp	Zero if recipient is ignoring undo messages; otherwise, if the value passed in *UAS_datatype* was UADT_PTR or UADT_VMCHAIN, **ax:bp** contains a VMChain or DBItem which may be used to undo the action. If neither of the above cases is true, **ax:bp** is destroyed..

Destroyed: cx, dx

Library: Objects\gProcC.def

MSG_GEN_PROCESS_UNDO_CHECK_IF_IGNORING

This message checks to see if the system is currently ignoring undo actions.

Source: Edit control.

Pass: None.

Returns:	ax	Non-zero if ignoring actions.

Library: Objects\gProcC.def

MSG_GEN_PROCESS_UNDO_END_CHAIN

This message notifies the process of the end of an undo-able action. Note that all this message does is decrement a count—the current undo chain is terminated when the count goes from one to zero. This allows a function to

perform a number of undo-able actions and have them all grouped as a single undo-able action.

Source: Any object wanting to register an action for undo.

Destination: **GenProcessClass** only.

Pass: **cx** Non-zero if you want to delete the chain if it has no actions; zero if the chain is OK without actions (actions will be added later).

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gProcC.def**

■ MSG_GEN_PROCESS_UNDO_FLUSH_ACTIONS

This message flushes the current undo chain (frees all undo actions, notifies edit control that there is no undo item).

Source: Any object using undo.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Should not be intercepted.

Library: **Objects\gProcC.def**

■ MSG_GEN_PROCESS_UNDO_GET_CONTEXT

This message gets the current undo context.

Source: Any object using undo.

Pass: None.

Returns: **cx:dx** Current context.

Destroyed: **ax, bp**

Library: **Objects\gProcC.def**

Object Assembly Reference

MSG_GEN_PROCESS_UNDO_GET_FILE

This message returns a VM file handle to store undo actions. This message is useful to access undo data in either a huge array or DB item. You may also use **GenProcessUndoGetFile()** to retrieve this file instead.

Source: Any object wanting to access the undo file.

Pass: None.

Returns: **ax** File handle of VM file with undo information.

Interception: Should not be intercepted.

Library: **Objects\gProcC.def**

MSG_GEN_PROCESS_UNDO_IGNORE_ACTIONS

This message causes a process to reject any undo messages.

Source: Edit control.

Pass: **cx** Non-zero to flush the queue.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gProcC.def**

MSG_GEN_PROCESS_UNDO_PLAYBACK_CHAIN

This message plays back the current undo chain, one action at a time. It simultaneously creates a "redo" chain, so the undone action can be redone.

Source: Edit control.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gProcC.def**

MSG_GEN_PROCESS_UNDO_SET_CONTEXT

This message sets the current undo context. This allows the application to have separate undo chains associated with various documents or modes. This should be sent out before any other undo related messages. The document control automatically takes care of this when a document gets the model exclusive.

Passing NULL_UNDO_CONTEXT as the new context will trigger some zealous EC code if any other undo messages are sent while the context is null.

Source: Any object using undo.

Pass: `cx.dx` New context (this has no meaning to the undo mechanism—it's just a value).

Returns: `cx.dx` Old context.

Destroyed: `ax, bp`

Interception: Generally, should not be intercepted. Applications wanting to override the default behavior should at least flush out the current undo actions, as they will probably not be valid in the new context.

Library: **Objects\gProcC.def**

■ MSG_GEN_PROCESS_UNDO_START_CHAIN

This message notifies the process of the start of an undo-able action. Note that all this message does is increment a count—a new undo chain is created when the count goes from zero to one. This allows a function to perform a number of undo-able actions and have them all grouped as a single undo-able action.

Source: Any object wanting to register an action for undo.

Destination: **GenProcessClass** only.

Pass: `ss:bp` A pointer to a **StartUndoableActionStruct**.

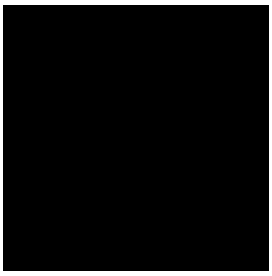
Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gProcC.def**

Object Assembly Reference

GenClass



GenClass 65

2

GenClass

Instance Data

```
GI_link                 LinkPart <>
   UIC Syntax:  genLink   = linkComp kbdPath;
```

genLink (*GI_link*) is an internal field.

```
GI_comp                 CompPart <>
   UIC Syntax:  children  = compositeComp genLink;
```

children (GI_comp) assigns child objects to the current object (the parent). *children* takes a list of generic objects separated by commas as arguments in its instance field. It then sets each of those arguments as children of the current object. In Espire, the *children* arguments will be used to create an internal generic tree structure, reassigning present links using both the *children* instance field and an internal instance field, *genLink* (*GI_link*). The *genLink* instance field is therefore an elemental instance field for every generic object but is rarely, if ever, accessed directly.

```
GI_visMoniker           lptr
   UIC Syntax:  moniker   = visMonikerComp;
```

moniker (GI_visMoniker) chooses the title for the generic object, usually added in an appropriate location within the object by the specific UI. The moniker can be text, in the form of a null-terminated text string, or as a graphics string, usually set up separately.

```
GI_kbdAccelerator    KeyboardShortcut <>
   UIC Syntax:  kbdAccelerator   = kbdAcceleratorComp;
```

kbdAccelerator (GI_kbdAccelerator) sets a given keystroke as a keyboard accelerator for the object. Usually, an object is activated in a manner relating to its specific UI implementation. Pressing the accelerator keystroke will activate the object in the same manner as if its default activation action had occurred. The instance field takes a modifier (*alt*, *shift*, *ctrl*, or combinations of these) and a character (a valid keyboard character) as its arguments. You can further specify the specific keyboard set to use.

```
GI_attrs                GenAttrs
   UIC Syntax:  genAttributes    = bitfieldComp byte {
                   signalInteractionComplete,
                   initiatesBusyState,
                   initiatesInputHoldUp,
                   initiatesInputIgnore,
                   readOnly,
                   kbdPath,
                   targetable,
                   notifyVisibility
                   } : {};
```

genAttributes (GI_attrs) defines how actions performed by the generic object will affect user interface input and output. You may set any, all, or none of these attributes within your object declaration.

```
GI_states               GenStates
   UIC Syntax:  genStates     = bitfieldComp byte {
                   usable,
                   enabled
                   } : usable, enabled;
```

genStates (GI_states) defines in what state a generic object will be represented. Default states are *usable* and *enabled*. You may set any, all, or none of these states within your object declaration.

Variable Instance Data

```
ATTR_GEN_PROPERTY                                   vardata
ATTR_GEN_NOT_PROPERTY                               vardata
```

ATTR_GEN_PROPERTY and ATTR_GEN_NOT_PROPERTY affect a generic object's ability to behave as a properties group. These attributes override the property group's default behavior. Generic objects marked as properties in which case in delayed or immediate mode.

```
ATTR_GEN_DESTINATION_CLASS       vardata DestinationClassArgs
```

ATTR_GEN_DESTINATION_CLASS specifies an object class that should handle all messages sent out along a destination path. The type **DestinationClassArgs** contains a pointer to a **ClassStruct**. Typically, this attribute is used with a **TravelOption** destination to form a search path. The message for that generic object travels down a path specified in the **TravelOption** until it encounters an object class of type ATTR_GEN_DESTINATION_CLASS; at that point, the message will be handled.

Object Assembly Reference

```
ATTR_GEN_INIT_FILE_KEY                          vardata char
ATTR_GEN_INIT_FILE_CATEGORY                     vardata char
ATTR_GEN_INIT_FILE_PROPAGATE_TO_CHILDREN        vardata
ATTR_GEN_USES_HIERARCHICAL_INIT_FILE_CATEGORY vardata
```

ATTR_GEN_INIT_FILE_KEY, ATTR_GEN_INIT_FILE_CATEGORY, ATTR_GEN_INIT_FILE_PROPAGATE_TO_CHILDREN, and ATTR_GEN_USES_HIERARCHICAL_INIT_FILE_CATEGORY allow a generic object to load options from the .INI file upon startup (after receiving a MSG_META_ATTACH). Generic objects with these attributes will receive a MSG_GEN_LOAD_OPTIONS (with the category and key of the proper entry in the .INI file) and may intercept that message to provide additional behavior.

```
ATTR_GEN_POSITION                               vardata Point
ATTR_GEN_POSITION_X                             vardata sword
ATTR_GEN_POSITION_Y                             vardata sword
```

ATTR_GEN_POSITION, ATTR_GEN_POSITION_X, and ATTR_GEN_POSITION_Y override default geometry management of an object and set a precise position for the object to appear within the UI, offset from the parent object. ATTR_GEN_POSITION specifies both a horizontal and a vertical position to place the object. ATTR_GEN_POSITION_X and ATTR_GEN_POSITION_Y specify either a horizontal or vertical position for the object. You should avoid using these attributes because they override default behavior and may result in different (and potentially unsightly) appearances under different specific UIs.

```
ATTR_GEN_SEND_APPLY_MSG_ON_APPLY_EVEN_IF_NOT_MODIFIED vardata
ATTR_GEN_SEND_APPLY_MSG_ON_APPLY_EVEN_IF_NOT_ENABLED vardata
```

ATTR_GEN_SEND_APPLY_MSG_ON_APPLY_EVEN_IF_NOT_MODIFIED and ATTR_GEN_SEND_APPLY_MSG_ON_APPLY_EVEN_IF_NOT_ENABLED affect generic objects that contain an apply message—a message that an object sends out when it wishes to apply changes made within that object. By default, an apply message will not be sent out if the object's state has not changed since the last apply or if the object is not enabled; these attributes tell the object to send out its apply message regardless of these conditions.

```
ATTR_GEN_VISIBILITY_DATA                        vardata dword
ATTR_GEN_VISIBILITY_MESSAGE                     vardata word
ATTR_GEN_VISIBILITY_DESTINATION                 vardata optr
```

ATTR_GEN_VISIBILITY_DATA, ATTR_GEN_VISIBILITY_MESSAGE, and ATTR_GEN_VISIBILITY_DESTINATION set the data, message or destination to use with the visibility notification mechanism in **GenClass**. The visibility mechanism allows an application to know when an object becomes visible

and not visible. An object provides this notification if its GA_NOTIFY_VISIBILITY bit is set.

```
ATTR_GEN_PATH_DATA                              vardata GenFilePath
```

ATTR_GEN_PATH_DATA stores an object's generic path information in the form of a **GenFilePath** structure. This structure contains disk and path information including path name and file name; generic objects can use this attribute to store current or default paths and files.

```
ATTR_GEN_FEATURE_LINK                           vardata lptr
```

ATTR_GEN_FEATURE_LINK contains a link to the next feature used by the GenControl mechanism when a controller's features map to multiple objects. It is unlikely you will use this attribute.

```
ATTR_GEN_DEFAULT_MONIKER          vardata GenDefaultMonikerType
```

ATTR_GEN_DEFAULT_MONIKER specifies a default moniker for a generic object. This is generally used for graphic monikers that occur several places within the system.

```
ATTR_GEN_OUTPUT_TRAVEL_START                    vardata optr
```

ATTR_GEN_OUTPUT_TRAVEL_START allows you to set a specific object to begin the **TravelOption**. Normally, objects that contain a **TravelOption** in their destination field use their own object as the point of reference for sending messages along travel paths. (I.e., sending a message TO_FOCUS would send a message along the focus path, starting at the sending object.) This can be used for optimization purposes, enabling you to skip several layers of hierarchies.

```
ATTR_GEN_HELP_FILE                              vardata char
ATTR_GEN_HELP_TYPE                              vardata byte
ATTR_GEN_HELP_FILE_FROM_INIT_FILE               vardata
ATTR_GEN_FOCUS_HELP                             vardata optr
                noreloc ATTR_GEN_FOCUS_HELP
ATTR_GEN_FOCUS_HELP_LIB                         vardata optr
ATTR_GEN_HELP_CONTEXT                           vardata char
```

ATTR_GEN_HELP_FILE, ATTR_GEN_HELP_TYPE, ATTR_GEN_HELP_FILE_FROM_INIT_FILE, ATTR_GEN_FOCUS_HELP, ATTR_GEN_FOCUS_HELP_LIB, and ATTR_GEN_HELP_CONTEXT specify the configuration of the help files for a generic object. Focus help starts help notification when the generic object gains the focus.

```
ATTR_GEN_WINDOW_CUSTOM_LAYER_ID                 vardata hptr
ATTR_GEN_WINDOW_CUSTOM_WINDOW_PRIORITY          vardata WinPriority
ATTR_GEN_WINDOW_CUSTOM_LAYER_PRIORITY           vardata LayerPriority
```

```
ATTR_GEN_WINDOW_CUSTOM_PARENT                vardata hptr.Window
ATTR_GEN_CUSTOM_WINDOW                       vardata
```

ATTR_GEN_WINDOW_CUSTOM_LAYER_ID, ATTR_GEN_WINDOW_CUSTOM_WINDOW_PRIORITY, ATTR_GEN_WINDOW_CUSTOM_LAYER_PRIORITY, ATTR_GEN_WINDOW_CUSTOM_PARENT, and ATTR_GEN_CUSTOM_WINDOW specify a window's interaction and place within the window system.

```
ATTR_GEN_WINDOW_ACCEPT_INK_EVEN_IF_NOT_FOCUSED vardata
ATTR_GEN_WINDOW_KBD_OVERRIDE           vardata KeyboardOverride
ATTR_GEN_WINDOW_KBD_POSITION           vardata Point
```

ATTR_GEN_WINDOW_ACCEPT_INK_EVEN_IF_NOT_FOCUSED, ATTR_GEN_WINDOW_KBD_OVERRIDE, and ATTR_GEN_WINDOW_KBD_POSITION affect how input flows within a generic window.

Geometry Hints

Many of the hints provided in **GenClass** manage the geometry of generic objects. These hints are listed below, but they are described within "Managing UI Geometry," Chapter 12 of the Concepts Book. Other hints which are not related to geometry are listed after these (and described within this reference).

```
HINT_DONT_ALLOW_CHILDREN_TO_WRAP             vardata
HINT_ALLOW_CHILDREN_TO_WRAP                  vardata
HINT_WRAP_AFTER_CHILD_COUNT                  vardata word
HINT_WRAP_AFTER_CHILD_COUNT_IF_VERTICAL_SCREEN vardata word

HINT_BOTTOM_JUSTIFY_CHILDREN                 vardata
HINT_LEFT_JUSTIFY_CHILDREN                   vardata
HINT_RIGHT_JUSTIFY_CHILDREN                  vardata
HINT_TOP_JUSTIFY_CHILDREN                    vardata
HINT_FULL_JUSTIFY_CHILDREN_HORIZONTALLY      vardata
HINT_FULL_JUSTIFY_CHILDREN_VERTICALLY        vardata
HINT_DONT_FULL_JUSTIFY_CHILDREN              vardata

HINT_CUSTOM_CHILD_SPACING                    vardata SpecSizeSpec
HINT_CUSTOM_CHILD_SPACING_IF_LIMITED_SPACE   vardata SpecSizeSpec
HINT_DONT_INCLUDE_ENDS_IN_CHILD_SPACING      vardata
HINT_INCLUDE_ENDS_IN_CHILD_SPACING           vardata
HINT_MINIMIZE_CHILD_SPACING                  vardata
```

```
HINT_LEFT_JUSTIFY_MONIKERS                  vardata
HINT_CENTER_MONIKER                         vardata
HINT_DO_NOT_USE_MONIKER                     vardata
HINT_ALIGN_LEFT_MONIKER_EDGE_WITH_CHILD     vardata

HINT_CENTER_CHILDREN_HORIZONTALLY           vardata
HINT_CENTER_CHILDREN_ON_MONIKERS            vardata
HINT_CENTER_CHILDREN_VERTICALLY             vardata

HINT_ALIGN_LEFT_EDGE_WITH_OBJECT            vardata optr
HINT_ALIGN_TOP_EDGE_WITH_OBJECT             vardata optr
HINT_ALIGN_RIGHT_EDGE_WITH_OBJECT           vardata optr
HINT_ALIGN_BOTTOM_EDGE_WITH_OBJECT          vardata optr

HINT_FIXED_SIZE                         vardata CompSizeHintArgs
HINT_INITIAL_SIZE                       vardata CompSizeHintArgs
HINT_MAXIMUM_SIZE                       vardata CompSizeHintArgs
HINT_MINIMUM_SIZE                       vardata CompSizeHintArgs
HINT_SIZE_WINDOW_AS_DESIRED                 vardata

HINT_DRAW_IN_BOX                            vardata

HINT_EXPAND_HEIGHT_TO_FIT_PARENT            vardata
HINT_EXPAND_WIDTH_TO_FIT_PARENT             vardata

HINT_MAKE_REPLY_BAR                         vardata

HINT_NO_TALLER_THAN_CHILDREN_REQUIRE        vardata
HINT_NO_WIDER_THAN_CHILDREN_REQUIRE         vardata

HINT_ORIENT_CHILDREN_HORIZONTALLY           vardata
HINT_ORIENT_CHILDREN_VERTICALLY             vardata
HINT_ORIENT_CHILDREN_ALONG_LARGER_DIMENSION vardata
HINT_SAME_ORIENTATION_AS_PARENT             vardata

HINT_PLACE_MONIKER_ABOVE                    vardata
HINT_PLACE_MONIKER_BELOW                    vardata
HINT_PLACE_MONIKER_TO_LEFT                  vardata
HINT_PLACE_MONIKER_TO_RIGHT                 vardata
HINT_PLACE_MONIKER_ALONG_LARGER_DIMENSION   vardata
HINT_NO_BORDER_ON_MONIKERS                  vardata

HINT_SEEK_MENU_BAR                          vardata
HINT_AVOID_MENU_BAR                         vardata
HINT_SEEK_REPLY_BAR                         vardata

HINT_SEEK_X_SCROLLER_AREA                   vardata
HINT_SEEK_Y_SCROLLER_AREA                   vardata
HINT_SEEK_LEFT_OF_VIEW                      vardata
```

```
HINT_SEEK_TOP_OF_VIEW                          vardata
HINT_SEEK_RIGHT_OF_VIEW                        vardata
HINT_SEEK_BOTTOM_OF_VIEW                       vardata
HINT_SEEK_TITLE_BAR_LEFT                       vardata
HINT_SEEK_TITLE_BAR_RIGHT                      vardata

HINT_USE_INITIAL_BOUNDS_WHEN_RESTORED          vardata
HINT_DIVIDE_WIDTH_EQUALLY                      vardata
HINT_DIVIDE_HEIGHT_EQUALLY                     vardata

HINT_KEEP_INITIALLY_ONSCREEN                   vardata
HINT_DONT_KEEP_INITIALLY_ONSCREEN              vardata
HINT_KEEP_PARTIALLY_ONSCREEN                   vardata
HINT_DONT_KEEP_PARTIALLY_ONSCREEN              vardata
HINT_KEEP_ENTIRELY_ONSCREEN                    vardata
HINT_KEEP_ENTIRELY_ONSCREEN_WITH_MARGIN        vardata

HINT_POPS_UP_TO_RIGHT                          vardata
HINT_POPS_UP_BELOW                             vardata

HINT_NOT_MOVABLE                               vardata
HINT_POSITION_WINDOW_AT_RATIO_OF_PARENT vardata SpecWinSizePair
HINT_POSITION_WINDOW_AT_MOUSE                  vardata

HINT_WINDOW_NO_TITLE_BAR                       vardata
HINT_WINDOW_NO_SYS_MENU                        vardata
HINT_WINDOW_MINIMIZE_TITLE_BAR                 vardata
HINT_WINDOW_ALWAYS_DRAW_WITH_FOCUS             vardata

HINT_STAGGER_WINDOW                            vardata
HINT_CENTER_WINDOW                             vardata
HINT_TILE_WINDOW                               vardata
HINT_EXTEND_WINDOW_TO_BOTTOM_RIGHT             vardata
HINT_EXTEND_WINDOW_NEAR_BOTTOM_RIGHT           vardata
HINT_SIZE_WINDOW_AS_RATIO_OF_PARENT    vardata SpecWinSizePair
HINT_SIZE_WINDOW_AS_RATIO_OF_FIELD     vardata SpecWinSizePair
HINT_WINDOW_NO_CONSTRAINTS                     vardata
HINT_POSITION_ICON_AS_RATIO_OF_FIELD   vardata SpecWinSizePair
```

Other Hints

Other hints in **GenClass** do not affect geometry. These hints are listed below and are explained within this chapter. Within **GenClass**, some hints are internal; these are not listed



HINT_FREQUENTLY_USED vardata
HINT_INFREQUENTLY_USED vardata
HINT_AN_ADVANCED_FEATURE vardata

HINT_FREQUENTLY_USED, HINT_INFREQUENTLY_USED, and HINT_AN_ADVANCED_FEATURE are descriptive hints that describe the generic object's role within the UI. It is, of course, left up to the specific UI to implement any special behavior based on these hints.

HINT_DEFAULT_DEFAULT_ACTION vardata
HINT_ENSURE_TEMPORARY_DEFAULT vardata
HINT_PREVENT_DEFAULT_OVERRIDES vardata

HINT_DEFAULT_DEFAULT_ACTION, HINT_ENSURE_TEMPORARY_DEFAULT, and HINT_PREVENT_DEFAULT_OVERRIDES affect the default activation of objects within the UI. Objects with default activation can be activated indirectly (e.g., pressing the ENTER key within a dialog box activates a default trigger).

HINT_SAME_CATEGORY_AS_PARENT vardata

HINT_SAME_CATEGORY_AS_PARENT indicates to the specific UI that this object should be treated in the same manner as its parent. Currently, this hint only affects GenInteractions. By default, GIV_SUB_GROUP GenInteractions within a GIV_POPUP menu have separator lines drawn around them to set them apart from other children. This hint removes those lines.

HINT_USE_TEXT_MONIKER vardata
HINT_USE_ICONIC_MONIKER vardata

HINT_USE_TEXT_MONIKER and HINT_USE_ICONIC_MONIKER indicates that the generic object should choose either a text moniker or an iconic moniker if there exists a choice within the object's moniker list. Currently, these hints are not implemented.

HINT_DEFAULT_FOCUS vardata
HINT_DEFAULT_TARGET vardata
HINT_DEFAULT_MODEL vardata

HINT_DEFAULT_FOCUS, HINT_DEFAULT_TARGET, and HINT_DEFAULT_MODEL specify that the generic object is the default focus, target, or model node at this level in the application.

HINT_PRESERVE_FOCUS vardata
HINT_DO_NOT_PRESERVE_FOCUS vardata

Object Assembly Reference

HINT_PRESERVE_FOCUS and HINT_DO_NOT_PRESERVE_FOCUS affect how the focus node changes after an object's activation. For a complete discussion of the focus and target hierarchies, see "Input," Chapter 11 of the Concepts Book.

`HINT_GENERAL_CONSUMER_MODE` `vardata`

HINT_GENERAL_CONSUMER_MODE instructs the specific UI to show the object in GCM mode. This hint takes an argument of type **GeneralConsumerModeFlags**; this record contains two three-bit flags: GCMF_LEFT_ICON and GCMF_RIGHT_ICON, which specify which icon to use on both the left and right sides of the title bar. Each icon must be of type **GCMIcon**; current types are GCMI_NONE, GCMI_EXIT, and GCMI_HELP.

`HINT_NEVER_ADOPT_MENUS` `vardata`
`HINT_ALWAYS_ADOPT_MENUS` `vardata`

HINT_NEVER_ADOPT_MENUS and HINT_ALWAYS_ADOPT_MENUS affect the behavior of menus placed within a GenDisplay. Menus can either sit within the GenDisplay or appear in the associated GenPrimary when that GenDisplay is the target. Menus within GenDisplays are discouraged, however, as they are a fairly advanced UI metaphor.

`HINT_NAVIGATION_ID` `vardata word`
`HINT_NAVIGATION_NEXT_ID` `vardata word`

HINT_NAVIGATION_ID and HINT_NAVIGATION_NEXT_ID allow the field navigation path within a windowed object. HINT_NAVIGATION_NEXT_ID sets the object to navigate to from the current object. An object with a matching HINT_NAVIGATION_ID must exist.

`HINT_DISMISS_WHEN_DISABLED` `vardata`

HINT_DISMISS_WHEN_DISABLED instructs the specific UI to dismiss the object when it is no longer fully enabled. This is typically useful for independently-displayable windows that should be dismissed in such cases.

`HINT_TOOLBOX` `vardata`

HINT_TOOLBOX indicates that the generic object and all objects below it in the visual hierarchy are part of a "toolbox" and should draw themselves appropriately. Objects in toolboxes are typically much smaller than other components.

`HINT_SHOW_SHORTCUT` `vardata`
`HINT_DONT_SHOW_SHORTCUT` `vardata`
`HINT_DRAW_SHORTCUT_BELOW` `vardata`

HINT_SHOW_SHORTCUT, HINT_DONT_SHOW_SHORTCUT, and HINT_DRAW_SHORTCUT_BELOW affect the display of keyboard accelerators for generic objects.

```
HINT_CAN_CLIP_MONIKER_WIDTH                 vardata
HINT_CAN_CLIP_MONIKER_HEIGHT                vardata
HINT_SHOW_ENTIRE_MONIKER                    vardata
```

HINT_CAN_CLIP_MONIKER_WIDTH and HINT_CAN_CLIP_MONIKER_HEIGHT instruct the specific UI to clip the object's moniker height or width if not enough space is provided for them. Objects with HINT_SHOW_ENTIRE_MONIKER will force the object to leave enough space to show its entire moniker, if possible.

```
HINT_GADGET_BACKGROUND_COLORS          vardata BackgroundColors
HINT_GADGET_TEXT_COLOR                 vardata TextColors
```

HINT_GADGET_BACKGROUND_COLORS causes grouping objects in toolboxes to draw custom background colors. The hint takes an argument of type **BackgroundColors**. Two colors can each be chosen for both the unselected state and the selected state; they will be blended together depending on the state of the object. If both colors are set to the same value, a solid color results. HINT_GADGET_TEXT_COLOR selects the text color of a visual moniker, typically within a toolbox. This hint takes an argument of type **TextColors**.

```
HINT_IF_SYSTEM_ATTRS                        vardata SystemAttrs
HINT_ENDIF                                  vardata
HINT_ELSE                                   vardata
```

HINT_IF_SYSTEM_ATTRS, HINT_ENDIF, and HINT_ELSE may be used together to conditionally add hints to an object based on certain system criteria. If the **SystemAttrs** set in the HINT_IF_SYSTEM_ATTRS field are true for the current system, then the hints that follow (until a HINT_ENDIF is encountered) are included. If no HINT_ENDIF is encountered, then only the next hint is included. If the **SystemAttrs** do not match the current system, the following group of hints is deleted.

Remember that these vardata entries are hints; they can be ignored by the specific UI and their implementation varies from object to object. Some hints, for example, are only relevant for certain object classes. Your mileage may vary.

Object Assembly Reference

Messages

MSG_GEN_ACTIVATE

This message manually activates an object as if the user had activated it in the manner defined by the specific UI. For example, this message may activate GenTriggers without use of the mouse. This message causes all normal visual cues associated with the activation.

Source: Unrestricted.

Destination: Usually sent by an object to itself.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\genC.def

MSG_GEN_ACTIVATE_INTERACTION_DEFAULT

This message acts on a default exclusive object within a window, forcing that object to activate as in MSG_GEN_ACTIVATE. This message travels up the visible tree to the first windowed object and sends a MSG_GEN_ACTIVATE to the default object selected there. (This default object is typically marked with the hint HINT_DEFAULT_DEFAULT_ACTION.)

Source: Unrestricted.

Destination: Any GS_USABLE **GenClass** object (usually within a dialog box).

Pass: Nothing.

Returns: CF set if the default action was activated.

Destroyed: ax, cx, dx, bp

Library: Objects\genC.def

MSG_GEN_ADD_CHILD

This message adds the passed object as a child of the recipient. The child object must not be GS_USABLE before being added to the generic tree. Make sure not to add a child that is already a child of some other parent. It is also illegal to add an object that is already a child of the parent. If necessary, check first if the specific child currently exists using MSG_GEN_FIND_CHILD.

In most cases a routine that adds a new child will follow three phases: checking whether the child exists, adding the child, and setting the child GS_USABLE.

The child object, if already specifically initialized, must be unbuilt before being added to the parent. This ensures that the object will be built out correctly. The internal keyboard search path attribute, GA_KBD_SEARCH_PATH, is also cleared and reset for the child.

Pass this message the optr of the child object to add, along with the **CompChildFlags** to use. If CCF_MARK_DIRTY is specified, the new linkage will be saved to state when the application is detached. You must pass a CCF_REFERENCE in **CompChildFlags** to specify the position to add the child. The special constants CCO_FIRST and CCO_LAST, which are special cases of CCF_REFERENCE, will add the object as the first or last child of the parent, respectively.

Note that the object must currently exist. MSG_GEN_ADD_CHILD merely sets up the correct linkage and reconfigures your UI. Note also that successive additions of children using the flag CCO_FIRST will result in a "reverse order" of children (the last added will be the first child, the first added will be the last).

Source: Unrestricted.

Destination: Any generic object.

Pass:		
	cx:dx	The optr of the object to add to the current object's children. This child must not be usable.
	bp	**CompChildFlags** to use when adding the child.

Returns: Nothing.

Destroyed: **ax, bp**

Library: **Objects\genC.def**

MSG_GEN_ADD_CHILD_UPWARD_LINK_ONLY

This message sets a parent link to a passed child object without adding a composite link from the parent to the child. This is a "one way" link in that

the parent does not have knowledge of its new child. Therefore, it must be used with caution.

Source: Unrestricted.

Destination: Any generic object.

Pass: `cx:dx` Optr of the child to add with an upward link.

Returns: Nothing.

Destroyed: `ax, bp`

Library: **Objects\genC.def**

MSG_GEN_ADD_GEOMETRY_HINT

This message adds a geometry hint to a generic object. The object may be GS_USABLE at the time, in which case the object is redrawn according to the new geometry configuration. The hint can be any of the following:

HINT_ALLOW_CHILDREN_TO_WRAP
HINT_BOTTOM_JUSTIFY_CHILDREN
HINT_CENTER_CHILDREN_HORIZONTALLY
HINT_CENTER_CHILDREN_ON_MONIKERS
HINT_CENTER_CHILDREN_VERTICALLY
HINT_CENTER_MONIKER
HINT_CUSTOM_CHILD_SPACING
HINT_DONT_ALLOW_CHILDREN_TO_WRAP
HINT_DONT_FULL_JUSTIFY_CHILDREN
HINT_DONT_INCLUDE_ENDS_IN_CHILD_SPACING
HINT_DO_NOT_USE_MONIKER
HINT_DRAW_IN_BOX
HINT_EXPAND_HEIGHT_TO_FIT_PARENT
HINT_EXPAND_WIDTH_TO_FIT_PARENT
HINT_FULL_JUSTIFY_CHILDREN_HORIZONTALLY
HINT_FULL_JUSTIFY_CHILDREN_VERTICALLY
HINT_INCLUDE_ENDS_IN_CHILD_SPACING
HINT_LEFT_JUSTIFY_CHILDREN
HINT_LEFT_JUSTIFY_MONIKERS
HINT_NO_TALLER_THAN_CHILDREN_REQUIRE
HINT_NO_WIDER_THAN_CHILDREN_REQUIRE
HINT_ORIENT_CHILDREN_HORIZONTALLY
HINT_ORIENT_CHILDREN_VERTICALLY
HINT_PLACE_MONIKER_ABOVE
HINT_ALIGN_LEFT_MONIKER_EDGE_WITH_CHILD
HINT_PLACE_MONIKER_TO_LEFT
HINT_PLACE_MONIKER_TO_RIGHT

HINT_RIGHT_JUSTIFY_CHILDREN
HINT_TOP_JUSTIFY_CHILDREN
HINT_WRAP_AFTER_CHILD_COUNT
HINT_DIVIDE_WIDTH_EQUALLY
HINT_DIVIDE_HEIGHT_EQUALLY
HINT_USE_INITIAL_BOUNDS_WHEN_RESTORED
HINT_KEEP_INITIALLY_ONSCREEN
HINT_DONT_KEEP_INITIALLY_ONSCREEN
HINT_KEEP_PARTIALLY_ONSCREEN
HINT_KEEP_ENTIRELY_ONSCREEN
HINT_KEEP_ENTIRELY_ONSCREEN_WITH_MARGIN
HINT_DONT_KEEP_PARTIALLY_ONSCREEN
HINT_POSITION_WINDOW_AT_RATIO_OF_PARENT
HINT_STAGGER_WINDOW
HINT_CENTER_WINDOW
HINT_TILE_WINDOW
HINT_POSITION_WINDOW_AT_MOUSE
HINT_EXTEND_WINDOW_TO_BOTTOM_RIGHT
HINT_EXTEND_WINDOW_NEAR_BOTTOM_RIGHT
HINT_SIZE_WINDOW_AS_DESIRED
HINT_CAN_CLIP_MONIKER_WIDTH
HINT_CAN_CLIP_MONIKER_HEIGHT
HINT_SHOW_ENTIRE_MONIKER
ATTR_GEN_POSITION
ATTR_GEN_POSITION_X
ATTR_GEN_POSITION_Y

Source: Unrestricted.

Destination: Any generic object.

Pass:

cx	Geometry Hint.
bp	Associated word of data for hint, if any.
dl	**VisUpdateMode**.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

■ MSG_GEN_APPLY

This message is sent to property gadgets to cause them to dispatch their "apply" message and at the same time reset any modified states associated with that object. If this message is sent to a GenInteraction within a dialog box, the GenInteraction will propagate this message to all of its children.

This message may be sent by a GIT_PROPERTIES Interaction upon receipt of MSG_GEN_GUP_INTERACTION_COMMAND with IC_APPLY. You may send this message yourself if you wish for changes in state within the dialog box to be reflected in your application.

Source: Usually a GIT_PROPERTIES Interaction (with an IC_APPLY trigger). You may send this message yourself to force generic objects to apply their changes.

Destination: Any GenInteraction within a dialog box, or any gadget with properties (GenItemGroup, GenValue, etc.).

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

■ MSG_GEN_BRANCH_REPLACE_OUTPUT_OPTR_CONSTANT

This message is a special case of MSG_GEN_BRANCH_REPLACE_PARAMS where the **BranchReplaceParamType** is BRPT_OUTPUT_OPTR with a search parameter of a constant value set up beforehand. You must pass this message the optr to replace the search constant found.

Source: Unrestricted.

Destination: Any generic object.

Pass:

cx:dx	The optr to replace all instance fields with destinations matching the search constant passed in **bp**.
bp	The constant to search for and replace with the optr in **cx:dx**.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Generally not intercepted. This message invokes MSG_GEN_BRANCH_REPLACE_PARAMS so intercept that instead.

Library: **Objects\genC.def**

■ MSG_GEN_BRANCH_REPLACE_PARAMS

This message travels down a generic branch to all of an object's children; the message replaces all instance data specified in the **BranchReplaceParams** field *BRP_type* with replacement instance data. A typical way to implement

this is to set up a template UI branch with MSG_GEN_COPY_TREE. Most often, you will use this message to search and replace optrs set within your template. This is easily done by setting these optrs to certain constant values and then searching for those values in **BranchReplaceParams** field *BRP_type*. If you only wish to replace your destination optrs, use MSG_GEN_BRANCH_REPLACE_OUTPUT_OPTR_CONSTANT, which is just a specialized case of this message.

When replacing optrs, the constant BRPT_DUMMY_OPTR_START should be used as the **BranchReplaceParams** field *BRP_searchParam* value. If multiple optrs are to be replaced, enumerated values based on that value should be used for other optrs. These values will be constants, and since the **TravelOption** types are also constants, care must be taken to replace these dummy values before anything else is done on the generic branch; otherwise, their output may be interpreted as a **TravelOption**.

This message may not be supported by several subclasses of **GenClass**.

Source: Unrestricted.

Destination: Any generic object.

Pass:

ss:bp	Offset to **BranchReplaceParams** structure.
dx	Size of **BranchReplaceParams** structure.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: This message is handled by most generic objects (although not all). If you wish to intercept this message, you should first call the superclass to search and replace any **GenClass** instance data, if applicable, and then search and replace any instance data in the intercepting class.

Library: **Objects\genC.def**

■ MSG_GEN_BRING_TO_TOP

This message brings a windowed object to the front if it is in a maximized or overlapping arrangement with other window objects. This message may also

grab the focus and target exclusives for the window involved, if the specific UI allows such behavior.

Source: Unrestricted.

Destination: Any GS_USABLE **GenClass** object. (Only GenDisplays, independently displayable GenInteractions, and GenPrimarys are affected.)

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\genC.def

MSG_GEN_CALL_APPLICATION

This message calls the GenApplication object associated with the recipient. This message allows return values and should be cast to whatever return values are expected of the passed message.

Source: Unrestricted.

Destination: Any generic object.

Pass: cx — Handle of **ClassedEvent** to call on Application.

Returns: CF — Set if GenApplication object was not found; otherwise, the CF is set as it was returned by the passed event.

ax, cx, dx, bp — As returned by the passed event handle. (The event handle is freed.)

Destroyed: ax, cx, dx, bp

Library: Objects\genC.def

MSG_GEN_CALL_PARENT

This message delivers an event to the generic parent of the recipient. This message must pass a classed event that the parent object will handle. You

should use this message if return values are expected. The event will be freed after it is sent.

Source: Unrestricted.

Destination: Any generic object.

Pass:	**cx**	Handle of ClassedEvent to call on parent.
Returns:	CF	Clear if no parent was found; otherwise CF is as returned by the event. **ax, cx, dx**, and **bp** are returned as they were from the classed event. (The event handle is freed.)
Destroyed:	**ax, cx, dx, bp**	
Library:	**Objects\genC.def**	

MSG_GEN_CALL_SYSTEM

This message calls the GenSystem object (the root object) associated with your application and sends the event.

Source: Unrestricted.

Destination: Any generic object.

Pass:	**cx**	**ClassedEvent** to call on GenSystem object.
Returns:	CF	As returned by message handler.
	ax, cx, dx, bp	As returned by message handler.
Destroyed:	**Determined by message handler (see above)**	
Library:	**Objects\genC.def**	

MSG_GEN_CHANGE_ACCELERATOR

This message changes the keyboard accelerator using constants defined in the input library. In most cases, use MSG_GEN_SET_KBD_ACCELERATOR instead. MSG_GEN_CHANGE_ACCELERATOR is useful in some cases where you wish to tinker slightly with the keyboard shortcut's usage (as in a case

where an accelerator of *ctrl z* should change to *alt z* in the duplication of a template object).

Source: Unrestricted.

Destination: Any generic object.

Pass: **cx** Mask to clear. (**KeyboardShortcut**.)
dx Mask to OR in (bits to set). (**KeyboardShortcut**.)

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

■ MSG_GEN_CHECK_IF_FULLY_ENABLED

This message checks whether an object is fully enabled. An object is only fully enabled (ready for user interaction) when both it and all of its ancestors are enabled.

Source: Unrestricted.

Destination: Any generic object.

Pass: Nothing.

Returns: CF Set if the object is fully enabled.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

■ MSG_GEN_CHECK_IF_FULLY_USABLE

This message checks whether an object is fully usable. The object and all of its parents must be GS_USABLE for the object to be fully usable.

Pass: Nothing.

Returns: CF Set if the object is fully usable.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

■ MSG_GEN_COPY_TREE

This message copies an entire branch of a generic object tree, starting at the object first called. Pass this message the handle of the destination object block. The object receiving the message must not be GS_USABLE.

You may pass *parentChunk* null if you wish the block to remain unattached to the generic tree. Note that if a generic tree is copied using MSG_GEN_COPY_TREE, the objects contained therein not only will have a new resource handle but will also have new chunk handles. Note that this is different from using **ObjDuplicateResource**, which will keep the same chunk offsets into the object block.

Unlike **ObjDuplicateResource**, this message will copy the entire branch desired, even if those objects reside in separate resources. This message is therefore more flexible than **ObjDuplicateResource**, which only copies a given object block.

GenClass provides the default behavior of broadcasting this message down a generic tree, where each object will make a copy of itself in the destination block. It creates an object chunk the same size as the object chunk being copied, copies over the entire contents of any instance fields and creates an outside chunk for a visual moniker, if any.

If you have a subclass of a generic object which references (and therefore might need to create) a chunk outside the object, you must be sure to intercept this message and allocate, copy over, and update any references to this chunk.

Source: Unrestricted.

Destination: Any generic object.

Pass:

cx	Block to copy the tree into. This block cannot be the same as the source object block.
dx	The chunk handle of a generic object in the destination block. The recipient of this message will be copied into the chunk handle passed in **cx**. If zero, copy tree into block without linking it to a particular chunk.
bp	**CompChildFlags** to use when adding the tree.

Returns: **cx:dx** Optr of new object created.

Destroyed: **ax, bp**

Interception: This message is handled by all generic objects. If you wish to intercept this message, you should first call the superclass to copy over the **GenClass** instance data and then copy over any lmem chunks that are referenced by instance data of the intercepting class.

Library: **Objects\genC.def**

MSG_GEN_COUNT_CHILDREN

This message returns the number of children of the recipient.

Source: Unrestricted.

Destination: Any generic object.

Pass: Nothing.

Returns: **dx** Number of generic children.

Destroyed: **ax, cx, bp**

Library: **Objects\genC.def**

MSG_GEN_CREATE_VIS_MONIKER

This message creates a visual moniker chunk within the resource block of the object sent this message. The moniker is copied from the source specified but is not attached to any object in the new object block.

Source: Unrestricted

Destination: Any **GenClass** object

Pass: **ss:bp** Pointer to **CreateVisMonikerFrame** structure.
dx Size of **CreateVisMonikerFrame** structure.

Returns: **ax** Chunk handle of new visual moniker.

Destroyed: **cx, dx, bp**

Library: **Objects\genC.def**

MSG_GEN_DESTROY

This message destroys a generic branch starting at the object called. The message sets all required generic objects not usable, removes them from the generic tree, and destroys them, including any chunks associated with the specific object destroyed. This message is the safest way to destroy a generic branch.

The message will leave the block that the objects previously resided in intact (except for objects removed, of course), but any chunks coming from those resources will be marked dirty and will be resized to zero.

It is usually a good idea to destroy generic branches that do not need to be saved to state when an application detaches or exits. This ensures that when an application is reattached, it will not contain the previous links. This keeps

the state file more compact and avoids the possibility of a child not being saved to state while retaining the parent's child pointer.

The only valid flag in **CompChildFlags** is CCF_MARK_DIRTY, which will mark the links as dirty and save them to state.

Source: Unrestricted.

Destination: Any generic object.

Pass:
- `dl` — **VisUpdateMode** to determine when the generic tree is to be visually updated. May not be VUM_MANUAL.
- `bp` — **CompChildFlags** to use when destroying the branch (either CCF_MARK_DIRTY or zero).

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: This message is handled by all generic objects. If you wish to intercept this message, you should first destroy any chunks that are referenced by instance data of the intercepting class and then call the superclass to destroy the object's **GenClass** instance data.

Library: **Objects\genC.def**

MSG_GEN_DESTROY_AND_FREE_BLOCK

This is a utility message used to destroy a generic branch which resides completely within one block. The block will be freed. The object called with this message must be the only top object in the block. This message is called by **UserDestroyDialog**.

If any object within the block resides on the application active list, or if the object is not a window, a slow, non-optimized approach is taken: the branch is set not usable, removed from the generic tree, and sent MSG_META_BLOCK_FREE. If no object of the block can be found on this list, an optimized approach is taken: the window is taken off the screen; focus, target, model and mouse exclusives are released; the linkage changed to be

Object Assembly Reference

one-way upward; and the block sent MSG_META_BLOCK_FREE. In other words, the whole unbuild process is carefully avoided.

Source: Unrestricted

Destination: Top object within a block to be freed.

Pass: Nothing.

Returns:

CF	Set if the message was handled; clear otherwise.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

■ MSG_GEN_DRAW_MONIKER

This message draws a moniker for an object according to the parameters passed. The moniker must currently exist. You can use this message to change the way you wish a visual moniker to appear within a generic object without actually changing the visual moniker itself.

Source: Unrestricted.

Destination: Any GS_USABLE generic object.

Pass:

ss:bp	Pointer to **GenMonikerMessageFrame** structure.
dx	Size of **GenMonikerMessageFrame** structure.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

■ MSG_GEN_FIND_CHILD

This message checks whether the object is a child of the recipient. If so, it returns the child's position. If you wish to find out if a specific child currently exists at a certain position, use MSG_GEN_FIND_CHILD_AT_POSITION.

Source: Unrestricted.

Destination: Any generic object.

Pass:

cx:dx	The optr of child to search for.

Returns:

CF	Set if the child was not found.
bp	Child position (zero-based).
cx, dx	Unchanged.

Destroyed: **ax**

Library: **Objects\genC.def**

MSG_GEN_FIND_CHILD_AT_POSITION

This message returns the optr of the child at *position*, if any. Pass the message the zero-based (zero indicates the first child) position to check. If no child is found, the message returns zero.

Source: Unrestricted.

Destination: Any generic object.

Pass:	**cx**	The position (zero-based) of the child to search for.
Returns:	**cx:dx**	Optr of child (or zero if no child was found).
Destroyed:	**ax, bp**	
Library:	**Objects\genC.def**	

MSG_GEN_FIND_MONIKER

This message scans a list of monikers and selects a specific moniker (or the most appropriate moniker). The moniker selected is determined according to the passed **VisMonikerSearchFlags**. This message is usually used by the specific UI to select an application moniker from a list.

If **dx** is not zero, the message will use the GenApplication's moniker list instead of the recipient's. This is useful for finding the icon used for an iconified application.

Note that this message returns an optr, not a chunk handle, as this message is most often used to find a moniker outside of the current object block. This message returns zero if no moniker is found.

Source: Unrestricted.

Destination: Any generic object.

Pass:	**dx**	Non-zero to use the moniker list of the application associated with this object.
	bp	**VisMonikerSearchFlags** to use for the search criteria. If VMSF_COPY_CHUNK is specified in **bp**, a block handle must

		be supplied in **cx**. The selected moniker will be copied into that block.
	cx	Block to copy a moniker into, if VMSF_COPY_CHUNK is specified in the **bp**.
Returns:	**cx:dx**	Optr of VisMoniker (or null if none).
Destroyed:	**ax, bp**	
Library:	**Objects\genC.def**	

MSG_GEN_FIND_OBJECT_WITH_TEXT_MONIKER

This message recursively searches through the generic tree (starting at the object sent the message) for an object whose visual moniker matches the passed text. This match does not need to be complete (unless the flag GFTMF_EXACT_MATCH is passed); the message will return the first object whose initial characters match the given text. The message must pass **GenFindObjectWithMonikerFlags**.

```
GenFindObjectWithMonikerFlags record
GFTMF_EXACT_MATCH:1
GFTMF_SKIP_THIS_NODE:1
:14
```

If GFTMF_EXACT_MATCH is passed, the text and the visual moniker text must match exactly (null-terminated). If GFTMF_SKIP_THIS_NODE is passed, the object sent the message will not be checked; only its children will be checked.

Source: Unrestricted.

Destination: Any generic object.

Pass:	**cx:dx**	Null-terminated text.
	bp	**GenFindObjectWithMonikerFlags**.
Returns:	CF	Set if match is found.
	cx:dx	(If CF is set) Optr of matching object.
Destroyed:	**ax, bp**	
Library:	**Objects\genC.def**	

MSG_GEN_FIND_PARENT

This message returns the parent of the recipient object, if any.

Source: Unrestricted.

Destination: Any generic object.

Pass: Nothing.

Returns: `cx:dx` Optr of parent (or null if none).

Destroyed: `ax, bp`

Library: **Objects\genC.def**

MSG_GEN_GET_ATTRIBUTES

This message retrieves the *genAttributes* instance data for the object the message is sent to. This message returns a byte length bitfield.

Source: Unrestricted.

Destination: Any generic object.

Pass: Nothing.

Returns: `cl` **GenAttrs** bitfield.

Destroyed: `ax, ch, dx, bp`

Library: **Objects\genC.def**

MSG_GEN_GET_ENABLED

This message returns the enabled state of the object the message is sent to.

Source: Unrestricted.

Destination: Any generic object.

Pass: Nothing.

Returns: `CF` Set if enabled, clear if not enabled.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\genC.def**

MSG_GEN_GET_FIXED_SIZE

This message retrieves the size stored in HINT_FIXED_SIZE. If HINT_FIXED_SIZE is not set, the structure passed will remain empty.

Source: Unrestricted.

Destination: Any generic object.

Pass: Nothing.

Returns:

cx	**SpecWidth** or 0 if none.
dx	**SpecHeight** or 0 if none.
ax	**SpecNumberOfChildren**, if specified.

Destroyed: **bp**

Library: **Objects\genC.def**

MSG_GEN_GET_INITIAL_SIZE

This message retrieves the initial size specifications of a generic object stored in the hint HINT_INITIAL_SIZE.

Source: Unrestricted.

Destination: Any generic object.

Pass: Nothing.

Returns:

cx	**SpecWidth** or 0 if none.
dx	**SpecHeight** or 0 if none.
ax	**SpecNumberOfChildren**, if specified.

Destroyed: **bp**

Library: **Objects\genC.def**

MSG_GEN_GET_KBD_ACCELERATOR

This message returns the object's current *kbdAccelerator*. The message returns a word-length data structure (**KeyboardShortcut**). You can then manipulate or use this word of data to set other object's keyboard

accelerators. Make sure that you do not have two objects sharing the same keyboard accelerator.

Source: Unrestricted.

Destination: Any generic object.

Pass: Nothing.

Returns: **cx** **KeyboardShortcut** or zero if no shortcut exists.

Destroyed: **ax, dx, bp**

Library: **Objects\genC.def**

MSG_GEN_GET_MAXIMUM_SIZE

This message retrieves the recipient's maximum size stored in the hint HINT_MAXIMUM_SIZE.

Source: Unrestricted.

Destination: Any generic object.

Pass: Nothing.

Returns: **cx** **SpecWidth,** or 0 if none.
dx **SpecHeight,** or 0 if none.
ax **SpecNumberOfChildren**, if specified.

Destroyed: **bp**

Library: **Objects\genC.def**

MSG_GEN_GET_MINIMUM_SIZE

This message retrieves the recipient's minimum size specifications stored in the hint HINT_MINIMUM_SIZE.

Source: Unrestricted.

Destination: Any generic object.

Pass: Nothing.

Returns: **cx** **SpecWidth** or 0 if none.
dx **SpecHeight** or 0 if none.
ax **SpecNumberOfChildren**, if specified.

Destroyed: **bp**

Library: **Objects\genC.def**

Object Assembly Reference

MSG_GEN_GET_MONIKER_POS

This message returns the x and y coordinates of the moniker for the object as if the moniker were drawn according to the passed parameters (the same as for MSG_GEN_DRAW_MONIKER).

This message, along with MSG_GEN_DRAW_MONIKER and MSG_GEN_GET_MONIKER_SIZE is most useful in the construction and modification of custom gadgets.

Source: Unrestricted.

Destination: Any generic object.

Pass: See MSG_GEN_DRAW_MONIKER.

Returns:

ax	*x* origin for moniker.
bp	*y* origin for moniker.

Destroyed: **cx, dx**

Library: **Objects\genC.def**

MSG_GEN_GET_MONIKER_SIZE

This message returns the width and height (in points) of the moniker for the object. This message, along with MSG_GEN_DRAW_MONIKER and MSG_GEN_GET_MONIKER_POS, is most useful in the construction and modification of custom gadgets.

Source: Unrestricted.

Destination: Any generic object.

Pass:

dx	The system text height, if known; otherwise, pass zero.
bp	The GState handle of the GState to draw to.

Returns:

cx	Width of moniker.
dx	Height of moniker.

Destroyed: **ax, bp**

Library: **Objects\genC.def**

MSG_GEN_GET_USABLE

This message checks the *usable* bit of the *genStates* field.

Source: Unrestricted.

Destination: Any generic object.

Pass: Nothing.

Returns: CF Set if GS_USABLE; clear if not GS_USABLE.

Destroyed: ax, cx, dx, bp

Library: Objects\genC.def

MSG_GEN_GET_VIS_MONIKER

This message retrieves the instance data in the object's current *GI_visMoniker* instance field. This message returns the ChunkHandle of the moniker data structure. You can then use that chunk handle to manipulate the visual moniker directly or to copy the moniker for use by other objects.

Source: Unrestricted.

Destination: Any generic object.

Pass: Nothing.

Returns: ax Chunk handle of **VisMoniker** in use.

Destroyed: cx, dx, bp

Library: Objects\genC.def

MSG_GEN_GUP_CALL_OBJECT_OF_CLASS

This message performs a generic upward query, passing the classed event upward until it reaches an object of the passed class. This message allows return values and should be cast into whatever return values are expected of the passed message. This is most easily done by enclosing the message sent within parentheses.

An object of the expected class should be present or the event will not be delivered to any object. To check for the existence of such a class, use MSG_GEN_GUP_TEST_FOR_OBJECT_OF_CLASS.

Source: Unrestricted.

Destination: Any generic object.

Pass: **cx** Handle of the classed event to deliver to an object up the generic tree.

Returns: CF, **ax, cx, dx, bp**
As returned from the call. (Event handle is freed.)

Destroyed: **As returned from the call (see above)**

Library: **Objects\genC.def**

MSG_GEN_GUP_FIND_OBJECT_OF_CLASS

This message performs a function similar to that of MSG_GEN_GUP_TEST_FOR_OBJECT_OF_CLASS, but it returns the optr of the class found, if any. This optr can then be used as the recipient of other messages. You should not use an optr returned by this message at a later time, as the object may have been removed.

Source: Unrestricted.

Destination: Any generic object.

Pass: **cx:dx** **ClassStruct** to search for.

Returns: CF Set if object of the passed class is found.
(If CF is set):
cx:dx Optr of object found. (Zero if not found.)

Destroyed: **ax, bp**

Library: **Objects\genC.def**

MSG_GEN_GUP_FINISH_QUIT

Message to be used by nodes that implement a GCN active list. This should be used if MSG_META_QUIT is handled and sent to the active list items by that node. When a node has finished sending a MSG_META_QUIT to the active

list or if the quit is aborted, this allows the notification of the above node in the generic tree.

Source: Sent by active list nodes that implement quit mechanism.

Destination: Any generic object involved in quit mechanism.

Pass: **cx** Abort flag. Pass true to abort. This will be passed on to MSG_META_FINISH_QUIT.
dx Pass true to notify parent of finished quit sequence.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Generally not intercepted. Default handler sends MSG_META_FINISH_QUIT to self, then sends to generic parent if flag indicates such. MSG_META_FINISH_QUIT is normally the message to intercept to know when a quit is aborted or finished.

Library: **Objects\genC.def**

MSG_GEN_GUP_INTERACTION_COMMAND

This message travels up the generic tree until it reaches a non-GIT_ORGANIZATIONAL GenInteraction. Once there, it performs the indicated **InteractionCommand**. See **InteractionCommand** declaration for the different command types and how specific user interfaces interpret the commands.

Source: Unrestricted. Normally sent by a MSG_GEN_INTERACTION_ACTIVATE_COMMAND handler or by a custom trigger's action message handler to perform some default command after perform some application-specific function.

Destination: Any generic object. Will travel up generic tree, stopping at first non organizational interaction.

Pass: **cx** Standard **InteractionCommand**.

Returns: CF Set if the message was handled.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

Object Assembly Reference

MSG_GEN_GUP_QUERY

This message is used to query up the generic composite tree.

Source: Anyone.

Destination: Any usable generic object, though only certain objects will answer certain queries.

Pass: `cx` **GenUpwardQueryType**.

The **GenUpwardQueryType** should be one of the following:

GUQT_DELAYED_OPERATION
: This query type determines if a gadget should be operating in "delayed mode," i.e., whether it should wait for a MSG_GEN_APPLY before sending notification to its output, or whether it should send notification whenever it changes state.

GUQT_FIELD
: Returns field object in **cx:dx** and window (if realized) handle in **bp**. This is a reasonably safe thing to do, since the field window will remain open until all applications within the field are shut down. Only field objects or substitutes for them should intercept this message.

GUQT_SCREEN
: Returns screen object in **cx:dx** and window (if any) handle in **bp**. This is a reasonably safe thing to do, since the screen window will remain open until all applications within the field are shut down. Screen object being returned will be default screen for the field.

Returns:

CF	Set if the query has been responded to.
cx:dx	Optr of object requested. (See above.)
bp	Window handle in which requested object resides. (See above.)

Destroyed: **ax (cx, dx, bp)**

Library: **Objects\genC.def**

MSG_GEN_GUP_SEND_TO_OBJECT_OF_CLASS

This message performs a generic upward query, passing the classed event upward until it reaches an object of the described class. Since this message performs a send, it does not allow return values. An object of the expected

class should be present; otherwise, the event will not be delivered to any object. To check for the existence of such a class, use MSG_GEN_GUP_TEST_FOR_OBJECT_OF_CLASS.

Source: Unrestricted.

Destination: Any generic object.

Pass: `cx` Handle of the classed event to deliver to an object up the generic tree.

Returns: Nothing. (Event handle freed.)

Destroyed: `ax, cx, dx, bp`

Library: **Objects\genC.def**

MSG_GEN_GUP_TEST_FOR_OBJECT_OF_CLASS

This message searches up the generic tree for any object of the specific class. This message is useful for testing the existence of required classes before sending MSG_GEN_GUP_CALL_OBJECT_OF_CLASS and MSG_GEN_GUP_SEND_TO_OBJECT_OF_CLASS.

Source: Unrestricted.

Destination: Any generic object.

Pass: `cx:dx` segment:offset of **ClassStruct** to search for among the object's parents.

Returns: `CF` Set if the an object of the passed class was found; clear, otherwise.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\genC.def**

MSG_GEN_LOAD_OPTIONS

This message instructs the generic object to load a value from the GEOS.INI file. It is called automatically by the handler in **GenClass** for MSG_META_LOAD_OPTIONS. The message scans the object's vardata for an ATTR_GEN_INIT_FILE_KEY and an ATTR_GEN_INIT_FILE_CATEGORY to scan the .INI file. If no file category is found in the object's vardata, the

handler will search up the generic tree for a parent with an ATTR_GEN_INIT_FILE_CATEGORY.

Source: Sent by **GenClass** on MSG_META_ATTACH. **GenClass** generates the **GenOptionsParams** structure by looking in the object's vardata (and by querying up the generic tree, if necessary).

Destination: Any generic object.

Pass: `ss:bp` **GenOptionsParams** structure (automatically generated by **GenClass**). See MSG_GEN_SAVE_OPTIONS.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: Various generic classes intercept this and provide default behavior

Library: **Objects\genC.def**

MSG_GEN_LOWER_TO_BOTTOM

This message lowers a windowed object to the bottom if it is in an overlapping arrangement with other windowed objects. This message may also release the focus and target exclusives for the window involved, if the specific UI allows such behavior.

Source: Unrestricted.

Destination: Any GS_USABLE **GenClass** object. (Only GenDisplays, independently displayable GenInteractions, and GenPrimarys are affected.)

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\genC.def**

MSG_GEN_MAKE_APPLYABLE

This message is typically sent to a dialog GenInteraction to enable its default "apply" and "reset" buttons. This message also sets up the proper user and

actual states for the object. This message is useful in the implementation of custom gadgets.

Source: Unrestricted.

Destination: Any GS_USABLE **GenClass** object. (This message is only meaningful for objects within a GIT_PROPERTIES GenInteraction, however.)

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

MSG_GEN_MAKE_NOT_APPLYABLE

This message is typically sent to a dialog GenInteraction to disable its default "apply" and "reset" buttons. This message is useful in the implementation of custom gadgets.

Source: Unrestricted.

Destination: Any GS_USABLE **GenClass** object. (This message is only meaningful for objects within a GIT_PROPERTIES GenInteraction, however.)

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

MSG_GEN_MOVE_CHILD

This message moves the given object from the location it currently occupies among its siblings to another location among its siblings. The object will still remain a child of the same parent.

Pass this message the optr of the child to move along with a word of **CompChildFlags**. If you pass the flag CCF_MARK_DIRTY, the new linkage will be saved to state when the application is detached. You must also pass a CCF_REFERENCE so the object will be moved to the position specified. CCO_FIRST and CCO_LAST will move the object to the first or last position, respectively.

If no flags are specified, the object will be moved to the parent object's first position without marking the linkages dirty. Note that for successive moves of children this will result in a "reverse order" of the children.

Note that this message only moves a child among its siblings. To move an object from one parent of the generic tree to another (different) parent, you must use MSG_GEN_REMOVE (or MSG_GEN_REMOVE_CHILD) and MSG_GEN_ADD_CHILD.

Source: Unrestricted.

Destination: Any generic object.

Pass: `cx:dx` The optr of the child to move.
`bp` **CompChildFlags** to use when moving the child.

Returns: Nothing.

Destroyed: `ax, bp (cx, dx unchanged)`

Library: **Objects\genC.def**

MSG_GEN_NAVIGATE_TO_NEXT_FIELD

This message moves the navigation cursor from its current object to the next valid object. This message is useful if you are trapping keyboard characters using MSG_META_KBD_CHAR and wish to use special characters to navigate the cursor to various fields within a View.

Source: Normally sent by the specific UI, although your application could also send this.

Destination: Any GS_USABLE **GenClass** object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: Generally not intercepted. To alter navigation functionality, intercept MSG_GEN_NAVIGATION_QUERY instead.

Library: **Objects\genC.def**

MSG_GEN_NAVIGATE_TO_PREVIOUS_FIELD

This message moves the navigation cursor from its current object to the previous valid object. This message is useful if you are trapping keyboard

characters using MSG_META_KBD_CHAR and wish to use special characters to navigate the cursor to various fields within a View.

Source: Normally sent by the specific UI, although your application could also send this.

Destination: Any GS_USABLE **GenClass** object.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Generally not intercepted. To alter navigation functionality, intercept MSG_GEN_NAVIGATION_QUERY instead.

Library: **Objects\genC.def**

■ MSG_GEN_NAVIGATION_QUERY

This message can be used by applications to initiate navigation-oriented queries, as the MSG_GEN_NAVIGATE_TO_[NEXT|PREVIOUS]_FIELD messages do.

This structure's return values determine how it fits into the navigation network. If the carry flag is set, then the object which should receive the focus was found, and **cx:dx** will hold the optr of this object.

Object Assembly Reference

If the carry flag is not set, then the focusable object has not yet been found; **cx:dx** contains the field of the next object to try., **bp** contains the updated navigation flags which should be passed in the next query.

Source: Unrestricted.

Destination: Any generic object.

Pass:	**bp**	**NavigateFlags**.
	cx:dx	Object which originated this query.
Returns:	CF	Set if this object should be given the focus; clear to continue the query.
	cx:dx	(If CF set) object which is replying and which should be given the focus. (If CF clear) next object to query.
	bp	**NavigateFlags**.
	al	Set if the object is focusable via backtracing (is previous to object through backwards navigation).
Destroyed:	**ah**	
Library:	**Objects\genC.def**	

■ MSG_GEN_OUTPUT_ACTION

This message sends an event via the destination optr (if any) of the object sent the message. The default handler (which is rarely subclassed) also scans the vardata looking for an ATTR_GEN_DESTINATION_CLASS and records that class in a classed event.

This message is usually sent at a low level and dispatches a message via the destination optr of the object. You may intercept MSG_GEN_OUTPUT_ACTION if you wish knowledge of when an object is being activated (although you may wish to intercept MSG_GEN_ACTIVATE in

those cases). If you do send this message yourself, you must either pass a destination optr or leave it null, depending on the object being activated.

Source: Normally sent by an object to itself.

Destination: Normally sent by an object to itself.

Pass:

bp	Handle of a classed event. The stored destination is ignored if an optr is passed in **cx:dx**.
cx:dx	The destination optr to send event to. OR
bp	Handle of a classed event. The class stored will be overwritten if an ATTR_GEN_DESTINATION_CLASS is found within the object, with the class stored in that attribute.
cx	0. (This is required so that dx is recognized as a **TravelOption**.)
dx	**TravelOption** to use with MSG_META_SEND_CLASSED_EVENT, which will be called either on this object or the one specified with ATTR_GEN_OUTPUT_TRAVEL_START (if present).

Returns: Nothing. (The event handle is freed.)

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

MSG_GEN_PATH_GET

This message sets the path associated with the object. Normally, a complete path name must be passed. Note that this path string should not include a drive specifier, as this is implied by the passed disk handle. If the passed **DiskHandle** is actually a **StandardPath** value, then the path string will be taken to be relative to this standard path.

Source: Unrestricted.

Destination: Generic object which has an associated path.

Pass:

dx:bp	Address to copy to. This location will store a null-terminated path name. This location may not be in the same block as the object receiving this message. (If null, the default handler will use root directory of disk handle or directory associated

Object Assembly Reference

with standard path.) If **dx** is zero, the path is copied to a block on the global heap and **cx** is ignored.

cx Size of buffer (or zero if null-terminated).

Returns: CF Set if an error occurred; otherwise clear.

(If CF is set):

ax Number of bytes required. (If zero, the path is invalid.)

cx Disk handle of path.

(If CF is clear and **dx:bp** contains a null-terminated path):

dx:bp Path name. The pointer to this location is preserved.

(If CF is clear and dx was passed as zero):

dx Handle of block containing the path.

cx Disk handle of path.

Destroyed: **ax**

Library: **Objects\genC.def**

MSG_GEN_PATH_GET_DISK_HANDLE

This message returns the disk handle for the path bound to the object.

Source: Unrestricted.

Destination: Generic object which has an associated path.

Pass: Nothing.

Returns: **cx** Disk handle of path (or zero is path is invalid).

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

MSG_GEN_PATH_SET

This message sets the path associated with the object. Normally, a complete path name must be passed. Note that this path string should not include a drive specifier, as this is implied by the passed disk handle. If the passed **DiskHandle** is actually a **StandardPath** value, then the path string will be taken to be relative to this standard path.

Source: Unrestricted.

Destination: Generic object which has an associated path.

Pass: **cx:dx** Null-terminated pathname; may not be in the same block as the object receiving this message. (If null, default handler

will use root directory of disk handle or directory associated with standard path.)

	bx	Disk handle of path, or **StandardPath** constant. (If **bx** is zero, the object's current disk handle is used.)

Returns:

CF	Set if path couldn't be set; clear otherwise.
(If CF is set):	
ax	error code (**FileError**).

Destroyed: **ax (unless CF was set)**

Interception: Generally not intercepted. If the object is specifically grown, the message will be forwarded to the specific class after it has been acted on by **GenClass**.

Library: **Objects\genC.def**

MSG_GEN_POST_APPLY

A properties dialog will send this message after MSG_GEN_APPLY. This can be used to clean up after application of changes.

Source: Normally sent by a properties dialog after sending MSG_GEN_APPLY.

Destination: Same as MSG_GEN_APPLY.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Can be intercepted to handle clean up work after MSG_GEN_APPLY.

Library: **Objects\genC.def**

MSG_GEN_PRE_APPLY

A properties dialog will send this message before MSG_GEN_APPLY. Geodes may use this to do validation of settings or to prepare for applying of changes.

Source: Normally sent by properties dialog before sending MSG_GEN_APPLY.

Destination: Same as MSG_GEN_APPLY.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Must be intercepted to handle properties validation.

Library: **Objects\genC.def**

Object Assembly Reference

MSG_GEN_QUERY_NEED_TO_BE_ON_ACTIVE_LIST

This message queries whether an object still needs to be on an active list. The active list mechanism sends this message if a mechanism no longer needs an object to remain on the active list; it checks whether any other mechanism also needs the object to be on the active list also. Windowed specific UI objects, for example, will return *true* if ATTR_INITIATE_ON_STARTUP is present on the object, because this optional attribute requires the object to be on the active list.

Source: Unrestricted, though it is generally called on itself. Specifically, if an object no longer needs to remain on the active list for a particular reason, this message should be called on the object to see if it should remain on the active list for any other reason. If so, the caller should not remove the object from the active list. The only exception to this would be if the object were being destroyed, in which the object should be removed in any case.

Destination: Any generic object.

Pass: Nothing.

Returns: CF Set if object should be on the active list.

Destroyed: **ax, cx, dx, bp**

Interception: Object classes which add the object to the active list must intercept this message and return *true* if the class desires the object to remain on the active list. If the message is intercepted and this is not the case, it should be sent to the superclass for handling.

Library: **Objects\genC.def**



MSG_GEN_RELOC_MONIKER_LIST

This method (un)relocates a passed moniker list.

Source: Anyone.

Destination: Any GenClass object.

Pass:

cx	Chunk handle of moniker list.
dx	Block whose owner's relocation table should be used. This will normally be the block that the moniker list resides within.
bp	Zero to relocate, one to unrelocate.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

MSG_GEN_REMOVE

This message removes the receiving object from the generic tree. The object to be removed need not be set not usable.

Source: Anyone.

Destination: Any generic object.

Pass:

dl	Visual update mode. VUM_MANUAL is not allowed.
bp	Set (mask) CCF_MARK_DIRTY to mark the links dirty.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

MSG_GEN_REMOVE_CHILD

This message removes the passed object from the recipient. A child must be marked not GS_USABLE in order to be removed. The child must currently exist, so your routine should check this using MSG_GEN_FIND_CHILD.

Pass this message the optr of the child object to be removed along with a word of **CompChildFlags**. If CCF_MARK_DIRTY is specified, the updated linkage will be saved to state when the application is detached.

Source: Unrestricted.

Destination: Any generic object.

Pass:

cx:dx	The optr of the child to remove. This child must be non-usable in order to be removed. The child must also exist as a child of the recipient.
bp	**CompChildFlags** to use when removing the child.

Returns: Nothing. (**cx** and **dx** are unchanged.)

Destroyed: **ax, bp**

Library: **Objects\genC.def**

MSG_GEN_REMOVE_GEOMETRY_HINT

This message removes a geometry hint from an object's instance data. The object may be GS_USABLE at the time, in which case it will be redrawn according to the new geometry configuration.

Source: Unrestricted.

Destination: Any generic object.

Pass:

cx	The hint to remove.
dl	**VisUpdateMode** to determine when the object will be redrawn.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

MSG_GEN_REPLACE_VIS_MONIKER

This message copies a visual moniker from a source to the object sent the message. The message returns the chunk handle of the newly copied moniker. The object's current visual moniker is overwritten by this message; therefore, if more than one object shares the overwritten block, the other object should be updated with MSG_GEN_USE_VIS_MONIKER. This message can copy a visual moniker from a variety of sources.

This message is a general, all-purpose, moniker replacement device. If your visual moniker source is a **visMoniker** structure or a null-terminated text

string, you may be able to use the simpler messages MSG_GEN_REPLACE_VIS_MONIKER_OPTR and MSG_GEN_REPLACE_VIS_MONIKER_TEXT.

Source: Unrestricted.

Destination: Any generic object.

Pass: **ss:bp** Pointer to a **ReplaceVisMonikerFrame** structure.
dx Size of **ReplaceVisMonikerFrame**.

The **ReplaceVisMonikerFrame** structure is defined as follows:

```
ReplaceVisMonikerFrame struct
```

RVMF_source Specifies the source of the moniker to be used during the copy process. This source may be a pointer to a text string, an optr pointing to a visual moniker outside the current block, or a variety of other sources. The source type should be specified in *RVMF_sourceType*.

RVMF_sourceType Specifies the type of pointer referencing the *source*. Valid **VisMonikerSourceType** types are VMST_FPTR, VMST_OPTR, and VMST_HPTR.

RVMF_dataType Specifies the type of data referenced by the *RVMF_source*. Valid **VisMonikerDataType** types include VMDT_VIS_MONIKER, VMDT_TEXT, VMDT_GSTRING, or VMDT_TOKEN.

RVMF_length The length of the moniker source data, in bytes. If the source data is a text string and this length is zero, the string is assumed to be null-terminated.

RVMF_width If the moniker is a GString, this specifies the width of the moniker in points. Otherwise, set this to zero.

RVMF_height If moniker is a GString, this specifies the height of the moniker in points. Otherwise, set this to zero.

RVMF_updateMode Specifies the **VisUpdateMode** to use when visually updating the changes to the screen. See "VisClass,"

Chapter 23 of the Objects book for more information on **VisUpdateMode** types.

```
ReplaceVisMonikerFrame ends
```

Returns: **ax** Chunk handle of object's visual moniker.

Destroyed: **cx, dx, bp**

Library: **Objects\genC.def**

MSG_GEN_REPLACE_VIS_MONIKER_OPTR

This message is a simplified version of MSG_GEN_REPLACE_VIS_MONIKER. If your visual moniker source is an existing **visMoniker** structure, you can use this message to replace an object's moniker.

Source: Unrestricted.

Destination: Any generic object.

Pass: **cx:dx** The optr of the visMoniker structure to use in the replacement operation.

bp Specifies the **VisUpdateMode** to use when visually updating the changes to the screen.

Returns: **ax** Chunk handle of the object's visual moniker.

Destroyed: **cx, dx, bp**

Library: **Objects\genC.def**

MSG_GEN_REPLACE_VIS_MONIKER_TEXT

This message is a simplified version of MSG_GEN_REPLACE_VIS_MONIKER. If your visual moniker source is a pointer to a null-terminated text string,

you can use this message to replace an object's moniker without having to pass the arguments of MSG_GEN_REPLACE_VIS_MONIKER.

Source: Unrestricted.

Destination: Any generic object.

Pass:
- **cx:dx** — A pointer to a null-terminated character string to use in the replacement operation.
- **bp** — Specifies the **VisUpdateMode** to use when visually updating the changes to the screen.

Returns:
- **ax** — Chunk handle of the object's visual moniker.

Destroyed: **cx, dx, bp**

Library: **Objects\genC.def**

MSG_GEN_RESET

This message, if sent to a GenInteraction within a GIT_PROPERTIES dialog box, will propagate to all children below the receiving Interaction. This message provides a convenient means to reset properties within gadgets. There is no default handling of MSG_GEN_RESET by generic objects. You must provide the reset behavior yourself by subclassing individual objects.

Source: Usually a GIT_PROPERTIES GenInteraction (with an IC_RESET trigger), or your application.

Destination: Any GenInteraction within a dialog box, or any gadget subclassed to handle MSG_GEN_RESET.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Your subclassed generic object should intercept this message to provide its own custom reset behavior.

Library: **Objects\genC.def**

MSG_GEN_RESET_TO_INITIAL_SIZE

This message forces the recipient and all of its children to be resized according to their initial size specifications. If the objects contain HINT_INITIAL_SIZE, they will be resized according to the hint; otherwise

they will be resized according to their default geometry. This message is useful for activating sizes set with MSG_GEN_SET_INITIAL_SIZE.

Source: Unrestricted.

Destination: Any GS_USABLE generic object.

Pass: `dl` **VisUpdateMode** to determine when the object will be redrawn.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\genC.def**

MSG_GEN_SAVE_OPTIONS

This message instructs the generic object to save its options to the initialization file.

Source: Sent by **GenClass** on MSG_META_SAVE_OPTIONS. **GenClass** generates the proper **GenOptionsParams** structure by looking in the object's vardata (and querying up the generic tree, if necessary).

Destination: Any generic object.

Pass: `ss:bp` **GenOptionsParams** structure (automatically generated by **GenClass**).

Returns: Nothing. (structure filled in.)

Destroyed: `ax, cx, dx, bp`

Interception: Various generic classes intercept this and provide default behavior

Library: **Objects\genC.def**

MSG_GEN_SEND_TO_CHILDREN

This message sends an encapsulated event to all children of the generic object receiving it. This message cannot return values. The event will be freed after it is sent.

Source: Unrestricted.

Destination: Any generic object.

Pass: **cx** Handle of the classed event to deliver to all children.

Returns: Nothing. (Event handle freed.)

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

MSG_GEN_SEND_TO_PARENT

This message sends an encapsulated event to the parent but expects no return values. The event will be freed after it is sent.

Source: Unrestricted.

Destination: Any generic object.

Pass: **cx** Handle of the classed event to deliver to parent of this object.

Returns: Nothing. (Event handle freed.)

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

MSG_GEN_SEND_TO_PROCESS

This message sends the event to the Process object of your application. Because the message may cross threads, return values are not allowed.

Source: Unrestricted.

Destination: Any generic object.

Pass: **cx** The classed event to deliver to the object's GenProcess object.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

MSG_GEN_SET_ATTRS

This message sets the recipient's *genAttributes* field. This message takes two arguments: the attributes to set and the attributes to clear. There is no need to repeat attributes that have been previously set. Note that these attributes will not take effect until the object is activated in the normal manner. (Sending this message does not in itself initiate the activity described).

You should only send this message while an object is not GS_USABLE, because these attributes are only checked when an object is specifically built. Setting the attributes of a GS_USABLE object may cause an error.

Source: Unrestricted.

Destination: Any non-usable generic object.

Pass: `cl` **GenAttributes** to set in *GI_attrs*.
`ch` **GenAttributes** to clear in *GI_attrs*.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\genC.def**

MSG_GEN_SET_ENABLED

This message sets an object GS_ENABLED. (This message has no effect on an object already GS_ENABLED.) You must pass this message a **VisUpdateMode**. Sending this message allows the object to receive user input. Only send this message if your application will be ready to interact with the object after the period specified in the **VisUpdateMode** has passed.

Source: Unrestricted.

Destination: Any generic object.

Pass: `dl` **VisUpdateMode** to use when updating changes to the screen.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\genC.def**

MSG_GEN_SET_FIXED_SIZE

This message sets the fixed size of an object. An object with a fixed size is forced to remain at the size set with HINT_FIXED_SIZE. This message alters this fixed size by creating or modifying HINT_FIXED_SIZE. Due to the nature of hints, there is no guarantee that the specific UI will implement this message in the expected fashion.

Source: Unrestricted.

Destination: Any generic object.

Pass:

ss:bp	**SetSizeArgs** specifying the height, width and update mode.
dx	Size of **SetSizeArgs**,

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

MSG_GEN_SET_INITIAL_SIZE

This message sets the initial size of a generic object (its size on being first built). This is only done by creating or modifying the hint HINT_INITIAL_SIZE; there is no guarantee that the specific UI will implement this message in the expected fashion.

The message must pass a **VisUpdateMode**. This message will only affect the size of an object when it first opens; it will not modify the geometry of a currently built object. For the defined geometry to take effect on an already built object, the object must be set not usable and then set usable again.

Source: Unrestricted.

Destination: Any generic object.

Pass:

ss:bp	**SetSizeArgs**.
dx	Size of **SetSizeArgs**.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

MSG_GEN_SET_KBD_ACCELERATOR

This message sets the *kbdAccelerator* of the object the message is sent to.

Source: Unrestricted.

Destination: Any generic object.

Pass:

cx	**KeyboardShortcut** to set *kbdAccelerator* of the generic object to.
dl	**VisUpdateMode** to determine when redrawing occurs.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

MSG_GEN_SET_KBD_OVERRIDE

This message sets an object's ATTR_GEN_WINDOW_KBD_OVERRIDE to the passed **KeyboardOverride** value.

Source: Unrestricted.

Destination: Any windowed generic object.

Pass:

cx	**KeyboardOverride**.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

MSG_GEN_SET_KBD_POSITION

This message sets an object's ATTR_GEN_WINDOW_KBD_POSITION to the passed **Point** values.

Source: Unrestricted.

Destination: Any windowed generic object.

Pass:

cx	*x* coordinate to position the keyboard (relative to the window).
dx	*y* coordinate to position the keyboard (relative to the window).

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**



MSG_GEN_SET_MAXIMUM_SIZE

This message sets the maximum allowable size of an object. An object with a maximum size is not allowed to grow larger than the bounds set with HINT_MAXIMUM_SIZE. This message alters the maximum size by creating or modifying the hint HINT_MAXIMUM_SIZE. Due to the nature of hints, there is no guarantee that the specific UI will implement this message in the expected fashion.

This message will modify the geometry of a currently built object, forcing that object to be unbuilt and then built again. Note that the maximum size of an object has nothing to do with the maximized state of an object but merely the maximum allowable size of an object in its normal usable state.

Source: Unrestricted.

Destination: Any generic object.

Pass: `ss:bp` **SetSizeArgs**.
`dx` Size of **SetSizeArgs**.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\genC.def**

MSG_GEN_SET_MINIMUM_SIZE

This message sets the minimum allowable size of an object. An object with a minimum size is not allowed to shrink smaller than the bounds set in HINT_MINIMUM_SIZE. This message alters the minimum size by creating or modifying the hint HINT_MINIMUM_SIZE. Due to the nature of hints, there is no guarantee that the specific UI will implement this message in the expected fashion.

This message will modify the geometry of a currently built object, forcing that object to be unbuilt and then built again. Note that the minimum size of an object has nothing to do with the minimized state of an object (for

GenDisplays, the iconification of an object) but merely the minimum allowable size of an object in its normal usable state.

Source: Unrestricted.

Destination: Any generic object.

Pass: `ss:bp` **SetSizeArgs**.
`dx` Size of **SetSizeArgs**.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\genC.def**

■ MSG_GEN_SET_NOT_ENABLED

This message sets the object not enabled (clears the object's GS_ENABLED bit.) You must pass the message a **VisUpdateMode**. In most specific UIs a disabled state is implemented by "graying out" the object. The user will be unable to interact with the object according to the **VisUpdateMode** passed.

Source: Unrestricted.

Destination: Any generic object.

Pass: `dl` **VisUpdateMode** to use when updating changes to the screen. May not be VUM_MANUAL.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\genC.def**

MSG_GEN_SET_NOT_USABLE

This message sets an object not usable (clears the GS_USABLE bit). Because an object may be visually unbuilt by this message, the **VisUpdateMode** VUM_MANUAL is not allowed.

Source: Unrestricted.

Destination: Any generic object.

Pass: `dl` **VisUpdateMode** to use when updating changes to the screen. May not be VUM_MANUAL.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\genC.def**

MSG_GEN_SET_USABLE

This message sets an object GS_USABLE. (This message has no effect on an object already GS_USABLE.) Objects may be set GS_USABLE only after they have been attached to a generic tree. Setting an object usable forces that object to be specifically built. If the object's associated window is realized, the object will be visually built and updated. Setting an object GS_USABLE that is not attached to the generic tree will cause an error.

Setting an object usable allows that object to become part of the user interface. Before the object can be used, though, it must be *fully usable*. An object becomes fully usable only if all of its ancestors are GS_USABLE. If any ancestor is not GS_USABLE, then the entire branch below it will not be fully usable. You can check if an object is fully usable with the message MSG_GEN_CHECK_IF_FULLY_USABLE.

Conversely, setting a node GS_USABLE, in which every branch object below it is already GS_USABLE will make that entire branch fully usable. This is

useful for bringing up an entire section of a generic tree by setting a single object usable.

Source: Unrestricted.

Destination: Any generic object.

Pass: `dl` **VisUpdateMode** to use when updating changes to the screen.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\genC.def**

MSG_GEN_SET_WIN_CONSTRAIN

This message changes a window's constrain type to the passed **WindowConstrainType**.

Source: Unrestricted.

Destination: Any GS_USABLE GenPrimary or GenInteraction object.

Pass: `dl` **VisUpdateMode**.
`dh` **WinConstrainType**.
WCT_NONE
WCT_KEEP_PARTIALLY_VISIBLE
WCT_KEEP_VISIBLE
WCT_KEEP_VISIBLE_WITH_MARGIN

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\genC.def**

MSG_GEN_SET_WIN_POSITION

This message sets a window's position. The message must be passed a **WinPositionType** along with two *x* and *y* coordinates passed in

SpecWinSizeSpec structures. The window's upper left corner will be drawn at the supplied coordinates.

Source: Unrestricted.

Destination: Any GS_USABLE GenDisplay, GenPrimary, or independently-displayable GenInteraction.

Pass:

dl	**VisUpdateMode.**
dh	**WinPositionType**.
cx	The new *x* coordinates of the window.
bp	The new *y* coordinates of the window.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

MSG_GEN_SET_WIN_SIZE

This message sets a window's size. The message must be passed a **WinPositionType** along with two *x* and *y* coordinates passed in **SpecWinSizeSpec** structures.

Source: Unrestricted.

Destination: Any GS_USABLE GenDisplay, GenPrimary, or independently-displayable GenInteraction.

Pass:

dl	**VisUpdateMode.**
dh	**WinPositionType**.
cx	The new *x* coordinates of the window.
bp	The new *y* coordinates of the window.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\genC.def**

MSG_GEN_UPDATE_VISUAL

This message updates the visual tree at the node specified and is used for updating objects modified with VUM_MANUAL.

Source: Unrestricted.

Destination: Any generic object.

Pass: `dl` **VisUpdateMode** to determine when the object should be redrawn.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\genC.def**

MSG_GEN_USE_VIS_MONIKER

This message allows an object to reference a visual moniker; this message does not destroy the chunk of the object's current visual moniker. The moniker must reside in the same block as the object. Use MSG_GEN_REPLACE_VIS_MONIKER if you wish to set a moniker from a source outside the object block or if you wish to use a new moniker, overwriting the old one.

You cannot pass a moniker list's chunk handle using this message. See MSG_GEN_FIND_MONIKER for information on selecting monikers from within a list.

Source: Unrestricted.

Destination: Any generic object.

Pass: `cx` The chunk handle of the visual moniker to use.
`dl` **VisUpdateMode** to determine when the moniker will be redrawn.

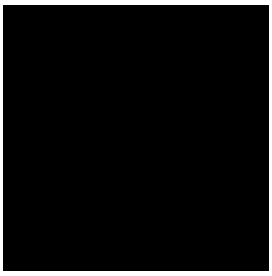
Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\genC.def**

Object Assembly Reference

GenApplication



GenApplicationClass 127

3

GenApplicationClass

Instance Data

```
GAI_appRef              AppInstanceReference
                  noreloc GAI_appRef.AIR_diskHandle
   UIC Syntax:   static       typeComp “AppInstanceReference” : "";
```

GAI_appRef is internal. It stores information needed to reload this application. If the application is detached, this instance field contains information necessary to reload this application to its state at detachment. You cannot set this field from within the **.ui** file.

```
GAI_appMode             dw    (?)
   UIC Syntax:   static       wordComp : 0;
```

GAI_appMode stores the message that should be sent to the application's Process object to bring the application back from a saved state. This is initially null and is set by the **GenProcessClass** object as soon as it is determined. You cannot set this field from within the **.ui** file.

```
GAI_launchFlags         AppLaunchFlags
   UIC Syntax:   static       byteComp : 0;
```

GAI_launchFlags stores the **AppLaunchFlags** that govern how the application should be run. These flags are used internally and are set when the application is first launched. You cannot set this field from within the **.ui** file.

```
GAI_optFlags            ApplicationOptFlags
   UIC Syntax:   appOptFlags  = bitFieldComp byte {
                    multipleInitFileCategories
                    } : {};
```

appOptFlags (*GAI_optFlags*) contains optimization flags. The only current optimization flag *multipleInitFileCategories* indicates that the application uses more than one .INI category; this speeds searches of the .INI file.

```
GAI_appFeatures         word
   UIC Syntax:   appFeatures  = wordComp : 0;
```

appFeatures (GAI_appFeatures) stores a word representing the application's features as determined by the user's level of expertise. This field is used primarily by hints in GenControl objects and is rarely used directly otherwise.

```
GAI_specificUI        hptr
   UIC Syntax:  static       wordComp : 0;
```

GAI_specificUI stores the handle of the specific UI under which this application is running. This is determined and set by the system when the application is launched. You cannot set this field from within the **.ui** file.

```
GAI_states            ApplicationStates
   UIC Syntax:  appState     = bitFieldComp word {
                    unused1,
                    unused2,
                    unused3,
                    singleInstance,
                    quitDetaching,
                    avoidTransparentDetach,
                    trasnparentDetaching,
                    unused4,
                    quitting,
                    detaching,
                    focusable,
                    modelable,
                    notUserInteractable,
                    receivedAppObjectDetach,
                    attachedToStateFile
                    } : focusable, modelable;
```

GAI_states stores the **ApplicationStates** of the GenApplication. Some of these states may be set from within the **.ui** file, but most are set by the system. The syntax of these flags in their Assembly definitions is given after the Espire syntax.

singleInstance (AS_SINGLE_INSTANCE) indicates that the application is not capable of being launched multiple times. You should not need to set this.

quitDetaching (AS_QUIT_DETACHING) indicates that the detach sequence has been initiated as the result of a QUIT. This bit will only be set if *quitting* is also set; the bit is set in the UI thread at the same time MSG_META_DETACH is sent to the process. Therefore, this bit represents an intermediate step between *quitting* and *detaching*.

avoidTransparentDetach (AS_AVOID_TRANSPARENT_DETACH) indicates that the application should not be transparently detached. If the application is running within UILM_TRANSPARENT mode, then the application will not detach when another application is launched.

trasnparentDetaching (AS_TRANSPARENT_DETACHING) indicates that the application is being transparently detached. An application can be transparently detached if another application is started in this application's field and that field is marked UILM_TRANSPARENT.

quitting (AS_QUITTING) indicates that the application is currently quitting.

detaching (AS_DETACHING) indicates that the application object has received MSG_META_DETACH and is detaching.

focusable (AS_FOCUSABLE) indicates that the application may receive the focus exclusive from its field parent. When launched, if this flag is set, the application automatically grabs the focus. This flag is set by default.

modelable (AS_MODELABLE) indicates that the application may receive the model exclusive from its field parent. When launched, if this flag is set, the application will automatically grab the model exclusive. This flag is set by default.

notUserInteractable (AS_NOT_USER_INTERACTABLE) indicates that the application should not be interactable with the user. This prevents the user from navigating to non-visible applications or otherwise selecting the application.

receivedAppObjectDetach (AS_RECEIVED_APP_OBJECT_DETACH) indicates that the application has received a detach message.

attachedToStateFile (AS_ATTACHED_TO_STATE_FILE) indicates that the application is currently attached to a state file.

```
GAI_attachFlags       AppAttachFlags
   UIC Syntax:  static       wordComp : 0;
```

GAI_attachFlags stores the **AppAttachFlags** relating to restoring the application from a state file when attached. You cannot set this field from within the **.ui** file.

```
   UIC Syntax:  default genStates = default -usable;
```

The default *genStates* of a GenApplication object are modified so that the object is not *usable* initially.

```
UIC Syntax:  default genAttributes = default +targetable;
```

The default *genAttributes* of a GenApplication object are modified so that the object is additionally marked *targetable* initially.

```
GAI_appLevel          UIInterfaceLevel        UIIL_ADVANCED
   UIC Syntax:  appLevel     = wordComp : "UIIL_ADVANCED";
```

An application's user interface level is stored in the GenApplication's *GAI_appLevel* instance data entry. Each **UIInterfaceLevel** corresponds to a certain group of features. Changing the UI level changes the group of features that may be displayed.

```
GAI_iacpConnects      lptr.ChunkArrayHeader   0
   UIC Syntax:  static       nptrComp : null;
```

GAI_iacpConnects stores the chunk handle to an array of active IACP connections. This chunk stores the IACP connection value referring to the remote application and the type of IACP connection (i.e. a connection that is enabled during a MSG_GEN_PROCESS_OPEN_APPLICATION, for example). These values are manipulated internally and there is no need to access them. You will instead use a variety of messages provided with **GenApplicationClass** to register and unregister for application notification. You cannot set this field from within the **.ui** file.

Variable Instance Data

```
HINT_APP_IS_ENTERTAINING                    vardata
HINT_APP_IS_EDUCATIONAL                     vardata
HINT_APP_IS_PRODUCTIVITY_ORIENTED           vardata
```

GenApplication provides several hints that indicate the type of application. HINT_APP_IS_ENTERTAINING, HINT_APP_IS_EDUCATIONAL and HINT_APP_IS_PRODUCTIVITY_ORIENTED are provided for this purpose.

```
ATTR_GEN_APPLICATION_PRINT_CONTROL          vardata optr
```

ATTR_GEN_APPLICATION_PRINT_CONTROL stores the optr of the object to act as the destination for any messages sent to the **GenApplicationTravelOption** TO_PRINT_CONTROL. Specifically, this attribute is designed to allow remote printing capabilities.

```
ATTR_GEN_APPLICATION_KBD_OBJ                vardata optr
```

ATTR_GEN_APPLICATION_KBD_OBJ stores the optr of the object to act as the application's floating keyboard. This object must be a subclass of **GenInteractionClass** and must be in the generic tree below the application object. MSG_GEN_APPLICATION_DISPLAY_FLOATING_KEYBOARD will display this keyboard.

`ATTR_GEN_APPLICATION_SAVE_OPTIONS_TRIGGER vardata optr`

ATTR_GEN_APPLICATION_SAVE_OPTIONS_TRIGGER contains the optr of the Save Options trigger within the options menu. If you have a custom Save Options trigger, you should add the optr of this object in this field.

Messages

■ MSG_GEN_APPLICATION_ACCEPT_INPUT

This message undoes a previous MSG_GEN_APPLICATION_IGNORE_INPUT, allowing the GenApplication to once again handle input events normally.

Source: Infrequently used.

Destination: The GenApplication object to resume input handling.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_APP_MODE_COMPLETE

This message is sent to the application when its life as a user-interactable application is complete. The default behavior is to continue shutting down the process if there are no IACP connections active.

Source: Sent by the GenProcess object after it receives a MSG_META_ACK from detaching the application.

Destination: GenApplication object.

Pass: **dx:bp** Optr to which a MSG_META_ACK is sent, informing the object that its life as a user-interactable application is complete.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Generally not intercepted; If you have other server connections which you want taken into account before shutting the application completely down, you should intercept MSG_GEN_APPLICATION_IACP_GET_NUMBER_OF_CONNECTIONS instead.

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_BRING_UP_HELP

This message brings up help for an application. Normally, this is accomplished by sending a message to the focus object telling it to bring up a help window with the focus' help context.

Source: Unrestricted, though generally from an application Help icon or <F1>.

Destination: GenApplication object.

Pass: **dx:bp** Original acknowledgment optr from whomever is detaching the application (usually generated by the system).

Returns: Nothing.

Destroyed: **ax**

Interception: Generally not intercepted, though it may be useful if for some reason you do not wish to bring up help (e.g. it does not exist for this application).

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_BRING_WINDOW_TO_TOP

This message brings the passed window to the front of the screen.

Source: Unrestricted.

Destination: Any GenApplication object.

Pass: `cx:dx` The optr of the window object to be brought to the front of the screen.

Returns: Nothing.

Destroyed: `ax, bp`

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_BUILD_DIALOG_FROM_TEMPLATE

This message builds a dialog box, adding it below the GenApplication object, and sets it *usable*. If the dialog is no longer needed, it should be destroyed by sending MSG_GEN_REMOVE (passing a **VisUpdateMode** of VUM_NOW) and lastly sending MSG_META_BLOCK_FREE to the created block.

Pass: `cx:dx` Optr of the dialog box template object.

Returns: `cx:dx` Optr of the created dialog box.
`bp` **CustomDialogType**.

Destroyed: `ax`

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_BUILD_STANDARD_DIALOG

This message builds a standard dialog box for the application, adding it below the GenApplication object, and sets it *usable*. If the dialog is no longer needed, it should be destroyed by sending MSG_GEN_REMOVE (passing a

VisUpdateMode of VUM_NOW) and lastly sending MSG_META_BLOCK_FREE to the created block.

Source: Infrequently used.

Destination: The GenApplication to use the dialog box.

Pass:	**ss:bp**	**StandardDialogParams**.
	dx	Size of **StandardDialogParams**.
Returns:	**cx:dx**	Optr of the created standard dialog box.
	bp	**CustomDialogBoxFlags**.
Destroyed:	**ax**	
Library:	**Objects\gAppC.def**	

MSG_GEN_APPLICATION_CHECK_IF_ALWAYS_INTERACTABLE_OBJECT

This message checks if the passed object should always remain interactable.

Source: Unrestricted.

Destination: GenApplication object.

Pass:	**cx:dx**	Optr of the object whose interactable state is to be checked.
Returns:	CF	Set if the object is always interactable; clear if not.
Destroyed:	**ax, cx, dx, bp**	
Library:	**Objects\gAppC.def**	

MSG_GEN_APPLICATION_CHECK_IF_RUNNING_USER_DO_DIALOG

This message checks whether an application's thread is currently blocked waiting to return from a call to **UserDoDialog**.

Source: Unrestricted.

Destination: GenApplication object.

Pass:	Nothing.	
Returns:	**ax**	Set if a thread is blocked for use of **UserDoDialog**.
Destroyed:	**cx, dx, bp**	
Library:	**Objects\gAppC.def**	

MSG_GEN_APPLICATION_DO_STANDARD_DIALOG

This message creates and brings up a standard dialog box within the application.

When the application is finished with the dialog box, the message stored in *GADDP_message* is sent to the destination in *GADDP_finishOD* with **cx** set to the appropriate **InteractionCommand**. If *GADDP_dialog.SDP_type* is SDBF_CUSTOM and *GADDP_dialog.SDP_customFlags*.CDBF_INTERACTION_TYPE is GIT_MULTIPLE_RESPONSE, the dialog box may return a custom value in **cx** instead.

Pass: **ss:bp** **GenAppDoDialogParams**.
dx Size of **GenAppDoDialogParams**.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_FIELD_QUIT_ABORTED

This message is sent to applications by their parent GenField object if '*quitOnClose*' was aborted by one of the applications.

Source: GenField object.

Destination: All GenApplication objects below that GenField at the time of the aborted quit.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_FIND_MONIKER

This message finds the specified moniker in the GenApplication's **VisMonikerList** and optionally copies it into a specified destination block.

Source: Infrequently used.

Destination: The GenApplication to get the moniker from.

Pass:	**cx**	The handle of the destination block into which the chunk will be copied. For this to work, you must pass VMSF_COPY_CHUNK in **bp**.
	bp	A record of **VisMonikerSearchFlags** indicating what type of moniker to find in the moniker list and what to do with it when it is found.
	dh	The display type of the moniker to search for.
Returns:	**cx:dx**	The optr of the GenApplication's **VisMonikerList** chunk.
	bp	Unchanged.

Destroyed: **ax**

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_FLOATING_KEYBOARD_CLOSED

This message is sent by the floating keyboard to the application whenever it is closed.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_GET_APP_FEATURES

This message retrieves the set of features set for the application.

Source: Unrestricted—typically a GenControl object finding out the application's UI level.

Destination: The GenApplication running the controller.

Pass: Nothing.

Returns: **ax** Feature bitfield for that application.
dx **UIInterfaceLevel** for that application.

Destroyed: **cx, bp**

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_GET_APP_INSTANCE_REFERENCE

This message retrieves the values in *GAI_appRef*.

Source: Rarely used.

Destination: Any GenApplication object.

Returns: **dx** Handle to a **AppInstanceReference** block.

Destroyed: **ax, cx, bp**

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_GET_APP_MODE_MESSAGE

This message returns the message number stored in the GenApplication's *GAI_appMode* field.

Source: Rarely used.

Destination: Any GenApplication object.

Pass: Nothing.

Returns: **cx** Message.

Destroyed: **ax, dx, bp**

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_GET_ATTACH_FLAGS

This message retrieves the contents of the GenApplication's *GAI_attachFlags* field.

Source: Rarely used.

Destination: Any GenApplication object.

Pass: Nothing.

Returns: `cx` **AppAttachFlags**.

Destroyed: `ax, dx, bp`

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_GET_DISPLAY_SCHEME

This message gets the current display scheme used by the application.

Source: Infrequently used.

Destination: Any GenApplication.

Pass: Nothing.

Returns: Contents of a **DisplayScheme** structure in the following registers:

`al`	*DS_colorScheme* (not to be used by applications)
`ah`	*DS_displayType*
`cx`	*DS_unused*
`dx`	*DS_fontID*
`bp`	*DS_pointSize*
`CF`	Set.

Pass: Nothing.

Library: **Objects\gAppC.def**

Object Assembly Reference

MSG_GEN_APPLICATION_GET_GCN_LIST_OF_LISTS

This message retrieves the GenApplication's GCN list of lists chunk handle. This chunk handle may then be used with a number of kernel routines for GCN list management or to perform operations on individual GCN lists.

Source: Any object in the GenApplication's thread.

Destination: Any GenApplication object.

Pass: Nothing.

Returns: **ax** Chunk handle of the GCN list of lists (or zero if none exists).

Destroyed: **cx, dx, bp**

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_GET_LAUNCH_FLAGS

This message retrieves the contents of the GenApplication's *GAI_launchFlags* field.

Source: Rarely used.

Destination: Any GenApplication object.

Pass: Nothing.

Returns: **al** **AppLaunchFlags**.

Destroyed: **ah, cx, dx, bp**

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_GET_MEASUREMENT_TYPE

This message gets the measurement currently used by the application.

Source: Infrequently used.

Destination: The GenApplication whose measurement is to be retrieved.

Pass: Nothing.

Returns: **al** **MeasurementType** (either metric or US).
ah **AppMeasurementType**.

Destroyed: **cx, dx, bp**

Library: **Objects\gAppC.def**



MSG_GEN_APPLICATION_GET_MODAL_WIN

This message returns the current top modal window for the application, if any is present.

Source: Unrestricted.

Destination: GenApplication object.

Pass: Nothing.

Returns: **cx:dx** Optr of top modal windowed object

Destroyed: **ax, bp**

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_GET_STATE

This message retrieves the current application state, stored in *GAI_states*.

Source: Rarely used.

Destination: Any GenApplication object.

Pass: Nothing.

Returns: **ax** The **ApplicationStates** record stored in *GAI_states*.

Destroyed: **cx, dx, bp**

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_HOLD_UP_INPUT

This message causes the GenApplication to mark itself busy and redirect input events to a special “hold-up” queue. When the application is ready to resume normal activity, it first handles the messages in the hold-up queue before handling new input messages.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_IACP_COMPLETE_CONNECTIONS

This message completes all pending IACP connections, accepting any queued messages that have been waiting to be handled. If you subclass it, be sure to call the superclass at some point.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION _IACP_GET_NUMBER_OF_APP_MODE_CONNECTIONS

This message retrieves the number of connections which require that the application be open in app-mode (as opposed to engine mode). This message is used to check whether an application can be closed down into engine mode even if some IACP connections are still open.

Source: This message is called by the **GenProcessClass** when the UI has finished processing MSG_META_QUIT to determine if the application should really close down to engine mode.

Destination: GenApplication object of application.

Pass: Nothing.

Returns:

`cx`	Number of application-mode IACP connections.
`dx, bp`	Unchanged.

Destroyed: `ax`

Interception: Only intercept if you have other server objects besides your GenApplication object. You should call the superclass first and then add the number of connections to other objects onto the result returned by **GenApplicationClass**.

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_IACP_GET_NUMBER_OF_CONNECTIONS

This message returns the number of active engine or app-mode IACP connections for a given application. This message is used to check whether an application open only for IACP purposes may be closed.

Source: Called by GenProcessClass when the UI has finished processing MSG_META_QUIT for the application; this determines if the application should exit at this point or if there are client applications that need the application to stay open.

Destination: GenApplication object of the application.

Pass: Nothing.

Returns:

cx	Number of open connections. If non-zero, application will remain open.
dx, bp	Unchanged.

Destroyed: **ax**

Interception: Only intercept if you have other server objects beside your GenApplication object. If intercepting, call the superclass first and then add the number of connections to the other objects onto the result returned by **GenApplicationClass**.

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_IACP_REGISTER

This message is sent by a GenApplication object to itself when it registers for IACP. It is not a message meant to be sent externally to an application to register it for IACP. Instead, you can subclass this message and register the object with other lists.

Source: Sent by the GenApplication object to itself.

Destination: The GenApplication object to register with IACP.

Pass: **cl** **IACPServerMode** in which the server is operating.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: May be intercepted if there are other lists with which you want to register the application, or other server objects. You must make sure to call the superclass, however.

Library: **Objects\gAppC.def**

■ MSG_GEN_APPLICATION_IACP_SHUTDOWN_ALL_CONNECTIONS

This message shuts down all IACP connections for a given application, either on the server or the client side of the connection.

Source: Sent by the GenApplication object to itself.

Destination: GenApplication object of the application.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax

Interception: May be intercepted to allow connections to other server objects to be shut down. You must call the superclass at some point to ensure that application connections are shut down as well.

Library: **Objects\gAppC.def**

■ MSG_GEN_APPLICATION_IACP_UNREGISTER

This message is sent by a GenApplication object to itself when it unregisters for IACP. It is not a message meant to be sent externally to an application to unregister it for IACP. Instead, you can subclass this message and unregister the object with other lists.

Source: Sent by the GenApplication object to itself.

Destination: The GenApplication object to unregister with IACP.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax

Interception: May be intercepted if there are other lists with which you need to unregister the application, or other server objects. You must make sure to call the superclass, however.

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_IGNORE_INPUT

This message causes the GenApplication to consume all input events it receives rather than handle them. This message may be used during debugging as a *last* resort to help find synchronization problems.

Source: Infrequently used.

Destination: The GenApplication object to consume input events.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gAppC.def

MSG_GEN_APPLICATION_INITIATE_UI_QUIT

This message causes the GenApplication to begin quitting. The application will automatically go through the entire quit sequence.

Source: Infrequently used.

Destination: The GenApplication of the application to be shut down.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gAppC.def

MSG_GEN_APPLICATION_INK_QUERY_REPLY

This message is sent to an Application object in reply to a MSG_META_QUERY_IF_PRESS_IS_INK. It indicates whether the object that was queried can or can not handle Ink presses. The GenApplication object responds by sending a message to the UI.

Source: Sent by an object in response to MSG_META_QUERY_IF_PRESS_IS_INK.

Destination: The GenApplication object associated with the sender.

Pass: cx **InkReturnValue** indicating whether the object queried can handle Ink input or not. Will be one of IRV_NO_INK,

IRV_INK_WITH_STANDARD_OVERRIDE, IRV_DESIRES_INK, or IRV_WAIT.

bp	**InkDestinationInfo** containing the GState, if any, to be used when drawing Ink.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gAppC.def

MSG_GEN_APPLICATION_INSTALL_TOKEN

This message instructs the GenApplication object to set its token into the token database file.

Source: Infrequently used.

Destination: The GenApplication to have its token installed.

Pass: Nothing.

Returns: Nothing. (The token will be installed into the token database.)

Destroyed: ax, cx, dx, bp

Library: Objects\gAppC.def

MSG_GEN_APPLICATION_LOWER_WINDOW_TO_BOTTOM

This message sends the specified window to the back of the screen, behind other window objects.

Source: Unrestricted.

Destination: Any GenApplication object.

Pass:	**cx:dx**	The optr of the window object to be sent to the back of the screen.

Returns: Nothing.

Destroyed: ax, bp

Library: Objects\gAppC.def

MSG_GEN_APPLICATION_MARK_APP_COMPLETELY_BUSY

This message is rarely used and forces a busy state over the application regardless of other states. It should be used only when a time-intensive task is going on in the UI and the program can not handle input during that time.

Source: Infrequently used.

Destination: The GenApplication object to be marked completely busy.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gAppC.def

MSG_GEN_APPLICATION_MARK_APP_NOT_COMPLETELY_BUSY

This message undoes MSG_GEN_APPLICATION_MARK_COMPLETELY_BUSY, allowing the application to once again handle user input.

Source: Infrequently used.

Destination: The GenApplication object to be marked not busy.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gAppC.def

MSG_GEN_APPLICATION_MARK_BUSY

This message marks the application busy and changes the cursor image.

Source: Sent automatically by objects with GA_INITIATES_BUSY_STATE set.

Destination: The GenApplication object running the sender.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gAppC.def

MSG_GEN_APPLICATION_MARK_NOT_BUSY

This message marks the application not busy, removing the effect of a previous MSG_GEN_APPLICATION_MARK_BUSY.

Source: Sent automatically by objects with GA_INITIATES_BUSY_STATE set.

Destination: The GenApplication object running the sender.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gAppC.def

MSG_GEN_APPLICATION_NOTIFY_MODAL_WIN_CHANGE

This message is called on a GenApplication object by the UI whenever the application should check to see if there is a change in modal status. The behavior is to look for the top system-modal window owned by the application and then the top application-modal window within the application's layer.

This message sets the AS_ATTACHED_TO_STATE_FILE in the GenApplication's *GAI_states* field.

Source: Sent by the UI.

Destination: Any GenApplication object.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gAppC.def

MSG_GEN_APPLICATION_OPEN_COMPLETE

This message is sent by the GenApplication object to itself when it has finished opening (after it has set itself usable). It is sent via the queue and indicates that the application's UI is fully usable.

Source: A GenApplication after it has set itself GS_USABLE.

Destination: Sent to itself.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gAppC.def

MSG_GEN_APPLICATION_QUERY_UI

This message is used to determine which UI should be used at a given point in the generic tree for a certain type of object.

Source: Infrequently used.

Destination: Any GenApplication object.

Pass: Nothing.

Returns: ax The handle of the specific UI library geode.

Destroyed: cx, dx, bp

Library: Objects\gAppC.def

MSG_GEN_APPLICATION_QUIT_AFTER_UI

This message is called from the MSG_META_QUIT handler in GenProcessClass, after the UI has finished its MSG_GEN_APPLICATION_INITIATE_UI_QUIT sequence. This message is the application's last chance to abort a quit before the DETACH sequence begins. The default behavior is to abort the QUIT if the application is still open for

the user (i.e. not ALF_OPEN_FOR_IACP_CONNECTION_ONLY) or if an IACP connection remains that requires the application to remain open.

Source: **GenProcessClass**.

Destination: GenApplication object of the application.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Interception: May be intercepted and not sent to the superclass to abort the QUIT.

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_REMOVE_ALL_BLOCKING_DIALOGS

This message brings down all application-specific blocking dialogs (i.e. all dialogs that are brought up via **UserDoDialog** and **UserStandardDialog**.)

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_RESUME_INPUT

This message causes a GenApplication to resume normal input handling after it has been held up with MSG_GEN_APPLICATION_HOLD_UP_INPUT.

Source: Used infrequently.

Destination: The GenApplication object running the sender.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_SEND_APP_INSTANCE_REFERENCE_TO_FIELD

This message causes the GenApplication to send the contents of its *GAI_appRef* field off to its parent GenField object.

Source: Rarely used.

Destination: Any GenApplication object.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_SET_APP_FEATURES

This message sets a new set of features into the GenApplication's *GAI_appFeatures* record. This message in turn generates a MSG_GEN_APPLICATION_UPDATE_APP_FEATURES for your application object to intercept. (The message handler for that message must in turn send MSG_GEN_APPLICATION_UPDATE_FEATURES_VIA_TABLE to activate the UI changes specified in the features list.)

Source: Unrestricted—typically a system function.

Destination: The GenApplication having its features set.

Pass: **cx** The new word-sized record of application features to set.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_SET_APP_INSTANCE_REFERENCE

This message sets the *GAI_appRef* field to the passed structure.

Source: Rarely used.

Destination: Any GenApplication object.

Pass: **dx** The handle of the **AppInstanceReference** data block.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gAppC.def**

■ MSG_GEN_APPLICATION_SET_APP_LEVEL

This message sets the **UIInterfaceLevel** in the application's *GAI_appLevel* instance field.

Pass: `cx` **UIInterfaceLevel.**

Returns: Nothing.

Destroyed: `ax, dx, bp`

Library: **Objects\gAppC.def**

■ MSG_GEN_APPLICATION_SET_APP_MODE_MESSAGE

This message stores a message into the GenApplication's *GAI_appMode* field. Generally, this message indicates the current mode of the application. Should the application be shut down and restored, this message will be sent to the Process object to restore the state to the same mode.

Source: Rarely used.

Destination: Any GenApplication object.

Pass: `cx` The message number to be stored in *GAI_appMode*.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gAppC.def**

■ MSG_GEN_APPLICATION_SET_ATTACHED_TO_STATE_FILE

This message sets the AS_ATTACHED_TO_STATE_FILE in the GenApplication's *GAI_states* field.

Source: Sent by the UI or the kernel.

Destination: The GenApplication object that has been attached to a state file.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_SET_MEASUREMENT_TYPE

This message sets the measurement type used by the application.

Source: Infrequently used.

Destination: The GenApplication to have the new measurement type.

Pass: `cl` **AppMeasurementType** to set for the application. Can be AMT_US, AMT_METRIC, or AMT_DEFAULT.

Returns: `cx, dx, bp` Unchanged.

Destroyed: `ax`

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_SET_NOT_ATTACHED_TO_STATE_FILE

This message clears the AS_ATTACHED_TO_STATE_FILE in the GenApplication's *GAI_states* field.

Source: Sent by the UI or the kernel.

Destination: The GenApplication object that has been detached to a state file.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_SET_NOT_QUITTING

This message clears the AS_QUITTING bit in the application's *GAI_states* bitfield.

Source: Sent by the UI or the kernel.

Destination: A GenApplication object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_SET_NOT_USER_INTERACTABLE

This message sets the AS_NOT_USER_INTERACTABLE flag in the application's *GAI_states* field.

Source: Infrequently used.

Destination: The GenApplication to be made not interactable.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_SET_STATE

This message alters a GenApplication's *GAI_states* flags. This message should only be used to set flags that aren't set internally by the UI. Flags that can be altered are the AS_FOCUSABLE, AS_MODELABLE, AS_NOT_USER_DETACHABLE and AS_AVOID_TRANSPARENT_DETACH state bits.

This message does not reject attempts to set internal bits; therefore, be careful in using this message and only use it to set the external bits mentioned above.

Source: Unrestricted. This message is also used internally.

Destination: Any GenApplication object.

Pass:

cx	**ApplicationStates** to set.
dx	**ApplicationStates** to clear.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: May intercept, but must pass to superclass at some point.

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_SET_TASK_ENTRY_MONIKER

This message changes the moniker which is used in the GenField's task list. The task list menu will be updated if currently visible.

Source: Infrequently used.

Destination: The GenApplication to have its moniker changed.

Pass: `cx:dx` Optr of the **VisMoniker** (or chunk containing the **VisMonikerList**) that will be set into the task list.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_SET_USER_INTERACTABLE

This message clears the AS_NOT_USER_INTERACTABLE flag in the application's *GAI_states* field.

Source: Infrequently used.

Destination: The GenApplication to be made interactable.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_TOGGLE_CURRENT_MENU_BAR

This message toggles the GIV_POPOUT state of the current GenPrimary's menu bar. This message only takes effect if the menu bar is toggleable (i.e. if UIWO_POPOUT_MENU_BAR is set).

Source: Unrestricted.

Destination: GenApplication object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gAppC.def**

MSG_GEN_APPLICATION_TOGGLE_CURSOR

This message toggles the cursor for a text object.

Source: Infrequently used.

Destination: The GenApplication of the text object.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gAppC.def

MSG_GEN_APPLICATION_TOGGLE_EXPRESS_MENU

This message toggles (opens or closes) the parent field's express menu.

Source: Unrestricted.

Destination: GenApplication object.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gAppC.def

MSG_GEN_APPLICATION_TOGGLE_FLOATING_KEYBOARD

This message toggles the state of the floating keyboard within the current application. Applications may subclass this to bring up their own PenInputControl (or equivalent) object. Otherwise the application object will create its own.

Source: Unrestricted, though generally only supported on pen-based systems.

Destination: GenApplication object.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gAppC.def

■ MSG_GEN_APPLICATION_UPDATE_APP_FEATURES

This message is sent by the application to itself when it is told to change either its features or its **UIInterfacelevel**. This message is passed a number of parameters, most of which should simply be passed to MSG_GEN_APPLICATION_UPDATE_FEATURES_VIA_TABLE. If you have reparent objects (or un-reparent objects), you must set them up here.

Source: Sent by an application object to itself in response to a MSG_GEN_APPLICATION_SET_APP_FEATURES or MSG_GEN_APPLICATION_SET_APP_LEVEL.

Destination: The GenApplication object.

Pass: **ss:bp** **GenAppUpdateFeatureParams** containing:

GAUFP_featuresOn
The set of features to set on.

GAUFP_featuresChanged
The set of features changed (deleted).

GAUFP_level The new **UIInterfaceLevel**.

GAUFP_oldLevel
The previous **UIInterfaceLevel**.

GAUFP_appOpening
Set if the application is starting.

GAUFP_table Table of **GenAppUsabilityTuple** entries that must be updated when the user level changes. This table is usually set up as global data and maps each user level feature to a **GenAppUsabilityTuple**.

GAUFP_tableLength
The number of table entries in *table*.

GAUFP_levelTable
This table contains the GenAppUsabilityTuples corresponding to objects that contain their *own* features and that must be notified when the user level changes. This is so that those objects can generate their own feature updates. Typically, controllers are included among these objects.

GAUFP_reparentObject
The optr of the object to be reparented to another UI location. You must supply this object if you have a **GenAppUsabilityTuple** entry that contains a GAUC_REPARENT entry.

GAUFP_unreparentObject
The optr of the object to be unreparented. If a null optr is

passed, the object will by default be moved up and added as the next sibling of its current parent.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: To set an application's features, you must intercept this message and send MSG_GEN_APPLICATION_UPDATE_FEATURES_VIA_TABLE; there is no default message handler. This message is provided as a convenient point to intercept and change features before executing the changes.

Library: **Objects\gAppC.def**

■ MSG_GEN_APPLICATION_UPDATE_FEATURES_VIA_TABLE

This message is called to update the application's features to reflect a new set of features.

Source: Typically, your message handler for MSG_GEN_APPLICATION_UPDATE_APP_FEATURES.

Destination: The GenApplication object.

Pass: **ss:bp** **GenAppUpdateFeatureParams**.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Generally not intercepted. Intercept MSG_GEN_APPLICATION_UPDATE_APP_FEATURES instead.

Library: **Objects\gAppC.def**

■ MSG_GEN_APPLICATION_VISIBILITY_NOTIFICATION

This message is sent by the specifically built part of a UI object when it either becomes visible or non-visible.

Source: The specific UI.

Destination: The GenApplication that has become visible or not visible.

Pass: **cx:dx** The optr of the object sending the notification message.
bp A Boolean indicating the state of the object: *true* if open, *false* if closed.

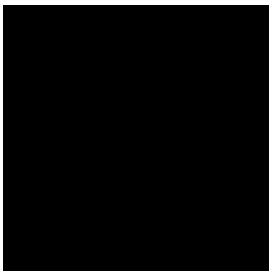
Returns: Nothing.

Destroyed: **ax, bp**

Library: **Objects\gAppC.def**

Object Assembly Reference

GenDisplay



4

GenDisplayClass 161

GenPrimaryClass 166

GenDisplayClass

Instance Data

```
GDI_attributes       GenDisplayAttrs (mask GDA_USER_DISMISSABLE)
   UIC Syntax:  attributes = bitFieldComp byte { userDismissable }
                    : userDismissable;
```

attributes (*GDI_attributes*) contains the attributes flags. There is only one flag defined for this field, namely *userDismissable* (GDA_USER_DISMISSABLE), which is on by default.

```
GDI_document         optr
   UIC Syntax:  document = optrComp : null;
```

document (*GDI_document*) contains the optr of the GenDocument object associated with this display.

Messages

■ MSG_GEN_DISPLAY_CLOSE

This message is activates the specific UI's way of closing a window (e.g. in Motif, double-clicking the window's control button in the upper-left corner). The **GenDisplayClass** handler does nothing but send a MSG_GEN_DOCUMENT_CLOSE to the Document object specified in *GDI_document*. The **GenPrimaryClass** handler closes the application.

Source: Unrestricted.

Destination: GenDisplay.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: If the Display is not associated with a GenDocument, you will need to subclass this message for it to have any effect at all. If the display is associated with a GenDocument object, you should probably subclass your Document object's MSG_GEN_DOCUMENT_CLOSE instead. Primary objects

may subclass this message if they want to alter or replace the default behavior (of closing the application).

Library: **Objects\gDispC.def**

■ MSG_GEN_DISPLAY_GET_ATTRS

This message retrieves the *GDI_attributes* field from the destination object.

Source: Unrestricted.

Destination: Any GenDisplay or GenPrimary object.

Pass: Nothing.

Returns: `cl` **GenDisplayAttrs**

Destroyed: `ax, ch, dx, bp`

Library: **Objects\gDispC.def**

■ MSG_GEN_DISPLAY_GET_DOCUMENT

This message returns the optr of the GenDocument associated with a given GenDisplay. This is equal to the value of the GenDisplay's *GDI_document* field.

Source: Unrestricted—ordinarily sent only by Document Control objects.

Destination: GenDisplay.

Pass: Nothing.

Returns: `cx:dx` optr of document object.

Destroyed: `ax, bp`

Library: **Objects\gDispC.def**

■ MSG_GEN_DISPLAY_GET_MAXIMIZED

This message indicates whether the recipient is maximized.

Source: Unrestricted.

Destination: Any GenDisplay or GenPrimary object.

Pass: Nothing.

Returns: `CF` Set if object is maximized.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gDispC.def**

Object Assembly Reference

MSG_GEN_DISPLAY_GET_MINIMIZED

This message indicates whether the recipient is minimized.

Source: Unrestricted.

Destination: Any GenDisplay or GenPrimary object.

Pass: Nothing.

Returns: CF Set if object is minimized.

Destroyed: ax, cx, dx, bp

Library: Objects\gDispC.def

MSG_GEN_DISPLAY_SET_ATTRS

This message changes the **GenDisplayAttrs** field of the destination object.

Source: Unrestricted.

Destination: Any GenDisplay or GenPrimary object.

Pass: cl Record of **GenDisplayAttrs** flags. There is only one flag defined, namely GDA_USER_DISMISSABLE.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gDispC.def

MSG_GEN_DISPLAY_SET_MAXIMIZED

This message instructs a display or primary object to maximize itself. If the window is already maximized, the message has no effect.

Source: Unrestricted.

Destination: Any GenDisplay or GenPrimary object.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Interception: You should not change the behavior of this message. You may intercept this message to find out when a window is being maximized; however, you should make sure to pass this message on to the superclass.

Library: Objects\gDispC.def

MSG_GEN_DISPLAY_SET_MINIMIZED

This message instructs a display or primary object to minimize itself. The result depends on the specific UI. Primary windows are usually iconified; display windows might be iconified or temporarily removed. If the window is already minimized, the message has no effect.

Source: Unrestricted.

Destination: Any GenDisplay or GenPrimary object.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Interception: You should not change the behavior of this message. You may intercept this message to find out when a window is being minimized; however, you should make sure to pass this message on to the superclass.

Library: Objects\gDispC.def

MSG_GEN_DISPLAY_SET_NOT_MAXIMIZED

This message instructs a display or primary object to de-maximize itself. It will generally be restored to its position and configuration as of the time it was maximized. If the window is not maximized, the message has no effect.

Source: Unrestricted.

Destination: Any GenDisplay or GenPrimary object.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Interception: You should not change the behavior of this message. You may intercept this message to find out when a window is being de-maximized; however, you should make sure to pass this message on to the superclass.

Library: Objects\gDispC.def

MSG_GEN_DISPLAY_SET_NOT_MINIMIZED

This message instructs a display or primary object to de-minimize itself. It will generally be restored to its position and configuration as of the time it was minimized. If the window is not minimized, the message has no effect.

Source: Unrestricted.

Destination: Any GenDisplay or GenPrimary object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: You should not change the behavior of this message. You may intercept this message to find out when a window is being de-minimized; however, you should make sure to pass this message on to the superclass.

Library: **Objects\gDispC.def**

MSG_GEN_DISPLAY_UPDATE_FROM_DOCUMENT

This message instructs a GenDisplay to update its instance data from its associated GenDocument object (if any). This message is ordinarily sent by the Document Control objects.

Source: Unrestricted—ordinarily sent only by Document Control objects.

Destination: GenDisplay.

Pass: `ss:bp` **DocumentUpdateParams**

Returns: Nothing.

Destroyed: `ax, cx, dx`

Interception: Normally not intercepted. If you subclass this message to add special updating behavior, be sure to end with an **ObjCallSuperNoLock**.

Library: **Objects\gDispC.def**

GenPrimaryClass

Instance Data

There are no instance data fields defined specifically for this class.

Messages

MSG_GEN_PRIMARY_GET_LONG_TERM_MONIKER

Use this message to find out the moniker of a GenPrimary object. The message returns the chunk handle of the moniker; the moniker is in the same block as the GenPrimary object.

Source: Unrestricted.

Destination: GenPrimary.

Pass: Nothing.

Returns: **ax** The chunk handle of the primary's long-term moniker. The chunk is in the same object block as the GenPrimary.

Destroyed: **cx, dx, bp**

Library: **Objects\gPrimC.def**

MSG_GEN_PRIMARY_REPLACE_LONG_TERM_MONIKER

This message is used to replace a primary's long term moniker with a new one. This message's arguments are precisely like those to the message MSG_GEN_REPLACE_VIS_MONIKER. Note that a long term moniker is ordinarily a simple text string.

Source: Unrestricted.

Destination: GenPrimary.

Pass: Same as MSG_GEN_REPLACE_VIS_MONIKER.

Returns: **ax** Chunk handle of object's long term moniker.

Destroyed: **cx, dx, bp**

Library: **Objects\gPrimC.def**

MSG_GEN_PRIMARY_USE_LONG_TERM_MONIKER

This message instructs a primary window to change its long-term moniker. The new long-term moniker must already be in a chunk in the same object block as the Primary. The chunk containing the obsolete long-term moniker will not be freed; you must do this manually.

Source: Unrestricted.

Destination: GenPrimary.

Pass: `cx` **ChunkHandle** of chunk in same object block as the GenPrimary. The chunk should contain the new long-term moniker.

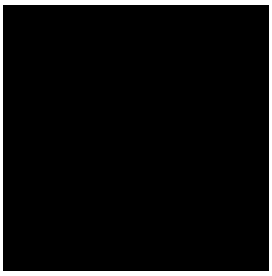
Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gPrimC.def**

Object Assembly Reference

GenTrigger



GenTriggerClass 171

5

GenTriggerClass

Instance Data

```
GTI_destination                         optr
GTI_actionMsg                           word
   UIC Syntax:  action    = actionComp : null;
```

action (*GTI_actionMsg* and *GTI_destination*) sets the message for the trigger to send out upon activation and the object or process for the trigger to send its message to upon activation. *action* takes two parameters, separated by commas; note that these are two separate instance fields within the object. The first parameter to *action* is the message name; the second is the object or process name.

E.g.:

```
action = MSG_MY_MESSAGE, process;
```

The activation itself is determined by the specific UI. The destination part of the *action* may be a **TravelOption** or a **GenTravelOption**.

Variable Instance Data

```
ATTR_GEN_TRIGGER_ACTION_DATA            vardata word
```

ATTR_GEN_TRIGGER_ACTION_DATA assigns data to pass with trigger's particular message. The data may be enclosed in brackets if it is more than one word of data. If the data is less than or equal to three words of data, it is passed in these registers in the following order: **cl, ch, dl, dh, bp low, bp high**. If the data is more than three words, it is placed on the stack and the message is sent using MF_STACK. This attribute does not apply to a message defined with ATTR_GEN_TRIGGER_CUSTOM_DOUBLE_PRESS.

```
ATTR_GEN_TRIGGER_INTERACTION_COMMAND    vardata InteractionCommand
```

A GenTrigger may perform special actions within a GenInteraction object. The ATTR_GEN_TRIGGER_INTERACTION_COMMAND attribute sets the

special activity for these types of triggers. This attribute indicates that the GenTrigger is an Interaction command trigger for the parent GenInteraction object. ATTR_GEN_TRIGGER_INTERACTION_COMMAND takes a word of data (of type **InteractionCommand**) specifying the command type of the trigger. A trigger with an **InteractionCommand** can perform one of several standard actions. Most of these actions are specific to dialog boxes.

`ATTR_GEN_TRIGGER_CUSTOM_DOUBLE_PRESS` `vardata word`

Your trigger sends out the message in the *actionMsg* field whenever the user clicks on the trigger, whether that click is a single or a double-click. If you wish your trigger to send out an alternate message if the user double-clicks, use ATTR_GEN_TRIGGER_CUSTOM_DOUBLE_PRESS.

`ATTR_GEN_TRIGGER_IMMEDIATE_ACTION` `vardata`

This attribute dispatches the message immediately to its event queue before any other action, including UI functionality.

Hints

`HINT_TRIGGER_BRINGS_UP_WINDOW` `vardata`

This hint indicates that the trigger's *action* results in the display of another window. The specific UI may use this hint to alter the display of the trigger's moniker.

`HINT_TRIGGER_DESTRUCTIVE_ACTION` `vardata`

This hint indicates that the trigger's *action* is potentially destructive. Trigger's with this attribute will not be made the default action within a dialog box.

Object Assembly Reference

Messages

MSG_GEN_TRIGGER_GET_ACTION_MSG

This message retrieves the *GTI_actionMsg* instance data of the trigger.

Source: Unrestricted.

Destination: Any GenTrigger object.

Pass: Nothing.

Returns: **cx** Message in *GTI_actionMsg*.

Destroyed: **ax, dx, bp**

Library: **Objects\gTrigC.def**

MSG_GEN_TRIGGER_GET_DESTINATION

This message returns the *GTI_destination* instance data field of the GenTrigger.

Source: Unrestricted.

Destination: Any GenTrigger object.

Pass: Nothing.

Returns: **cx:dx** The optr of destination object (in *GTI_destination*).

Destroyed: **ax, bp**

Library: **Objects\gTrigC.def**

MSG_GEN_TRIGGER_MAKE_DEFAULT_ACTION

This message sets the trigger to be the default action in its windowed parent. It is the message equivalent of HINT_DEFAULT_DEFAULT_ACTION, defined in **GenClass**.

Source: Unrestricted.

Destination: Any GenTrigger object.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gTrigC.def**

MSG_GEN_TRIGGER_SEND_ACTION

This message causes a GenTrigger to send its action message specified in *GTI_actionMsg* to the output specified in *GTI_destination* if the object is GS_USABLE. This message is sent automatically by the UI as part of the default behavior when a GenTrigger is activated by the specific UI. If you wish to simulate a double-click, sending the message included in an object's ATTR_GEN_TRIGGER_CUSTOM_DOUBLE_PRESS, pass this message with **cl** set to a non-zero value. Double-press messages may not pass any data in a trigger's ATTR_GEN_TRIGGER_ACTION_DATA.

Slightly more useful is the **GenClass** message MSG_GEN_ACTIVATE, which activates the trigger at the specific UI's level, performing any default activation behavior (such as making the trigger blink to show that it is being activated).

Source: Unrestricted. Usually the specific UI.

Destination: Any usable GenTrigger object.

Pass: **cl** Pass non-zero to use the message stored in ATTR_GEN_TRIGGER_CUSTOM_DOUBLE_PRESS rather than message in *GTI_actionMsg*.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gTrigC.def**

MSG_GEN_TRIGGER_SET_ACTION_MSG

This message sets the *GTI_actionMsg* instance field of a GenTrigger to the message passed.

Source: Unrestricted.

Destination: Any GenTrigger object.

Pass: **cx** Message to assign *GTI_actionMsg* to.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gTrigC.def**

Object Assembly Reference

MSG_GEN_TRIGGER_SET_DESTINATION

This message sets the *GTI_destination* instance field of the GenTrigger to the optr passed.

Source: Unrestricted.

Destination: Any GenTrigger object.

Pass: `cx:dx` The optr of the new destination.

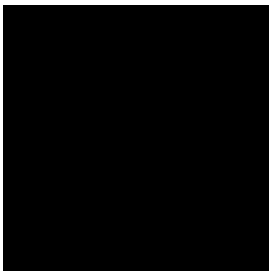
Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gTrigC.def**

Object Assembly Reference

GenGlyph



GenGlyphClass 179

6

GenGlyphClass

Instance Data

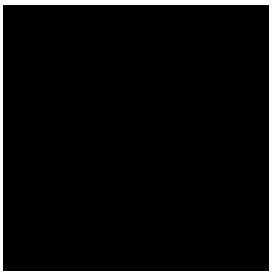
GenGlyphClass has no instance data defined.

Messages

GenGlyphClass has no messages defined.

Object Assembly Reference

GenInteraction



7

GenInteractionClass 183

GenInteractionClass

Instance Data

```
GII_type        GenInteractionType              (GIT_ORGANIZATIONAL)
   UIC Syntax:  type          = enumComp byte {
                   organizational,
                   properties,
                   progress,
                   command,
                   notification,
                   affirmation,
                   multipleResponse
                   } : organizational;
```

type (*GII_type*) describes the function of the GenInteraction.

organizational (GIT_ORGANIZATIONAL) indicates that the Interaction is merely organizational. This allows the Interaction to provide geometry management (and not much else) to its children. Any objects within an organizational Interaction should be able to perform their actions independently. Organizational Interactions may appear in any visual form: sub-groups, menus, or dialog boxes.

properties (GIT_PROPERTIES) indicates that the Interaction contains properties (attributes) that the user can change. Typically, a properties Interaction will operate in one of two modes: immediate or delayed. If the *visibility* attribute causes this Interaction to appear as a dialog box, the specific UI may create "Apply" and "Reset" triggers. In delayed mode, an apply trigger allows the Interaction to process changes made within the properties Interaction as a group rather than individually; the reset trigger allows the Interaction to reset the properties to their former state. If a popup is GIT_PROPERTIES, it will operate in immediate mode even if hints attempt to override this.

progress (GIT_PROGRESS) indicates that the Interaction displays a progress report for some operation. For example, a dialog box may

appear showing a disk copy operation with the time remaining to its completion. If the *visibility* attribute causes this Interaction to appear as a dialog box, the specific UI may create a "Stop" trigger that halts the operation in progress and the progress reporting. Your application is responsible for stopping the Interaction from communicating its progress report and may dismiss the Interaction if the specific UI desires.

command (GIT_COMMAND) indicates that the Interaction contains commands that the application provides. If the *visibility* attribute causes this Interaction to appear as a dialog box, the specific UI may create a "Close" trigger to dismiss the dialog. You will have to supply your own specific command triggers within a GIT_COMMAND dialog box.

notification (GIT_NOTIFICATION) indicates that the Interaction sends notification of some event. If the *visibility* attribute causes this Interaction to appear as a dialog box, the specific UI may create an "OK" trigger that the user can press to acknowledge the notification.

affirmation (GIT_AFFIRMATION) indicates that the Interaction asks for confirmation of an operation. If the *visibility* attribute causes this Interaction to appear as a dialog box, the specific UI may create "Yes" and "No" triggers. Depending upon the Specific UI, these triggers may dismiss the dialog box.

multipleResponse (GIT_MULTIPLE_RESPONSE) indicates that this Interaction may include multiple items that the user can respond with. (Your application should add custom response triggers using the GenTrigger's vardata ATTR_GEN_TRIGGER_INTERACTION_COMMAND.) If you wish these triggers to appear in a dialog box reply bar, use HINT_SEEK_REPLY_BAR.

```
GII_visibility GenInteractionVisibility      (GIV_SUB_GROUP)
   UIC Syntax:  visibility   = enumComp byte {
                    noPreference,
                    popup,
                    subGroup,
                    controlGroup,
                    dialog,
                    popout
                    } : subGroup;
```

Object Assembly Reference

visibility (*GII_visibility)* describes the visual implementation of the GenInteraction. Typically, this affects where and how the object and its children are arranged within the UI. A GenInteraction's visibility may also affect the behavior of the children, though this is generally a function of the *GII_type* field.

noPreference (GIV_NO_PREFERENCE) indicates that there is no visual preferences for this Interaction. The specific UI will create the Interaction based on hints, the types of children it contains, and its location in the generic tree.

popup (GIV_POPUP) indicates that this Interaction is temporary and only appears on-screen for the duration of a single selection. In most specific UIs, the popup will stay up until either an object in the Interaction is activated or until the user clicks somewhere else. Some specific UIs allow a menu to be pinned, however. The most common manifestation of a popup is a menu. Popups typically appear within appropriate "menu bars" of an application but may be independently displayable under different specific UIs.

subGroup (GIV_SUB_GROUP) indicates that the Interaction serves as a visual grouping of objects within a larger window. This visibility creates an Interaction within the parent window and is therefore not independently displayable. This visibility is the default case.

controlGroup (GIV_CONTROL_GROUP) indicates that this Interaction contains controls and therefore should not appear as a popup Interaction. This prevents the Interaction from disappearing before the user can select an entry in the control group. A control group Interaction may appear as either a sub-group or dialog, depending on the specific UI and its position within the application's generic object tree.

dialog (GIV_DIALOG)indicates that this Interaction should appear as a dialog box. A dialog box is a temporary window used to display controls or request information from the user. A dialog box can typically contain any number or variety of other generic objects. A dialog box may be either modal or non-modal.

popout (GIV_POPOUT) indicates that the GenInteraction may act as either a sub-group or as a dialog box, depending on its state. Usually, popouts act as normal sub-group Interactions until the user or

the application "pops" them out into a dialog box. Popouts are normally "popped out" into their dialog box state by double-clicking within the confines of the GenInteraction. This popout state is reflected with either the presence or absence of the ATTR_GEN_INTERACTION_POPPED_OUT vardata entry.

```
GII_attrs       GenInteractionAttrs
   UIC Syntax:  attributes    = bitFieldComp byte {
                    notUserInitiatable,
                    initiatedViaUserDoDialog,
                    modal,
                    sysModal
                    } : {};
```

attrs (GII_attrs) define attributes that affect how a GenInteraction is initiated, and whether input to other parts of the application (or system) is affected.

notUserInitiatable (GIA_NOT_USER_INITIATABLE) indicates that the user cannot bring up the dialog directly. Unless this attribute is set, most specific UIs will place a default activation trigger within a proper place in the UI to initiate the dialog box. This attribute prevents this trigger from appearing. To bring up a dialog box set *notUserInitiatable*, your application must send the dialog a MSG_GEN_INTERACTION_INITIATE. Usually, these Interactions should be direct children of either a GenPrimary or GenApplication.

initiatedViaUserDoDialog (GIA_INITIATED_VIA_USER_DO_DIALOG) indicates that the application brings up this Interaction through **UserDoDialog()**. You should use this routine in special cases where you need the user to respond to a dialog box before continuing with your application thread. (This routine blocks the calling thread until the user responds to the dialog box.) Any dialog boxes marked with this attribute should *not* be initiated with MSG_GEN_INTERACTION_INITIATE. The UI won't include a default activation trigger for dialog boxes with this attribute.

modal (GIA_MODAL) indicates that while this dialog is up, all other input to the application should be ignored. This attribute will create an application modal dialog box.

Object Assembly Reference

sysModal (GIA_SYS_MODAL) indicates that while this dialog is up, all other input to the system (including other applications) should be ignored. This attribute will create a system modal dialog box.

Hints

HINT_INTERACTION_FREQUENT_USAGE vardata

HINT_INTERACTION_FREQUENT_USAGE indicates that the GenInteraction is frequently used, and the specific UI may alter default functionality to reflect this. In most cases, this prevents dialog boxes from being automatically dismissed when the user is done interacting with them.

HINT_INTERACTION_INFREQUENT_USAGE vardata

HINT_INTERACTION_INFREQUENT_USAGE indicates that the GenInteraction is infrequently used. In some specific UIs, this prevents sub-menus from automatically cascading.

HINT_INTERACTION_MAKE_RESIZABLE vardata

HINT_INTERACTION_MAKE_RESIZABLE indicates that this GenInteraction may be resized even if it not normally allowed to do so. This hint usually only applies to dialog boxes.

HINT_INTERACTION_CANNOT_BE_DEFAULT vardata

HINT_INTERACTION_CANNOT_BE_DEFAULT indicates that this GenInteraction should not be activated as a system default. This hint prevents children of this GenInteraction from being activated as system defaults also. You should use this hint to mark groups of objects that may be potentially destructive; the user will be prevented from accidentally activating any objects in the GenInteraction.

HINT_INTERACTION_MODAL vardata

HINT_INTERACTION_MODAL indicates that this GenInteraction (a dialog box) should be application modal. This hint should be used in place of the *attributes modal* field in cases where modality is not needed for functionality, but is needed to present a cleaner UI to the user. Ideally, you should be able to go back and remove these hints by reworking the UI at some later time.

HINT_INTERACTION_NO_DISTURB vardata

HINT_INTERACTION_NO_DISTURB indicates that this GenInteraction, if initiated, should be brought on-screen without disturbing the focus of the application. Usually, this hint is placed on notification type dialog boxes (such as new-mail notifications) so that the dialog box doesn't irritate the user by drawing away the focus.

`HINT_INTERACTION_DEFAULT_ACTION_IS_NAVIGATE_TO_NEXT_FIELD vardata`

HINT_INTERACTION_DEFAULT_ACTION_IS_NAVIGATE_TO_NEXT_FIELD indicates that the default action for this Interaction group is to navigate to the next field. If something activates the GenInteraction's default action (for example by double-clicking within its confines) objects within the Interaction will not be activated; the focus will travel to the next field.

`HINT_CUSTOM_SYS_MENU vardata word`

HINT_CUSTOM_SYS_MENU indicates that this GenInteraction menu is a custom system menu. Depending on the specific UI, the standard system menu will be added as a sub-menu of the custom system menu. This hint takes an integer value that specifies the child position to place the standard system menu (0 being the first position).

`HINT_INTERACTION_MAXIMIZABLE vardata`

HINT_INTERACTION_MAXIMIZABLE indicates that this GenInteraction should be maximizable. This hint also allows the GenInteraction to become restorable to its non-maximizable state.

`HINT_INTERACTION_POPOUT_HIDDEN_ON_STARTUP vardata`

HINT_INTERACTION_POPOUT_HIDDEN_ON_STARTUP indicates that the *popout* GenInteraction should be initially off-screen.

Variable Instance Data

`ATTR_GEN_INTERACTION_GROUP_TYPE vardata GenInteractionGroupType`

ATTR_GEN_INTERACTION_GROUP_TYPE indicates that this GenInteraction (typically a menu) is a special group that the specific UI should be aware of. This optional attribute takes a **GenInteractionGroupType** as its argument.

`ATTR_GEN_INTERACTION_OVERRIDE_INPUT_RESTRICTIONS vardata`

Object Assembly Reference

ATTR_GEN_INTERACTION_OVERRIDE_INPUT_RESTRICTIONS instructs the specific UI to override any input restrictions in place on a modal dialog box. Many dialog types (*progress*, *notification*, *affirmation* and *multipleResponse*) override these input restrictions by default anyway; this attribute is for use in cases where this override is not provided.

`ATTR_GEN_INTERACTION_ABIDE_BY_INPUT_RESTRICTIONS vardata`

ATTR_GEN_INTERACTION_ABIDE_BY_INPUT_RESTRICTIONS instructs the specific UI to abide by any input restrictions in place on a modal dialog box. In general, this changes the default behavior for many dialog types (*progress*, *notification*, *affirmation* and *multipleResponse*) which normally override any input restrictions be default.

`ATTR_GEN_INTERACTION_POPPED_OUT vardata`

ATTR_GEN_INTERACTION_POPPED_OUT indicates that a *popout* GenInteraction is in its popped-out state. Absence of this optional attribute indicates that the popout is in its popped-in state. This attribute can be set initially, in which case the Popout will appear popped out. This attribute is also set internally by the UI whenever the state of the *popout* changes.

`ATTR_GEN_INTERACTION_POPOUT_NOT_CLOSABLE vardata`

ATTR_GEN_INTERACTION_POPOUT_NOT_CLOSABLE indicates that a *popout*, when popped out in its dialog box state, will not be closable; you will only be able to pop it back into its sub-group state.

Messages

MSG_GEN_INTERACTION_ACTIVATE_COMMAND

This message activates a GenTrigger having the passed command in its ATTR_GEN_TRIGGER_INTERACTION_COMMAND; if no such trigger exists, this message calls MSG_GEN_GUP_INTERACTION_COMMAND to the Interaction itself with the supplied command value. Note that the

InteractionCommand type IC_INTERACTION_COMPLETE should not be used as this is a notification rather than a command.

Source: Unrestricted.

Destination: Any non GIT_ORGANIZATIONAL GenInteraction object.

Pass: **cx** InteractionCommand to activate (except IC_INTERACTION_COMPLETE and IC_NULL).

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: May be intercepted to change or supplement default functionality of standard or custom interaction commands. Standard response triggers will use this message, so intercepting this message will allow changing the default behavior of these triggers without having to replace them with your own triggers.

Library: **Objects\gInterC.def**

■ MSG_GEN_INTERACTION_DISABLE_DISCARDING

This message informs a GenInteraction dialog with HINT_INTERACTION_DISCARD_WHEN_CLOSED that it shouldn't be discarded until it is brought off-screen. This message is useful for ensuring that dialog boxes that need initialization aren't discarded between the time that they are being initialized and the time that they physically appear on-screen.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gInterC.def**

MSG_GEN_INTERACTION_GET_ATTRS

This message retrieves the current *GII_attrs* instance field from the Interaction sent the message.

Source: Unrestricted.

Destination: Any GenInteraction object.

Pass: Nothing.

Returns: `cl` **GenInteractionAttrs** in use by the GenInteraction.

Destroyed: `ax, ch, dx, bp`

Library: **Objects\gInterC.def**

MSG_GEN_INTERACTION_GET_TYPE

This message returns the current **GenInteractionType** stored in the *GII_type* instance field for the Interaction.

Source: Unrestricted.

Destination: Any GenInteraction object.

Pass: Nothing.

Returns: `cl` **GenInteractionType** in use by the GenInteraction.

Destroyed: `ax, ch, dx, bp`

Library: **Objects\gInterC.def**

MSG_GEN_INTERACTION_GET_VISIBILITY

This message retrieves the current **GenInteractionVisibility** stored in the *GII_visibility* instance field of an Interaction.

Source: Unrestricted.

Destination: Any GenInteraction object

Pass: Nothing.

Returns: `cl` **GenInteractionVisibility**.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gInterC.def**

MSG_GEN_INTERACTION_INITIATE

This message brings an Interaction on-screen, giving it focus and target if possible. The Interaction must be both within the generic tree and set GS_USABLE before receiving this message. The Interaction must also be fully usable and enabled before it can be initiated. Interactions marked GIA_INITIATED_VIA_USER_DO_DIALOG should not be sent this message. To initiate those dialogs, you must use **UserDoDialog**.

You may intercept this message to determine when a dialog is coming on-screen. This is useful if you need to set up UI gadgetry in the dialog to show some initial status. The gadgetry should be set up before calling the superclass to avoid unnecessary visual updating of the object.

Source: Unrestricted.

Destination: Any GS_USABLE GenInteraction, though usually only meaningful for GIV_DIALOG interactions.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: May be intercepted to find out when an Interaction is appearing on-screen. This is useful for cases in which the UI gadgetry of a dialog box needs to be set up before being visually built. The superclass should be called after setting up the UI to avoid unnecessary visual changes once the interaction is on-screen.

Library: **Objects\gInterC.def**

MSG_GEN_INTERACTION_INITIATE_NO_DISTURB

This message brings an Interaction on-screen but does not give the dialog the focus or target. This message may place the dialog behind other windows. The Interaction must be both within the generic tree and set GS_USABLE before receiving this message. The Interaction must also be fully usable and enabled before it can be initiated. Interactions marked

GIA_INITIATED_VIA_USER_DO_DIALOG should not be sent this message. To initiate those dialogs, you must use **UserDoDialog**.

Source: Unrestricted.

Destination: Any GS_USABLE GenInteraction, though usually only meaningful for GIV_DIALOG Interactions. Not allowed for GIA_MODAL or GIA_INITIATED_VIA_USER_DO_DIALOG Interactions.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Interception: May be intercepted to find out when an Interaction is appearing on-screen. This is useful for cases in which the UI gadgetry of a dialog box needs to be set up before being visually built. The superclass should be called after setting up the UI to avoid unnecessary visual changes once the interaction is on-screen.

Library: Objects\gInterC.def

■ MSG_GEN_INTERACTION_POP_IN

This message 'pops in' a GIV_POPOUT GenInteraction into its default sub-group state. This message has no effect if the GenInteraction is not a GIV_POPOUT or if the GenInteraction is already 'popped in.'

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gInterC.def

■ MSG_GEN_INTERACTION_POP_OUT

This message 'pops out' a GIV_POPOUT GenInteraction into its dialog box state. This message has no effect if the GenInteraction is not a GIV_POPOUT or if the GenInteraction is already 'popped out.'

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gInterC.def

MSG_GEN_INTERACTION_SET_ATTRS

This message sets the *GII_attrs* instance data for the GenInteraction sent the message. The Interaction must not be GS_USABLE when receiving this message. The new attributes will take effect when the Interaction is next made GS_USABLE.

Source: Unrestricted.

Destination: Any non-usable GenInteraction object.

Pass: `cl` **GenInteractionAttrs** to set.
`ch` **GenInteractionAttrs** to clear.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gInterC.def**

MSG_GEN_INTERACTION_SET_TYPE

This message sets the *GII_type* instance data for the GenInteraction. This message must pass a valid **GenInteractionType** enumerated type to the Interaction object. The Interaction must not be GS_USABLE when receiving this message; the new type will take effect when the Interaction is next made GS_USABLE.

Source: Unrestricted.

Destination: Any non-usable GenInteraction object.

Pass: `cl` **GenInteractionType** for the Interaction.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gInterC.def**

MSG_GEN_INTERACTION_SET_VISIBILITY

This message sets the *GII_visibility* instance data for the GenInteraction. This message must pass a valid **GenInteractionVisibility** type to the Interaction object. The Interaction must not be GS_USABLE when receiving

this message. The new visibility will take effect when the Interaction is next made GS_USABLE.

Source: Unrestricted.

Destination: Any non-usable GenInteraction object.

Pass: `cl` **GenInteractionVisibility** for interaction.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gInterC.def**

■ MSG_GEN_INTERACTION_TEST_INPUT_RESTRICTABILITY

This message is called on modal windows by the GenApplication object to find out the status of the various input restricting mechanisms (input hold-up, input ignore, busy states, etc.) and whether they should be overridden or not.

Source: Part of the input flow/modality mechanism—sent by the GenApplication object to the modal window.

Destination: The modal GenInteraction object.

Pass: `cx:dx` The optr of the windowed object that may or may not be restricted.

Returns: `CF` Set if input restrictions should be overridden; clear otherwise. If set, this indicates that mouse and keyboard input will flow to the object regardless of hold-up, ignore, or busy states.

Destroyed: `ax, cx, dx, bp`

Interception: May be intercepted to allow the passed windowed object to override its input restrictions in certain cases.

Library: **Objects\gInterC.def**

■ MSG_GEN_INTERACTION_TOGGLE_POPOUT

This message toggles the state of a GIV_POPOUT GenInteraction from its 'popped in' state to its 'popped out' state (or 'popped out' and hidden from

view) and vice-versa. The specific UI has control over how this message is implemented.

Pass: Nothing.

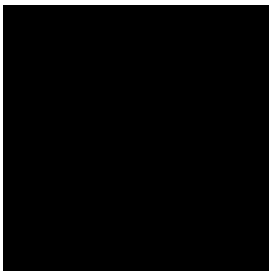
Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gInterC.def

Object Assembly Reference

GenValue



GenValueClass 199

8

GenValueClass

Instance Data

```
GVLI_value              WWFixed     0
   UIC Syntax:  valueFrac     = wordComp : 0;
   UIC Syntax:  value         = wordComp : 0;
```

value and *valueFrac (GVLI_value)* store the current numerical value of the GenValue. By default, it is an integer constant defined by the application. Depending on the *displayFormat*, it can be interpreted as an integer, a decimal, or distance unit. *value* takes the integer portion of the value; *valueFrac* takes the fractional portion.

```
GVLI_minimum            WWFixed     0
   UIC Syntax:  minimumFrac   = wordComp : 0;
   UIC Syntax:  minimum       = wordComp : 0;
```

minimum and *minimumFrac (GVLI_minimum)* store the minimum possible value that the GenValue may display. This value may be positive or negative. The default value is zero.

```
GVLI_maximum            WWFixed     <0ffffh, 7fffh>
   UIC Syntax:  maximumFrac   = wordComp : 0;
   UIC Syntax:  maximum       = wordComp : 32766;
```

maximum and *maximumFrac (GVLI_maximum)* store the maximum possible value that the GenValue may display. This value may be positive or negative. The default value is 32766.

```
GVLI_increment          WWFixed     <0,1>
   UIC Syntax:  incrementFrac = wordComp : 0;
   UIC Syntax:  increment     = wordComp : 1;
```

increment and *incrementFrac (GVLI_increment*) store the value to increment (or decrement) the GenValue when its value is changed by UI controls rather than direct text input. The default value is one.

```
GVLI_stateFlags      GenValueStateFlags 0
   UIC Syntax:  stateFlags   = bitFieldComp byte {
                    indeterminate,
                    modified,
                    } : {};
```

stateFlags (GVLI_stateFlags) specifies the **GenValueStateFlags** for the GenValue to operate under. These flags affect whether the data within the GenValue is indeterminate (not necessarily true) or modified (changed since the last MSG_GEN_APPLY).

```
GVLI_displayFormat   GenValueDisplayFormat
   UIC Syntax:  displayFormat    = enumComp byte {
                    integer,
                    decimal,
                    points,
                    inches,
                    centimeters,
                    millimeters,
                    picas,
                    europeanPoints,
                    ciceros,
                    pointsOrMillimeters,
                    inchesOrCentimeters
                    } : integer;
```

displayFormat (GVLI_displayFormat) specifies the units of measurement (**GenValueDisplayFormat**) that the numerical value represents. By default, this is an integer, but it could also be a decimal or a unit of measurement (such as inches, points, or centimeters).

```
GVLI_destination     optr
   UIC Syntax:  destination  = optrComp : null;
```

destination (GVLI_destination) stores the object or process to send the message upon GenValue changes. This can be an optr to an object or a valid **TravelOption**.

```
GVLI_applyMsg        word 0
   UIC Syntax:  applyMsg     = wordComp : 0;
```

applyMsg (GVLI_applyMsg) stores the message to send out upon GenValue changes (i.e. whenever it receives MSG_GEN_APPLY). There is no default message.

Variable Instance Data

`ATTR_GEN_VALUE_STATUS_MSG` `vardata word`

ATTR_GEN_VALUE_STATUS_MSG sets a status message for a GenValue. A status message allows your object to communicate with other objects when changes occur without sending out its apply message (*applyMsg*).

`ATTR_GEN_VALUE_DECIMAL_PLACES` `vardata word`

ATTR_GEN_VALUE_DECIMAL_PLACES specifies the number of decimal places to display to the right of the decimal point if the *displayFormat* allows fractional quantities.

`ATTR_GEN_VALUE_METRIC_INCREMENT` `vardata WWFixed`

ATTR_GEN_VALUE_METRIC_INCREMENT allows you to specify a particular metric increment to use besides the default if the *displayFormat* is GVDF_POINTS_OR_MILLIMETERS or GVDF_INCHES_OR_CENTIMETERS.

`ATTR_GEN_VALUE_SET_MODIFIED_ON_REDUNDANT_SELECTION vardata`

ATTR_GEN_VALUE_SET_MODIFIED_ON_REDUNDANT_SELECTION specifies that the GenValue should be marked modified whether or not a change in the value has occurred. This will result in that value being applied whenever it receives a MSG_GEN_APPLY. (The default behavior for when no change in state occurs, no message will be sent out.)

Hints

`HINT_VALUE_INCREMENTABLE` `vardata`
`HINT_VALUE_NOT_INCREMENTABLE` `vardata`

HINT_VALUE_INCREMENTABLE and HINT_VALUE_NOT_INCREMENTABLE specify whether increment (and decrement) gadgets are appropriate for this GenValue. By default, GenValues are incrementable.

`HINT_VALUE_CUSTOM_RETURN_PRESS` `vardata`

HINT_VALUE_CUSTOM_RETURN_PRESS stores a Message to send out if the user hits return within the textual portion of a GenValue. Normally, this action triggers the interaction default (sending the apply message).

HINT_VALUE_DISPLAYS_RANGE vardata WWFixed

HINT_VALUE_DISPLAYS_RANGE indicates that this GenValue displays a range of values between its minimum and maximum. In most cases, this suggests that the GenValue use sliders or some other UI gadget that is able to show the width of a range. It is up to the specific UI to support range values. If this hint is present, *value* refers to the starting point of the range of values, whose width is stored in the **WWFixed** value here. The maximum *value* in this case would be *maximum* minus the range. If this hint is not present, the range "length" is presumed to be zero, even in gadgets that specify a range length by default.

HINT_VALUE_DISPLAY_INTERVALS vardata GenValueIntervals

HINT_VALUE_DISPLAY_INTERVALS indicates that intervals should be displayed along an object's range. This hint is used most often in analog (e.g. slider) type displays. If possible, hash marks will appear along the GenValue's display at the intervals specified by **GenValueIntervals**. This structure stores entries for both major intervals and minor intervals. In general, minor intervals will have smaller tick marks than major intervals.

```
GenValueIntervals struc
      GVI_numMajorIntervals word
      GVI_numMinorIntervals word
GenValueIntervals ends
```

If either *GVI_numMajorIntervals* or *GVI_numMinorIntervals* is zero, only one set of marks will appear.

HINT_VALUE_CONSTRAIN_TO_INTERVALS vardata

HINT_VALUE_CONSTRAIN_TO_INTERVALS suggests that the value within *GVLI_value* constrain itself to the *GVI_numMinorIntervals* interval within HINT_VALUE_DISPLAY_INTERVALS.

HINT_VALUE_SHOW_MIN_AND_MAX vardata

HINT_VALUE_SHOW_MIN_AND_MAX instructs the GenValue to display its minimum and maximum values, if possible.

HINT_VALUE_IMMEDIATE_DRAG_NOTIFICATION vardata

HINT_VALUE_IMMEDIATE_DRAG_NOTIFICATION instructs the GenValue to send out its status and/or apply messages constantly during a drag operation

(each time the value changes). It is up to the specific UI to support this behavior.

`HINT_VALUE_DELAYED_DRAG_NOTIFICATION` `vardata`

HINT_VALUE_DELAYED_DRAG_NOTIFICATION instructs the GenValue to delay sending out status and/or apply message until the user releases the mouse after the drag operation. It is up to the specific UI to support this behavior.

`HINT_VALUE_ORIENT_HORIZONTALLY` `vardata`
`HINT_VALUE_ORIENT_VERTICALLY` `vardata`

HINT_VALUE_ORIENT_HORIZONTALLY instructs the specific UI to arrange an analog GenValue, if available, in the horizontal dimension. HINT_VALUE_ORIENT_VERTICALLY instructs the specific UI to arrange an analog GenValue, if available, in the vertical dimension.

`HINT_VALUE_ANALOG_DISPLAY` `vardata`
`HINT_VALUE_DIGITAL_DISPLAY` `vardata`
`HINT_VALUE_NO_DIGITAL_DISPLAY` `vardata`
`HINT_VALUE_NO_ANALOG_DISPLAY` `vardata`

HINT_VALUE_ANALOG_DISPLAY indicates that the GenValue should be displayed in an analog fashion, if applicable. HINT_VALUE_DIGITAL_DISPLAY indicates that the GenValue should be displayed in a digital fashion (i.e. numerically). Similarly, HINT_VALUE_NO_DIGITAL_DISPLAY and HINT_VALUE_NO_ANALOG_DISPLAY indicate that a digital or analog display is not appropriate.

`HINT_VALUE_NOT_DIGITALLY_EDITABLE` `vardata`

HINT_VALUE_NOT_DIGITALLY_EDITABLE instructs the UI to disallow editing of the text within a numerical GenValue. This hint is only applicable if some other UI means of changing the value is available. To remove a numeric display of values entirely, use HINT_VALUE_NO_DIGITAL_DISPLAY.

`HINT_VALUE_DO_NOT_MAKE_LARGER_ON_PEN_SYSTEMS` `vardata`

HINT_VALUE_DO_NOT_MAKE_LARGER_ON_PEN_SYSTEMS instructs the specific UI to avoid expanding a GenValue object to accept pen input, if possible. By default, text areas of GenValues grow larger under pen systems to allow ink strokes. This hint usually indicates that non-ink means of entering values is available.

Messages

MSG_GEN_VALUE_DECREMENT

This message decreases the value of *GVLI_value* within the GenValue by the increment in *GVLI_increment*. This message will clear the indeterminate flag of the GenValue but will not affect its modified flag.

Source: Unrestricted. This message is also used internally when responding to user actions.

Destination: Any GenValue object.

Pass: Nothing.

Returns: Nothing. *GVLI_value* will be decremented (or set to the minimum if decreasing the value would push it below the minimum).

Destroyed: ax, cx, dx, bp

Library: Objects\gValueC.def

MSG_GEN_VALUE_GET_APPLY_MSG

This message returns the GenValue's *GVLI_applyMsg*.

Source: Unrestricted.

Destination: Any GenValue object.

Pass: Nothing.

Returns: ax — The current apply message of the GenValue.

Destroyed: cx, dx, bp

Library: Objects\gValueC.def

MSG_GEN_VALUE_GET_DESTINATION

This message returns the current destination object (or process) that the GenValue sends its apply messages to.

Source: Unrestricted.

Destination: Any GenValue object.

Pass: Nothing.

Returns: cx:dx — The destination optr (*GVLI_destination*) of the GenValue.

Destroyed: ax, bp

Library: Objects\gValueC.def

MSG_GEN_VALUE_GET_DISPLAY_FORMAT

This message returns the *GVLI_displayFormat* field of the GenValue.

Source: Unrestricted.

Destination: Any GenValue object.

Pass: Nothing.

Returns: **al** **GenValueDisplayFormat** of the GenValue.

Destroyed: **an, cx, dx, bp**

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_GET_INCREMENT

This message returns the value of the *GVLI_increment* field of the GenValue.

Source: Unrestricted.

Destination: Any GenValue object.

Pass: Nothing.

Returns: **dx.cx** The fixed point value of *GVLI_increment*.

Destroyed: **ax, bp**

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_GET_MAXIMUM

This message returns the fixed point value within the *GVLI_maximum* instance field of the GenValue.

Source: Unrestricted.

Destination: Any GenValue object.

Pass: Nothing.

Returns: **dx.cx** The fixed point maximum value in *GVLI_maximum*.

Destroyed: **ax, bp**

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_GET_MAX_TEXT_LEN

This message retrieves the maximum number of characters allowed to be typed into a textual GenValue. By default, this number is calculated from the minimums and maximums specified. Subclasses of GenValue may wish to

intercept this message to allow different text lengths (such as values represented with floating point or enumerated type instance data).

Source: The GenValue object sends this to itself when it needs to figure out its maximum text length.

Destination: Itself.

Pass: Nothing.

Returns: **ax** Maximum number of allowable characters.

Destroyed: **cx, dx, bp**

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_GET_MINIMUM

This message returns the fixed point value within the *GVLI_minimum* instance field of the GenValue.

Source: Unrestricted.

Destination: Any GenValue object.

Pass: Nothing.

Returns: **dx.cx** The fixed point minimum value in *GVLI_minimum*.

Destroyed: **ax, bp**

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_GET_TEXT_FILTER

This message retrieves the text filtering in use on a GenValue object. By default, GenValues use numeric-only filtering with no spaces and no tabs for numbers and a filtering of no tabs for distance units.

Source: Unrestricted. This message is normally sent by a GenValue to itself when building its textual component.

Destination: Any GenValue object.

Pass: Nothing.

Returns: **al** **VisTextFilters** in use by the textual portion of the GenValue.

Destroyed: **ah, cx, dx, bp**

Interception: Usually, you will want to intercept this message if you subclass **GenValueClass** and provide your own custom filtering.

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_GET_VALUE

This message returns the value stored in the GenValue's *GVLI_value* instance field. This returned value will be a fixed point number.

Source: Unrestricted.

Destination: Any GenValue object.

Pass: Nothing.

Returns: **dx.cx** The fixed point numerical value of the *GVLI_value* instance field.

Destroyed: **ax, bp**

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_GET_VALUE_RATIO

This message gets the value of a GenValue (*GVLI_value*) as a ratio of its distance between the minimum value and the maximum value. It returns this ratio as a dword (0000.0000h meaning 0%, ffff.ffffh meaning 100%).

Source: Unrestricted.

Destination: Any GenValue object.

Pass: **bp** **GenValueType** of the instance data field to get the value ratio for.

Returns: **dx.cx** The ratio as a **WWFixed** value.

Interception: You may intercept subclasses of **GenValueClass** to allow custom text formats to be displayed on the screen. Someone subclassing this message for this reason would also want to subclass MSG_GEN_VALUE_SET_VALUE_FROM_TEXT to correctly parse text to yield a new value.

Destroyed: **ax, bp**

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_GET_VALUE_TEXT

This message retrieves a fixed point value (either *GVLI_value*, *GVLI_minimum*, *GVLI_maximum*, or *GVLI_increment*) from the GenValue and stores its textual representation (in a null-terminated text string) in the

passed buffer. This message is not affected by the indeterminate state of the GenValue.

Source: Unrestricted.

Destination: Any GenValue object.

Pass:
- `cx:dx` — The pointer to the buffer to store the null-terminated text string. This buffer must be at least GEN_VALUE_MAX_TEXT_LEN bytes long.
- `bp` — The **GenValueType**, specifying the instance data to retrieve and convert into text.

Returns:
- `cx:dx` — The *buffer* filled in with text.

Interception: Can be intercepted by a subclass of GenValue to allow custom text formats to be displayed on the screen. In this case you would also subclass MSG_GEN_VALUE_SET_VALUE_FROM_TEXT.

Destroyed: `ax, bp`

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_INCREMENT

This message increases the value of *GVLI_value* within the GenValue by the increment in *GVLI_increment*. This message will clear the indeterminate flag of the GenValue but will not affect its modified flag.

Source: Unrestricted. This message is also used internally when responding to user actions.

Destination: Any GenValue object.

Pass: Nothing.

Returns:
- `CF` — Set if the GenValue's instance data changed. *GVLI_value* will be incremented (or set to the maximum if increasing the value would push it over the maximum).

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_IS_INDETERMINATE

This message checks whether a GenValue is indeterminate.

Source: Unrestricted.

Destination: Any GenValue object.

Pass: Nothing.

Returns: CF Set if GenValue is indeterminate; clear if it is not.

Destroyed: ax, cx, dx, bp

Library: Objects\gValueC.def

MSG_GEN_VALUE_IS_MODIFIED

This message checks whether a GenValue has been modified since the last MSG_GEN_APPLY.

Source: Unrestricted.

Destination: Any GenValue object.

Pass: Nothing.

Returns: CF Set if GenValue is modified; clear if it is not.

Destroyed: ax, cx, dx, bp

Library: Objects\gValueC.def

MSG_GEN_VALUE_SELECT_TEXT

This message selects a GenValue's text, if allowed by the specific UI.

Source: Unrestricted.

Destination: Any GenValue object.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gValueC.def

MSG_GEN_VALUE_SEND_STATUS_MSG

This message sends the status message stored in the object's ATTR_GEN_VALUE_STATUS_MSG instance field. You should pass this

message the modified State you wish to send. This modified state may or not reflect the GVSF_MODIFIED flag in the GenValue's *GVLI_stateFlags*.

Source: Unrestricted.

Destination: Any GenValue object.

Pass: **cx** Non-zero if this message should pass the modified bit (GVSF_MODIFIED) set; zero if it should pass GVSF_MODIFIED cleared.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_SET_APPLY_MSG

This message sets the apply message (in *GVLI_applyMsg*) for a GenValue.

Source: Unrestricted.

Destination: Any GenValue object.

Pass: **cx** The apply message to set for the GenValue.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_SET_DESTINATION

This message sets the *GVLI_destination* field of the range to the passed optr. The object can be a pointer to a specific object in the system (i.e. the GenProcess object) or can be a pointer to a generic location in the system (i.e. a **TravelOption**).

Source: Unrestricted.

Destination: Any GenValue object.

Pass: **cx:dx** The optr of the new destination object.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_SET_DISPLAY_FORMAT

This message sets the *GVLI_displayFormat* of the GenValue to the given format. The current value in *GVLI_value* is unaffected, but the display will be updated to reflect the new display format.

Source: Unrestricted. This message is also used internally when the GenValue is being built.

Destination: Any GenValue object.

Pass: `cl` **GenValueDisplayFormat** to set the GenValue to.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_SET_INCREMENT

This message sets the *GVLI_increment* field to the passed *value*.

Source: Unrestricted. This message is also used internally when the GenValue is being built.

Destination: Any GenValue object.

Pass: `dx.cx` signed <integer>.<fraction> fixed point value to set *GVLI_increment* to.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_SET_INDETERMINATE_STATE

This message sets the indeterminate state for a GenValue. Pass a non-zero value in **cx** to mark the GenValue indeterminate, zero to mark it not

indeterminate. The GenValue will not be marked modified after this message.

Source: Unrestricted.

Destination: Any GenValue object.

Pass: **cx** Non-zero to set the GVSF_INDETERMINATE flag in the GenValue's *GVLI_stateFlags*, zero to clear the GVSF_INDETERMINATE flag.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_SET_INTEGER_VALUE

This message sets the *GVLI_value* to the passed integer (word-sized) value. *GVLI_value* will then contain this value in its high (integral) word and zero in its low (fractional) word. The modified state of the GenValue will be cleared.

Source: Unrestricted. This message is also used internally when responding to user actions.

Destination: Any GenValue object.

Pass: **cx** The signed integer value to set *GVLI_value* to.
bp Non-zero to mark the GenValue indeterminate, zero to mark it non indeterminate.

Returns: CF Set if *GVLI_value* was changed.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_SET_MAXIMUM

This message sets the *GVLI_maximum* field of the GenValue to the passed fixed point value. If you set a new maximum that places the current value (in

GVLI_value) above that maximum, *GVLI_value* will be adjusted to this maximum.

Source: Unrestricted. This message is also used internally when the GenValue is being built.

Destination: Any GenValue object.

Pass: **dx.cx** Signed <integer>.<fraction> fixed point value to set *GVLI_maximum* to.

Returns: Nothing.

Destroyed: **ax, bp**

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_SET_MINIMUM

This message sets the *GVLI_minimum* instance field of the GenValue to the passed fixed point value. If you set a new minimum that places the current value (in *GVLI_value*) below that minimum, *GVLI_value* will be adjusted to this minimum.

Source: Unrestricted. This message is also used internally when the GenValue is being built.

Destination: Any GenValue object.

Pass: **dx.cx** Signed <integer>.<fraction> fixed point value to set *GVLI_minimum* to.

Returns: Nothing.

Destroyed: **ax, bp**

Library: **Objects\gValueC.def**

Object Assembly Reference

MSG_GEN_VALUE_SET_MODIFIED_STATE

This message sets the modified state for a GenValue. Pass TRUE to mark the GenValue modified, FALSE to mark it not modified.

Source: Unrestricted.

Destination: Any GenValue object.

Pass: **cx** Non-zero to set the GVSF_MODIFIED flag in the GenValue's *GVLI_stateFlags*, zero to clear the GVSF_MODIFIED flag.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_SET_OUT_OF_DATE

This message sets a GenValue's GVSF_OUT_OF_DATE flag.

Source: Unrestricted.

Destination: Usually itself.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_SET_VALUE

This message sets the *GVLI_value* field of the GenValue to the passed *value*. This message clears a GenValue's modified state in its *GVLI_stateFlags*. To

mark the GenValue modified send MSG_GEN_VALUE_SET_MODIFIED_STATE after sending this message.

Source: Unrestricted. This message is also used internally when responding to user actions.

Destination: Any GenValue object.

Pass: **dx.cx** The signed <integer>.<fraction> fixed point value to set *GVLI_value* to. If you only need an integral value, consider using MSG_GEN_VALUE_SET_INTEGER_VALUE instead.

bp TRUE to mark the GenValue indeterminate, FALSE to mark it non indeterminate.

Returns: CF Set if *GVLI_value* was modified.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_SET_VALUE_FROM_RATIO

This message sets the value of a GenValue (*GVLI_value*) as a ratio of its distance between the minimum value and the maximum value. It returns this ratio as a dword (0000.0000h meaning 0%, ffff.ffffh meaning 100%).

Source: Unrestricted.

Destination: Any GenValue object.

Pass: **dx.cx** WWFixed ratio (

Pass: **bp** **GenValueType** of the instance data field to get the value ratio for.

Returns: CF Set if GenValue instance data was changed.

Destroyed: **ax, cx, dx, bp**

Interception: You may intercept subclasses of **GenValueClass** to allow custom text formats to be displayed on the screen. Someone subclassing this message for this reason would also want to subclass MSG_GEN_VALUE_SET_VALUE_FROM_TEXT to correctly parse text to yield a new value.

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_SET_VALUE_FROM_TEXT

This message sets a fixed point value (either *GVLI_value*, *GVLI_minimum*, *GVLI_maximum*, or *GVLI_increment*) of the GenValue from a textual

representation (in a null-terminated text string) in the passed buffer. This message clears the indeterminate state of the GenValue but does not change its modified state.

Source: Unrestricted.

Destination: Any GenValue object.

Pass: `cx:dx` The pointer to a null-terminated text string to set the GenValue's text to.
`bp` The **GenValueType**, specifying the instance data to convert into a fixed point value from the passed text.

Returns: `CF` Set if GenValue instance data changed.

Destroyed: `ax, cx, dx, bp`

Interception: Can be intercepted by a subclass of GenValue to allow custom text formats to be displayed on the screen. In this case, you would also subclass MSG_GEN_VALUE_GET_VALUE_TEXT.

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_SET_VALUE_TO_MAXIMUM

This message sets the value of *GVLI_value* to the maximum value in *GVLI_maximum*. This message will clear the indeterminate flag of the GenValue but will not affect its modified flag.

Source: Unrestricted.

Destination: Any GenValue object.

Pass: Nothing.

Returns: `CF` Set if GenValue instance data changed. *GVLI_value* will be set to *GVLI_maximum*.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gValueC.def**

MSG_GEN_VALUE_SET_VALUE_TO_MINIMUM

This message sets the value of *GVLI_value* to the minimum value in *GVLI_minimum*. This message will clear the indeterminate flag of the GenValue but will not affect its modified flag.

Source: Unrestricted.

Destination: Any GenValue object.

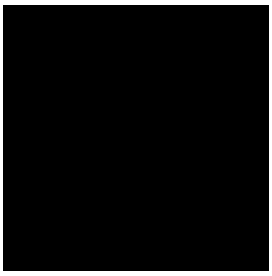
Pass: Nothing.

Returns: CF — Set if GenValue instance data changed. *GVLI_value* will be set to *GVLI_minimum*.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gValueC.def**

GenView



GenViewClass 221

GenViewControlClass 241

9

GenViewClass

Instance Data

```
GVI_origin                    PointDWFixed
          UIC Syntax:   originX = typeComp “DWFixed” : “0,0”
                        originY = typeComp “DWFixed” : “0,0”;
```

originX and *originY* (*GVI_origin*): The GenView must know where in the coordinate space it is located. To do this, it maintains an origin—the upper left coordinates of the window's scrollers. That is, the origin is the upper-left position in the document. The origin is stored in the field *GVI_origin*, a **PointDWFixed** structure containing two **DWFixed** structures.

```
GVI_docBounds        RectDWord        <0, 0, 0, 0>
          UIC Syntax:   docLeft =     dwordComp: 0;
                        docTop =      dwordComp: 0;
                        docRight =    dwordComp : 0;
                        docBottom =   dwordComp : 0;
```

docLeft, *docTop*, *docRight*, and *docBottom* (*GVI_docBounds*): Because most documents will not actually require the entire document space, the view has an instance data field that defines the view's scrollable boundaries, *GVI_docBounds*. This field is a **RectDWord** structure containing four signed dwords, each of which represents a specific boundary of the document.

```
GVI_increment        PointDWord <20, 15>
          UIC Syntax:   horizIncrement =    dwordComp : 20;
                        vertIncrement =     dwordComp : 15;
```

horizIncrement and *vertIncrement* (*GVI_increment*): Every view has an increment associated with it; this increment may be used in scrolling and sizing the view window. The increment has both a horizontal and a vertical component which are separate from each other. The increment must be specified in document coordinates (points) and is represented by a signed long number. By default, the horizontal increment is set to 20 and the vertical to 15. The increment values are stored in the *GVI_increment* field. This field is a structure of type **PointDWord**.

```
GVI_scaleFactor      PointWWFixed <<0, 1>, <0, 1>>
          UIC Syntax:   scaleX = typeComp “WWFixed” : “0, 1”;
                        scaleY = typeComp “WWFixed” : “0, 1”;
```

scaleX and *scaleY* (*GVI_scaleFactor*): Many applications implement scaling to allow users to view and modify their documents at various scale factors. This scaling behavior, apart from the implementation of the view menu, is almost entirely internal to the view. The application simply needs to instruct the view in what scale factor to use; the view will provide the proper geometry and math involved in displaying, scrolling, and resizing your document. Every view has a scale factor stored as a **PointWWFixed** structure in the *GVI_scaleFactor* instance field. The default scale factor is 100 percent.

```
GVI_color           ColorQuad <C_WHITE, 0, 0, 0>
         UIC Syntax:  backColorR =     byteComp : C_WHITE;
                      backColorFlags = byteComp : 0;
                      backColorG =     byteComp : 0;
                      backColorB =     byteComp : 0;
```

backColorR, *backColorFlags*, *backColorG*, and *backColorB* (*GVI_color*): If a view is managing generic objects or is displaying text, it may be most appropriate for the view to appear with the same background color as its parent Primary window. To do this, set the GVA_SAME_COLOR_AS_PARENT_WIN flag in the *GVI_attrs* field. Otherwise, you will probably want to set the view to something other than the parent's color. To this end, you can set a color value in the *GVI_color* field.

```
GVI_attrs           GenViewAttrs (mask GVA_FOCUSABLE)
         UIC Syntax:  viewAttributes = bitFieldComp word {
                      controlled,
                      genericContents,
                      trackScrolling,
                      dragScrolling,
                      noWinFrame,
                      sameColorAsParentWin,
                      viewSizeFollowsContentSize,
                      windowCoordinateMouseEvents,
                      dontSendPtrEvents,
                      dontSendKbdReleases,
                      sendAllKbdChars,
                      focusable,
                      scaleToFit,
                      adjustForAspectRatio
                      } : focusable;
```

viewAttributes (*GVI_attrs*) controls many aspects of the view, including several features related to scrolling, input events, and the content object. It is implemented as a bitfield record, and any or all of its attributes may be set at once.

Object Assembly Reference

```
GVI_horizAttrs                  GenViewDimensionAttrs   0
        UIC Syntax:  horizAttributes = bitFieldComp byte {
                     scrollable,
                     splittable,
                     tailOriented,
                     dontDisplayScrollbar,
                     noLargerThanContent,
                     noSmallerThanContent,
                     sizeAMultipleOfIncrement,
                     keepAspectRatio } : {};

GVI_vertAttrs                   GenViewDimensionAttrs   0
        UIC Syntax:  vertAttributes = bitFieldComp byte {
                     scrollable,
                     splittable,
                     tailOriented,
                     dontDisplayScrollbar,
                     noLargerThanContent,
                     noSmallerThanContent,
                     sizeAMultipleOfIncrement,
                     keepAspectRatio } : {};
```

vertAttributes and *horizAttributes* (*GVI_horizAttrs* and *GVI_vertAttrs*) are called the "dimension attributes." They determine the characteristics of the view in each dimension-whether it is scrollable or splittable, whether a scrollbar should be displayed, and whether certain rules should be applied to sizing and scaling.

```
GVI_inkType          GenViewInkType
        UIC Syntax:  inkType = byteComp : 0;
```

inkType (*GVI_inkType*): The GenView accepts Ink input events and can pass them to the content either as ink or as mouse events. How the view determines which type of events are desired is mandated by the value of the instance field *GVI_inkType*.

```
GVI_content                     optr
        UIC Syntax:  output = optrComp : null;
```

output (*GVI_content*) determines what object is designated as its content. By default, an application's Process object acts as the content. Because the output field defaults to the Process object, you do not have to explicitly set it unless you plan on using a GenContent or VisContent.

```
GVI_horizLink                   optr
        UIC Syntax:  horizLink = optrComp : null;

GVI_vertLink                    optr
        UIC Syntax:  vertLink = optrComp : null;
```

horizLink and *vertLink* (*GVI_horizLink* and *GVI_vertLink*): Views may be linked together either in the horizontal or the vertical dimension. Classed events may then be sent to the view's links, causing the events to propagate through all the links. Most scrolling events will be propagated through the links.

Each view may have one horizontal and one vertical link, though no links are required. The links are stored as optrs in the instance fields *GVI_horizLink* and *GVI_vertLink*, and links should be circular. In other words, if three views are linked in a single dimension, the links should be as follows: view one points to view two, view two points to view three, and view three points back to view one.

`ATTR_GEN_VIEW_INK_DESTINATION_INFO vardata InkDestinationInfoParams`

Applications that handle Ink input can also set extended information about the Ink. The type of extended information you can set includes the color of the Ink as it's drawn, the thickness of the Ink brush, etc.You can pre-set the extended Ink information with the vardata attribute ATTR_GEN_VIEW_INK_DESTINATION_INFO.

`ATTR_GEN_VIEW_PAGE_SIZE vardata XYSize`

This attribute takes an argument of type **XYSize**. The view will scale itself to fit this page size rather than its entire document size.

`ATTR_GEN_VIEW_SCALE_TO_FIT_BASED_ON_X vardata`

This attribute causes the view to scale itself based on the horizontal size rather than the vertical size. The vertical scale factor will follow the horizontal scale factor.

`ATTR_GEN_VIEW_SCALE_TO_FIT_BOTH_DIMENSIONS vardata`

This attribute causes the view to scale both dimensions independently when scaling to fit. The horizontal and vertical scale factors may end up different.

`ATTR_GEN_VIEW_DO_NOT_WIN_SCROLL vardata`

Typically, the view will also automatically scroll its window to the new origin when the origin changes. You can override this behavior, forcing the view not to scroll on origin changes, by setting the vardata attribute ATTR_GEN_VIEW_DO_NOT_WIN_SCROLL. This attribute, in effect, disconnects the scrollers from the visible position of the document.

`HINT_VIEW_IMMEDIATE_DRAG_UPDATES vardata`

HINT_VIEW_IMMEDIATE_DRAG_UPDATES causes the view to be updated constantly during drags.

Object Assembly Reference

HINT_VIEW_DELAYED_DRAG_UPDATES vardata

HINT_VIEW_DELAYED_DRAG_UPDATES causes the view to be updated only at the end of drags.

HINT_VIEW_REMOVE_SCROLLERS_WHEN_NOT_SCROLLABLE vardata

HINT_VIEW_REMOVE_SCROLLERS_WHEN_NOT_SCROLLABLE suggests that the view not display scrollbars when the document bounds are smaller than the view size.

HINT_VIEW_SHOW_SCROLLERS_WHEN_NOT_SCROLLABLE vardata

HINT_VIEW_SHOW_SCROLLERS_WHEN_NOT_SCROLLABLE suggests that the view display its scrollbars even when the document bounds are smaller than the view size.

ATTR_GEN_VIEW_DOES_NOT_ACCEPT_TEXT_INPUT vardata

On many pen systems, focusable GenViews will bring up an on-screen keyboard on the assumption that the user may wish to enter text to objects within the view. ATTR_GEN_VIEW_DOES_NOT_ACCEPT_TEXT_INPUT signals that the view does not accept text input and therefore should not cause a floating keyboard to come up. Non-textual keyboard events (e.g. up-arrow used for scrolling) will still be transmitted to the GenView.

HINT_VIEW_SHARES_SPACE_WITH_VIEW_ABOVE vardata optr

HINT_VIEW_SHARES_SPACE_WITH_VIEW_BELOW vardata optr

HINT_VIEW_SHARES_SPACE_WITH_VIEW_TO_LEFT vardata optr

HINT_VIEW_SHARES_SPACE_WITH_VIEW_TO_RIGHT vardata optr

These hints suggest that the view can share real estate with a neighboring view.

Messages

MSG_GEN_VIEW_ALLOW_GLOBAL_TRANSFER

This message indicates to the view that it should relinquish the mouse grab to allow a quick-transfer operation to continue. Pointer events will then be

sent to other objects in the system, allowing them to accept or reject the transfer.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\gViewC.def

MSG_GEN_VIEW_CALC_WIN_SIZE

This message is sent by the view to itself when it is first being initialized. It takes a suggested size and returns a new suggested size, adjusted for specific conditions.

Pass:
- **cx** — Suggested width of the window.
- **dx** — Suggested height of the window.

Returns:
- **cx** — New suggested width of the window.
- **dx** — New suggested height of the window.

Destroyed: ax, bp

Library: Objects\gViewC.def

MSG_GEN_VIEW_GET_ATTRS

This message returns a word representing the current *GVI_attrs* record. To check whether an attribute is set, mask the returned word value against the attribute.

Pass: Nothing.

Returns: **cx** — **VisAttrs** field.

Destroyed: ax, dx, bp.

Library: Objects\gViewC.def

Object Assembly Reference

MSG_GEN_VIEW_GET_COLOR

This message returns all four fields of the view's background color in a single dword value.

Pass: Nothing.

Returns:

cl	Red color value.
ch	**ColorFlag**.
dl	Green color value.
dh	Blue color value.

Destroyed: **ax, bp.**

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_GET_CONTENT

This message returns the optr of the object currently set as the view's content.

Pass: Nothing.

Returns: ^l**cx**:**dx** Content

Destroyed: **ax, bp.**

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_GET_DIMENSION_ATTRS

This message returns the GenView's current dimension attribute records.

Pass: Nothing.

Returns:

cl	**GenViewDimensionAttrs** withnew horizontal attributes.
ch	**GenViewDimensionAttrs** with new vertical attributes.

Destroyed: **ax, dx, bp.**

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_GET_DOC_BOUNDS

This message returns the current boundaries of the view's scrollable document space.

Pass: **cx**:**dx** Buffer of size **RectDWord** in which result is to be returned.

Returns: **cx**:**dx** **RectDWord** structure containing current scrollable bounds.

Destroyed: **ax, bp.**

Library: **Objects\gViewC.def**



MSG_GEN_VIEW_GET_INCREMENT

This message returns the view's current horizontal and vertical increments in document coordinates.

Pass: **cx**:**dx** Buffer of size **PointDWord** in which to store result.

Returns: **cx**:**dx** Buffer filled with current increment amount.

Destroyed: **ax, bp.**

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_GET_ORIGIN

This message returns a **PointDWord** structure containing the current *x* and *y* coordinates of the view's origin.

Pass: **cx**:**dx** Buffer of size **PointDWord** in which to return origin.

Returns: **cx**:**dx** **PointDWord** buffer filled with origin point coordinates.

Destroyed: **ax, bp.**

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_GET_SCALE_FACTOR

This message returns the current scale factors of the view.

Pass: Nothing.

Returns: **dx**.**cx** *x* scale factor as a **WWFixed** number.
bp.**ax** *y* scale factor as a **WWFixed** number.

Destroyed: Nothing.

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_GET_VISIBLE_RECT

This message returns the rectangle currently visible in the view, in document coordinates.

Pass: **cx**:**dx** Buffer of size **RectDWord**.

Returns: **cx**:**dx** **RectDWord** holding bounds of visible area (or null if object not yet built).

Destroyed: **ax**, **bp**.

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_GET_WINDOW

This message returns the window handle of the view window. As an alternative, you can store the view's window handle when MSG_META_CONTENT_VIEW_WIN_OPENED is sent upon view startup and discard it when you receive MSG_META_CONTENT_VIEW_CLOSING.

Pass: Nothing.

Returns: **cx** Window handle, or null if none.

Destroyed: **ax, dx, bp.**

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_INITIATE_DRAG_SCROLL

This message instructs the view to begin drag scrolling until the pressed mouse button is let go by the user. When drag scrolling is enabled, the select button initiates it; this message allows initiation by any mouse button.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_MAKE_RECT_VISIBLE

This message causes the view to scroll until a given portion of a passed rectangle is visible in the display window. If the rectangle is already partially visible, the view will not scroll unless instructed to do so by the flags arguments. If the view is not visibly initialized, this message will have no effect.

Pass: **ss:bp** **MakeRectVisibleParams** structure.
dx sizeof **MakeRectVisibleParams** structure.

Returns: Nothing.

Destroyed: **ax, bx, dx, bp.**

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_REDRAW_CONTENT

This message causes the GenView to redraw its entire content. This is essentially the same as exposing the entire view window.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\gViewC.def

MSG_GEN_VIEW_RESET_EXTENDED_INK_TYPE

This message resets the extended Ink type to default values.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\gViewC.def

MSG_GEN_VIEW_SCALE_LOW

This low-level message is similar to MSG_GEN_VIEW_SET_SCALE_FACTOR except that it does not propagate to linked views. It is generally encapsulated by the GenView when another scaling message is received, then dispatched to linked views later.

Pass: ss:bp — **ScaleViewParams** structure.
dx — sizeof **ScaleViewParams**.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\gViewC.def

MSG_GEN_VIEW_SCROLL

This message causes the view to scroll a given amount in both the *x* and *y* directions.

Pass: `ss:bp` **PointDWord** structure containing distance by which to scroll.
`dx` sizeof **PointDWord**.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_SCROLL_BOTTOM

Causes the view to scroll to the bottom of the document.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_SCROLL_DOWN

Causes the view to scroll down one increment.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_SCROLL_LEFT

Causes the view to scroll left one increment.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_SCROLL_LEFT_EDGE

Causes the view to scroll to the document's left edge.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\gViewC.def

MSG_GEN_VIEW_SCROLL_PAGE_DOWN

Causes the view to scroll down one window height.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\gViewC.def

MSG_GEN_VIEW_SCROLL_PAGE_LEFT

Causes the view to scroll left one window width.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\gViewC.def

MSG_GEN_VIEW_SCROLL_PAGE_RIGHT

Causes the view to scroll right one window width.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\gViewC.def

MSG_GEN_VIEW_SCROLL_PAGE_UP

Causes the view to scroll up one window height.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\gViewC.def

MSG_GEN_VIEW_SCROLL_RIGHT

Causes the view to scroll right one increment.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\gViewC.def

MSG_GEN_VIEW_SCROLL_RIGHT_EDGE

Causes the view to scroll to the document's right edge.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\gViewC.def

MSG_GEN_VIEW_SCROLL_SET_X_ORIGIN

Causes the view to scroll to a given horizontal point and sets the *x* component of the origin to the passed value. It keeps the *y* component of the origin fixed.

Pass: dx:cx New *x* origin as a signed **DWord** value.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\gViewC.def

MSG_GEN_VIEW_SCROLL_SET_Y_ORIGIN

Causes the view to scroll to a given vertical point and sets the *y* component of the origin to the passed value. It keeps the *x* component of the origin fixed.

Pass: **dx**:**cx** New *y* origin as a signed **DWord** value.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_SCROLL_TOP

Causes the view to scroll to the top of the document.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_SCROLL_UP

Causes the view to scroll up one increment.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_SEND_NOTIFICATION

This message is part of the GenViewControl mechanism and is used almost exclusively by the GenView. It sends the special notification from the GenView to the controller, if needed.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_SET_ATTRS

This message sets the GenViewControlAttrs in *GVCI_attrs*. Attributes noted in both parameters will be turned off for the controller.

Pass:
- **cx** — **GenViewAttrs** to set.
- **dx** — **GenViewAttrs** to clear.
- **bp** — **VisUpdateMode**.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_SET_COLOR

This message sets the background color of the view. It will not affect the document or printing. The color may be specified either by standard GEOS color index or by RGB value.

Pass:
- **cl** — Red color value.
- **ch** — **ColorFlag**.
- **dl** — Green color value.
- **dh** — Blue color value.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_SET_CONTENT

This message sets the view's *GVI_content* instance data field to a new optr. The current content will receive messages as if the view were shutting down, and the new content will receive messages as if the view were being created anew.

Pass:
- ^l**cx**:**dx** — New object to act as content (zero to set none).

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_SET_CONTROLLED_ATTRS

This message sets attributes controlled by the GenViewControl.

Pass:	**cx**	**GenViewControlAttrs** field.
	dx	Scale factor.
Returns:	Nothing.	
Destroyed:	**ax, cx, dx, bp.**	
Library:	**Objects\gViewC.def**	

MSG_GEN_VIEW_SET_DIMENSION_ATTRS

This message changes the *GVI_horizAttrs* and *GVI_vertAttrs* records of the view.

Pass:	**cl**	**GenViewDimensionAttrs** with horizontal attributes to set.
	ch	**GenViewDimensionAttrs** with horizontal attributes to clear.
	dl	**GenViewDimensionAttrs** with vertical attributes to set.
	dh	**GenViewDimensionAttrs** with vertical attributes to clear.
Returns:	Nothing.	
Destroyed:	**ax, cx, dx, bp.**	
Library:	**Objects\gViewC.def**	

MSG_GEN_VIEW_SET_DOC_BOUNDS

This message sets the view's scrollable document bounds to those passed. Each of the four parameters must be given in document coordinates (points), and all are signed long integers. If the new document bounds cause the view to show invalid coordinates, the view will scroll until it is showing valid document space.

Pass:	**ss**:**bp**	New scrollable document bounds, represented as a **RectDWord** structure. Zero if no constrain requested.
	dx	sizeof **RectDWord**.
Returns:	Nothing.	
Destroyed:	**ax, cx, dx, bp.**	
Library:	**Objects\gViewC.def**	

MSG_GEN_VIEW_SET_DRAG_BOUNDS

This message sets a temporary rectangle in which drag scrolling can operate if the user should not drag scroll across the entire document.

Pass: `ss:bp` **RectDWord** structure with draggable bounds.
`dx` sizeof **RectDWord**.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_SET_EXTENDED_INK_TYPE

This message sets the extended Ink type for the GenView. The extended Ink type is stored in the vardata ATTR_GEN_VIEW_INK_DESTINATION_INFO.

Pass: `ss:bp` Pointer to InkDestinationInfoParams structure.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_SET_INCREMENT

This message sets the horizontal and vertical increment values for the view. If the increment had been used before for scrolling, sizing, or scaling, all the same operations will subsequently use the new increment. The two increment values must be in document coordinates (1/72 inch).

Pass: `ss:bp` **PointDWord** structure with new increment amount.
`dx` sizeof **PointDWord**.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_SET_INK_TYPE

This message sets the type of Ink input expected by the view's content.

Pass: `cl` **GenViewInkType** value.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_SET_ORIGIN

This message changes the view's current origin, causing it to scroll immediately to the new location. The origin represents the upper-left corner of the view in the scrollable document space and is made up of two signed long integers. Each coordinate must be in document coordinates (points).

Pass:	**ss**:**bp**	**PointDWord** structure containing new origin.
	dx	sizeof **PointDWord**.
Returns:	Nothing.	
Destroyed:	**ax, cx, dx, bp.**	
Library:	**Objects\gViewC.def**	

MSG_GEN_VIEW_SET_PTR_IMAGE

If you want to change the pointer's image while it is over your view or while the view has the focus, use MSG_GEN_VIEW_SET_PTR_IMAGE. To use this message, you must create the new pointer image in a sharable memory block, lock the block, and pass the pointer to the locked block. To reset your request (return to the default pointer), send the same message with the pointer zeroed.

Pass:	^l**cx**:**dx**	Object pointer of **PointerDef** structure in sharable memory block. If **cx** is zero and **dx** is zero, that means there is no pointer image requested for the new. If **cx** is zero and **dx** is not zero, then **dx** will be interpreted as a **PtrImageValue** value.
	bp	**PointerImageLevel** value. Ignored if **cx** is zero and **dx** is PIV_UPDATE.
Returns:	Nothing.	
Destroyed:	**ax, cx, dx, bp.**	
Library:	**Objects\gViewC.def**	

MSG_GEN_VIEW_SET_SCALE_FACTOR

This message changes the view's scale factor, causing it to redraw at the new scale. The view will continue to display your document at the new scale until it is either shut down or instructed to change the scale factor again.

Pass: **ss**:**bp** **ScaleViewParams** structure.
dx sizeof **ScaleViewParams**.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_SETUP_TRACKING_ARGS

This message takes the **TrackScrollingParams** structure and fills in remaining fields not passed by MSG_META_CONTENT_TRACK_SCROLLING.

Pass: **cx**:**dx** pointer to **TrackScrollingParams** structure.

Returns: **cx**, **bp** Preserved.

Destroyed: **ax, dx.**

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_SUSPEND_UPDATE

This message causes the view to suspend sending MSG_META_EXPOSED until it receives MSG_GEN_VIEW_UNSUSPEND_UPDATE.
Suspend-unsuspend pairs can be nested.

Pass: Nothing.

Returns: CF Set if view window was suspended.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_TRACKING_COMPLETE

This message sets the altered scrolling arguments and makes them official, instructing the view to implement them.

Pass: `ss:bp` **TrackScrollingParams** structure.
`dx` sizeof **TrackScrollingParams**.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_UNSUSPEND_UPDATE

This message allows the view to send a MSG_META_EXPOSED again after being suspended with MSG_GEN_VIEW_SUSPEND_UPDATE.

Pass: `Nothing.`

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\gViewC.def**

MSG_GEN_VIEW_UPDATE_CONTENT_TARGET_INFO

This message is sent to the view from **VisContentClass** contents whenever target information within the view changes. This is done so that MSG_META_GET_TARGET_EXCL may be used to retrieve target information about the portion of the target hierarchy which is within the content itself. (The UI can't directly call objects running in other threads, so this message is required to update it with the latest information.)

Pass: `ss:bp` **ViewTargetInfo** structure.
`dx` sizeof **ViewTargetInfo**.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\gViewC.def**

GenViewControlClass

Instance Data

```
GVCI_minZoom          word    DEFAULT_ZOOM_MINIMUM
          UIC Syntax:  minZoom = wordComp : 25
```

minZoom (*GVCI_minZoom*) determines the minimum percentage zoom allowed.

```
GVCI_maxZoom          word    DEFAULT_ZOOM_MAXIMUM
          UIC Syntax:  maxZoom = wordComp : 200;
```

maxZoom (*GVCI_maxZoom*) indicates the maximum zoom percentage allowed.

```
GVCI_scale            word    100
          UIC Syntax:  scale = wordComp : 100;
```

scale (*GVCI_scale*) indicates the current view scale factor setting.

```
GVCI_attrs            GenViewControlAttrs
                    mask GVCA_SHOW_HORIZONTAL or mask
                    GVCA_SHOW_VERTICAL or mask GVCA_APPLY_TO_ALL
          UIC Syntax:  viewControlAttrs = bitFieldComp word {
                       adjustForAspectRatio,
                       applyToAll,
                       showHorizontal,
                       showVertical
                       } : showHorizontal, showVertical, applyToAll
```

viewControlAttrs (*GVCI_attrs*) indicates which attributes the controller should implement on startup.

Messages

MSG_GEN_VIEW_CONTROL_SET_ATTRS

This message sets the **GenViewControlAttrs** in *GVCI_attrs*. Attributes noted in both parameters will be turned off for the controller.

Pass: **cx** **GenViewControlAttrs** to set.
dx **GenViewControlAttrs** to clear.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: **Objects\gViewCC.def**

MSG_GEN_VIEW_CONTROL_SET_MAXIMUM_SCALE_FACTOR

This message sets the maximum settable scale factor for the view controller.

Pass: **cx** Maximum scale factor.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: **Objects\gViewCC.def**

MSG_GEN_VIEW_CONTROL_SET_MINIMUM_SCALE_FACTOR

This message sets the view controller's minimum allowable scale factor.

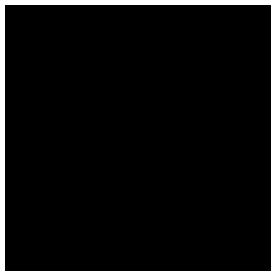
Pass: **cx** Minimum scale factor.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: **Objects\gViewCC.def**

Text Objects



10

GenTextClass 245

VisTextClass 254

GenTextClass

Instance Data

```
GTXI_text            lptr
   UIC Syntax:  text                  = nptrComp : null;
```

text (GTXI_text) stores a pointer to the text string.

```
GTXI_maxLength       sword               -1
   UIC Syntax:  maxLength             = wordComp : -1;
```

maxLength (GTXI_maxLength) stores the maximum number of characters that can be entered.

```
GTXI_attrs           GenTextAttrs (GTA_USE_TAB_FOR_NAVIGATION)
   UIC Syntax:  textDisplayAttributes  = bitFieldComp byte {
                   singleLineText,
                   useTabForNavigation,
                   initScrolling,
                   noWordWrapping,
                   allowTextOffEnd,
                   tailOriented,
                   dontScrollToChanges
                   } : useTabForNavigation;
```

textDisplayAttributes (GTXI_attrs) stores the text attributes in use by the text object. Initially, GenText objects have *useTabForNavigation* set.

```
GTXI_stateFlags      GenTextStateFlags 0
   UIC Syntax:  stateFlags            = bitFieldComp byte {
                   indeterminate,
                   modified
                   } : {};
```

stateFlags (GTXI_stateFlags) stores the **GenTextStates** in use by the text object.

```
GTXI_destination     optr
   UIC Syntax:  destination           = optrComp : null;
```

destination (GTXI_destination) stores the destination of any messages sent out by the object.

```
GTXI_applyMsg        word              0
   UIC Syntax:  applyMsg              = wordComp : 0;
```

applyMsg (GTXI_applyMsg) stores the message to send out to the destination on MSG_GEN_APPLY.

Variable Instance Data

ATTR_GEN_TEXT_STATUS_MSG vardata word

ATTR_GEN_TEXT_STATUS_MSG sets a status message for this GenText. A status message allows the GenText to send notification other than its apply message.

ATTR_GEN_TEXT_SET_MODIFIED_ON_REDUNDANT_SELECTION vardata

ATTR_GEN_TEXT_SET_MODIFIED_ON_REDUNDANT_SELECTION sets the text dirty (modified) on any modification, even if it does not change the text or the text's attributes.

ATTR_GEN_TEXT_SELECTABLE vardata

ATTR_GEN_TEXT_SELECTABLE should be set if text should be selectable whether or not the text is editable. By default, non-editable text is not selectable. Editable text is always selectable.

ATTR_GEN_TEXT_DEFAULT_CHAR_ATTR vardata VisTextDefaultCharAttr
ATTR_GEN_TEXT_DEFAULT_PARA_ATTR vardata VisTextDefaultParaAttr

ATTR_GEN_TEXT_DEFAULT_CHAR_ATTR stores singular default character attributes of type **VisTextDefaultCharAttr** in a word-length record. ATTR_GEN_TEXT_DEFAULT_PARA_ATTR stores singular default paragraph attributes of type **VisTextDefaultParaAttr** in a word-length record.

ATTR_GEN_TEXT_CHAR_ATTR vardata lptr

ATTR_GEN_TEXT_CHAR_ATTR stores the singular character attributes for the text object. This vardata field stores the ChunkHandle to a single **VisTextCharAttr** structure.

ATTR_GEN_TEXT_MULTIPLE_CHAR_ATTR_RUNS vardata lptr

ATTR_GEN_TEXT_MULTIPLE_CHAR_ATTR_RUNS stores the multiple character attributes for the text object. This vardata field will store the ChunkHandle to a chunk array of character style runs (each of type **VisTextCharAttr**).

ATTR_GEN_TEXT_PARA_ATTR vardata lptr

Object Assembly Reference

ATTR_GEN_TEXT_PARA_ATTR stores the singular paragraph attributes for the text object. This vardata field stores the ChunkHandle to a single **VisTextParaAttr** structure.

`ATTR_GEN_TEXT_MULTIPLE_PARA_ATTR_RUNS vardata lptr`

ATTR_GEN_TEXT_MULTIPLE_PARA_ATTR_RUNS stores the multiple paragraph attributes for the text object. This vardata field will store the ChunkHandle to a chunk array of paragraph style runs (each of type **VisTextParaAttr**).

`ATTR_GEN_TEXT_EXTENDED_FILTER vardata word`

ATTR_GEN_TEXT_EXTENDED_FILTER stores the **VisTextExtendedFilterTypes** to use instead of the default **VisTextFilters**. Extended filters offer more powerful (and complex) filtering mechanisms for accepting and rejecting text into the GenText.

`ATTR_GEN_TEXT_TYPE_RUNS vardata word`
`ATTR_GEN_TEXT_GRAPHIC_RUNS vardata word`

ATTR_GEN_TEXT_TYPE_RUNS specifies the type runs for this text object. You should not need type runs unless you need hyperlink capabilities. ATTR_GEN_TEXT_GRAPHIC_RUNS specifies a graphics run to use within the text object.

`ATTR_GEN_TEXT_REGION_ARRAY vardata word`
`ATTR_GEN_TEXT_STYLE_ARRAY vardata word`

ATTR_GEN_TEXT_REGION_ARRAY specifies the array of regions defined for this text object. Text within the GenText will flow from each of the defined regions to the next region. ATTR_GEN_TEXT_STYLE_ARRAY specifies the name array of styles used in the GenText's style sheets.

`ATTR_GEN_TEXT_RUNS_ITEM_GROUP vardata optr`

ATTR_GEN_TEXT_RUNS_ITEM_GROUP indicates that this text object is linked to an item group (list). When the user hits Enter (or the equivalent), the text object sends a message to the item group; similarly, the item group will attempt to update the text object on changes.

`ATTR_GEN_TEXT_DO_NOT_INTERACT_WITH_SEARCH_CONTROL vardata`

ATTR_GEN_TEXT_DO_NOT_INTERACT_WITH_SEARCH_CONTROL indicates that the text object should not send notification to the search control object.

`ATTR_GEN_TEXT_SET_OBJECT_ENABLED_WHEN_TEXT_EXISTS vardata optr`

ATTR_GEN_TEXT_SET_OBJECT_ENABLED_WHEN_TEXT_EXISTS makes the specified object enabled any time the GenText object has text in it.

ATTR_GEN_TEXT_LEGAL_DOS_PATH vardata

ATTR_GEN_TEXT_LEGAL_DOS_PATH makes sure that only legal DOS paths are used.

ATTR_GEN_TEXT_DATE vardata
ATTR_GEN_TEXT_TIME vardata

ATTR_GEN_TEXT_DATE and ATTR_GEN_TEXT_TIME allows short date and time characters and spaces; the text object will be affected by the localization driver.

ATTR_GEN_TEXT_MAKE_UPPERCASE vardata

ATTR_GEN_TEXT_MAKE_UPPERCASE indicates that in addition to any other filters, any alphabetic characters are forced into uppercase.

ATTR_GEN_TEXT_ALLOW_COLUMN_BREAKS vardata

ATTR_GEN_TEXT_ALLOW_COLUMN_BREAKS allows all characters including column breaks. Column breaks are normally filtered out in a GenText object. The column breaks usually will appear as new lines (like carriage returns) in the text object.

ATTR_GEN_TEXT_DASHED_ALPHA_NUMERIC vardata

ATTR_GEN_TEXT_DASHED_ALPHA_NUMERIC allows only uppercase alphabetic characters and spaces but also allows dashes.

ATTR_GEN_TEXT_NORMAL_ASCII vardata

ATTR_GEN_TEXT_NORMAL_ASCII allows only normal ASCII characters (no extended ASCII characters).

ATTR_GEN_TEXT_LEGAL_DOS_VOLUME_NAMES vardata

ATTR_GEN_TEXT_LEGAL_DOS_VOLUME_NAMES allows only characters that may appear in a legal DOS volume name.

ATTR_GEN_TEXT_DOS_CHARACTER_SET vardata

ATTR_GEN_TEXT_DOS_CHARACTER_SET allows only characters in the legal DOS character set.

ATTR_GEN_TEXT_NO_SPACES vardata
ATTR_GEN_TEXT_ALLOW_SPACES vardata

ATTR_GEN_TEXT_NO_SPACES and ATTR_GEN_TEXT_ALLOW_SPACES govern whether spaces are allowed in the text object.

ATTR_GEN_TEXT_ALPHA vardata
ATTR_GEN_TEXT_NUMERIC vardata
ATTR_GEN_TEXT_SIGNED_NUMERIC vardata

```
ATTR_GEN_TEXT_SIGNED_DECIMAL                    vardata
ATTR_GEN_TEXT_FLOAT_DECIMAL                     vardata
ATTR_GEN_TEXT_ALPHA_NUMERIC                     vardata
```

ATTR_GEN_TEXT_ALPHA, ATTR_GEN_TEXT_NUMERIC, ATTR_GEN_TEXT_SIGNED_NUMERIC, ATTR_GEN_TEXT_SIGNED_DECIMAL, ATTR_GEN_TEXT_FLOAT_DECIMAL, and ATTR_GEN_TEXT_ALPHA_NUMERIC govern the character types allowed.

```
ATTR_GEN_TEXT_LEGAL_FILENAMES                   vardata
```

ATTR_GEN_TEXT_LEGAL_FILENAMES allows any characters legal to DOS file names.

```
ATTR_GEN_TEXT_NEVER_MAKE_SCROLLABLE             vardata
```

ATTR_GEN_TEXT_NEVER_MAKE_SCROLLABLE forces the text object to expand larger as the text grows without forcing it into scrollable text.

```
ATTR_GEN_TEXT_DOES_NOT_ACCEPT_INK               vardata
```

ATTR_GEN_TEXT_DOES_NOT_ACCEPT_INK indicates that this text object will not accept ink under any circumstances.

```
ATTR_GEN_TEXT_CUSTOM_FILTER vardata lptr.VisTextCustomFilterData
```

ATTR_GEN_TEXT_CUSTOM_FILTER contains the ChunkHandle of an array of **VisTextCustomFilterData** structures. Each of these structures contains a range of character *values* to filter out. These values are determined from the **Chars** enumerated type. The text object figures out the number of filters in place by getting the size of the chunk.

Hints

```
HINT_TEXT_WASH_COLOR                            vardata ColorQuad
HINT_TEXT_WHITE_WASH_COLOR                      vardata
HINT_TEXT_AUTO_HYPHENATE                        vardata
HINT_TEXT_SELECT_TEXT                           vardata
HINT_TEXT_CURSOR_AT_START                       vardata
HINT_TEXT_CURSOR_AT_END                         vardata
HINT_TEXT_FRAME                                 vardata
HINT_TEXT_NO_FRAME                              vardata
HINT_TEXT_ALLOW_UNDO                            vardata
HINT_TEXT_ALLOW_SMART_QUOTES                    vardata
HINT_TEXT_DO_NOT_MAKE_LARGER_ON_PEN_SYSTEMS vardata
```

HINT_TEXT_WASH_COLOR indicates the background color for the GenText to exhibit. This hint takes a **ColorQuad** argument.



HINT_TEXT_WHITE_WASH_COLOR indicates the background color of the GenText should be white.

Messages

GenTextClass contains a number of messages that deal specifically with the instance data defined for that class. You may also use a large number of messages defined for VisTextClass, as GenTextClass is defined off of that class also.

There also exist many MetaClass messages that are imported in VisTextClass. Many of these also work with GenTextClass. Consult those specific messages for more information.

MSG_GEN_TEXT_GET_APPLY_MSG

This message retrieves the current apply message (*GTXI_applyMsg*) of a GenText object.

Source: Unrestricted.

Destination: Any GenText object.

Pass: Nothing.

Returns: **ax** The apply message of the GenText object.

Destroyed: **cx, dx, bp**

Library: **Objects\gTextC.def**

MSG_GEN_TEXT_GET_ATTRS

This message retrieves the GenText object's **GenTextAttrs** (*GTXI_attrs*).

Source: Unrestricted.

Destination: Any GenText object.

Pass: Nothing.

Returns: **al** **GenTextAttrs** in use by the GenText object(*GTXI_attrs*).

Destroyed: **ah, cx, dx, bp**

Library: **Objects\gTextC.def**

Object Assembly Reference

MSG_GEN_TEXT_GET_DESTINATION

This message returns the current destination object (*GTXI_destination*) of a GenText object.

Source: Unrestricted.

Destination: Any GenText object.

Pass: Nothing.

Returns: `cx:dx` The optr of the GenText's destination object (*GTXI_destination*).

Destroyed: `ax, bp`

Library: **Objects\gTextC.def**

MSG_GEN_TEXT_IS_INDETERMINATE

This message checks whether a GenText object is indeterminate in state.

Source: Unrestricted.

Destination: Any GenText object.

Pass: Nothing.

Returns: `CF` Set if the GenText object is in an indeterminate state; clear, otherwise.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gTextC.def**

MSG_GEN_TEXT_IS_MODIFIED

This message checks whether a GenText object has been modified.

Source: Unrestricted.

Destination: Any GenText object.

Pass: Nothing.

Returns: `CF` Set if the GenText object is in a modified state; clear, otherwise.

Destroyed: `ax, cx, dx, bp`

Include: **Objects\gTextC.def**

MSG_GEN_TEXT_SEND_STATUS_MSG

This message causes a GenText object to send out its status message, stored in the text object's ATTR_GEN_TEXT_STATUS_MSG vardata field. This message will still function even if the text object is not enabled (or usable).

Source: Unrestricted.

Destination: Any GenText object.

Pass: `cx` Non-zero if GVSF_MODIFIED bit should be passed with the status message.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gTextC.def**

MSG_GEN_TEXT_SET_APPLY_MSG

This message sets a new apply message (*GTXI_applyMsg*) for the text object.

Source: Unrestricted.

Destination: Any GenText object.

Pass: `cx` The new apply message.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gTextC.def**

MSG_GEN_TEXT_SET_ATTRS

This message sets a GenText object's **GenTextAttrs** (*GTXI_attrs*). The GenText object must not be GS_USABLE when sent this message.

Source: Unrestricted.

Destination: Any non-usable GenText object.

Pass: `cl` **GenTextAttrs** to add.
`ch` **GenTextAttrs** to remove. An attribute set in both parameters will be cleared.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gTextC.def**

Object Assembly Reference

MSG_GEN_TEXT_SET_DESTINATION

This message sets the destination object or process (*GTXI_destination*) of a GenText object.

Source: Unrestricted.

Destination: Any GenText object.

Pass: **cx:dx** The optr of the new destination object or process.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gTextC.def**

MSG_GEN_TEXT_SET_FROM_ITEM_GROUP

If the GenText object contains the ATTR_GEN_TEXT_RUNS_ITEM_GROUP attribute, this message is sent to the GenText in response to a selection within the GenItemGroup.

Source: GenItemGroup

Destination: A GenText object with ATTR_GEN_TEXT_RUNS_ITEM_GROUP set to the specific GenItemGroup in question.

Pass: **cx** GenItem identifier of the selection made.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gTextC.def**

MSG_GEN_TEXT_SET_INDETERMINATE_STATE

This message sets the GenText object's indeterminate state (the GTSF_INDETERMINATE flag in *GTXI_stateFlags*). This message does not affect the stored text.

Source: Unrestricted.

Destination: Any GenText object.

Pass: **cx** Non-zero to set the text object indeterminate (GTSF_INDETERMINATE); zero otherwise.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gTextC.def**

■ MSG_GEN_TEXT_SET_MODIFIED_STATE

This message sets the GenText object's modified state (the GTSF_MODIFIED flag in *GTXI_stateFlags*). This message does not affect the stored text.

Source: Unrestricted.

Destination: Any GenText object.

Pass: **cx** Non-zero to set the text object indeterminate (GTSF_MODIFIED); zero otherwise.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gTextC.def**

VisTextClass

Data Storage Instance Data Fields

The following instance data fields contain references to data storage locations:

```
VTI_text                lptr.char
   UIC Syntax:  text          = nptrComp : null;
```

text (VTI_text) stores the chunk handle containing the text string.

```
VTI_charAttrRuns      word VIS_TEXT_INITIAL_CHAR_ATTR
   UIC Syntax:  charAttrRuns = wordComp : "VIS_TEXT_INITIAL_CHAR_ATTR";
```

charAttrRuns (VTI_charAttrRuns) stores the character attributes in use by the text object. This will either be a **VisTextDefaultCharAttr** record or a pointer to a run of **VisTextCharAttr** structures.

```
VTI_paraAttrRuns      word VIS_TEXT_INITIAL_PARA_ATTR
   UIC Syntax:  paraAttrRuns = wordComp : "VIS_TEXT_INITIAL_PARA_ATTR";
```

paraAttrRuns (VTI_paraAttrRuns) stores the paragraph attributes in use by the text object. This will either be a **VisTextDefaultParaAttr** record or a pointer to a run of **VisTextParaAttr** structures.

```
VTI_vmFile            word         0
   UIC Syntax:  vmFile        = wordComp : 0;
```

vmFile (VTI_vmFile) stores the handle of the text object's associated VM file

```
VTI_lines            word        0
   UIC Syntax:  static    nptrComp : null;
```

VTI_lines stores the LMem handle to store line structures. This is dealt with internally; it cannot be set in the .**ui** file.

External Instance Data Fields

The following instance data fields store external attributes that may be accessed and altered.

```
VTI_storageFlags     VisTextStorageFlags (mask
VTSF_DEFAULT_CHAR_ATTR or mask VTSF_DEFAULT_PARA_ATTR)
   UIC Syntax:  storage   = bitFieldComp byte {
                    large,
                    multipleCharAttr,
                    multipleParaAttr,
                    types,
                    graphics,
                    defaultCharAttr,
                    defaultParaAttr,
                    styles
                    } : defaultCharAttr, defaultParaAttr;
```

storage (VTI_storageFlags) describes how data is stored within the text object. The value of this field determines what is contained within the data storage instance data fields mentioned previously. By default all VisText objects use *defaultCharAttr* and *defaultParaAttr*.

```
VTI_features         VisTextFeatures 0
   UIC Syntax:  features  = bitFieldComp word {
                    noWordWrap,
                    autoHyphenate,
                    allowSmartQuotes,
                    allowUndo,
                    showHiddenText,
                    outlineMode,
                    dontShowSoftPageBreaks,
                    dontShowGraphics
                    } : {};
```

features (VTI_features) contains what **VisTextFeatures** are in use by the text object.

```
VTI_state            VisTextStates 0
   UIC Syntax:  states    = bitFieldComp byte {
                   editable,
                   selectable,
                   targetable,
                   oneLine,
                   subclassVirtPhys,
                   overstrikeMode,
                   dirty
                   } : {};
```

states (VTI_state) stores the **VisTextStates** of the text object.

Internal Instance Data Fields

The following instance data fields are primarily internal and should not be accessed:

```
VTI_intFlags         VisTextIntFlags 0
   UIC Syntax:  static    byteComp : 0;
```

VTI_intFlags stores internal flags. It cannot be set in the .**ui** file.

```
VTI_intSelFlags      VisTextIntSelFlags 0
   UIC Syntax:  static    byteComp : 0;
```

VTI_intSelFlags stores internal selection flags.It cannot be set in the .**ui** file.

```
VTI_gsRefCount       GSRefCountAndFlags 0
   UIC Syntax:  static    byteComp : 0;
```

VTI_gsRefCount stores the reference count used by the gstate. This is dealt with internally; it cannot be set in the .**ui** file.

```
VTI_gstate           hptr.GState 0
noreloc VTI_gstate
   UIC Syntax:  static    hptrComp : null;
```

VTI_gstate stores the temporary gstate used by the text object. This is dealt with internally; it cannot be set in the .**ui** file.

```
VTI_gstateRegion     word -1
   UIC Syntax:  static    wordComp : -1;
```

VTI_gstateRegion stores the region to which the gstate has been translated. This is dealt with internally; it cannot be set in the .**ui** file.

Selection Instance Data Fields

The following instance data fields store selection information.

```
VTI_selectStart       dword 0
   UIC Syntax:  selectStart  = dwordComp : 0;
```

selectStart (VTI_selectStart) stores the offset to the selection start.

```
VTI_selectEnd         dword 0
   UIC Syntax:  selectEnd    = dwordComp : 0;
```

selectEnd (*VTI_selectEnd*) stores the offset to the selection end.

```
VTI_startEventPos     PointDWord
   UIC Syntax:  static    typeComp "PointDWord" : "0,0";
```

VTI_startEventPos stores the start event position which holds the coordinate position of the original mouse event. This is dealt with internally; it cannot be set in the .**ui** file.

```
VTI_selectMinStart    dword
   UIC Syntax:  static    dwordComp : 0;
```

VTI_selectMinStart stores the minimum selection start. This indicates the minimum range that must remain selected during drag-selection. The selection will never be smaller than this. This is dealt with internally; it cannot be set in the .**ui** file.

```
VTI_selectMinEnd      dword
   UIC Syntax:  static    dwordComp : 0
```

VTI_selectMinEnd stores the minimum selection end. This indicates the minimum range that must remain selected during drag-selection. The selection will never be smaller than this. This is dealt with internally; it cannot be set in the .**ui** file.

```
VTI_lastOffset        dword
   UIC Syntax:  static    dwordComp : 0;
```

VTI_lastOffset stores the last character offset which the mouse was over. This is used for optimizations; if the character offset of the mouse event hasn't changed, the selection can't have changed. This is dealt with internally; it cannot be set in the .**ui** file.

```
VTI_goalPosition      word
   UIC Syntax:  static    wordComp : 0;
```

VTI_goalPosition stores the current horizontal offset of the cursor within the text. As the cursor is moved up or down, this value indicates where the selection should be extended to. This is dealt with internally; it cannot be set in the .**ui** file.

Other Instance Data Fields

The following miscellaneous instance fields are mainly for internal use.

```
VTI_cursorPos          Point
   UIC Syntax:      static    wordComp : 0;
                    static    wordComp : 0
```

VTI_cursorPos stores the current cursor position. Note that in UIC, this is stored as two word length positions, one for horizontal displacement and one for vertical displacement. This is dealt with internally; it cannot be set in the .**ui** file.

```
VTI_cursorRegion      word
   UIC Syntax:   static    wordComp : 0;
```

VTI_cursorRegion stores the current region in which the cursor is visible. This is dealt with internally; it cannot be set in the .**ui** file.

```
VTI_leftOffset        sword
   UIC Syntax:   static    wordComp : 0;
```

VTI_leftOffset stores the current cursor offset from left edge of text. This is dealt with internally; it cannot be set in the .**ui** file.

```
VTI_lrMargin          byte
   UIC Syntax:   lrMargin  = byteComp : 0;
```

lrMargin (VTI_lrMargin) stores the current left/right margins in eighths of a point.

```
VTI_tbMargin          byte
   UIC Syntax:   tbMargin  = byteComp : 0;
```

tbMargin (VTI_tbMargin) stores the current top/bottom margins in eighths of a point.

```
VTI_washColor         ColorQuad <C_WHITE, 0, 0, 0>
   UIC Syntax:   washColor = ColorQuad : {
                     redOrIndex = "C_WHITE";
                     }
```

washColor (VTI_washColor) stores the background color and map mode of the text object.

```
VTI_maxLength         word 10000
   UIC Syntax:   maxLength = wordComp : 10000;
```

maxLength (VTI_maxLength) stores the maximum number of characters that can exist within the text object. This only applies to small text objects. Large text objects can not have a limit set on their size.

Object Assembly Reference

```
VTI_filters             VisTextFilters
   UIC Syntax:  filters   = bitFieldComp byte {
                   noSpaces,
                   noTabsOrCRs,
                   filterClass:6 = {
                      none,
                      alpha,
                      numeric,
                      signedNumeric,
                      signedDecimal,
                      floatDecimal,
                      alphaNumeric,
                      fileNames,
                      dosFilenames,
                      dosPath,
                      date,
                      time,
                      dashedAlphaNumeric,
                      normalASCII,
                      dosVolumeNames,
                      dosCharacterSet
                      }
                   } : filterClass none;
```

filters (VTI_filters) stores any pre-defined keystroke filters to use on text entered into the text object.

```
VTI_output              optr
   UIC Syntax:  output    = optrComp : null;
```

output (*VTI_output)* stores the destination optr of messages sent out by the text object.

```
VTI_height              WBFixed <>
   UIC Syntax:      static    byteComp : 0;
                    static    wordComp : 0;
```

VTI_height stores the text object's height in points. In UIC, the byte length value contains the fractional point size and the word-length value contains the integral height of the point size. This is dealt with internally; it cannot be set in the .**ui** file.

```
VTI_lastWidth           word -1
   UIC Syntax:  static    wordComp : -1;
```

VTI_lastWidth stores the cached width. This value is used to optimize resizing the object via the geometry `manager`. This is dealt with internally; it cannot be set in the .**ui** file.

```
VTI_timerHandle         word
   UIC Syntax:  static    wordComp : 0;
```

```
VTI_timerID          word
   UIC Syntax:  static    wordComp : 0;
```

VTI_timerHandle and *VTI_timerID* store internal timing information related to cursor blinking. They cannot be set in the **.ui** file.

Variable Instance Data

```
ATTR_VIS_TEXT_TYPE_RUNS                          vardata word
ATTR_VIS_TEXT_GRAPHIC_RUNS                       vardata word
```

ATTR_VIS_TEXT_TYPE_RUNS specifies the Type runs for this text object. You should not need type runs unless you need hyperlink capabilities.

ATTR_VIS_TEXT_GRAPHIC_RUNS specifies a graphics run to use within the text object.

```
ATTR_VIS_TEXT_STYLE_ARRAY                        vardata word
ATTR_VIS_TEXT_REGION_ARRAY                       vardata word
```

ATTR_VIS_TEXT_REGION_ARRAY specifies the array of regions defined for this text object. Text within the VisText will flow from each of the defined regions to the next region. ATTR_VIS_TEXT_STYLE_ARRAY specifies the name array of styles used in the VisText's style sheets.

```
ATTR_VIS_TEXT_SELECTED_TAB                       vardata word
```

ATTR_VIS_TEXT_SELECTED_TAB specifies the position of the currently-selected tab, if any.

```
ATTR_VIS_TEXT_DO_NOT_INTERACT_WITH_SEARCH_CONTROL vardata
```

ATTR_VIS_TEXT_DO_NOT_INTERACT_WITH_SEARCH_CONTROL instructs the text object not to send the search-control notification.

```
ATTR_VIS_TEXT_EXTENDED_FILTERS   vardata VisTextExtendedFilterType
```

ATTR_VIS_TEXT_EXTENDED_FILTERS stores the **VisTextExtendedFilterType** to use instead of the **VisTextFilters** stored within *VTI_filters*. Extended filters offer more powerful (and complex) filtering mechanisms for accepting and rejecting text. Including this attribute will affect the performance of your text object, however.

```
ATTR_VIS_TEXT_SUSPEND_DATA               vardata VisTextSuspendData
```

ATTR_VIS_TEXT_SUSPEND_DATA stores data to be used by the text object during a SUSPEND/UNSUSPEND operation (**VisTextSuspendData**).

ATTR_VIS_TEXT_SEND_CONTEXT_NOTIFICATIONS_EVEN_IF_NOT_FOCUSED
vardata

ATTR_VIS_TEXT_SEND_CONTEXT_NOTIFICATIONS_EVEN_IF_NOT_FOCUSED sends out context notification even if the object is not focused. Applications must make sure that objects with this attribute do not get modified while another text object has the focus.

ATTR_VIS_TEXT_UPDATE_VIA_PROCESS vardata

ATTR_VIS_TEXT_UPDATE_VIA_PROCESS indicates that status updates must be sent via the process object (usually because there are targetable text objects in multiple threads).

ATTR_VIS_TEXT_DOES_NOT_ACCEPT_INK vardata

ATTR_VIS_TEXT_DOES_NOT_ACCEPT_INK indicates that the VisText object will not accept ink.

ATTR_VIS_TEXT_MINIMUM_SINGLE_LINE_HEIGHT vardata WBFixed

ATTR_VIS_TEXT_MINIMUM_SINGLE_LINE_HEIGHT defines the height of one-line text objects so that custom graphics, etc. can be placed inside the text object.

ATTR_VIS_TEXT_ADD_SPACE_TO_ALL_LINES vardata word

ATTR_VIS_TEXT_ADD_SPACE_TO_ALL_LINES adds additional spacing above all lines in a text object, and it takes a word argument. This optional attribute is intended to be added to objects that use accent characters that may draw above the top of the text's "font box." This spacing value differs depending on the system that text is implemented on; only use this attribute if redraws are not drawing your text characters properly. This attribute will also affect any text lines that contain borders, adding additional space between the text and the border.

Imported Messages From MetaClass

VisTextClass imports a number of messages from MetaClass. The range that these are imported from is pointed out in the first line of the reference entry. These messages may be used by both GenText and VisText objects, unless otherwise stated.



MSG_ABORT_ACTIVE_SEARCH

Imported in the MetaSearchSpellMessages range.

This message is sent out to the GAGCNLT_APP_TARGET_NOTIFY_SEARCH_SPELL_CHANGE GCN list to inform the search box that the user has aborted the current operation.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

MSG_ABORT_ACTIVE_SPELL

Imported in MetaSearchSpellMessages range.

This message is sent out to the GAGCNLT_APP_TARGET_NOTIFY_SEARCH_SPELL_CHANGE GCN list to inform the spell box that the user has aborted the current operation.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

MSG_META_CONTEXT

Imported in MetaSearchSpellMessages range.

This message returns the current context to the SpellControl object.

Source: A text object receiving MSG_META_GET_CONTEXT.

Destination: The object that sent the request.

Pass: **bp** Block containing **ContextData** structure storing information on the context, and the null-terminated context string.

```
ContextData struct
    CD_object       optr
    CD_numChars     dword
    CD_range        VisTextRange
    CD_selection    VisTextRange
    CD_contextData  label     char
ContextData ends
```

CD_object stores the optr of the object that the context is coming from.

CD_numChars stores the number of characters in the text object.

CD_range stores the range of characters that this context represents within the text object.

CD_selection stores the range of characters that represent the current text selection.

The null-terminated context data follows this structure.

Returns: Nothing. (Block is freed by the SpellControl.)

Destroyed: **ax, cx, dx, bp**

Library: **Objects\vTextC.def**

MSG_META_DISPLAY_OBJECT_FOR_SEARCH_SPELL

Imported in MetaSearchSpellMessages range.

During a search/spell session, the SpellControl may search through several text objects. When it finds a match within an object, the text object sends this message to itself to bring itself on the screen.

Source: Sent by a text object to itself during a search or spell check session.

Destination: The text object sends this message to itself.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

MSG_META_GENERATE_CONTEXT_NOTIFICATION

Imported in MetaSearchSpellMessages range.

This message creates a context block and sends it to the GAGCNLT_NOTIFY_TEXT_CONTEXT GCN list.

Pass: `ss:bp` GetContextParams (GCP_replyObj is ignored.)

Returns: Nothing.

Destroyed: `Nothing.`

Library: **Objects\vTextC.def**

MSG_META_GET_CONTEXT

Imported in MetaSearchSpellMessages range.

Object Assembly Reference

This message is sent by the Spell controller to find out the current context of the text object. Other objects may also send this message to the text object. The text object will return MSG_META_CONTEXT to the requestor.

Source: The Spell controller. (Other objects may want to send this to retrieve the text context displayed in the spell box.)

Destination: *GCI_output* of the Spell controller (usually a text object).

Pass: **ss:bp** Pointer to a **GetContextParams** structure.

```
GetContextParams struct
    GCP_replyObj            optr
    GCP_numCharsToGet       word
    GCP_location            ContextLocation
    GCP_position            dword
GetContextParams ends
```

GCP_position
Position of the context. The context returned depends on this value and the **ContextLocation** passed in *location*.

GCP_location
ContextLocation. This may be one of the following values:

CL_STARTING_AT_POSITION
CL_ENDING_AT_POSITION
CL_CENTERED_AROUND_POSITION
CL_CENTERED_AROUND_SELECTION
CL_CENTERED_AROUND_SELECTION_START
CL_SELECTED_WORD

GCP_numCharsToGet
Maximum number of characters to return.

GCP_replyObj
Optr of the object to reply to via MSG_META_CONTEXT.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: You may intercept to alter the nature of the context request.

Library: **Objects\vTextC.def**

MSG_META_GET_OBJECT_FOR_SEARCH_SPELL

Imported in MetaSearchSpellMessages range.

This message is sent by a text object to itself when a search reaches the end of the current object and needs to determine where to continue the search.

Source: Any text object.

Destination: The text object sends this to itself.

Pass:

`bp`	**GetSearchSpellObjectOption** specifying the next object to continue the search at. This value may be GSSOT_FIRST_OBJECT, GSSOT_LAST_OBJECT, GSSOT_NEXT_OBJECT, or GSSOT_PREV_OBJECT.
`cx:dx`	Optr of the object that is currently being searched through. If you intercept this message, you can use this information to set the *option* above.

Returns: `cx:dx` Optr to requested object.

Destroyed: `ax, bp`

Interception: May intercept if you want to direct where a search or spell check operation should continue to.

Library: **Objects\vTextC.def**

■ MSG_META_TEXT_CR_FILTERED

Imported in MetaTextMessages range.

This message is sent to *VTI_output* when a Carriage Return is filtered out in the text input stream.

Source: Sent by the system when a CR is filtered out of the text object.

Destination: *VTI_output* of the text object's instance data. The message is also sent to the text object itself.

Pass:

`cx`	The character value (CR).
`dl`	**CharFlags**.
`dh`	**ShiftState**.
`bp.low`	**ToggleState**.
`bp.high`	The scan code.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: Intercept to receive notification that a Carriage Return has been filtered out of the text object.

Library: **Objects\vTextC.def**

Object Assembly Reference

MSG_META_TEXT_EMPTY_STATUS_CHANGED

Imported in MetaTextMessages range.

This message is sent out when the text object is either becoming empty or not empty.

Source: Sent by the system when the user modifies the text object.

Destination: *VTI_output* of the text object's instance data. The message is also sent to the text object itself first.

Pass:

cx:dx	Optr of the text object whose empty status is changing.
bp	Non-zero if the text object is becoming non-empty.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Intercept to receive notification when the empty status of the text object is changing.

Library: **Objects\vTextC.def**

MSG_META_TEXT_GAINED_FOCUS

Imported in MetaTextMessages range.

This message is sent when the text object gains the focus of the application.

Source: Sent by the system when the user modifies the text object.

Destination: *VTI_output* of the text object's instance data.

Pass:

cx:dx	Optr of the text object.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Intercept to receive notification of when the text object gains the focus.

Library: **Objects\vTextC.def**

MSG_META_TEXT_GAINED_TARGET

Imported in MetaTextMessages range.

This message is sent when the text object gains the target of an application.

Source: Sent by the system when the user modifies the text object.

Destination: *VTI_output* of the text object's instance data.

Pass: `cx:dx` Optr of the text object.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: Intercept to receive notification of when the text object gains the target.

Library: **Objects\vTextC.def**

MSG_META_TEXT_LOST_FOCUS

Imported in MetaTextMessages range.

This message is sent when the text object loses the focus of the application.

Source: Sent by the system when the user modifies the text object.

Destination: *VTI_output* of the text object's instance data.

Pass: `cx:dx` Optr of the text object.

Returns: Nothing.

Interception: Intercept to receive notification of when the text object loses the focus.

Library: **Objects\vTextC.def**

MSG_META_TEXT_LOST_TARGET

Imported in MetaTextMessages range.

This message is sent when the text object loses the target of an application.

Source: Sent by the system when the user modifies the text object.

Destination: *VTI_output* of the text object's instance data.

Pass: `cx:dx` Optr of the text object.

Returns: Nothing.

Interception: Intercept to receive notification of when the text object loses the target.

Library: **Objects\vTextC.def**

MSG_META_TEXT_NOT_USER_MODIFIED

Imported in MetaTextMessages range.

This message is sent to *VTI_output* when the VTS_MODIFIED state of the text object is cleared, either by the system, or by an application inspired action.

Source: Sent by the system when the user modifies the text object.

Destination: *VTI_output* of the text object's instance data. The message is also sent to the text object itself first.

Pass: `cx:dx` Optr of the text object marked not modified.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: Intercept to receive notification of when a text object is marked not modified.

Library: **Objects\vTextC.def**

MSG_META_TEXT_TAB_FILTERED

Imported in MetaTextMessages range.

This message is sent to *VTI_output* when a Tab is filtered out in the text input stream.

Source: Sent by the system when the user modifies the text object.

Destination: *VTI_output* of the text object's instance data. The message is also sent to the text object itself.

Pass:

`cx`	The character value (Tab).
`dl`	**CharFlags**.
`dh`	**ShiftState**.
`bp.low`	**ToggleState**.
`bp.high`	The scan code.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: Intercept to receive notification that a Tab has been filtered out of the text object.

Library: **Objects\vTextC.def**

MSG_META_TEXT_USER_MODIFIED

Imported in MetaTextMessages range.

MSG_META_TEXT_USER_MODIFIED is sent to *VTI_output* when the user modifies text within the object. The message is also sent to the text object itself (to mark itself modified) after it is sent to the destination, so the state

of the text's VTS_MODIFIED bit may not accurately reflect the state of the text object. If you need to intercept this message and also need to perform an operation using the text object's modified state.

Source: Sent by the system when the user modifies the text object.

Destination: *VTI_output* of the text object's instance data. The message is also sent to the text object itself.

Pass: **cx:dx** Optr of the text object modified.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: May intercept to receive notification of user modification of text. Because modification of the text object's VTS_MODIFIED bit may be asynchronous, you should send any operations that depend on this information (such as clearing the VTS_MODIFIED bit) to the text object using MF_FORCE_QUEUE .

Library: **Objects\vTextC.def**

MSG_REPLACE_ALL_OCCURRENCES

Imported in MetaSearchSpellMessages range.

This message is sent by the SearchReplace controller when an object is starting a search operation that should replace all found matches (in the search string) with the passed replacement string.

Source: Search controller.

Destination: *GCI_output* of the search controller (usually a text object).

Pass: **dx** Handle of block containing the **SearchReplaceStruct**. The null-terminated search string immediately follows this structure. The null-terminated replace string follows that string. This block should be freed by the message handler.

cx Non-zero if the replace-all operation should proceed from the beginning of the document; normally, the replace-all operation begins after the current position.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Intercept to find out the nature of the replacement operation.

Library: **Objects\vTextC.def**

Object Assembly Reference

MSG_REPLACE_ALL_OCCURRENCES_IN_SELECTION

Imported in MetaSearchSpellMessages range.

This message is sent by the SearchReplace controller when an object is starting a search operation that should replace all found matches within a selection (in the search string) with the passed replacement string.

Source: Search controller.

Destination: *GCI_output* of the search controller (usually a text object).

Pass: **dx** Handle of block containing the **SearchReplaceStruct**. The null-terminated search string immediately follows this structure. The null-terminated replace string follows that string. This block should be freed by the message handler.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Intercept to find out the nature of the replacement operation.

Library: **Objects\vTextC.def**

MSG_REPLACE_CURRENT

Imported in MetaSearchSpellMessages range.

This message is sent by the SearchReplace controller when an object is starting a search operation that should replace the found match with the passed replacement string. This message involves a single replacement operation. To perform a replace-all operation, use MSG_REPLACE_ALL_OCCURENCES.

Source: Search controller.

Destination: *GCI_output* of the search controller (usually a text object).

Pass: **dx** Handle of block containing the **SearchReplaceStruct**. The null-terminated search string immediately follows this structure. The null-terminated replace string follows that string. This block should be freed by the message handler.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Intercept to find out the nature of the replacement operation.

Library: **Objects\vTextC.def**

MSG_SEARCH

Imported in MetaSearchSpellMessages range.

This message is sent by the SearchReplace controller when an object is starting a search operation. You may intercept this message to find out the nature and specifics of the search.

Source: Search controller.

Destination: *GCI_output* of the search controller (usually a text object).

Pass: **dx** Handle of block containing the **SearchReplaceStruct**. The search string immediately follows this structure in the passed block. This block should be freed by the message handler.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Intercept to find out the nature of the search and replace operation.

Library: **Objects\vTextC.def**

MSG_SPELL_CHECK

Imported in MetaSearchSpellMessages range.

This message is sent by the Spell controller to continue spell checking from the current position in the document.

Source: Spell controller.

Destination: *GCI_output* of the spell controller.

Pass: **ss:bp** Pointer to **SpellCheckInfo** structure.
dx Size of **SpellCheckInfo** structure.

```
SpellCheckInfo struct
    SCI_ICBuff      hptr
    SCI_options     SpellCheckOptions
    SCI_numChars    dword
    SCI_replyObj    optr
SpellCheckInfo ends
```

SCI_ICbuff Handle of buffer to pass spell check library.
SCI_options **SpellCheckOptions** in use for this operation.
SCI_numChars Number of characters to check (if *options* has SCO_CHECK_NUM_CHARS set).
SCI_replyObj Object to send MSG_SPELL_CHECK_COMPLETED.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Intercept to find out the nature of the spell checking operation.

Library: **Objects\vTextC.def**

Messages

The following messages are defined specifically within VisTextClass. (They are not imported from MetaClass). Most of these messages may also be used by GenText objects. The compatibility of these messages with VisText and GenText objects is pointed out in the “Destination:” field.

MSG_VIS_TEXT_ADD_PARA_ATTR

This message adds a given set of paragraph attributes passed in the **VisTextParaAttr** buffer to the paragraph “run” for that text object. The text

object must not be in "default" paragraph attribute mode; it must already be storing runs of **VisTextParaAttr** structures.

Source: Unrestricted.

Destination: Any Text object that is storing runs of **VisTextParaAttr** structures.

Pass:
ss:bp	Pointer to a **VisTextParaAttr** buffer.
dx	Size of **VisTextParaAttr** buffer.

Returns:
bp	Unchanged.

Destroyed: **cx, dx**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_APPEND_BLOCK

This message appends text to a text object; the text is added at the end of the current text. None of the previous text is changed.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass:
dx	Handle of the text block.
cx	The length of the text in characters or zero, if null-terminated.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_APPEND_DB_ITEM

This message appends text to a text object; the text is added at the end of the current text. None of the previous text is changed. The text is assumed to be null-terminated.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass:
dx	The handle of the VM file containing the text.
bp	The database group containing the text.
cx	The database item.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_APPEND_HUGE_ARRAY

This message appends text to a text object; the text is added at the end of the current text. None of the previous text is changed.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass:

dx	The VM file containing the huge array.
bp	The block handle of the huge array.
cx	The length of the text in characters or zero, if null-terminated.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_APPEND_OPTR

This message adds text to a text object; the text is added at the end of the current text. None of the previous text is changed.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass:

dx:bp	The optr of the text chunk.
cx	The length of the text in characters or zero, if null-terminated.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_APPEND_PTR

This message appends text to a text object; the text is added at the end of the current text. None of the previous text is changed.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass:

dx:bp	The pointer to the character string.
cx	The length of the text in characters or zero, if null-terminated.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_APPEND_VM_BLOCK

This message appends text to a text object; the text is added at the end of the current text. None of the previous text is changed.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass:

dx	Handle of the VM file containing the text.
bp	Handle of the block within the VM file.
cx	The length of the text in characters or zero, if null-terminated.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\vTextC.def**

■ MSG_VIS_TEXT_DELETE_ALL

This message deletes the entire contents of a text object's text chunk. The chunk will be resized to zero.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass: None.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

■ MSG_VIS_TEXT_DELETE_SELECTION

This message deletes the currently selected text. The text after the deletion will be automatically repositioned, and the text chunk will be resized.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass: None.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

■ MSG_VIS_TEXT_DO_KEY_FUNCTION

This message allows the text object to mimic certain text operations that normally are the result of user keystrokes.

Note: the **VisTextKeyFunction** types VTKF_SELECT_LINE, VTKF_SELECT_PARAGRAPH, and VTKF_SELECT_OBJECT will be supported in an upcoming release.

Source: Unrestricted.

Destination: Any text object

Pass: `cx` **VisTextKeyFunction** to perform.
`bp` Data for that key function (if needed).

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_ENTER_INSERT_MODE

This message enters the text object into insert mode (clearing the VTS_OVERSTRIKE_MODE flag).

Source: Unrestricted.

Destination: Any text object.

Pass: `cx` Set if this message was called internally by the text object.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_ENTER_OVERSTRIKE_MODE

This message enters the text object into overstrike mode (setting the VTS_OVERSTRIKE_MODE flag).

Source: Unrestricted.

Destination: Any text object.

Pass: `cx` Set if this message was called internally by the text object.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

Object Assembly Reference

MSG_VIS_TEXT_FILTER_VIA_CHARACTER

This message allows the text object to filter text on a character by character basis. This message is called for each character that is entered into the text object.

This message will be generated if the text object includes ATTR_VIS_TEXT_EXTENDED_FILTER in its instance data, with its value set to the **VisTextExtendedFilterType** VTEFT_CHARACTER_LEVELER_LEVEL.

Source: Sent by the system if text object contains ATTR_VIS_TEXT_EXTENDED_FILTER set to VTEFT_CHARACTER_LEVELER_LEVEL.

Destination: Any text object

Pass:

`cx`	Character value that is being filtered. You can pass this value on to the superclass or filter it out.

Returns:

`cx`	Replacement character, or zero to reject the replacement.

Destroyed: `ax, dx, bp`

Interception: Must intercept to provide character by character filtering.

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_GET_ALL_BLOCK

This message retrieves the entire text of a text object and copies it into the passed data block.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass:

`dx`	Handle of the memory block into which the text will be copied. Pass a null handle to have the message allocate a new block.

Returns:

`cx`	The handle of the resized (or new) block. The block will always exist upon return and will contain at least the terminating null character.
`ax`	String length, not counting the null character.
`dx, bp`	Unchanged.

Destroyed: `Nothing.`

Library: **Objects\vTextC.def**

■ MSG_VIS_TEXT_GET_ALL_DB_ITEM

This message retrieves the entire text of a text object and copies it into the passed database item.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass:	**dx**	The VM file handle of the VM file.
	bp	The database group number (may be DB_UNGROUPED—if so, a new group will be returned as part of the return value).
	cx	The item number of the database item into which the text will be copied. Pass zero to allocate a new database item in the specified *group*.
Returns:	**cx**	The DB item of the chunk in which the text was placed, representing the resized (or new) DB item into which the text was copied. The item will always exist (assuming a valid VM file was specified) upon return, and it will contain at least the terminating null character.
	bp	The DB group.
	ax	The string length (not counting the null-terminator).
	dx	Unchanged.

Destroyed: **Nothing.**

Library: **Objects\vTextC.def**

■ MSG_VIS_TEXT_GET_ALL_HUGE_ARRAY

This message retrieves the entire text of a text object and copies it into the passed huge array block.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass:	**dx**	The VM file handle of the VM file containing the huge array.
	bp	The VM block handle of the huge array. Pass a null handle to have the message allocate a new huge array.
Returns:	**cx**	The VM block handle of the first block of the resized (or new) huge array. The huge array will always exist upon return

Object Assembly Reference

(assuming a valid VM file was specified), and it will contain at least the terminating null character.

dx:ax	String length (not counting the null-terminator).
bp	Unchanged.

Destroyed: **Nothing.**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_GET_ALL_OPTR

This message retrieves the entire text of a text object and copies it into the chunk specified by *o*. The text within the text object is unchanged.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass:	**dx:bp**	The optr of the chunk into which the text will be copied. Pass a valid memory handle with a null chunk handle to have the message allocate a new chunk.
Returns:	**cx**	The chunk handle of the resized (or new) chunk. The chunk will always exist upon return and will contain at least the terminating null character.
	ax	String length (not counting the null-terminator).
	dx, bp	Unchanged.

Destroyed: **Nothing.**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_GET_ALL_PTR

This message retrieves the entire text of a text object and copies it to the buffer passed. The text within the text object is unchanged.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass:	**dx:bp**	The pointer to a locked or fixed buffer. The buffer must be large enough to accommodate all the text.
Returns:	**cx**	The length of the null-terminated string not counting the null character.
	dx:bp	(Unchanged) The pointer to the text buffer.

Destroyed: **ax**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_GET_ALL_VM_BLOCK

This message retrieves the entire text of a text object and copies it into the passed VM block.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass:
- **dx** — The VM file handle of the VM file.
- **bp** — The VM block handle of the VM block into which the text will be copied. Pass a null handle to have the message allocate a new VM block in the file.

Returns:
- **cx** — The VM block handle of the resized (or new) VM block. The VM block will always exist (assuming a valid VM file was passed), and it will contain at least the terminating null character.
- **ax** — String length (not counting the null-terminator).
- **dx, bp** — Unchanged.

Destroyed: **Nothing.**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_GET_AVERAGE_CHAR_WIDTH

This message returns the average character width for the first font/character attribute run.

Source: Unrestricted.

Destination: Any text object.

Returns: Average character width (in points).

Pass: Nothing.

Returns:
- **cx** — Average character width.

Destroyed: **ax, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_GET_CHAR_ATTR

This message returns a buffer filled in with the **VisTextCharAttr** attributes of the given range of text.

This message passes a pointer to a **VisTextGetAttrParams** structure, shown below:

Structures:

```
VisTextGetAttrParams    struct
      VTGAP_range             VisTextRange
      VTGAP_attr              fptr
      VTGAP_return            fptr
      VTGAP_flags             VisTextGetAttrFlags
VisTextGetAttrParams    ends
```

VTGAP_range stores the **VisTextRange** of text for which we are retrieving the character attributes.

VTGAP_attr stores a pointer to a **VisTextCharAttr** structure. If VTGAF_MERGE_WITH_PASSED is passed in *VTGAP_flags*, then the passed **VisTextCharAttr** structure will be merged with the range of text that this message is sent to. (If this flag is not passed, any information initially in that buffer will be ignored.)

VTGAP_return stores a pointer to a **VisTextCharAttrDiffs** structure. Over the passed range, different characters may exhibit different attributes; a pointer to a **VisTextCharAttrDiffs** structure is passed to store information about attributes that are different across the range of text. This structure show below) contains a list of **VisTextCharAttrFlags** specifying what attributes are multiply present. The structure also contains the bitfields of the **TextStyle** and **VisTextExtendedStyles** present over the range of text.

```
VisTextCharAttrDiffs    struct
      VTCAD_diffs             VisTextCharAttrFlags
      VTCAD_extendedStyles    VisTextExtendedStyles
      VTCAD_textStyles        TextStyle
      even
VisTextCharAttrDiffs    ends
```

Possible **VisTextCharAttrFlags**:

VTCAF_MULTIPLE_FONT_IDS
VTCAF_MULTIPLE_POINT_SIZES
VTCAF_MULTIPLE_COLORS
VTCAF_MULTIPLE_GRAY_SCREENS
VTCAF_MULTIPLE_PATTERNS
VTCAF_MULTIPLE_TRACK_KERNINGS
VTCAF_MULTIPLE_FONT_WEIGHTS
VTCAF_MULTIPLE_FONT_WIDTHS
VTCAF_MULTIPLE_BG_COLORS
VTCAF_MULTIPLE_BG_GRAY_SCREENS



VTCAF_MULTIPLE_BG_PATTERNS
VTCAF_MULTIPLE_STYLES

VTGAP_flags stores the **VisTextGetAttrFlags**. Only one flag, VTGAF_MERGE_WITH_PASSED is defined, and that flag is explained above.

Source: Unrestricted.

Destination: Any text object.

Pass: `ss:bp` Pointer to a **VisTextGetAttrParams** structure.
`dx` Size of **VisTextGetAttrParams** structure.

Returns: `ax` The token of the specific character attribute run (word value) if the text object is storing runs of **VisTextCharAttr** structures. The *VTGAP_attrs* and *VTGAP_return* buffers are also filled with their relevant information.
`bp` Unchanged.

Destroyed: `dx`

Library: **Objects\vTextC.def**

■ MSG_VIS_TEXT_GET_FEATURES

This message retrieves the text object's **VisTextFeatures** (*VTI_features*).

Source: Unrestricted.

Destination: Any text object.

Pass: Nothing.

Returns: `cx` **VisTextFeatures** in use by the text object.

Destroyed: `ax, dx, bp`

Library: **Objects\vTextC.def**

Object Assembly Reference

MSG_VIS_TEXT_GET_FILTER

This message retrieves the current **VisTextFilters** in use by the text object (stored in *VTI_filters*).

Source: Unrestricted.

Destination: Any text object.

Pass: Nothing.

Returns: `cl` **VisTextFilters** in use by the text object.

Destroyed: `ax, ch, dx, bp`

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_GET_LINE_HEIGHT

This message returns the height of a line (in points) for a single-line text object *only*. The text object must also not reside within a document object.

Source: Unrestricted.

Destination: Single-line text objects that are not within document objects.

Pass: Nothing.

Returns: `ax` Height of line (in points).

Destroyed: cx, dx, bp

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_GET_LR_MARGIN

This message returns the current left/right margins. Both margins are the same and are thus represented by the same returned value.

Source: Unrestricted.

Destination: **VisTextClass** objects (not GenText objects).

Pass: Nothing.

Returns: `cl` The current left/right margin.

Destroyed: `ax, ch, dx, bp`

Library: **Objects\vTextC.def**



MSG_VIS_TEXT_GET_MAX_LENGTH

This message returns the maximum length of text in the text object (stored in *VTI_maxLength*).

Source: Unrestricted.

Destination: Any text object.

Pass: Nothing.

Returns: **cx** Maximum length of the text, in characters.

Destroyed: **ax, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_GET_MIN_WIDTH

This message returns the minimum width that can be supported for attributes of the text object.

Source: Unrestricted.

Destination: Any text object.

Pass: Nothing.

Returns: **cx** Minimum width (in points) that can be supported.

Destroyed: **ax, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_GET_ONE_LINE_WIDTH

This message calculates the width of a single-line text object. This width is calculated using the first encountered character attributes.

Source: Unrestricted.

Destination: Any single-line text object.

Pass: **cx** Number of characters (from start of text) to use in the calculation.

Returns: **cx** Width of the text.

Destroyed: **ax, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_GET_OUTPUT

This message returns the current destination (output) for actions taken by the text object.

Source: Unrestricted.

Destination: Any text object.

Pass: Nothing.

Returns: **cx:dx** Optr of the object's destination (in the *VTI_output* field).

Destroyed: **ax, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_GET_PARA_ATTR

This message returns a buffer filled in with the **VisTextParaAttr** attributes of the given range of text.

This message passes a pointer to a **VisTextGetAttrParams** structure, shown below:

Structures:

```
VisTextGetAttrParams    struct
      VTGAP_range             VisTextRange
      VTGAP_attr              fptr
      VTGAP_return            fptr
      VTGAP_flags             VisTextGetAttrFlags
VisTextGetAttrParams    ends
```

VTGAP_range stores the **VisTextRange** of text for which we are retrieving the paragraph attributes.

VTGAP_attr stores a pointer to a **VisTextParaAttr** structure. If VTGAF_MERGE_WITH_PASSED is passed in *VTGAP_flags*, then the passed **VisTextParaAttr** structure will be merged with the range of text that this message is sent to. (If this flag is not passed, any information initially in that buffer will be ignored.)

VTGAP_return stores a pointer to a **VisTextParaAttrDiffs** structure. Over the passed range, different paragraphs may exhibit different attributes; a pointer to a **VisTextParaAttrDiffs** structure is passed to store information about attributes that are different across the range of text. This structure show below) contains a list of **VisTextParaAttrFlags** specifying what

attributes are multiply present. The structure also contains the bitfields of the **TextStyle** and **VisTextExtendedStyles** present over the range of text.

```
VisTextParaAttrDiffs    struct
VTPAD_diffs             VisTextParaAttrFlags
VTPAD_diffs2            VisTextParaAttrFlags2
VTPAD_borderDiffs       VisTextParaAttrBorderFlags
VTPAD_attributes        VisTextParaAttrAttributes
VTPAD_hyphenationInfo   VisTextHyphenationInfo
VTPAD_keepInfo          VisTextKeepInfo
VTPAD_dropCapInfo       VisTextDropCapInfo
even
VisTextCharAttrDiffs    ends
```

Possible **VisTextParaAttrFlags**:

VTPAF_MULTIPLE_LEFT_MARGINS
VTPAF_MULTIPLE_RIGHT_MARGINS
VTPAF_MULTIPLE_PARA_MARGINS
VTPAF_MULTIPLE_LINE_SPACINGS
VTPAF_MULTIPLE_DEFAULT_TABS
VTPAF_MULTIPLE_TOP_SPACING
VTPAF_MULTIPLE_BOTTOM_SPACING
VTPAF_MULTIPLE_LEADINGS
VTPAF_MULTIPLE_BG_COLORS
VTPAF_MULTIPLE_BG_GRAY_SCREENS
VTPAF_MULTIPLE_BG_PATTERNS
VTPAF_MULTIPLE_TAB_LISTS
VTPAF_MULTIPLE_STYLES
VTPAF_MULTIPLE_PREPEND_CHARS
VTPAF_MULTIPLE_STARTING_PARA_NUMBERS
VTPAF_MULTIPLE_NEXT_STYLES

Possible **VisTextParaAttrFlags2**:

VTPAF2_MULTIPLE_LANGUAGES
VTPAF2_MULTIPLE_JUSTIFICATION_TYPES

Possible **VisTextParaAttrBorderFlags**:

VTPABF_MULTIPLE_BORDER_LEFT
VTPABF_MULTIPLE_BORDER_TOP
VTPABF_MULTIPLE_BORDER_RIGHT
VTPABF_MULTIPLE_BORDER_BOTTOM
VTPABF_MULTIPLE_BORDER_DOUBLES
VTPABF_MULTIPLE_BORDER_DRAW_INNERS

Object Assembly Reference

VTPABF_MULTIPLE_BORDER_ANCHORS
VTPABF_MULTIPLE_BORDER_WIDTHS
VTPABF_MULTIPLE_BORDER_SPACINGS
VTPABF_MULTIPLE_BORDER_SHADOWS
VTPABF_MULTIPLE_BORDER_COLORS
VTPABF_MULTIPLE_BORDER_GRAY_SCREENS
VTPABF_MULTIPLE_BORDER_PATTERNS

VTGAP_flags stores the **VisTextGetAttrFlags**. Only one flag, VTGAF_MERGE_WITH_PASSED, is defined, and that flag is explained above.

Source: Unrestricted.

Destination: Any text object.

Pass:	**ss:bp**	Pointer to a **VisTextGetAttrParams** structure.
	dx	Size of **VisTextGetAttrParams** structure.
Returns:	**ax**	The token of the specific paragraph attribute run (word value) if the text object is storing runs of **VisTextParaAttr** structures. The *VTGAP_attrs* and *VTGAP_return* buffers are also filled with their relevant information.
	bp	Unchanged.
Destroyed:	**dx**	
Library:	**Objects\vTextC.def**	

MSG_VIS_TEXT_GET_RANGE

This message fills in a **VisTextRange** buffer based on the selection criteria passed in **cx** . The **VisTextRange** buffer must be initially filled in (with *VTR_start* as 0 and *VTR_end* as TEXT_ADDRESS_PAST_END). The context information specifies whether the range will be used for a character attribute change (in which case the normal selection positions will be used) or whether

the range will be used for a paragraph attribute change (in which case the boundaries of the selection will be kicked out to paragraph boundaries).

Source: Unrestricted.

Destination: Any text object.

Pass:	`dx:bp`	**VisTextRange** buffer. This buffer must be initially filled in with *VTR_start* as 0 and *VTR_end* as TEXT_ADDRESS_PAST_END.
	`cx`	**VisTextRangeContext**.
Returns:	`dx:bp`	The **VisTextRange** buffer filled in.
Destroyed:	`ax, cx`	
Library:	**Objects\vTextC.def**	

■ MSG_VIS_TEXT_GET_SELECTION_BLOCK

This message retrieves the currently selected text and copies it into the passed data block. The text remains selected and intact in the original text object.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass:	`dx`	Handle of the text block to place the text into. Pass a null handle to have the message allocate a new memory block.
Returns:	`cx`	The memory handle of the resized (or new) block containing the selected text. The block will contain at least the terminating null character.
	`ax`	String length (not counting the null-terminator).
	`dx, bp`	Unchanged.
Destroyed:	`Nothing.`	
Library:	**Objects\vTextC.def**	

Object Assembly Reference

■ MSG_VIS_TEXT_GET_SELECTION_DB_ITEM

This message retrieves the currently selected text and copies it into the given database item. The text remains selected and intact in the original text object.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass:

dx	The VM file handle of the VM file.
bp	The group number of the passed item (if DB_UNGROUPED, a group will be determined). Pass zero to have the message allocate a new database item in the specified group and file.

Returns:

cx	The DB item representing the resized (or new) database item. The item will contain at least the terminating null character.
bp	The DB group (which may be different if DB_UNGROUPED was passed above).
ax	String length (not counting the null-terminator).
dx	Unchanged.

Destroyed: **Nothing.**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_GET_SELECTION_HUGE_ARRAY

This message retrieves the currently selected text and copies it into the passed huge array. The text remains selected and intact in the original text object.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass:	**dx**	The VM file handle of the VM file.
	bp	The VM block handle of the huge array into which the text will be copied. Pass a null handle to have the message allocate a new huge array in the passed VM file.
Returns:	**cx**	The VM block handle of the resized (or new) huge array block. The huge array will contain at least the terminating null character.
	dx:ax	String length (not counting the null-terminator).
	bp	Unchanged.

Destroyed: **Nothing.**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_GET_SELECTION_OPTR

This message retrieves the currently selected text and copies it into the passed chunk. The text remains selected and intact in the original text object.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass:	**dx:bp**	The optr of the text chunk into which the text will be copied. Pass a valid memory handle and a null chunk handle to have the message allocate a new chunk in the give block.
Returns:	**cx**	The chunk handle of the resized (or new) chunk containing the text. The chunk will contain at least the terminating null character.
	ax	String length (not counting the null-terminator).
	dx, bp	Unchanged.

Destroyed: **Nothing.**

Library: **Objects\vTextC.def**

Object Assembly Reference

MSG_VIS_TEXT_GET_SELECTION_PTR

This message retrieves the currently selected text and stores it in the character string referenced by the passed pointer. The text remains selected and intact in the original text object. The passed buffer must be large enough to accommodate the text.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass:	**dx:bp**	The pointer to the character string.
Returns:	**cx**	The length of the string not including the terminating null character.
	dx:bp	Unchanged. The buffer will contain the null-terminated character string.

Destroyed: **ax**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_GET_SELECTION_RANGE

This message returns the range of the text object's current selection. You must pass this message a **VisTextRange** buffer for the message to fill in with the selection range.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass:	**dx:bp**	A pointer to a **VisTextRange** structure to fill in with the selection range.
Returns:	**dx:bp**	Unchanged. The **VisTextRange** buffer will be filled in.

Destroyed: **ax, cx**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_GET_SELECTION_VM_BLOCK

This message retrieves the currently selected text and copies it into the passed VM block. The text remains selected and intact in the original text object.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass:

dx	The VM file handle of the VM file.
bp	The VM block handle of the VM block into which the text will be copied. Pass a null block handle to have the message allocate a new VM block in the file.

Returns:

cx	The VM block handle of the resized (or new) VM block. The VM block will contain at least the terminating null character.
ax	String length (not counting the null-terminator).
dx, bp	Unchanged.

Destroyed: **Nothing.**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_GET_SIMPLE_MIN_WIDTH

This message returns the minimum width that can be supported for the text object's current text, font and character attributes. This message assumes that the caller is not worried about keeping a width for the largest border possible.

Source: Unrestricted.

Destination: Any text object.

Pass: Nothing.

Returns:

cx	Minimum width which can be supported.

Destroyed: **ax, dx, bp**

Library: **Objects\vTextC.def**

Object Assembly Reference

MSG_VIS_TEXT_GET_STATE

This message retrieves the text object's **VisTextStates** (*VTI_state*).

Source: Unrestricted.

Destination: Any text object.

Pass: Nothing.

Returns: `cl` **VisTextStates** in use by the text object.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_GET_TEXT_SIZE

This message returns the current size of text.

Source: Unrestricted.

Destination: Any text object.

Returns: `dx:ax` The size of the text within the text object.

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_GET_USER_MODIFIED_STATE

This message returns the state of the text object's VTS_MODIFIED flag in its *VTI_state* entry.

Source: Unrestricted.

Destination: Any text object.

Pass: Nothing.

Returns: `cx` Non-zero if object has been modified.
`ax, dx, bp` Unchanged.

Destroyed: `Nothing.`

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_LOAD_FROM_DB_ITEM

This message loads text saved with MSG_VIS_TEXT_SAVE_TO_DB_ITEM.

Source: Unrestricted.

Destination: Any text object.

Pass: `cx.dx` **DBItem** to load.
`bp` Handle of the VM file to use. If zero, the VM file in *VTI_vmFile* will be used.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_LOAD_FROM_DB_ITEM_WITH_STYLES

This message loads text and style sheets saved with MSG_VIS_TEXT_SAVE_TO_DB_ITEM_WITH_STYLES.

Source: Unrestricted.

Destination: Any text object.

Pass: `ss:bp` Pointer to a **VisTextLoadFromDBWithStylesParams** structure.

```
VisTextLoadFromDBWithStylesParams   struct
        VTLFDBWSP_params  fptr.StyleSheetParams
        VTLFDBWSP_dbItem  dword
        VTLFDBWSP_file    hptr
VisTextLoadFromDBWithStylesParams   ends
```

VTLFDBWSP_file
Handle of the VM file to use. If zero, the VM file in *VTI_vmFile* will be used.

VTLFDBWSP_dbItem
DBItem to load.

VTLFDBWSP_params
Pointer to a **StyleSheetParams** structure.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_MODIFY_EDITABLE_SELECTABLE

This message modifies the text object's VTS_EDITABLE and VTS_SELECTABLE states. These are the only **VisTextStates** you should modify with this message.

Source: Unrestricted.

Destination: Any text object.

Pass:

`cl`	The VTS_EDITABLE and/or VTS_SELECTABLE states to set.
`ch`	The VTS_EDITABLE and/or VTS_SELECTABLE states to clear.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_RECALC_AND_DRAW

This message recalculates and redraws a text object, usually after constraints on either its width or its height is made without its knowledge.

Source: Unrestricted.

Destination: Any text object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_REPLACE_ALL_BLOCK

This message replaces the text string within a text object (either a VisText or a GenText) with the text within the passed data block.

Source: Unrestricted.

Destination: Any GenText or VisText object.

Pass:

dx	The handle of the data block containing the text to use in the replacement operation.
cx	The length of the text or zero if null-terminated.

Returns: Nothing. The text object's text is replaced.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_REPLACE_ALL_DB_ITEM

This message replaces the text string within a text object (either a VisText or a GenText) with the text within the passed database item. The text is assumed to be null-terminated.

Source: Unrestricted.

Destination: Any GenText or VisText object.

Pass:

dx	The handle of the database item's associated VM file.
bp	The database group containing the text.
cx	The DB item containing the text.

Returns: Nothing. The text object's text is replaced.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_REPLACE_ALL_HUGE_ARRAY

This message replaces the text string within a text object (either a VisText or a GenText) with the text within the passed HugeArray.

Source: Unrestricted.

Destination: Any GenText or VisText object.

Pass:

dx	The handle of the huge array's associated VM file.
bp	The VM block handle of the huge array directory block.
cx	The length of the text or zero if null-terminated.

Returns: Nothing. The text object's text is replaced.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_REPLACE_ALL_OPTR

This message replaces the text string within a text object (either a VisText or a GenText) with the text referenced by the passed optr.

Source: Unrestricted.

Destination: Any GenText or VisText object.

Pass:

dx:bp	The optr of the chunk containing the text.
cx	Length of the text string or zero if null-terminated.

Returns: Nothing. The Text object's text is replaced.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_REPLACE_ALL_PTR

This message replaces the text string within a text object (either a VisText or a GenText object) with the text referenced by the passed pointer.

Pass:

dx:bp	A pointer to a text string.
cx	The text length (in characters) or zero if null-terminated.

Returns: Nothing. The Text object's text is replaced.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_REPLACE_ALL_VM_BLOCK

This message replaces the text string within a text object (either a VisText or a GenText) with the text within the passed data block.

Source: Unrestricted.

Destination: Any GenText or VisText object.

Pass:

dx	The handle of the VM file containing the text.
bp	The VM block handle of the block within the VM file.
cx	The length of the text or zero if null-terminated.

Returns: Nothing. The text object's text is replaced.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_REPLACE_SELECTION_BLOCK

This message replaces the current selection within a text object with the text contained in the passed block. If no text is currently selected, the text will be inserted at the current cursor position.

Source: Unrestricted.

Destination: Any GenText or VisText object.

Pass:

dx	Handle of the text block.
cx	The length of the text in characters or zero, if null-terminated.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_REPLACE_SELECTION_DB_ITEM

This message replaces the current selection within a text object with the text contained in the passed database item. If no text is currently selected, the

text will be inserted at the current cursor position. The text is assumed to be null-terminated.

Source: Unrestricted.

Destination: Any GenText or VisText object.

Pass:

dx	The handle of the VM file containing the text.
bp	The database group containing the text.
cx	The database item.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_REPLACE_SELECTION_HUGE_ARRAY

This message replaces the current selection within a text object with the text contained in the passed huge array. If no text is currently selected, the text will be inserted at the current cursor position.

Source: Unrestricted.

Destination: Any GenText or VisText object.

Pass:

dx	The VM file containing the huge array.
bp	The VM block handle of the huge array directory block.
cx	The length of the text in characters or zero, if null-terminated.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_REPLACE_SELECTION_OPTR

This message replaces the current selection within a text object with the text in the chunk specified by **dx:bp**. If no text is currently selected, the text will be inserted at the current cursor position.

Source: Unrestricted.

Destination: Any GenText or VisText object.

Pass:

dx:bp	The optr of the text chunk.
cx	The length of the text in characters or zero, if null-terminated.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_REPLACE_SELECTION_PTR

This message replaces the current selection within a text object with the text contained in the passed string pointer. If no text is currently selected, the text will be inserted at the current cursor position.

Source: Unrestricted.

Destination: Any GenText or VisText object.

Pass:

dx:bp	The pointer to the character string.
cx	The length of the text in characters or zero, if null-terminated.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_REPLACE_SELECTION_VM_BLOCK

This message replaces the current selection within a text object by the text contained in the passed VM block. If no text is currently selected, the text will be inserted at the current cursor position.

Source: Unrestricted.

Destination: Any GenText or VisText object.

Pass:

dx	Handle of the VM file containing the text.
bp	Handle of the block within the VM file.
cx	The length of the text in characters or zero, if null-terminated.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_REPLACE_TEXT

This message replaces the text string within a text object (either a VisText or a GenText object) with the text referenced within the passed structure.

Source: Unrestricted.

Destination: Any GenText or VisText object.

Pass:

ss:bp	A pointer to a **VisTextReplaceParameters** structure on the stack.

Returns:

CF	Set if an error occurred; clear if the Text object's text is replaced.
cx, dx, bp	Unchanged.

Destroyed: **ax**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_SAVE_TO_DB_ITEM

This message saves information about the current state of the text object. Data saved with this message can be retrieved with MSG_VIS_TEXT_LOAD_FROM_DB_ITEM.

Source: Unrestricted.

Destination: Any text object.

Pass: `cx.dx` DB group and item to store the text information. If this is left zero, the info will be saved to the VM file specified in *VTI_vmFile*.

`bp` **VisTextSaveDBFlags** specifying what aspect of the text's info to save.

Returns: `cx.dx` **DBItem** of saved text info.

Destroyed: `ax, bp`

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_SAVE_TO_DB_ITEM_WITH_STYLES

This message saves information about the current state of the text object, passing a **StyleSheetParams** structure.

VTST_RUNS_ONLY should be passed in the **VisTextSaveDBFlags** entries VTSDBF_CHAR_ATTR and VTSDBF_PARA_ATTR. Do not set VTSDBF_STYLE.

Source: Unrestricted.

Destination: Any text object.

Pass: `ss:bp` Pointer to a **VisTextSaveToDBWithStylesParams** structure.

```
VisTextSaveToDBWithStylesParams     struct
        VTSTDBWSP_params
                fptr.VisTextSaveStyleSheetParams
        VTSTDBWSP_dbItem        dword
        VTSTDBWSP_flags         VisTextSaveDBFlags
        VTSTDBWSP_xferFile      word
VisTextLoadFromDBWithStylesParams   ends
```

VTSTDBWSP_params
Pointer to a **VisTextSaveStyleSheetParams** structure.

VTSTDBWSP_dbItem
DBItem DB group and item to store the text information. If

this is left zero, the info will be saved to the VM file specified in *VTI_vmFile*.

VTSTDBWSP_flags
VisTextSaveToDBFlags specifying what aspect of the text's info to save.

VTSTDBWSP_xferFile
If non-zero, the **StyleSheetParams** structure is not initialized and this is the file to put the style sheet arrays into.

```
VisTextSaveStyleSheetParams   struct
        VTSSSP_common                   StyleSheetParams
        VTSSSP_graphicsElements         word
        VTSSSP_treeBlock                word
        VTSSSP_graphicTreeOffset        word
VisTextSaveStyleSheetParams   ends
```

VTSSSP_common stores a **StyleSheetParams** structure.

VTSSSP_graphicsElements stores the VM block of the graphics elements.

VTSSSP_treeBlock stores the segment address of the graphic element within the VM block.

VTSSSP_graphicTreeOffset stores the offset of the graphic element within that segment.

Returns: `cx.dx` **DBItem** saved.

Destroyed: `ax, bp`

Library: **Objects\vTextC.def**

■ MSG_VIS_TEXT_SELECT_ALL

This message selects the entire text of a text object as its selection.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_SELECT_END

This message places the cursor at the end of the text. Any current selection will be deselected.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_SELECT_RANGE

This message creates a selection for a text object. The message must pass the starting and ending character positions to mark as the text object's selection. Any previous selection will be deselected. Note that this message uses dword offsets into the text, and can therefore be used with LARGE model text objects.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass:

`ss:bp`	**VisTextRange** to select.
`dx`	Size of the **VisTextRange**.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_SELECT_RANGE_SMALL

This message creates a selection for a text object. The message must pass the starting and ending character positions to mark as the text object's selection.

Any previous selection will be deselected.This message will not work with LARGE model text objects.

Source: Unrestricted.

Destination: Any small-model VisText or GenText object. This message will not work with LARGE model text objects.

Pass:
- **cx** — The zero-based character position of the start of the selection.
- **dx** — The zero-based character position of the end of the selection. This must be equal to or larger than the start position.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_SELECT_RELATIVE

This message moves a selection from the current cursor position to a position relative to that position.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass:
- **cx** — The number of character positions from the current cursor position to start the new selection.
- **dx** — The number of character positions from the current cursor position to end the new selection. This value must be equal to or greater than that passed in **cx**.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_SELECT_START

This message places the cursor at the start of the text. Any current selection will be deselected.

Source: Unrestricted.

Destination: Any VisText or GenText object.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\vTextC.def

MSG_VIS_TEXT_SET_CHAR_ATTR

This message sets the character attributes passed in the **VisTextCharAttr** buffer over the specified range of the text object. If the text object is in "default" character attribute mode, it will force the text object to begin storing runs of **VisTextCharAttr** structures.

Source: Unrestricted.

Destination: Any Text object.

Pass: ss:bp — Pointer to a **VisTextSetCharAttrParams** buffer. This buffer contains the range of text to change and a pointer to a **VisTextCharAttr** structure to use for the changes.

dx — Size of **VisTextSetCharAttrParams** buffer.

```
VisTextSetCharAttrParamsstruct
        VTSCAP_range        VisTextRange
        VTSCAP_charAttr     fptr.VisTextCharAttr
VisTextSetCharAttrParamsends
```

Returns: bp — Unchanged.

Destroyed: ax, cx, dx

Library: Objects\vTextC.def

MSG_VIS_TEXT_SET_CHAR_ATTR_BY_DEFAULT

This message sets the character attributes passed in **VisTextDefaultCharAttr** over the specified range of the text object. If the text object is not in "default" character attribute mode (i.e. it is storing runs

of **VisTextCharAttr** structures) it will translate the default attributes into their matching **VisTextCharAttr** attributes.

Source: Unrestricted.

Destination: Any Text object.

Pass: `ss:bp` Pointer to a **VisTextSetCharAttrByDefaultParams** buffer. This buffer contains the range of text to change and a **VisTextDefaultCharAttr** record to use for the changes.
`dx` Size of **VisTextSetCharAttrByDefaultParams** buffer.

```
VisTextSetCharAttrByDefaultParams   struct
        VTSCABDP_range    VisTextRange
        VTSCABDP_charAttr VisTextDefaultCharAttr
VisTextSetCharAttrByDefaultParams   ends
```

Returns: `bp` Unchanged.

Destroyed: `ax, cx, dx`

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_SET_FEATURES

This message sets a text object's **VisTextFeatures** (*VTI_features*).

Source: Unrestricted.

Destination: Any text object.

Pass: `cx` **VisTextFeatures** to add.
`dx` **VisTextFeatures** to remove.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_SET_FILTER

This message sets the text object's **VisTextFilters** (stored in *VTI_filters*) to the passed value. Resetting this filter does not affect any text that already exists in the text object.

Source: Unrestricted.

Destination: Any text object.

Pass: `cl` **VisTextFilters** to set.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_SET_LR_MARGIN

This message sets the left/right margins of the text object. Both margins will be set to the same value. Do not use this message on GenText objects, as the specific UI is responsible for this behavior.

Source: Unrestricted.

Destination: **VisTextClass** objects (not GenText objects).

Pass: `cl` New left/right margins.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_SET_MAX_LENGTH

This message sets the maximum length of text within a text object (stored in *VTI_maxLength*). If the current text is too long, any excess text is removed.

Source: Unrestricted.

Destination: Any text object (except **VisLargeTextClass**).

Pass: `cx` Maximum length of the text object (in characters).

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_SET_NOT_USER_MODIFIED

This message clears the text object's VTS_USER_MODIFIED flag.

Source: Unrestricted.

Destination: Any text object.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\vTextC.def

MSG_VIS_TEXT_SET_OUTPUT

This message sets the destination object for actions taken by this text object.

Source: Unrestricted.

Destination: Any text object.

Pass: cx:dx — Optr of the new destination object (placed in object's *VTI_output* field).

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\vTextC.def

MSG_VIS_TEXT_SET_PARA_ATTR

This message sets the paragraph attributes passed in the **VisTextParaAttr** buffer over the specified range of the text object. If the text object is in "default" paragraph attribute mode, it will force the text object to begin storing runs of **VisTextParaAttr** structures.

Source: Unrestricted.

Destination: Any Text object.

Pass: ss:bp — Pointer to a **VisTextSetParaAttrParams** buffer. This buffer contains the range of text to change and a **VisTextParaAttr** record to use for the changes.
dx — Size of **VisTextSetParaAttrParams** buffer.

```
VisTextSetParaAttrParams            struct
      VTSPAP_range      VisTextRange
      VTSPAP_paraAttr   fptr.VisTextParaAttr
VisTextSetParaAttrParams            ends
```

Returns: **bp** Unchanged.

Destroyed: **ax, cx, dx**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_SET_PARA_ATTR_BY_DEFAULT

This message sets the paragraph attributes passed in **VisTextDefaultParaAttr** over the specified range of the text object. If the text object is not in "default" paragraph attribute mode (i.e. it is storing runs of **VisTextParaAttr** structures) it will translate the default attributes into their matching **VisTextParaAttr** attributes.

Source: Unrestricted.

Destination: Any Text object.

Pass: **ss:bp** Pointer to a **VisTextSetParaAttrByDefaultParams** buffer. This buffer contains the range of text to change and a **VisTextDefaultParaAttr** record to use for the changes.

dx Size of **VisTextSetParaAttrByDefaultParams** buffer.

```
VisTextSetParaAttrByDefaultParams   struct
      VTSPABDP_range    VisTextRange
      VTSPABDP_paraAttr VisTextDefaultParaAttr
VisTextSetParaAttrByDefaultParams   ends
```

Returns: **bp** Unchanged.

Destroyed: **ax, cx, dx**

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_SET_TB_MARGIN

This message sets the top/bottom margins of the text object. Both margins will be set to the same value. Do not use this message on GenText objects, as the specific UI is responsible for this behavior.

Source: Unrestricted.

Destination: **VisTextClass** objects (not GenText objects).

Pass: `cl` New top/bottom margins.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_SET_USER_MODIFIED

This message sets the text object's VTS_USER_MODIFIED flag. This message does *not* result in a MSG_META_TEXT_USER_MODIFIED notification; it merely modifies the flag.

Source: Unrestricted.

Destination: Any text object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**

MSG_VIS_TEXT_SET_VM_FILE

This message sets the text object's *VTI_vmFile* instance field to the passed VM file handle. This file handle will be used as the default for operations that require a VM file.

Source: Unrestricted.

Destination: Any text object.

Pass: `cx` VM file handle.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\vTextC.def**



Object Assembly Reference

The List Objects

11

GenBooleanClass 317

GenBooleanGroupClass 318

GenDynamicListClass 326

GenItemClass 330

GenItemGroupClass 331

GenBooleanClass

Instance Data

```
GBI_identifier                      word 0
   UIC Syntax:  identifier   = wordComp : 0;
```

identifier (GBI_identifier) sets the identifying keyword (a 16-bit mask representing a specific bit or bits within a word-length bitfield) that uniquely identifies this GenBoolean. This identifier is used by the GenBooleanGroup parent to reference the Boolean. No two Booleans may specify common bits set, although a single Boolean's identifier may have several bits set.

Messages

■ MSG_GEN_BOOLEAN_GET_IDENTIFIER

This message returns the identifier for the GenBoolean item.

Pass: Nothing.

Returns: **ax** Identifier.

Destroyed: **cx, dx, bp.**

Library: **Objects\gBoolC.def**

■ MSG_GEN_BOOLEAN_SET_IDENTIFIER

This message sets a new identifier for a GenBoolean object. No change is made to the GenBooleanGroup object. You must make sure that you are not referencing the GenBoolean with its old identifier before sending this message.

Pass: **cx** New identifier.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gBoolC.def**

GenBooleanGroupClass

Instance Data

```
GBGI_selectedBooleans            word 0
   UIC Syntax:  selectedBooleans       = wordComp : 0;
```

selectedBooleans (GBGI_selectedBooleans) is a record representing the selection state of the GenBooleanGroup's children. If a bit is set, the GenBoolean corresponding to that position is selected. Because of this field's size, no more than 16 GenBooleans may belong to a single GenBooleanGroup.

```
GBGI_indeterminateBooleans       word 0
   UIC Syntax:  indeterminateBooleans  = wordComp : 0;
```

indeterminateBooleans (GBGI_indeterminateBoolean) is a record representing the indeterminate state of the GenBooleanGroup's children. A GenBoolean is indeterminate if it cannot be considered either selected or unselected. If a bit is set, the GenBoolean corresponding to that position is indeterminate.

```
GBGI_modifiedBooleans            word 0
   UIC Syntax:  modifiedBooleans       = wordComp : 0;
```

modifiedBooleans (GBGI_modifiedBooleans) is a record representing the modified state of the GenBooleanGroup's children. A GenBoolean is modified if its state (selected, deselected, or indeterminate) has changed since the last apply. If a bit is set, the GenBoolean corresponding to that position has been marked modified.

```
GBGI_destination                 optr
   UIC Syntax:  destination            = optrComp : null;
```

destination (GBGI_destination) contains the optr of the object or process that handles messages sent out by the GenBooleanGroup. This destination object will receive the apply action stored in *GBGI_applyMsg* whenever the user changes the GenBooleanGroup and that state change is later applied.

```
GBGI_applyMsg                    word 0
   UIC Syntax:  applyMsg               = wordComp : 0;
```

applyMsg (GBGI_applyMsg) stores the apply action for the GenBooleanGroup to send out whenever a MSG_GEN_APPLY takes place, either through an immediate state change or through a delayed mode activation. The arguments passed in the notify message are:

cx Booleans currently selected or "true"

dx Booleans whose state is indeterminate

bp Booleans whose state has been modified.

The message need return nothing; it may destroy **ax**, **cx**, **dx**, and **bp**.

Variable Instance Data

`ATTR_GEN_BOOLEAN_GROUP_STATUS_MSG` `vardata word`

ATTR_GEN_BOOLEAN_GROUP_STATUS_MSG assigns a status message to your GenBooleanGroup. A status message allows objects to receive notice of a change in the BooleanGroup's current state without forcing an apply. This attribute is useful for cases where the state within your BooleanGroup may reflect information within other user interface objects and should therefore be updated whenever its user changes occur, regardless of the need to apply those changes.

`ATTR_GEN_BOOLEAN_GROUP_INIT_FILE_BOOLEAN` `vardata`

ATTR_GEN_BOOLEAN_GROUP_INIT_FILE_BOOLEAN forces the GenBooleanGroup to use "true" and "false" values in the GEOS.INI file.

`ATTR_GEN_BOOLEAN_GROUP_LINK` `vardata optr`

ATTR_GEN_BOOLEAN_GROUP_LINK allows two or more GenBooleanGroups to be linked together and act as one BooleanGroup. Each BooleanGroup may contain one or more booleans with constant values unique to all other booleans within the linked BooleanGroups. The links should also be circular, with the last group pointing back to the first. All of these BooleanGroups should have identical states so they will work correctly. Use this attribute to circumvent undesirable geometry constraints. All of the GenBooleanGroups should have identical states.

Hints

`HINT_BOOLEAN_GROUP_SCROLLABLE` `vardata`

HINT_BOOLEAN_GROUP_SCROLLABLE implements the Boolean Group as a scrollable list, if that feature is supported by the specific UI.

`HINT_BOOLEAN_GROUP_MINIMIZE_SIZE` `vardata`

HINT_BOOLEAN_GROUP_MINIMIZE_SIZE instructs the UI to minimize the size of the Boolean Group. Depending on the specific UI, the Boolean group may be placed within a popup list or made scrollable.

HINT_BOOLEAN_GROUP_CHECKBOX_STYLE vardata

HINT_BOOLEAN_GROUP_CHECKBOX_STYLE indicates that the Boolean Group should display its Booleans with the "checkbox" style, if that style is supported by the Specific UI.

HINT_BOOLEAN_GROUP_TOOLBOX_STYLE vardata

HINT_BOOLEAN_GROUP_TOOLBOX_STYLE indicates that the group should use the toolbox style, with a simple box around the Boolean's visual moniker. This style is the default for toolboxes in OSF/Motif, but not for other Booleans

Messages

■ MSG_GEN_BOOLEAN_GROUP_GET_APPLY_MSG

This message returns the current apply action in the GenBooleanGroup's *GBGI_applyMsg* instance field.

Pass: Nothing.

Returns: ax The apply message.

Destroyed: **cx, dx, bp.**

Library: **Objects\gBoolGC.def**

■ MSG_GEN_BOOLEAN_GROUP_GET_BOOLEAN_OPTR

This message returns the optr of the requested GenBoolean. You may then use this optr to alter that Boolean's state individually. If the GenBoolean with the matching identifier is not found, a null optr will be returned.

Pass: **cx** Identifier to look for.

Returns: CF Set if Boolean found.
cx:**dx** Object pointer of Boolean if found; null otherwise.

Destroyed: **ax, bp.**

Library: **Objects\gBoolGC.def**

MSG_GEN_BOOLEAN_GROUP_GET_DESTINATION

This message returns the current destination object for the BooleanGroup, as specified in the *GBGI_destination* instance field.

Pass: Nothing.

Returns: ^lcx:dx — The destination object.

Destroyed: ax, bp.

Library: Objects\gBoolGC.def

MSG_GEN_BOOLEAN_GROUP_GET_INDETERMINATE_BOOLEANS

This message returns the indeterminate items of a GenBooleanGroup by returning the contents of the *GBDI_indeterminateBooleans* instance field.

Pass: Nothing.

Returns: ax — Current indeterminate Booleans, represented by the OR-ed sum of identifiers.

Destroyed: cx, dx, bp.

Library: Objects\gBoolGC.def

MSG_GEN_BOOLEAN_GROUP_GET_MODIFIED_BOOLEANS

This message returns selections marked modified within a GenBooleanGroup since the last apply (by returning the contents of the *GBGI_modifiedBooleans* instance field).

Pass: Nothing.

Returns: ax — Modified Booleans, represented by the OR-ed sum of identifiers.
CF — Set if no Booleans modified.

Destroyed: cx, dx, bp.

Library: Objects\gBoolGC.def

MSG_GEN_BOOLEAN_GROUP_GET_SELECTED_BOOLEANS

This message returns the current state of all GenBooleans in a GenBooleanGroup (selected or unselected) by returning the contents of the *GBGI_selectedBooleans* instance field.

Pass: Nothing.

Returns:
- **ax** — Booleans which are selected or "true," represented by the OR-ed sum of identifiers.
- CF — Set if none selected.

Destroyed: **cx, dx, bp.**

Library: **Objects\gBoolGC.def**

MSG_GEN_BOOLEAN_GROUP_IS_BOOLEAN_INDETERMINATE

This message checks whether the GenBoolean with the passed identifier is indeterminate (by examining the *GBDI_indeterminateBooleans* instance field).

Pass:
- **cx** — Identifier of Boolean.

Returns:
- CF — Set if Boolean indeterminate.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gBoolGC.def**

MSG_GEN_BOOLEAN_GROUP_IS_BOOLEAN_MODIFIED

This message checks whether the GenBoolean with the passed identifier has been modified (by examining the GBGI_modifiedBooleans instance field).

Pass:
- **cx** — Identifier of Boolean.

Returns:
- CF — Set if Boolean is modified.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gBoolGC.def**

MSG_GEN_BOOLEAN_GROUP_IS_BOOLEAN_SELECTED

This message checks whether the GenBoolean with the passed identifier is selected.

Pass: `cx` Identifier of Boolean.

Returns: `CF` Set if selected.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\gBoolGC.def**

MSG_GEN_BOOLEAN_GROUP_MAKE_BOOLEAN_VISIBLE

This message, when sent to a scrollable BooleanGroup object, will make the specified GenBoolean visible in the list. If the passed identifier is unknown or is not usable, the BooleanGroup will do nothing.

Pass: `cx` Identifier of Boolean to ensure visible.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\gBoolGC.def**

MSG_GEN_BOOLEAN_GROUP_SEND_STATUS_MSG

This message causes the GenItemGroup to send out the status message to the destination object. This status message is stored in ATTR_GEN_ITEM_GROUP_STATUS_MSG.

Pass: `cx` Data to have passed in the "changed" field of the status message.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\gBoolGC.def**

MSG_GEN_BOOLEAN_GROUP_SET_APPLY_MSG

This message sets a new apply message in the GenBooleanGroup's *GBGI_applyMsg* instance field.

Pass: `cx` New message.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\gBoolGC.def**



MSG_GEN_BOOLEAN_GROUP_SET_BOOLEAN_INDETERMINATE_STATE

This message sets the GenItemGroup's GIGSF_INDETERMINATE flag in its *GIGI_stateFlags* instance field. Usually this message is used in conjunction with MSG_GEN_ITEM_GROUP_SET_MULTIPLE_SELECTIONS. (MSG_GEN_ITEM_GROUP_SET_NONE_SELECTED and MSG_GEN_ITEM_GROUP_SET_SINGLE_SELECTION set the indeterminate state themselves.)

Pass: **cx** Identifier of Boolean.
dx Non-zero to mark indeterminate, zero to mark not indeterminate.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gBoolGC.def**

MSG_GEN_BOOLEAN_GROUP_SET_BOOLEAN_MODIFIED_STATE

This message sets the modified state of an individual GenBoolean without affecting any other GenBooleans. You may mark GenBooleans modified even if they are not usable, not enabled, or not even present within the BooleanGroup.

Pass: **cx** Identifier of Boolean.
dx Non-zero to mark modified , zero to mark not modified.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gBoolGC.def**

MSG_GEN_BOOLEAN_GROUP_SET_BOOLEAN_STATE

This message sets an individual GenBoolean's selection state, leaving other GenBooleans unaffected. You may alter the selection state of individual GenBooleans even if they are not usable, not enabled, or not even within the BooleanGroup. (This is useful for linked BooleanGroups.)

Pass: **cx** Identifier of Boolean.
dx Non-zero if selected, or "true."

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gBoolGC.def**

MSG_GEN_BOOLEAN_GROUP_SET_DESTINATION

This message sets a new destination object for a BooleanGroup. The apply and status messages of this BooleanGroup will be sent to this new destination.

Pass: **cx**:**dx** Destination object.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gBoolGC.def**

MSG_GEN_BOOLEAN_GROUP_SET_GROUP_MODIFIED_STATE

GenBooleans are normally marked modified any time their state is altered, and they are marked not modified after these changes have been applied. You may also mark GenBooleans modified (or mark them nor modified) with this message. You should pass this message both the GenBooleans to set and the GenBooleans to clear (in the *GBGI_modifiedBooleans* instance field).

Pass: **cx** Bitfield of Booleans to mark modified.
dx Bitfield of Booleans to mark not modified.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gBoolGC.def**

MSG_GEN_BOOLEAN_GROUP_SET_GROUP_STATE

This message sets new selections and indeterminate states for a GenBooleanGroup (altering the contents of the *GBGI_selectedBooleans* and *GBGI_indeterminateBooleans* instance field). You should pass the records indicating the GenBooleans to select and/or mark indeterminate. Any GenBooleans which are not set will be cleared.

Pass: **cx** Booleans to mark selected, or "true," represented as an OR-ed sum of identifiers.
dx Booleans to mark unidentified, represented as an OR-ed sum of identifiers.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gBoolGC.def**

GenDynamicListClass

Instance Data

```
GDLI_numItems                              word 0
   UIC Syntax:  numItems            = wordComp : 0;
```

numItems (GDLI_numItems) stores the number of items in the dynamic list. This field should be set to the initial number of monikers needed within your GenDynamicList. If this value is initially unknown (i.e. is retrieved dynamically) then you should set this value after the number of items becomes known (using MSG_GEN_DYNAMIC_LIST_INITIALIZE). If this instance field is zero, no items will be drawn.

```
GDLI_queryMsg         word 0
   UIC Syntax:  queryMsg = wordComp : 0;
```

queryMsg (GDLI_queryMsg) stores the querying message that the dynamic list sends each time it needs to display an entry's moniker. (For example, when initially realized, a scrolling dynamic list may display only four out of 16 entries; this message will be sent out four times to retrieve the monikers for the items initially displayable.)

The query message will be handled by whatever object you set in *destination*. The handler should search a list of data for the visual monikers to display within the list. You will want to set up a custom handler for this message and the list of data to search through. Note that this list of data will serve as the GenDynamicList's "children"; there are no GenItems allowed as children of a GenDynamicList. The query message is based on the prototype GEN_DYNAMIC_LIST_QUERY_MSG.

Messages

MSG_GEN_DYNAMIC_LIST_ADD_ITEMS

This message adds an item after the position passed. All items that previously appeared at or after the position specified will have their positions incremented, and the total number of items will be incremented by the number of items added.

Usually, you will send this message after a change in the moniker data occurs. The moniker for the new item will be requested when it is made visible.

Pass: **cx** Position to give the new entry or GDLP_LAST to add at end or GDLP_FIRST to insert at beginning.
dx Number of items to add.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gDListC.def**

MSG_GEN_DYNAMIC_LIST_GET_NUM_ITEMS

This message returns the current number of items for a dynamic list. If no items are within the list, this message returns zero.

Pass: Nothing.

Returns: **cx** Number of items.

Destroyed: **ax, dx, bp.**

Library: **Objects\gDListC.def**

MSG_GEN_DYNAMIC_LIST_INITIALIZE

This message initializes a dynamic list, updating the display to reflect a new number of items or new data within the moniker list. If the list is already usable, this message invalidates all the current items and requests new monikers for all displayable items. The dynamic list will clear all previous selections; it will not scroll to any new selections. Use MSG_GEN_ITEM_GROUP_MAKE_ITEM_VISIBLE to scroll to a selected item.

This message is useful for cases when initially building a dynamic list without knowing the number of items; after you know the number of items, make sure to send the dynamic list this message.

Pass: **cx** New number of items or GDLI_NO_CHANGE.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gDListC.def**

MSG_GEN_DYNAMIC_LIST_REMOVE_ITEM_LIST

This message removes a list of items from a dynamic list. Therefore, unlike MSG_GEN_DYNAMIC_LIST_REMOVE_ITEMS, this message can remove separate items that occur at different positions in the list. The message

updates other item positions appropriately, causing a visual refresh, if necessary, and updates the total number of items.

Pass:	**cx:dx**	Buffer containing list of item positions.
	bp	Number of items to remove.
Returns:	Nothing.	
Destroyed:	**ax, cx, dx, bp.**	
Library:	**Objects\gDListC.def**	

MSG_GEN_DYNAMIC_LIST_REMOVE_ITEMS

This message removes the specified item(s) from a dynamic list at the position passed. All items that previously appeared after those deleted will have their positions decremented, and the total number of items will be decremented by the number of items removed.

Usually, you will send this message after a change in the moniker data. If the moniker for the deleted item is visible, a visual update will occur to reflect the list's new item positions. If the deleted item was selected, it will be deselected before being removed.

Pass:	**cx**	Position of first entry to remove or GDLP_LAST to remove from end or GDLP_FIRST to remove at beginning.
	dx	Number of items to remove.
Returns:	Nothing.	
Destroyed:	**ax, cx, dx, bp.**	
Library:	**Objects\gDListC.def**	

MSG_GEN_DYNAMIC_LIST_REPLACE_ITEM_MONIKER

This message sends a moniker to use for a particular list item and is usually sent in response to a moniker query by a dynamic list object. Within your query message handler (*GDLI_queryMsg*), you should access the data for the moniker to use and send off this message with the proper parameters. You may also use this message to change an item's moniker, guaranteeing that it is updated if currently visible.

If you pass this message the flag RIMF_NOT_ENABLED, the moniker for the item will be disabled (usually grayed out).

This is an all-purpose message that allows many different source types and many ways to reference the data in these sources. If you are using lists of visual monikers or simple text strings and you do not to perform any fancy operations on this data, you may use

MSG_GEN_DYNAMIC_LIST_REPLACE_ITEM_MONIKER_OPTR or MSG_GEN_DYNAMIC_LIST_REPLACE_ITEM_TEXT.

Pass: **ss:bp** **ReplaceItemMonikerFrame** structure.
dx sizeof **ReplaceItemMonikerFrame**.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gDListC.def**

MSG_GEN_DYNAMIC_LIST_REPLACE_ITEM_MONIKER_OPTR

This message is a simplified version of MSG_GEN_DYNAMIC_LIST_REPLACE_ITEM_MONIKER. The moniker must be of type VisMoniker and referenced by an optr. No special operations (such as marking the item disabled) are possible with this message. In those cases, use MSG_GEN_DYNAMIC_LIST_REPLACE_ITEM_MONIKER.

Pass: ^l**cx:dx** new VisMoniker.
bp Item number.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gDListC.def**

MSG_GEN_DYNAMIC_LIST_REPLACE_ITEM_TEXT

This message is a simplified version of MSG_GEN_DYNAMIC_LIST_REPLACE_ITEM_MONIKER. The moniker passed must be a null-terminated text string. No special operations (such as marking the item disabled) are possible with this message. In those cases, use MSG_GEN_DYNAMIC_LIST_REPLACE_ITEM_MONIKER.

Pass: **cx:dx** Far pointer to new null-terminated text string.
bp Item number.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gDListC.def**

GenItemClass

Instance Data

```
GII_identifier                          word 0
   UIC Syntax:  identifier    = wordComp : 0;
```

identifier (GII_identifier) sets the identifying keyword (usually an enumerated type) that uniquely identifies this GenItem. This identifier is used by the GenItemGroup parent to indicate the current selection and otherwise reference the item. Each GenItem identifier within the same GenItemGroup must evaluate to a unique value.

Messages

MSG_GEN_ITEM_GET_IDENTIFIER

This message returns the identifier stored in the item's *GII_identifier* instance field.

Pass: Nothing.

Returns: **ax** Identifier.

Destroyed: **cx, dx, bp.**

Library: **Objects\gItemC.def**

MSG_GEN_ITEM_SET_IDENTIFIER

This message sets a GenItem's GII_identifier instance field to a new value. There is no effect on the list itself. The item whose identifier is changed must not be currently selected. It is therefore up to the application to make sure that the item is de-selected before changing its identifier.

Pass: **cx** New identifier.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Warning: The new enumerated value should be unique among all of the items in the list. Use MSG_GEN_ITEM_GROUP_GET_UNIQUE_IDENTIFIER.

Library: **Objects\gItemC.def**

GenItemGroupClass

Instance Data

```
GIGI_behaviorType    GenItemGroupBehaviorType GIGBT_EXCLUSIVE
   UIC Syntax:  behaviorType     = enumComp byte {
                    exclusive,
                    exclusiveNone,
                    extendedSelection,
                    nonExclusive
                    } : exclusive;
```

behaviorType (GIGI_behaviorType) describes the selection behavior of the list. The default type is *exclusive*, which creates an exclusive list.

exclusive (GIGBT_EXCLUSIVE) specifies a list where only one option may be selected at a time. Once a selection has been made, making another selection will deselect the previous item. The user may not deselect a currently selected item. Many specific UIs represent these items with radio buttons; OSF/Motif represents these items with "diamond" settings. This type of list is the default for a GenItemGroup.

exclusiveNone (GIGBT_EXCLUSIVE_NONE) specifies an exclusive list also—at most one option may be selected at a time—but the user can also deselect an item, leaving no items selected. GenItemGroups show a "none-selected" state with GIGS_NONE in their *GIGI_selection* instance fields.

extendedSelection (GIGBT_EXTENDED_SELECTION) specifies a list that is normally exclusive, unless the user "extends" the selection. The user can extend the selection either by dragging the mouse through another selection or by clicking on a new selection in conjunction with a special keystroke. In OSF/Motif, clicking on an item while holding the SHIFT key will extend the selection (leaving any other selections unaffected) to the one clicked on. Extending a selection to a currently selected item will deselect that item.

nonExclusive (GIGBT_NON_EXCLUSIVE) specifies a list which does not exhibit any exclusive behavior. Any or all settings may be on at any time, and selecting any items will not affect the state of other settings. In many case, you may want to use a

GenBooleanGroup in place of a non-exclusive GenItemGroup. For example, a non-exclusive GenItemGroup cannot store individual indeterminate or modified states for its children; GenBooleanGroup stores this information.

```
GIGI_selection        word GIGS_NONE
   UIC Syntax:  selection = wordComp : "GIGS_NONE";
```

selection (GIGI_selection) contains the name of the current GenItem child that is selected. GenItemGroups use the identifying keywords stored in each GenItem's *identifier* instance field to reference those selections. If more than one item may be selected at a time (if *numSelections* is greater than one), this instance field will point to a list of GenItem identifiers rather than store a single identifier.

```
GIGI_numSelections   word 0
   UIC Syntax:  numSelections = wordComp : 0;
```

numSelections (GIGI_numSelections) contains the number of GenItems currently selected. For exclusive or exclusive-none lists, this number will be either one or zero. For multiple selection lists, this number may be greater than zero. Unless you have an item group that initially appears with more than one selection, you will not need to set this up in your object declaration.

```
GIGI_stateFlags      GenItemGroupStateFlags 0
   UIC Syntax:  stateFlags = bitFieldComp byte {
                    indeterminate, modified } : {};
```

stateFlags (GIGI_stateFlags) contains flags that affect the state of the GenItemGroup. These flags indicate whether a GenItemGroup is in an indeterminate or a modified state.

```
GIGI_destination     optr
   UIC Syntax:  destination = optrComp : null;
```

destination (GIGI_destination) contains the optr of the object or process to handle messages sent out by this GenItemGroup. This destination object will receive the notification message in the *applyMsg* instance field whenever state changes are applied within the GenItemGroup.

```
GIGI_applyMsg        word 0
   UIC Syntax:  applyMsg = wordComp : 0;
```

applyMsg (GIGI_applyMsg) stores the apply action for the GenItemGroup to send out whenever state changes should be applied, either through an immediate state change or through a delayed mode activation.

Object Assembly Reference

Variable Instance Data

ATTR_GEN_ITEM_GROUP_STATUS_MSG vardata word

ATTR_GEN_ITEM_GROUP_STATUS_MSG assigns a status message to your GenItemGroup. A status message allows objects to receive notice of a change in the GenItemGroup's current state without forcing an apply action. This attribute is useful for cases where the state of your GenItemGroup may reflect information in other user interface objects, and should therefore be updated whenever its state changes, regardless of the need to apply those changes.

ATTR_GEN_ITEM_GROUP_SET_MODIFIED_ON_REDUNDANT_SELECTION vardata

In exclusive or extended-selection lists, selecting an already-select item will not change the state of the GenItemGroup and will not mark it modified. ATTR_GEN_ITEM_GROUP_SET_MODIFIED_ON_REDUNDANT_SELECTION alters this default behavior for a GenItemGroup. Including this attribute in the object's instance data will cause the group to be marked modified whenever the user makes a redundant selection, thereby forcing an apply action to later take place. Use this attribute for operations you may wish to repeat even if the state of the GenItemGroup has not changed.

ATTR_GEN_ITEM_GROUP_CUSTOM_DOUBLE_PRESS vardata word

ATTR_GEN_ITEM_GROUP_CUSTOM_DOUBLE_PRESS allows a special activation message (stored in the word value) to be sent out on double press events; it is used only in exclusive GenBooleanGroup lists.

ATTR_GEN_ITEM_GROUP_LINK vardata optr

ATTR_GEN_ITEM_GROUP_LINK allows two or more GenItemGroups to be linked together and act as one GenItemGroup. Each GenItemGroup may contain one or more items with identifiers unique to all other items within the linked GenItemGroups. The links should be circular, with the last group pointing back to the first. All of these GenItemGroups should have identical states and behavior types so they will work correctly. Use this attribute to circumvent undesirable geometry constraints.

ATTR_GEN_ITEM_GROUP_INIT_FILE_BOOLEAN vardata

ATTR_GEN_ITEM_GROUP_INIT_FILE_BOOLEAN forces the GenItemGroup to use "true" and "false" in the .INI file rather than numerical values.

Hints

HINT_ITEM_GROUP_SCROLLABLE vardata

A scrolling list is a special version of a GenItemGroup that allows the user to scroll through a list of items which are not all visible at one time. Scrolling lists usually provide scrollbars or some other means to quickly scan through a list. You may make a GenItemGroup scrollable if the specific UI allows by including the hint HINT_ITEM_GROUP_SCROLLABLE in the list's object declaration.

`HINT_ITEM_GROUP_MINIMIZE_SIZE` `vardata`

`HINT_ITEM_GROUP_MINIMIZE_SIZE_IF_VERTICAL_SCREEN` `vardata`

HINT_ITEM_GROUP_MINIMIZE_SIZE and HINT_ITEM_GROUP_MINIMIZE_SIZE_IF_VERTICAL_SCREEN force the item group to use only the smallest space available. In OSF/Motif, for example, this would cause the item group to be a popup menu. The latter of the two hints takes effect only on screens that are taller than they are wide.

`HINT_ITEM_GROUP_RADIO_BUTTON_STYLE` `vardata`

`HINT_ITEM_GROUP_TOOLBOX_STYLE` `vardata`

HINT_ITEM_GROUP_RADIO_BUTTON_STYLE indicates that the Item Group should appear as "radio buttons" if the Specific UI allows such behavior. OSF/Motif does this by default. HINT_ITEM_GROUP_TOOLBOX_STYLE indicates that the item's should be drawn in toolbox style, with a simple box around the visual moniker.

`HINT_ITEM_GROUP_DISPLAY_CURRENT_SELECTION` `vardata`

HINT_ITEM_GROUP_DISPLAY_CURRENT_SELECTION signals that a minimized list should display the currently selected item's moniker when said list is in its minimized state.

Messages

MSG_GEN_ITEM_GROUP_GET_APPLY_MSG

This message returns the current apply message in the GenItemGroup's *GIGI_applyMsg* instance field.

Pass: Nothing.

Returns: ax The current apply message.

Destroyed: **cx, dx, bp.**

Library: **Objects\gItemGC.def**

Object Assembly Reference

MSG_GEN_ITEM_GROUP_GET_BEHAVIOR_TYPE

This message returns the current behavior type of a GenItemGroup (stored in the *GIGI_behaviorType* instance field).

Pass: Nothing.

Returns: `al` **GenItemGroupBehaviorType** value.

Destroyed: `ah, cx, dx, bp.`

Library: **Objects\gItemGC.def**

MSG_GEN_ITEM_GROUP_GET_DESTINATION

This message returns the optr of the current destination object as specified in the *GIGI_destination* instance field.

Pass: Nothing.

Returns: `cx:dx` Optr of the current destination object.

Destroyed: `ax, bp.`

Library: **Objects\gItemGC.def**

MSG_GEN_ITEM_GROUP_GET_ITEM_OPTR

This message returns the optr of a GenItem within a GenItemGroup. You may then use this optr for directly manipulating the GenItem. If an item with the requested identifier is not found as one of the GenItemGroup's children, this message will return a null optr.

Pass: `cx` Identifier to look for.

Returns: `CF` Carry set if item found.
`cx:dx` Optr of item if found; null otherwise.

Destroyed: `ax, bp.`

Library: **Objects\gItemGC.def**

MSG_GEN_ITEM_GROUP_GET_MULTIPLE_SELECTIONS

This message returns the current list of selections for non-exclusive and extended-selection GenItemGroups. The caller must allocate a buffer for the entries and pass the size of that buffer. If there is insufficient space in the passed buffer, no entries will be filled in. You should call MSG_GEN_ITEM_GET_NUM_SELECTIONS beforehand in order to allocate the correct maximum size for the buffer.

This message returns the number of selections filled into the buffer. The buffer is filled in with the proper identifiers. The list of identifiers within the buffer will not be null-terminated.

Pass:	**cx**:**dx**	Buffer to hold the selections.
	bp	Maximum number of selections to place in the buffer.
Returns:	**cx**:**dx**	Preserved; buffer is now filled in with the identifiers of the selections.
	ax	Number of selections.
Destroyed:	**bp.**	
Library:	**Objects\gItemGC.def**	

MSG_GEN_ITEM_GROUP_GET_NUM_SELECTIONS

This message returns the GenItemGroup's current number of selections (stored in *GIGI_numSelections*), or zero if there is no selection.

Pass:	Nothing.	
Returns:	**ax**	Number of selections.
Destroyed:	**cx, dx, bp.**	
Library:	**Objects\gItemGC.def**	

MSG_GEN_ITEM_GROUP_GET_SELECTION

This message returns the current selection identifier for the GenItemGroup. If there is no selection, this message returns GIGS_NONE. If there are multiple selections, this message only returns the first selection. Use MSG_GEN_ITEM_GROUP_GET_MULTIPLE_SELECTIONS for multiple selections. A selection may be returned even if the item is not usable, not enabled, or not in the GenItemGroup.

If you are using a GenItemGroup which may contain a selection of -1, you will want to call MSG_GEN_ITEM_GROUP_GET_NUM_SELECTIONS to differentiate between the item with -1 being selected and GIGS_NONE.

Pass:	Nothing.	
Returns:	**ax**	Current selection, or GIGS_NONE if no selections, or the first selection in the item group if there are multiple selections.
Destroyed:	**cx, dx, bp.**	
Library:	**Objects\gItemGC.def**	

MSG_GEN_ITEM_GROUP_GET_UNIQUE_IDENTIFIER

This message returns an identifier that is unique among the items in the GenItemGroup's group. It does not check across links, however.

Pass: Nothing.

Returns: ax — A unique identifier (not used among the generic children of this GenItemGroup).

Destroyed: cx, dx, bp.

Library: Objects\gItemGC.def

MSG_GEN_ITEM_GROUP_IS_INDETERMINATE

This message checks whether a GenItemGroup is in the indeterminate state; it checks the GIGSF_INDETERMINATE flag in the GenItemGroup's *GIGI_stateFlags*.

Pass: Nothing.

Returns: CF — Set if item group is indeterminate.

Destroyed: ax, cx, dx, bp.

Library: Objects\gItemGC.def

MSG_GEN_ITEM_GROUP_IS_ITEM_SELECTED

This message checks whether an item within a GenItemGroup is selected, even if that item is not usable, not enabled, or not within the GenItemGroup. The message returns CF set if the item is selected, CF clear if it is not selected.

Pass: cx — Identifier of item to check.

Returns: CF — Set if item is selected; clear otherwise.

Destroyed: ax, cx, dx, bp.

Library: Objects\gItemGC.def

MSG_GEN_ITEM_GROUP_IS_MODIFIED

This message checks whether a GenItemGroup has been modified by checking the GIGSF_MODIFIED flag in the GenItemGroup's *GIGI_stateFlags*.

Pass: Nothing.

Returns: CF — Set if item group is modified; clear otherwise.

Destroyed: ax, cx, dx, bp.

Library: Objects\gItemGC.def

MSG_GEN_ITEM_GROUP_MAKE_ITEM_VISIBLE

This message ensures that an item within a scrolling GenItemGroup is visible, by scrolling the list if necessary. This message has no effect if the GenItemGroup is not scrollable or is not usable.

Pass: **cx** Identifier of item to ensure visible.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gItemGC.def**

MSG_GEN_ITEM_GROUP_SEND_STATUS_MSG

This message causes the GenBooleanGroup to send out its status message stored in ATTR_GEN_ITEM_GROUP_STATUS_MSG. The message should include the current state of the GenBooleanGroup. (Base the status message on the prototype GEN_BOOLEAN_GROUP_STATUS_MSG to ensure this.) To mark individual GenBooleans as changed since the last apply action, pass this message a record of the changed GenBooleans.

Pass: **cx** Non-zero if GIGSF_MODIFIED bit should be passed set in status message

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gItemGC.def**

MSG_GEN_ITEM_GROUP_SET_APPLY_MSG

This message sets a new apply message in the GenBooleanGroup's *GBGI_applyMsg* instance field.

Pass: **cx** New notification message.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gItemGC.def**

MSG_GEN_ITEM_GROUP_SET_BEHAVIOR_TYPE

This message returns the current behavior type of a GenItemGroup (stored in the *GIGI_behaviorType* instance field).

Pass: `cl` New **GenItemGroupBehaviorType**.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\gItemGC.def**

MSG_GEN_ITEM_GROUP_SET_DESTINATION

This message sets a new destination object for the list, stored in the list's *GIGI_destination* field. The apply and status messages of this GenItemGroup will be sent to this new destination.

Pass: ^l**cx**:**dx** Destination object.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\gItemGC.def**

MSG_GEN_ITEM_GROUP_SET_INDETERMINATE_STATE

This message sets the GenItemGroup's GIGSF_INDETERMINATE flag in its *GIGI_stateFlags* instance field. Usually this message is used in conjunction with MSG_GEN_ITEM_GROUP_SET_MULTIPLE_SELECTIONS. (MSG_GEN_ITEM_GROUP_SET_NONE_SELECTED and MSG_GEN_ITEM_GROUP_SET_SINGLE_SELECTION set the indeterminate state themselves.)

Pass: `cx` Non-zero to set the item group indeterminate.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\gItemGC.def**

MSG_GEN_ITEM_GROUP_SET_ITEM_STATE

This message sets the state (selected or unselected) of a particular GenItem within a GenItemGroup. Other items will be unaffected by the new state. This message also clears the GenItemGroup's modified state. If an identifier

is passed which does not match an item within the GenItemGroup (or matches a not usable item), it will still be made the selection.

Pass: **cx** Identifier of item to select.
dx Non-zero if item should be selected; zero otherwise.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gItemGC.def**

MSG_GEN_ITEM_GROUP_SET_MODIFIED_STATE

This message marks a GenItemGroup modified by setting the GIGSF_MODIFIED flag in its *GIGI_stateFlags* instance field.

Pass: **cx** Non-zero to mark modified; zero to mark not modified.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gItemGC.def**

MSG_GEN_ITEM_GROUP_SET_MULTIPLE_SELECTIONS

This message sets multiple selections for a GenItemGroup. The caller must pass a pointer to a list of identifiers that should be selected, along with the number of selections to set. This message will clear the GenItemGroup's modified flag. If you wish to set the group modified send it a MSG_GEN_ITEM_GROUP_SET_MODIFIED_STATE.

Pass: **cx:dx** Buffer with identifiers of selections.
bp Number of selections.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gItemGC.def**

MSG_GEN_ITEM_GROUP_SET_NONE_SELECTED

This message sets a GenItemGroup to show no selections; it should not be sent to an exclusive GenItemGroup. This message will also clear the

GenItemGroup's modified flag. If you wish to set the modified flag, send the list a MSG_GEN_ITEM_GROUP_SET_MODIFIED_STATE.

Pass: **dx** Non-zero if indeterminate.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gItemGC.def**

MSG_GEN_ITEM_GROUP_SET_SINGLE_SELECTION

This message sets exactly one selection for a GenItemGroup. Any previously-selected items will be deselected, regardless of the GenItemGroup's behavior type. To set items without disturbing other settings, use MSG_GEN_ITEM_GROUP_SET_ITEM_STATE. If the list is on-screen, this message will cause a visual update. If the GenItemGroup is scrollable, this message will usually ensure that the item selected will appear on-screen.

This message clears the list's modified state. If you wish to set the group modified, send it a MSG_GEN_ITEM_GROUP_SET_MODIFIED_STATE. You may set a selection that is not usable, not enabled, or even not in the GenItemGroup. (This is useful for linked GenItemGroups.)

Pass: **cx** Identifier of the item to select.
dx Non-zero if indeterminate.

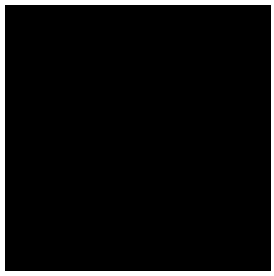
Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gItemGC.def**

Object Assembly Reference

Controllers



12

GenControlClass 345

GenToolControlClass 356

GenToolGroupClass 357

ColorSelectorClass 358

GenPageControlClass 367

GenControlClass

Instance Data

```
GCI_output      optr
   UIC Syntax:  output    = optrComp : null;
```

output (GCI_output) contains the optr of the object to which the controller is currently sending its "apply" messages. This field is typically set to a **GenTravelOption** (such as TO_APP_TARGET) or a **TravelOption** (such as TO_OBJ_BLOCK_OUTPUT).

Messages

■ MSG_GEN_CONTROL_ADD_APP_TOOLBOX_UI

This message adds the passed UI object to the controller's tool UI gadgetry as if it had been defined as a tool in the tool resource block. By default, the new object is added as the last child of the controller.

Source: Unrestricted—generated as part of the default functionality of MSG_GEN_CONTROL_GENERATE_TOOLBOX_UI if the controller has ATTR_GEN_CONTROL_APP_TOOLBOX_UI set.

Destination: Any GenControl object.

Pass: **cx:dx** The optr of the object to be added.

Returns: Nothing.

Destroyed: Nothing.

Interception: Should be intercepted by controllers that wish to add the new object somewhere other than as the last child.

Library: **Objects\gCtrlC.def**

MSG_GEN_CONTROL_ADD_APP_UI

This message adds the passed UI object to the controller's generic tree as if it had been originally defined in the default child block. By default, the new object is added as the last child of the controller.

Source: Unrestricted—generated as part of the default functionality of MSG_GEN_CONTROL_GENERATE_UI if the controller has ATTR_GEN_CONTROL_APP_UI set.

Destination: Any GenControl object.

Pass: **cx:dx** The optr of the object to be added.

Returns: Nothing.

Destroyed: Nothing.

Interception: Should be intercepted by controllers that wish to add the new object somewhere other than as the last child.

Library: **Objects\gCtrlC.def**

MSG_GEN_CONTROL_ADD_FEATURE

This message adds a feature to those currently supported by the controller. This is used for the default settings only, not for tool box UI. The controller is destroyed and then rebuilt with the new feature, causing it to be marked for saving to the state file in the new configuration.

Source: Unrestricted.

Destination: Any GenControl object.

Pass: **cx** A feature record with the flag of the feature to be added set.

Returns: Nothing.

Destroyed: Nothing.

Interception: Should not be intercepted.

Library: **Objects\gCtrlC.def**

MSG_GEN_CONTROL_ADD_TO_GCN_LISTS

This message adds the controller object to the GCN lists specified by its return value of MSG_GEN_CONTROL_GET_INFO. It will force a status update to ensure that the controller updates itself.

Source: Generated internally on startup and at other times.

Destination: Sent to self.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: Generally not intercepted. Subclasses should use this only to see when the controller is being added to its lists; it should call the superclass to do the actual addition.

Library: **Objects\gCtrlC.def**

MSG_GEN_CONTROL_ADD_TOOLBOX_FEATURE

This message adds a tool to those currently supported by the controller. This is used for the tool box settings only, not for the default UI features. The controller is destroyed and then rebuilt with the new tool, causing it to be marked for saving to the state file in the new configuration.

Source: Unrestricted.

Destination: Any GenControl object.

Pass: `cx` A feature record with the flag of the to be added set.

Returns: Nothing.

Destroyed: Nothing.

Interception: Should not be intercepted.

Library: **Objects\gCtrlC.def**

MSG_GEN_CONTROL_DESTROY_TOOLBOX_UI

Source: Sent by the tool box being destroyed as part of its specific UI destruction mechanism.

Destination: The GenControl object that owns the tool box.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Interception: Any controller class that subclasses the message MSG_GEN_CONTROL_GENERATE_TOOLBOX_UI must also subclass this message. The subclass must call its superclass at the end of its handler.

Interception: Should not be intercepted.

Library: **Objects\gCtrlC.def**

MSG_GEN_CONTROL_DESTROY_UI

This message destroys the UI gadgetry for the controller.

Source: Unrestricted—sent by self when being taken off the screen.

Destination: Any GenControl object

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Interception: Any subclass that intercepts MSG_GEN_CONTROL_GENERATE_UI must intercept this. The subclass must call the superclass at the end of the handler.

Library: **Objects\gCtrlC.def**

MSG_GEN_CONTROL_ENABLE_DISABLE

This message enables or disables the controller object as well as its default and tool box UI gadgetry.

Source: Sent by **GenControlClass** to the controller object when it receives MSG_META_NOTIFY_WITH_DATA_BLOCK.

Destination: Sent by controller to itself.

Pass:

cx	Either MSG_GEN_SET_ENABLED or MSG_GEN_SET_NOT_ENABLED as appropriate.
dl	A **VisUpdateMode** indicating when the visual update should occur.

Returns: Nothing.

Destroyed: Nothing.

Interception: Unlikely—typically should not be intercepted.

Library: **Objects\gCtrlC.def**

MSG_GEN_CONTROL_GENERATE_TOOLBOX_UI

This message generates the UI gadgetry for the controller's tool box.

Source: Sent by tool box object via the active list as part of its specific UI build mechanism.

Destination: The GenControl object that owns the tool box.

Pass:

cx:dx	The optr of GenInteraction that will be the parent of the controller's tools.

Returns: Nothing.

Destroyed: Nothing.

Interception: A subclass may intercept this to add UI gadgetry to the toolbox when the toolbox is built. The subclass must call the superclass at the beginning of the handler.

Warning: Any controller that subclasses this message must also subclass MSG_GEN_CONTROL_DESTROY_TOOLBOX_UI as well.

Library: **Objects\gCtrlC.def**



MSG_GEN_CONTROL_GENERATE_UI

This message generates the UI gadgetry for the controller.

Source: Unrestricted—sent by self as part of specific UI build.

Destination: Any GenControl object

Destroyed: Nothing.

Interception: If subclassed to add functionality, the subclass must call the superclass at the beginning of the handler.

Warning: If you intercept this message, you should also intercept MSG_GEN_CONTROL_DESTROY_UI, below.

Library: **Objects\gCtrlC.def**

MSG_GEN_CONTROL_GET_INFO

This message must be handled by all controllers. It takes an empty **GenControlBuildInfo** structure and fills it; this message is called in several circumstances by different objects and controller methods.

Source: Unrestricted—typically generated in **GenControlClass** methods.

Destination: Any controller object.

Pass: **cx:dx** A pointer to an empty **GenControlBuildInfo** structure.

Returns: The **GenControlBuildInfo** structure filled with the appropriate controller information.

Destroyed: Nothing.

Interception: Every controller class *must* intercept this message. There is no need to call the superclass anywhere in the handler.

Library: **Objects\gCtrlC.def**

Object Assembly Reference

MSG_GEN_CONTROL_GET_NORMAL_FEATURES

This message returns a structure indicating which of the default features of the controller are currently active, which are required, and which are prohibited.

Source: Unrestricted—typically GenToolControl.

Destination: Any GenControl object.

Pass: Nothing.

Returns:

ax	A record of flags indicating which features are normally active.
cx	A record of flags indicating which features are required.
dx	A record of flags indicating which features are prohibited to the controller.
bp	A record of flags indicating which features are supported by the controller.

Destroyed: Nothing.

Library: **Objects\gCtrlC.def**

MSG_GEN_CONTROL_GET_TOOLBOX_FEATURES

This message returns a structure indicating which of the controller's tools are currently active, which are required, and which are prohibited.

Source: Unrestricted—typically GenToolControl.

Destination: Any GenControl object.

Pass: Nothing.

Returns:

ax	A record of flags indicating which toolbox features are active.
cx	A record of flags indicating which toolbox features are required.
dx	A record of flags indicating which toolbox features are prohibited to the controller.
bp	A record of flags indicating which toolbox features are supported by the controller.

Destroyed: Nothing.

Library: **Objects\gCtrlC.def**

■ MSG_GEN_CONTROL_NOTIFY_INTERACTABLE

This message instructs the controller to take any actions necessary before being put on the screen and made interactable. It causes the controller to add itself to its GCN lists (if it wasn't already) and take any final steps to get ready to go on the screen.

Source: Generated internally—do not generate externally.

Destination: Sent to self.

Pass: `cx` A single record of flags indicating which portions of the controller will be made interactable.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: May be intercepted to circumvent default behavior.

Library: **Objects\gCtrlC.def**

■ MSG_GEN_CONTROL_NOTIFY_NOT_INTERACTABLE

This message indicates that the UI has determined that the controller can no longer be seen or interacted with by the user. The default behavior of the controller is to remove itself from its GCN lists for optimization.

Source: Generated internally—do not generate externally.

Destination: Sent to self.

Pass: `cx` A single record of flags indicating which portions of the controller are no longer interactable.

Destroyed: `ax, cx, dx, bp`

Interception: May be intercepted to circumvent default behavior.

Library: **Objects\gCtrlC.def**

MSG_GEN_CONTROL_REBUILD_NORMAL_UI

This message forces the normal UI of the controller to be rebuilt; each component will be visually destroyed and rebuilt.

Source: Unrestricted.

Destination: Any GenControl object.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Interception: Should not be intercepted.

Library: **Objects\gCtrlC.def**

MSG_GEN_CONTROL_REBUILD_TOOLBOX_UI

This message forces the toolbox UI of the controller to be rebuilt; each component will be visually destroyed and rebuilt.

Source: Unrestricted.

Destination: Any GenControl object.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Interception: Should not be intercepted.

Library: **Objects\gCtrlC.def**

MSG_GEN_CONTROL_REMOVE_FEATURE

This message removes a feature from those currently supported by the controller. It does not affect the active tools. The controller will be marked for



saving to the state file in the new configuration; it is destroyed and rebuilt with feature removed.

Source: Unrestricted.

Destination: Any GenControl object.

Pass: **cx** A feature record with the flag of the feature to be removed set.

Returns: Nothing.

Destroyed: Nothing.

Interception: Should not be intercepted.

Library: **Objects\gCtrlC.def**

MSG_GEN_CONTROL_REMOVE_FROM_GCN_LISTS

This message removes the controller from all its GCN lists.

Source: Generated internally on shutdown and at other times.

Destination: Sent to self.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: Generally not intercepted. Subclasses should use this only to see when the controller is being removed from its lists; it should call the superclass to do the actual removal.

Library: **Objects\gCtrlC.def**

MSG_GEN_CONTROL_REMOVE_TOOLBOX_FEATURE

This message removes a tool from those currently supported by the controller. It does not affect the default active feature list. The controller will

Object Assembly Reference

be marked for saving to the state file in the new configuration; it is destroyed and rebuilt with the tool removed.

Source: Unrestricted.

Destination: Any GenControl object.

Pass: **cx** A feature record with the flag of the tool to be removed set.

Returns: Nothing.

Destroyed: Nothing.

Interception: Should not be intercepted.

Library: **Objects\gCtrlC.def**

MSG_GEN_CONTROL_SCAN_FEATURE_HINTS

This message scans the feature hints set for the controller object to set the required and prohibited features.

Source: Unrestricted—Sent by **GenControlClass** to itself in numerous circumstances.

Destination: The GenControl object to be scanned.

Pass: **cx** An indicator whether the normal or tool box UI hints are to be scanned. This should be GCUIT_NORMAL for the normal hints, GCUIT_TOOLBOX for the tool box hints.

dx:bp A pointer to an empty **GenControlScanInfo** structure to be filled in by the handler. This structure is shown below.

Returns: **dx:bp** The pointer to the **GenControlScanInfo** structure filled in by the method.

Destroyed: Nothing.

Interception: Should not be intercepted.

Library: **Objects\gCtrlC.def**

MSG_GEN_CONTROL_UNBUILD_NORMAL_UI_IF_POSSIBLE

This internal message requests that the normal default controller UI gadgetry be removed and destroyed if they are not in use. This is an optimization intended to save swap space and perhaps memory space; it causes the objects to be loaded back in again when needed, however. Thus,

the optimization is only suited to machines with limited swap space (it originated due to these conditions on palmtop machines).

Source: Generated internally. Do not generate externally.

Destination: Sent to self.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: May be intercepted to avoid the optimization. May also need to be intercepted if a subclass creates its own UI gadgetry by intercepting MSG_GEN_CONTROL_GENERATE_UI.

Library: **Objects\gCtrlC.def**

MSG_GEN_CONTROL_UPDATE_UI

This message updates all UI components for the recipient controller.

Source: Sent by **GenControlClass** in its default handler for MSG_META_NOTIFY_WITH_DATA_BLOCK.

Destination: The GenControl object being updated.

Pass: `ss:bp` Pointer to a **GenControlUpdateUIParams** structure.

Returns: Nothing.

Destroyed: Nothing.

Library: **Objects\gCtrlC.def**

GenToolControlClass

Instance Data

```
GTCI_toolboxList     lptr.ToolboxInfo
   UIC Syntax:     toolboxList     = nptrComp : null;
```

toolboxList (*GTCI_toolboxList*) contains the chunk handle of the Tool Location Table.

```
GTCI_toolGroupList   lptr.ToolGroupInfo
   UIC Syntax:     toolGroupList   = nptrComp : null;
```

toolGroupList (*GTCI_toolGroupList*) contains the chunk handle of the Tool Group List.

GenToolGroupClass

Instance Data

```
GTGI_controller      optr
   UIC Syntax:  controller = optrComp : null;
```

controller (*GTGI_controller*) contains the optr of the GenControl object whose tools are managed by this GenToolGroup. This field is nearly always set in your **.uih** file.

Messages

MSG_GEN_TOOL_GROUP_SET_HIGHLIGHT

This message sets the highlight color of the tool group. This is used almost exclusively by the GenToolControl.

Source: GenToolControl object; can also be sent by others.

Destination: The GenToolGroup whose highlight color is to be set.

Pass: `cl` The new highlight type to be used by the tool group. The values are TGHT_INACTIVE_HIGHLIGHT, TGHT_ACTIVE_HIGHLIGHT, and TGHT_NO_HIGHLIGHT.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: Should not be intercepted.

Library: **Objects\gToolGC.def**

ColorSelectorClass

Instance Data

```
CSI_color                    ColorQuad
   UIC Syntax:  color = ColorQuad : {}
```

color (*CSI_color*) contains the currently selected color.

```
CSI_colorIndeterminate       byte
   UIC Syntax:  colorInd = byteComp : 0;
```

colorInd (*CSI_colorIndeterminate*) is true if the current color is indeterminate.

```
CSI_drawMask                 SystemDrawMask
   UIC Syntax:  drawMask = byteComp : 0;
```

drawMask (*CSI_drawMask*) contains the draw mask in use by the color.

```
CSI_drawMaskIndeterminate  byte
   UIC Syntax:  drawMaskInd = byteComp : 0;
```

drawMaskInd (*CSI_drawMaskIndeterminate*) is true if the current pattern is indeterminate.

```
CSI_pattern                  GraphicPattern
   UIC Syntax:  pattern = wordComp : 0;
```

pattern (*CSI_pattern*) contains the pattern in use by the color.

```
CSI_patternIndeterminate   byte
   UIC Syntax:  patternInd = byteComp : 0;
```

patternInd (*CSI_patternIndeterminate*) is true if the current pattern is indeterminate.

```
CSI_states                   ColorModifiedStates
   UIC Syntax:  states = byteComp : 0;
```

states (*CSI_states*) indicates which aspects have changed.

```
CSI_toolboxPrefs             ColorToolboxPreferences
   UIC Syntax:  toolboxPrefs = bitFieldComp byte {
                    unused : 2,
                    indexOrientation : 2 = {
                       areaOriented,
                       textOriented,
                       lineOriented
                    },
```

```
drawMaskOrientation : 2 = {
   areaOriented,
   textOriented,
   lineOriented
},
patternOrientation : 1 = {
   areaOriented,
   textOriented
},
} : isPopup;
```

toolboxPrefs (*CSI_toolboxPrefs*) contains the preferences for the color selector.

Imported Messages

MSG_META_COLORED_OBJECT_SET_COLOR

This message notifies the controlled object that it should set its color to the passed value. This message is part of the **MetaColoredMessages** range.

Source: ColorSelector object.

Destination: The current Target object.

Pass: **dxcx** The **ColorQuad** structure describing the color to be set.

Returns: Nothing.

Destroyed: Nothing.

Interception: Must be intercepted by the controlled object if it is to work with the ColorSelector controller.

Library: **Objects\colorC.def**

MSG_META_COLORED_OBJECT_SET_DRAW_MASK

This message notifies the controlled object that it should set its draw mask to the passed value. This message is part of the **MetaColoredMessages** range.

Source: ColorSelector object.

Destination: The current Target object.

Pass: `cl` The **SystemDrawMask** to be set as the object's draw mask.

Returns: Nothing.

Destroyed: Nothing.

Interception: Must be intercepted by the controlled object if it is to work with the ColorSelector controller.

Library: **Objects\colorC.def**

MSG_META_COLORED_OBJECT_SET_PATTERN

This message notifies the controlled object that it should sets its draw pattern to that passed. This message is part of the **MetaColoredMessages** range.

Source: ColorSelector object.

Destination: The current Target object.

Pass: `cx` The **GraphicPattern** to be set as the object's draw pattern.

Returns: Nothing.

Destroyed: Nothing.

Interception: Must be intercepted by the controlled object if it is to work with the ColorSelector controller.

Library: **Objects\colorC.def**

Messages

MSG_COLOR_SELECTOR_APPLY_COLOR

Makes the color selector send MSG_META_COLORED_OBJECT_SET_COLOR.

Source: Unrestricted.

Destination: Any ColorSelector object.

Pass: `cxdx` A **ColorQuad** value.

Returns: Nothing.

Destroyed: Nothing.

Library: **Objects\colorC.def**

MSG_COLOR_SELECTOR_APPLY_DRAW_MASK

Generates MSG_META_COLORED_OBJECT_SET_DRAW_MASK.

Source: Unrestricted.

Destination: Any ColorSelector object.

Pass: `cl` A **SystemDrawMask** value.

Returns: Nothing.

Destroyed: Nothing.

Interception: Generally not intercepted.

Library: **Objects\colorC.def**

MSG_COLOR_SELECTOR_APPLY_PATTERN

Generates MSG_META_COLORED_OBJECT_SET_PATTERN.

Source: Unrestricted.

Destination: Any ColorSelector object.

Pass: `cx` A **GraphicPattern** value.

Returns: Nothing.

Destroyed: Nothing.

Interception: Generally not intercepted.

Library: **Objects\colorC.def**



MSG_COLOR_SELECTOR_GET_COLOR

This message retrieves the controller's *CSI_color* field.

Source: Unrestricted.

Destination: Any ColorSelector object.

Pass: Nothing.

Returns:

dxcx	A **ColorQuad** structure specifying the color set in *CSI_color*.
bp	non-zero if indeterminate.

Destroyed: Nothing.

Interception: Generally not intercepted.

Library: **Objects\colorC.def**

MSG_COLOR_SELECTOR_GET_DRAW_MASK

This message retrieves the draw mask set in *CSI_drawMask*.

Source: Unrestricted.

Destination: Any ColorSelector object.

Returns:

cl	The **SystemDrawMask** set in *CSI_drawMask*.
dx	Non-zero if indeterminate.

Destroyed: Nothing.

Interception: Generally not intercepted.

Library: **Objects\colorC.def**

MSG_COLOR_SELECTOR_GET_FILLED_MONIKER

Returns the **VisMoniker** that should be used to represent the "do draw" item in the color selector.

Source: Unrestricted.

Destination: Any ColorSelector object.

Returns:

cx:dx	The optr of the **VisMoniker** chunk (or zero to use the default).

Destroyed: Nothing.

Interception: Generally not intercepted.

Library: **Objects\colorC.def**

MSG_COLOR_SELECTOR_GET_PATTERN

This message returns the pattern set in *CSI_pattern*.

Source: Unrestricted.

Destination: Any ColorSelector object.

Pass: Nothing.

Returns:

cx	The **GraphicPattern** currently set in *CSI_pattern*.
dx	Non-zero if indeterminate.

Destroyed: Nothing.

Interception: Generally not intercepted.

Library: **Objects\colorC.def**

MSG_COLOR_SELECTOR_GET_UNFILLED_MONIKER

Returns the **VisMoniker** that should be used to represent the "don't draw" item in the color selector.

Source: Unrestricted.

Destination: Any ColorSelector object.

Pass: Nothing.

Returns:

cx:dx	The optr of the **VisMoniker** chunk (or zero to use the default).

Destroyed: Nothing.

Interception: Generally not intercepted.

Library: **Objects\colorC.def**



MSG_COLOR_SELECTOR_SET_COLOR

This message sets the color in *CSI_color*.

Source: Unrestricted.

Destination: Any ColorSelector object.

Pass:

dxcx	The **ColorQuad** structure representing the new color for the *CSI_color* field.
bp	Non-zero if indeterminate.

Returns: Nothing.

Destroyed: Nothing.

Interception: Generally not intercepted.

Library: **Objects\colorC.def**

MSG_COLOR_SELECTOR_SET_DRAW_MASK

This message sets the draw mask stored in *CSI_drawMask*.

Source: Unrestricted.

Destination: Any ColorSelector object.

Pass:

cl	The new **SystemDrawMask** to be set into *CSI_drawMask*.
bp	Non-zero if indeterminate.

Returns: Nothing.

Destroyed: Nothing.

Interception: Generally not intercepted.

Library: **Objects\colorC.def**

MSG_COLOR_SELECTOR_SET_PATTERN

This message sets the drawing pattern stored in *CSI_pattern*.

Source: Unrestricted.

Destination: Any ColorSelector object.

Pass:

cx	The **GraphicPattern** value to be set.
dx	Non-zero if indeterminate.

Returns: Nothing.

Destroyed: Nothing.

Interception: Generally not intercepted.

Library: **Objects\colorC.def**

MSG_COLOR_SELECTOR_UPDATE_COLOR

Update the current color in the toolbox and normal UI. This message is normally sent from within a MSG_GEN_CONTROL_UPDATE_UI handler.

Source: Unrestricted—sent in a MSG_GEN_CONTROL_UPDATE_UI handler.

Destination: Any ColorSelector object.

Pass:

dxcx	The **ColorQuad** structure representing the new color for the *CSI_color* field.
bp	Non-zero if indeterminate.

Returns: Nothing.

Destroyed: Nothing.

Interception: Generally not intercepted.

Library: **Objects\colorC.def**

MSG_COLOR_SELECTOR_UPDATE_DRAW_MASK

Update the current draw mask in the toolbox and the UI. This is often sent from within a MSG_GEN_CONTROL_UPDATE_UI handler.

Source: Unrestricted—sent in a MSG_GEN_CONTROL_UPDATE_UI handler.

Destination: Any ColorSelector object.

Pass:

cl	The new **SystemDrawMask** to be set into *CSI_drawMask*.
bp	Non-zero if indeterminate.

Returns: Nothing.

Destroyed: Nothing.

Interception: Generally not intercepted.

Library: **Objects\colorC.def**

MSG_COLOR_SELECTOR_UPDATE_FILLED_STATUS

Makes the color selector update the draw mask and associated UI.

Source: Unrestricted.

Destination: Any ColorSelector object.

Pass:

cl	A **SystemDrawMask**.
dx	Non-zero if indeterminate.
bp	Non-zero to update toolbox UI as well.

Returns: Nothing.

Destroyed: Nothing.

Interception: Generally not intercepted.

Library: **Objects\colorC.def**

■ MSG_COLOR_SELECTOR_UPDATE_PATTERN

Update the current pattern in the toolbox and the UI. This is often sent from within a MSG_GEN_CONTROL_UPDATE_UI handler.

Source: Unrestricted—sent in a MSG_GEN_CONTROL_UPDATE_UI handler.

Destination: Any ColorSelector object.

Pass:

cx	The **GraphicPattern** value to be set.
dx	Non-zero if indeterminate.

Returns: Nothing.

Destroyed: Nothing.

Interception: Generally not intercepted.

Library: **Objects\colorC.def**

GenPageControlClass

Instance Data

This class has no new instance data fields.

Messages

This class has no public methods.

Object Assembly Reference

Document Control

13

GenDocumentClass 371

GenDocumentGroupClass 397

GenDocumentControlClass 419

GenDocumentClass

Instance Data

```
GDI_attrs       GenDocumentAttrs
   UIC Syntax:  attributes      = wordComp : 0;
```

attributes (*GDI_attrs*) contains flags indicating the status of the document.

```
GDI_type        GenDocumentType
   UIC Syntax:  type            = wordComp : 0;
```

type (*GDI_type*) indicates the type of document associated with this document object.

```
GDI_operation   GenDocumentOperation
   UIC Syntax:  operation       = wordComp : 0;
```

operation (*GDI_operation*) indicates what operation is currently being performed on that object (if any).

```
GDI_fileHandle word
   UIC Syntax:  static wordComp : 0;
```

GDI_fileHandle contains the handle of the file associated with this document object.

```
GDI_fileName    FileLongName
   UIC Syntax:  fileName        = typeComp \
                    "FileLongName" : "";
```

fileName (*GDI_fileName*) contains the GEOS name of the file associated with this document object.

```
GDI_display     hptr
   UIC Syntax:  display         = hptrComp : null;
```

display (*GDI_display*) contains the handle of the display associated with this document object.

Messages

MSG_GEN_DOCUMENT_ATTACH_FAILED

Notification that the document could not be re-opened when restoring from state.

Source: GenDocumentControl

Destination: GenDocument

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_ATTACH_UI_TO_DOCUMENT

Notification that a document has been opened, the UI has been created for it, and the document needs to be displayed.

Source: GenDocumentControl

Destination: GenDocument

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_AUTO_SAVE

Instruction to auto-save the document.

Source: GenDocument

Destination: GenDocument

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gDocC.def

■ MSG_GEN_DOCUMENT_CHANGE_PASSWORD

Change the password for this document.

Source: GenDocumentControl

Destination: GenDocument

Pass: `ss:bp` **GenDocumentChangePasswordParams**

Returns: Nothing.

Destroyed: `ax, cd, dx, bp`

Library: **Objects\gDocC.def**

■ MSG_GEN_DOCUMENT_CHANGE_TYPE

Change the type of this document.

Source: GenDocumentControl

Destination: GenDocument

Pass: `cx` **GenDocumentType**

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gDocC.def**

■ MSG_GEN_DOCUMENT_CHECK_FOR_MODIFICATIONS

Check if a (shared multiple) file has been modified.

Source: GenDocumentControl

Destination: GenDocument

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gDocC.def**

MSG_GEN_DOCUMENT_CLEAR_DEFAULT

Clear the default document.

Source: GenDocumentControl

Destination: GenDocument

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_CLEAR_EMPTY

Clear the empty document.

Source: GenDocumentControl

Destination: GenDocument

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_CLOSE

Close this document.

Source: GenDocumentControl

Destination: GenDocument

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_CLOSE_FILE

Close the document associated with a file without removing the document object (e.g. so the file can be renamed).

Source: GenDocumentControl

Destination: GenDocument

Pass: **bp** IACP connection requesting the close (0 if user-initiated).

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gDocC.def**

MSG_GEN_DOCUMENT_CLOSE_IF_CLEAN_UNNAMED

Close the document if it has never been saved (and is thus un-named) and has not been changed since it was created.

Source: GenDocumentControl

Destination: GenDocument

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gDocC.def**

MSG_GEN_DOCUMENT_CLOSE_IF_OPEN_FOR_IACP_ONLY

Close the document if it is open on screen not for the user, but rather only for an application-mode IACP connection.

Source: GenDocumentControl

Destination: GenDocument

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gDocC.def**



MSG_GEN_DOCUMENT_COMPUTE_ACCESS_FLAGS

Compute the **VMAccessFlags** or **FileAccessFlags** to pass to **VMOpen()**, **FileCreate()**, or **FileOpen()**.

Source: GenDocumentControl

Destination: GenDocument

Pass:	**ss:bp**	**DocumentCommonParams**
Returns:	**al**	**VMAccessFlags** or **FileAccessFlags**
Destroyed:	**ah**	
Library:	**Objects\gDocC.def**	

MSG_GEN_DOCUMENT_COPY_TO

Make a copy of this document.

Source: GenDocumentControl

Destination: GenDocument

Pass:	**dx**	Size of **DocumentCommonParams**
	ss:bp	**DocumentCommonParams**
Returns:	CF	Set if error.
	ax	Handle of new file.
Destroyed:	**cx, dx**	
Library:	**Objects\gDocC.def**	

MSG_GEN_DOCUMENT_CREATE_UI_FOR_DOCUMENT

The GenDocument object sends this message to itself after a document has been created or opened. Before this message is sent, the GenDocumentControl object will enable those objects on its *GDCI_enableDisableList*, and the GenDocumentGroup object will copy the GenDisplay block for the document (if one is defined).

Applications that use dynamic UI objects will commonly respond to this message by creating the objects for the newly-opened document. Applications

that use static UI objects will commonly respond to this message by enabling the objects.

Source: A GenDocument object.

Destination: The document object sends this message to itself.

Pass: Nothing.

Returns: CF Set if error.

Destroyed: ax, cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_DESTROY_UI_FOR_DOCUMENT

The GenDocumentGroup object sends this message out when a document is being closed, whether because a user closes the file or because the application is being closed. Applications will generally disable static display objects and delete dynamic display objects. Note that the GenDocumentControl object will automatically disable all objects in its *GDCI_enableDisableList*, and the GenDocumentGroup will delete the display block it created for a document, if any.

Source: A GenDocument object.

Destination: The document object sends this message to itself.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_DETACH_UI_FROM_DOCUMENT

The document object sends this message when a document is being closed, whether because a user closes the file or because the application is being closed. It also sends this message when GEOS is in the process of saving itself to a state file prior to shutting down. Applications generally respond to this

by detaching dynamic UI objects. Note that the GenDocumentControl object will automatically disable any objects in its *GDCI_enableDisableList*.

Source: A GenDocument object.

Destination: The document object sends this message to itself.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gDocC.def**

■ MSG_GEN_DOCUMENT_DISABLE_AUTO_SAVE

Sometimes an application needs to temporarily disable auto-save for a specific document (for example, if it is in the middle of making elaborate changes to the file). It can do this by sending this message to the document object. The document's GDA_PREVENT_AUTO_SAVE bit will be turned on, and auto-save will be disabled until the document receives a MSG_GEN_DOCUMENT_ENABLE_AUTO_SAVE.

Source: Unrestricted—objects subclassed from **GenDocumentClass** often send this message to themselves.

Destination: Any GenDocument object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: You should not subclass this message.

Library: **Objects\gDocC.def**

■ MSG_GEN_DOCUMENT_DOCUMENT_HAS_CHANGED

If the GDGA_AUTOMATIC_CHANGE_NOTIFICATION attribute of the GenDocumentGroup object is set to on, a timer will periodically check to see if any open documents have been changed by another application. If they have, the document object will send this message out. The application should

■ Object Assembly Reference

respond by redisplaying the data on the screen and rereading any cached data from the file.

Source: A GenDocument object.

Destination: The document object sends this message to itself.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gDocC.def**

MSG_GEN_DOCUMENT_EDIT_USER_NOTES

Edit the "User Notes" field of this document.

Source: GenDocumentControl

Destination: GenDocument

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gDocC.def**

MSG_GEN_DOCUMENT_ENABLE_AUTO_SAVE

This message turns off a document's GDA_PREVENT_AUTO_SAVE bit. If the bit is already off (i.e., auto-save is enabled), the message has no effect.

Source: Unrestricted—objects subclassed from **GenDocumentClass** often send this message to themselves.

Destination: Any GenDocument object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: You should not subclass this message.

Library: **Objects\gDocC.def**

MSG_GEN_DOCUMENT_EXPORT

Export the current document.

Source: GenDocumentControl

Destination: GenDocument

Pass: **ss:bp** **GenTranslationParams**

Returns: Nothing.

Destroyed: Nothing.

Library: **Objects\gDocC.def**

MSG_GEN_DOCUMENT_FILE_CHANGED_REINITIALIZE_CREATED_UI

Notification that the file has been changed for the document control. Applications that store cached document data in the created UI should reload that data on this message.

Source: GenDocumentControl

Destination: GenDocument

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gDocC.def**

MSG_GEN_DOCUMENT_GENERATE_NAME_FOR_NEW

Create a name for a new document.

Source: GenDocumentControl

Destination: GenDocument

Pass: **dx:bp** **DocumentCommonParams** buffer.
cx Number of times (from zero) this method has been called trying to create a unique name.

Returns: **cx** GEN_DOCUMENT_GENERATE_NAME_ERROR if error, GEN_DOCUMENT_GENERATE_NAME_CANCEL to cancel; otherwise unchanged.
dx:bp **DocumentCommonParams** buffer, with new name in appropriate fields.

Destroyed: **ax**

Library: **Objects\gDocC.def**

MSG_GEN_DOCUMENT_GET_ATTRS

Use this message to get the *GDI_attrs* flags for a given document. These attribute flags give information about the document's permissions as well as about any operations currently in progress.

Source: Unrestricted.

Destination: GenDocument

Pass: Nothing.

Returns: **ax** The object's word-sized *GDI_attrs* field.

Destroyed: Nothing.

Interception: You should not subclass this message.

Library: **Objects\gDocC.def**

MSG_GEN_DOCUMENT_GET_DISPLAY

Get the handle of the display associated with this document.

Source: Unrestricted.

Destination: GenDocument

Pass: Nothing.

Returns: **cx:dx** Document's display.

Destroyed: Nothing.

Library: **Objects\gDocC.def**

MSG_GEN_DOCUMENT_GET_FILE_HANDLE

This message returns the handle of the file associated with a given GenDocument object.

Source: Unrestricted.

Destination: GenDocument

Pass: Nothing.

Returns: **ax** File handle.

Destroyed: Nothing.

Interception: You should not subclass this message.

Library: **Objects\gDocC.def**

MSG_GEN_DOCUMENT_GET_FILE_NAME

This message instructs a GenDocument to write the name of its file (without the path) to the specified address.

Source: Unrestricted.

Destination: GenDocument

Destination: Any GenDocument object

Pass: **cx:dx** Buffer of length FILE_LONGNAME_BUFFER_SIZE

Returns: Writes file's virtual name into the passed buffer as a null-terminated string.

Destroyed: **ax**

Warning: Make sure the buffer passed is of length FILE_LONGNAME_BUFFER_SIZE; otherwise the method might overwrite other data.

Library: **Objects\gDocC.def**

MSG_GEN_DOCUMENT_GET_OPERATION

Use this message to find out what user action a given document object is in the midst of processing. This is useful if you are handling some message and want to find out the context in which that message was sent. Note that although the message returns a word-length value, the *GDI_operation* enumerated type is byte-length; it is thus safe to cast the return value to a byte-length variable.

Source: Unrestricted.

Destination: Any GenDocument object.

Pass: Nothing.

Returns: **ax** A member of the **GenDocumentOperation** enumerated type corresponding to the document object's current operation.

Destroyed: Nothing.

Library: **Objects\gDocC.def**

Object Assembly Reference

MSG_GEN_DOCUMENT_GET_VARIABLE

Get a piece of information about the document. This is used primarily by the Text object to implement variable graphics.

Source: Unrestricted.

Destination: GenDocument

Destination: Any GenDocument object.

Pass: `ss:bp` **GenDocumentVariableParams**

Returns: Result written to buffer indicated by **GenDocumentVariableParams**.

Destroyed: Nothing.

Library: **Objects\gDocC.def**

MSG_GEN_DOCUMENT_IMPORT

Notification that the user has imported a file.

Source: Unrestricted.

Destination: GenDocument

Destination: Any GenDocument object.

Pass: `ss:bp` **GenTranslationParams**

Returns: `CF` Set if error.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gDocC.def**

MSG_GEN_DOCUMENT_INITIALIZE_DOCUMENT_FILE

When a new document is created, the document object sends this message to itself. VM file based applications will generally respond to this message by allocating and initializing the map block. DOS file based applications will commonly initialize data structures for a default file. If an application maintains data caches for its files, it should initialize the caches at this point.

The application should not take any UI-related actions; those should be postponed until the messages

MSG_GEN_DOCUMENT_CREATE_UI_FOR_DOCUMENT and MSG_GEN_DOCUMENT_ATTACH_UI_TO_DOCUMENT are received.

Source: A GenDocument object.

Destination: The document object sends this message to itself.

Pass: Nothing.

Returns: CF Set if error.

Destroyed: ax, cx, dx, bp

Library: Objects\gDocC.def

■ MSG_GEN_DOCUMENT_MARK_DIRTY

Instruct the document object that it's file is dirty.

Source: Unrestricted.

Destination: Any GenDocument object.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gDocC.def

■ MSG_GEN_DOCUMENT_MOVE_TO

Move or rename this document's file.

Source: GenDocumentControl

Destination: Any GenDocument object.

Pass: dx size of **DocumentCommonParams**
ss:bp **DocumentCommonParams** structure

Returns: CF Set if error

Destroyed: cx, dx

Library: Objects\gDocC.def

Object Assembly Reference

■ MSG_GEN_DOCUMENT_NEW

Create a new document.

Source: GenDocumentControl

Destination: Any GenDocument object.

Pass:

dx	size of **DocumentCommonParams**
ss:bp	**DocumentCommonParams** structure

Returns:

CF	Set if error
ax	File handle

Destroyed: **cx, dx**

Library: **Objects\gDocC.def**

■ MSG_GEN_DOCUMENT_OPEN

Open a document. If *DCP_connection* is non-zero, MSG_META_IACP_DOC_OPEN is sent to the client of the connection.

Source: GenDocumentControl

Destination: Any GenDocument object.

Pass:

dx	size of **DocumentCommonParams**
ss:bp	**DocumentCommonParams** structure

Returns:

CF	Set if error
ax	File handle

Destroyed: **cx, dx**

Library: **Objects\gDocC.def**

■ MSG_GEN_DOCUMENT_PHYSICAL_CHECK_FOR_MODIFICATIONS

Check if the (shared-multiple) file has been modified by some other accesser.

Source: The GenDocument object.

Destination: The document object sends this message to itself.

Pass: Nothing.

Returns:

CF	Set if the file is modified.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gDocC.def**

MSG_GEN_DOCUMENT_PHYSICAL_CLOSE

Actually close the document object's file, calling the appropriate file system routine.

Source: The GenDocument object.

Destination: The document object sends this message to itself.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_PHYSICAL_COPY_TEMPLATE

Copy the file to an untitled file.

Source: The GenDocument object.

Destination: The document object sends this message to itself.

Pass: Nothing.

Returns:

CF	Set if error.
ax	File handle (if successful) or error code (if error).

Destroyed: cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_PHYSICAL_CREATE

Actually create the document object's file, calling the appropriate file system routine.

Source: The GenDocument object.

Destination: The document object sends this message to itself.

Pass: ss:bp **DocumentCommonParams**
Thread's current path set to directory in which file should be created

Returns:

CF	set if error
ax	File handle (if no error) or error code (if error)

Destroyed: cx, dx

Library: Objects\gDocC.def

Object Assembly Reference

MSG_GEN_DOCUMENT_PHYSICAL_DELETE

Actually delete the document object's file, calling the appropriate file system routine.

Source: The GenDocument object.

Destination: The document object sends this message to itself.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_PHYSICAL_OPEN

Open a file for this document.

Source: The GenDocument object.

Destination: The document object sends this message to itself.

Pass: ss:bp **DocumentCommonParams**
Thread's current path set to directory in which file should be created

Returns:

CF	set if error
ax	File handle (if no error) or error code (if error)
cx	Non-zero if template document opened
dx	Non-zero if a new file was created

Destroyed: Nothing.

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_PHYSICAL_REVERT

Revert the file to its last-saved version.

Source: The GenDocument object.

Destination: The document object sends this message to itself.

Pass: Nothing.

Returns:

CF	set if error
ax	Error code (if error)

Destroyed: cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_PHYSICAL_SAVE

This message is sent when the user saves a file.

Source: The GenDocument object.

Destination: The document object sends this message to itself.

Pass: Nothing.

Returns:

CF	Set if error.
ax	Error code (if error).

Destroyed: cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_PHYSICAL_SAVE_AS

This message is sent when the user performs a "Saves As" on the file.

Source: The GenDocument object.

Destination: The document object sends this message to itself.

Pass: Nothing.

Returns:

CF	Set if error.
ax	Error code (if error).

Destroyed: cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_PHYSICAL_SAVE_AS_FILE_HANDLE

Save the document to a different file, specified by a file handle.

Source: The GenDocument object.

Destination: The document object sends this message to itself.

Pass:

cx	Handle of new file.

Returns:

CF	Set if error.
ax	Error code (if error).

Destroyed: cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_PHYSICAL_UPDATE

This message is sent when the file is auto-saved (if this is enabled), and when the document is detached as part of a GEOS shutdown.

Source: A GenDocument object.

Destination: The document object sends this message to itself.

Pass: Nothing.

Returns:

CF	Set if error.
ax	Error code (if error).

Destroyed: cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_QUICK_BACKUP

Backup this document.

Source: GenDocumentControl.

Destination: GenDocument.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_READ_CACHED_DATA_FROM_FILE

The document object sends this message to itself when the document needs to read cached data. In particular, it sends this when a document is opened, when a document is reverted to its last-saved state, and when a document is re-opened as GEOS restores from state. If the application maintains a data cache, it should read the data from the file at this point. If the document does not cache data, it can ignore this message.

Note that if the document control notices that the file has changed on disk, it will not send this message; it will, however, send a

MSG_GEN_DOCUMENT_DOCUMENT_HAS_CHANGED. The handler for that message should reread the cache or call the handler for this message.

Source: A GenDocument object.

Destination: The document object sends this message to itself.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gDocC.def**

■ MSG_GEN_DOCUMENT_RECOVER_QUICK_BACKUP

Restore this document from its quick-backup.

Source: GenDocumentControl.

Destination: GenDocument.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Library: **Objects\gDocC.def**

■ MSG_GEN_DOCUMENT_RENAME

Rename this document.

Source: GenDocumentControl.

Destination: GenDocument.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gDocC.def**

Object Assembly Reference

MSG_GEN_DOCUMENT_REOPEN_FILE

Re-open the file associated with this document.

Source: GenDocumentControl.

Destination: GenDocument.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_REVERT

Revert to the backup version of this document.

Source: GenDocumentControl.

Destination: GenDocument.

Pass: Nothing.

Returns: CF Set if error.

Destroyed: ax, cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_REVERT_NO_PROMPT

Revert to the backup version of this document *without* prompting the user.

Source: GenDocumentControl.

Destination: GenDocument.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_SAVE

Save this document.

Source: GenDocumentControl.

Destination: GenDocument.

Pass: Nothing.

Returns: CF Set if error.

Destroyed: cx, dx, bp

Library: **Objects\gDocC.def**

MSG_GEN_DOCUMENT_SAVE_AS

Save this document with a new name.

Source: GenDocumentControl.

Destination: GenDocument.

Pass: **dx** size of **DocumentCommonParams**
ss:bp **DocumentCommonParams**

Returns: CF set if error
ax File handle (if no error) or error code (if error)

Destroyed: cx, dx

Library: **Objects\gDocC.def**

MSG_GEN_DOCUMENT_SAVE_AS_CANCELLED

Notification that a “Save As” operation has been canceled.

Source: GenDocumentControl.

Destination: GenDocument.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: **Objects\gDocC.def**

MSG_GEN_DOCUMENT_SAVE_AS_COMPLETED

Notification that a "Save As" operation has been completed successfully.

Source: GenDocumentControl.

Destination: GenDocument.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_SAVE_AS_TEMPLATE

Save this document as a template.

Source: GenDocumentControl.

Destination: GenDocument.

Pass:

ss:bp	**DocumentCommonParams**
dx	size of **DocumentCommonParams**

Returns:

CF	set if error
ax	File handle (if no error) or error code (if error)

Destroyed: cx, dx

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_SEARCH_FOR_DOC

Search for the specified document and bring it to the front if it exists. If *DCP_connection* is non-zero, MSG_META_IACP_DOC_OPEN is sent to the client of the connection.

Source: GenDocumentControl.

Destination: GenDocument.

Pass:

ss:bp	**DocumentCommonParams**

Returns:

CF	Set if document found.

Destroyed: cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_SET_DEFAULT

Save this document as the default document.

Source: GenDocumentControl.

Destination: GenDocument.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_SET_EMPTY

Save this document as the template for “empty” documents.

Source: GenDocumentControl.

Destination: GenDocument.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_TEST_FOR_FILE

Test to see if this document has the specified file open.

Source: GenDocumentControl.

Destination: GenDocument.

Pass:

cx	File handle against which to compare.
dx:bp	Address of buffer in which to store optr

Returns:

CF	Set if file is open and dirty.

Destroyed: ax

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_UPDATE

Update the document, flushing all cached data to the file.

Source: GenDocumentControl.

Destination: GenDocument.

Pass: Nothing.

Returns:

CF	Set if error.
ax	Error code (if error).

Destroyed: cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_UPDATE_EARLIER_COMPATIBLE_DOCUMENT

The document object sends this message when the user tries to open a document with the same major protocol number as the document control and a lower minor protocol number.

Source: A GenDocument object.

Destination: The document object sends this message to itself.

Pass: Nothing.

Returns:

CF	Set if error
ax	Non-zero to instruct document object to update the file's protocol number.

Destroyed: cx, dx, bp

Library: Objects\gDocC.def

MSG_GEN_DOCUMENT_UPDATE_EARLIER_INCOMPATIBLE_DOCUMENT

The document object sends this message when the user tries to open a document with a lower major protocol number than the document control.

Applications respond to this message by making any changes to the document necessary to make it compatible with the application.

Source: A GenDocument object.

Destination: The document object sends this message to itself.

Pass: Nothing.

Returns:

CF	Set if error
ax	Non-zero to instruct document object to update the file's protocol number.

Destination: cx, dx, bp

Library: Objects\gDocC.def

■ MSG_GEN_DOCUMENT_WRITE_CACHED_DATA_TO_FILE

The document object sends this message to itself when the document needs to read cached data. In particular, it sends this when a document is opened, when a document is reverted to its last-saved state, and when a document is re-opened as GEOS restores from state. If the application maintains a data cache, it should read the data from the file at this point. If the document does not cache data, it can ignore this message.

Note that if the document control notices that the file has changed on disk, it will not send this message; it will, however, send a MSG_GEN_DOCUMENT_DOCUMENT_HAS_CHANGED. The handler for that message should reread the cache or call the handler for this message.

Source: A GenDocument object.

Destination: The document object sends this message to itself.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gDocC.def

Object Assembly Reference

GenDocumentGroupClass

Instance Data

```
GDGI_attrs      GenDocumentGroupAttrs \
                   mask GDGA_VM_FILE or \
                   mask GDGA_NATIVE or \
                   mask GDGA_SUPPORTS_AUTO_SAVE or \
                   mask GDGA_AUTOMATIC_CHANGE_NOTIFICATION or \
                   mask GDGA_AUTOMATIC_DIRTY_NOTIFICATION or \
                   mask GDGA_APPLICATION_THREAD or \
                   mask GDGA_CONTENT_DOES_NOT_MANAGE_CHILDREN or \
                   mask GDGA_AUTOMATIC_UNDO_INTERACTION
   UIC Syntax:  attributes = bitFieldComp word {
                        vmFile,
                        native,
                        supportsAutoSave,
                        automaticChangeNotification,
                        automaticDirtyNotification,
                        applicationThread,
                        vmFileContainsObjects,
                        dontManageChildren,
                        largeContent,
                        automaticUndoInteraction
                   } : vmFile,
                   supportsAutoSave,
                   automaticChangeNotification,
                   automaticDirtyNotification,
                   applicationThread,
                   dontManageChildren,
                   automaticUndoInteraction;
```

attributes (*GDGI_attrs*) is a record that specifies certain basic characteristics of the documents to be managed. The attributes are set in the source code and are not changed at run-time. The attributes available include:

vmFile (GDGA_VM_FILE)
: This attribute is on if the documents to be opened are GEOS Virtual Memory files. By default, it is on.

native (GDGA_NATIVE)
: This attribute is on if the documents to be opened are in the file system's native format (i.e. they are not GEOS files). By default, it is off.

supportsAutoSave (GDGA_SUPPORTS_AUTO_SAVE)
: If this attribute is on, the documents will be periodically auto-saved. It works only with VM files. It works by periodically updating the file to disk. It should probably be turned off if "Save As" and "Revert" are disabled. By default, the attribute is on. The program can temporarily disable auto-save for a document by sending the document object MSG_GEN_DOCUMENT_DISABLE_AUTO_SAVE.

automaticChangeNotification (GDGA_AUTOMATIC_CHANGE_NOTIFICATION)
: If this attribute is on, the GenDocumentGroup object will periodically check all open documents to see if they have been changed by another process. If a document has changed, the document control will send MSG_META_DOC_OUTPUT_DOCUMENT_HAS_CHANGED to the application. This attribute is useful if the application may be reading multi-user files.

automaticDirtygeNotification (GDGA_AUTOMATIC_CHANGE_NOTIFICATION)
: This attribute is relevant only for GEOS files. If the attribute is on, whenever a file is marked "dirty," the file system will automatically notify the document control. The document control will then take appropriate actions (enable the "Save" trigger, etc.). The document control will also present a "Save changes before closing" dialog box if the document is closed before being saved. If GDGA_AUTOMATIC_DIRTY_NOTIFICATION is off, or if the documents are DOS files, the application will have to notify the document control when the document is dirty. It does this by sending a MSG_GEN_DOCUMENT_GROUP_MARK_DIRTY to the GenDocumentGroup object (under the procedure model), or by sending a MSG_GEN_DOCUMENT_MARK_DIRTY to the document object (under the object model). By default, GDGA_AUTOMATIC_DIRTY_NOTIFICATION is on.

applicationThread (GDGA_APPLICATION_THREAD)
: If this attribute is on, the GenDocumentGroup object is run by the application thread, as are its (document-object) children. By default, it is on.

vmFileContainsObjects (GDGA_VM_FILE_CONTAINS_OBJECTS)
: If the document control manages Virtual Memory Object files, this attribute should be set to on. By default, this attribute is off.

Object Assembly Reference

dontManageChildren (GDGA_CONTENT_DOES_NOT_MANAGE_CHILDREN)
The application's main VisContent, if any, does not manage its children. By default, this attribute is on.

largeContent (GDGA_LARGE_CONTENT)
The application's main VisContent uses the large model. By default, this attribute is off.

largeContent (GDGA_AUTOMATIC_UNDO_INTERACTION)
The application sends out undo set-context messages automatically as necessary.

```
GDGI_untitledName     lptr.char
   UIC Syntax:   defaultName      = nptrComp : null;
```

defaultName (*GDGI_untitledName*) contains the chunk handle of the chunk containing the name suggested when a new document is first saved.

```
GDGI_documentClass    fptr.ClassStruct GenDocumentClass
   UIC Syntax:   documentClass    = fptrComp : "GenDocumentClass";
```

documentClass (*GDGI_documentClass*) contains a pointer to the **ClassStruct** of the class definition used for document objects.

```
GDGI_genDocument      optr
   UIC Syntax:   genDocument      = optrComp: null;
```

genDocument (*GDGI_genDocument*) contains the optr of a document object to be duplicated for each new document (if any).

```
GDGI_output           optr
   UIC Syntax:   output           = optrComp : null;
```

output (*GDGI_output*) contains the optr of the output object for document control messages.

```
GDGI_documentControl optr
   UIC Syntax:   uiDC             = optrComp : null;
```

uiDC (*GDGI_documentControl*) contains the optr of the Document Control object.

```
GDGI_genView          optr
   UIC Syntax:   genView          = optrComp : null;
```

genView (*GDGI_genView*) contains the optr of the GenView object to be linked with the Document object (if any).

```
GDGI_genDisplay       optr
   UIC Syntax:   genDisplay       = optrComp : null;
```

genDisplay (*GDGI_genDisplay*) contains the optr to a block to be duplicated for each new document (if any).

```
GDGI_genDisplayGroup optr
   UIC Syntax:  genDisplayGroup  = optrComp : null;
```

genDisplayGroup (*GDGI_genDisplayGroup*) contains the optr of the GenDisplayGroup which manages the GenDisplays.

```
GDGI_protocolMajor   word  0
   UIC Syntax:  protocolMajor    = wordComp : 1;

GDGI_protocolMinor   word  1
   UIC Syntax:  protocolMinor    = wordComp : 0;
```

These fields specify the document protocol for documents created by this version of the application.

Messages

■ MSG_GEN_DOCUMENT_GROUP_GET_ATTRS

Use this message to find out the attributes of the GenDocumentGroup object. Note that the attributes cannot be changed at run-time; they can only be examined.

Source: Unrestricted.

Destination: Any GenDocumentGroup object.

Pass: Nothing.

Returns: **ax** **GenDocumentGroupAttrs** in *GDGI_attrs* field

Destroyed: **ax, cx, dx, bp**

Interception: You should not subclass this message.

Library: **Objects\gDocGrpC.def**

MSG_GEN_DOCUMENT_GROUP_GET_CREATOR

This message gets the token for the application; this token is the "creator token" for all files created by the document control.

Source: Unrestricted.

Destination: Any GenDocumentGroup object.

Pass: **cx:dx** Address of empty **GeodeToken** structure.

Returns: Creator token written to **cx:dx**.

Destroyed: **ax**

Interception: You should not subclass this message.

Library: **Objects\gDocGrpC.def**

MSG_GEN_DOCUMENT_GROUP_GET_DEFAULT_NAME

This message instructs the GenDocumentGroup object to copy the *GDGI_defaultName* attribute to the specified address. In addition, the message will return the *GDGI_attrs* word of the GenDocumentGroup object. If you just want the attributes, use MSG_GEN_DOCUMENT_GROUP_GET_ATTRS.

Source: Unrestricted.

Destination: Any GenDocumentGroup object.

Pass: **cx:dx** Pointer to **DocumentCommonParams** structure

Returns: **ax** **GenDocumentGroupAttrs**
cx:dx.*DCP_name*
Default document name
cx:dx.*DCP_path*, **cx:dx**.*DCP_diskHandle*
Directory holding the file whose name is in **cx:dx**.*DCP_name*

Destroyed: Nothing.

Interception: You should not subclass this message.

Library: **Objects\gDocGrpC.def**

MSG_GEN_DOCUMENT_GROUP_GET_DISPLAY

The *GDGI_genDisplay* field can be set to point to a GenDisplay object. If *GDGI_genDisplay* is not a null optr, then the document control will duplicate the resource containing the referenced GenDisplay whenever a new document object is created. The duplicate GenDisplay is made a child of the

GenDisplayGroup object indicated by *GDGI_genDisplayGroup*. By using this message, you can get an optr to that "template" display object. Any changes made to that object will be copied whenever a new document object is created.

Source: Unrestricted.

Destination: Any GenDocumentGroup object.

Pass: Nothing.

Returns: **cx:dx** optr of display object

Destroyed: Nothing.

Interception: You should not subclass this message.

Library: **Objects\gDocGrpC.def**

■ MSG_GEN_DOCUMENT_GROUP_GET_DISPLAY_GROUP

If a GenDisplayGroup object is used to manage GenDisplay objects, the GenDocumentGroup object will contain an optr to the GenDisplayGroup. By using this message, you can get an optr to the GenDisplayGroup object.

Source: Unrestricted.

Destination: Any GenDocumentGroup object.

Returns: **cx:dx** optr of the GenDisplayGroup object.

Destroyed: Nothing.

Interception: You should not subclass this message.

Library: **Objects\gDocGrpC.def**

■ MSG_GEN_DOCUMENT_GROUP_GET_DOC_BY_FILE

Use this message if you know the file handle of a document and you need to get an object-pointer to the document object corresponding to the file. In the

Object Assembly Reference

inverse situation (you know the object, and need to get the file handle), send MSG_GEN_DOCUMENT_GET_FILE_HANDLE directly to the document object.

Source: Unrestricted.

Destination: Any GenDocumentGroup object.

Pass: **cx** The handle of file whose document object is needed.

Returns: **cx:dx** optr of document object (or zero if no document object has that file open).

Destroyed: **ax**

Interception: You should not subclass this message.

Library: **Objects\gDocGrpC.def**

MSG_GEN_DOCUMENT_GROUP_GET_OUTPUT

Under the Procedural model of document control, the GenDocumentGroup sends messages to a designated output object. To get the optr of that output object, send this message to the GenDocumentGroup object.

Source: Unrestricted.

Destination: Any GenDocumentGroup object.

Returns: **cx:dx** optr of the document group's output object

Destroyed: **ax**

Interception: You should not subclass this message.

Library: **Objects\gDocGrpC.def**

MSG_GEN_DOCUMENT_GROUP_GET_PROTOCOL

Use this message to get the protocol number associated with the GenDocumentGroup. A protocol number is composed of two parts, a major

component and a minor component. This message returns a double-word; the high word is the major component, and the low word is the minor component.

Source: Unrestricted.

Destination: GenDocumentGroupClass.

Pass: Nothing.

Returns: **cx** Major document protocol number.
dx Minor document protocol number.

Destroyed: **ax**

Interception: You should not subclass this message.

Library: **Objects\gDocGrpC.def**

MSG_GEN_DOCUMENT_GROUP_GET_TEMPLATE_DIR

Get the path string for the template directory.

Source: Unrestricted.

Destination: GenDocumentGroupClass.

Pass: **cx:dx** Buffer to which to store the template directory.

Returns: Nothing (template directory written to **cx:dx**).

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gDocGrpC.def**

MSG_GEN_DOCUMENT_GROUP_GET_TOKEN

This message gets the document token values for all documents created by this document control.

Source: Unrestricted.

Destination: Any GenDocumentControl object.

Pass: **cx:dx** Address of an empty **GeodeToken** structure.

Returns: The document **GeodeToken** is written to the variable whose address is passed.

Destroyed: **ax**

Interception: You should not subclass this message.

Library: **Objects\gDocGrpC.def**

Object Assembly Reference

MSG_GEN_DOCUMENT_GROUP_GET_UI_ATTRS

Use this message to find out what the GenDocumentControl object's *GDCA_attrs* flags are. The attributes cannot be changed by a message; they can only be read.

Source: Unrestricted.

Destination: Any GenDocumentGroup object.

Pass: Nothing.

Returns: **ax** **GenDocumentControlAttrs**

Destroyed: Nothing.

Interception: You should not subclass this message.

Library: **Objects\gDocGrpC.def**

MSG_GEN_DOCUMENT_GROUP_GET_UI_FEATURES

Use this message to retrieve the current *GDCI_features* flags. The flags cannot be changed at run-time.

Source: Unrestricted.

Destination: Any GenDocumentControl object.

Pass: Nothing.

Returns: **ax** **GenDocumentControlFeatures**

Destroyed: Nothing.

Interception: You should not subclass this message.

Library: **Objects\gDocGrpC.def**

MSG_GEN_DOCUMENT_GROUP_GET_VIEW

The GenDocumentGroup object can be set to automatically link document objects to **GenView** objects. Use this message to find out what the

designated **GenView** is. If there is no such **GenView**, this message will return a null optr.

Source: Unrestricted.

Destination: Any GenDocumentGroup object.

Pass: Nothing.

Returns: **cx:dx** optr of the **GenView** object (specified in *GDGI_genView).*

Destroyed: Nothing.

Interception: You should not subclass this message.

Library: **Objects\gDocGrpC.def**

MSG_GEN_DOCUMENT_GROUP_IMPORT_NEW_DOC

Create a new document and import a file into it.

Source: Internal.

Destination: GenDocumentGroup.

Pass: **ss:bp** **ImpexTranslationParams**

Returns: **cx:dx** optr of new document object created (or zero if error)

Destroyed: **ax, bp**

Library: **Objects\gDocGrpC.def**

MSG_GEN_DOCUMENT_GROUP_MARK_DIRTY

This message notifies the GenDocumentGroup object that the specified document has been dirtied. The GenDocumentGroup will enable and disable file menu triggers as appropriate. If the argument is a null pointer, the target document will be marked dirty.

Source: Unrestricted.

Destination: Any GenDocumentGroup object.

Pass: **cx:dx** optr of document to mark dirty. If a null optr is passed, the target document will be dirtied.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: You should not subclass this message.

Library: **Objects\gDocGrpC.def**

Object Assembly Reference

MSG_GEN_DOCUMENT_GROUP_MARK_DIRTY_BY_FILE

This message notifies the GenDocumentGroup that the file with the specified handle has been dirtied. The GenDocumentGroup will enable and disable file menu triggers as appropriate. The document is specified by file handle, not document optr.

Source: Unrestricted

Destination: Any GenDocumentGroup object.

Pass: `cx` The handle of the file to dirty.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: You should not subclass this message.

Library: **Objects\gDocGrpC.def**

MSG_GEN_DOCUMENT_GROUP_NEW_DOC

Create a new document.

Source: Internal.

Destination: GenDocumentGroup.

Pass: `dx` Size of **DocumentCommonParams** structure
`ss:bp` **DocumentCommonParams** structure

Returns: `cx:dx` New Document object created (or zero if error)

Destroyed: `ax, bp`

Library: **Objects\gDocGrpC.def**

MSG_GEN_DOCUMENT_GROUP_OPEN_DEFAULT_DOC

Try to open the default document (whose name is passed).

Source: Internal.

Destination: GenDocumentGroup.

Pass: `dx` Size of **DocumentCommonParams** structure
`ss:bp` **DocumentCommonParams** structure

Returns: Nothing.

Destroyed: `ax, cx, dx`

Library: **Objects\gDocGrpC.def**

■ MSG_GEN_DOCUMENT_GROUP_OPEN_DOC

Open the specified document.

Source: Internal.

Destination: GenDocumentGroup.

Pass:		
	dx	Size of **DocumentCommonParams** structure
	ss:bp	**DocumentCommonParams** structure
Returns:		
	CF	Set if error
	cx:dx	New Document object created (or zero if error)

Destroyed: **ax**

Library: **Objects\gDocGrpC.def**

■ MSG_GEN_DOCUMENT_GROUP_SAVE_AS_CANCELLED

Notification that an attempt to "Save As" a document was cancelled.

Source: Internal.

Destination: GenDocumentGroup.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gDocGrpC.def**

■ MSG_GEN_DOCUMENT_GROUP_SEARCH_FOR_DOC

See if the specified document is open, and bring it to the front if it is.

Source: A GenApplication object.

Destination: A GenDocumentGroup object.

Pass:		
	ss:bp	A **DocumentCommonParams** structure.
Returns:		
	CF	Set if document found.

Destroyed: **ax, cx, dx, bp**

Interception: This message should not be subclassed.

Library: **Objects\gDocGrpC.def**

MSG_GEN_DOCUMENT_GROUP_SET_OUTPUT

Under the Procedural model of document control, the GenDocumentGroup object sends messages to a designated output object. Use this message to change the recipient of the output.

Source: Unrestricted.

Destination: Any GenDocumentGroup object.

Pass: **cx:dx** The optr of the object which will receive the output.

Returns: Nothing.

Destroyed: **ax**

Interception: You should not subclass this message.

Library: **Objects\gDocGrpC.def**

Messages Imported from MetaClass

The following messages are imported from MetaClass. They are sent to the GenDocumentGroup object's output object (if an output object is specified). None of these messages have default handlers, of course (since they might be sent to any object). If the application does not handle the messages, they won't be handled at all.

Note that each of these messages corresponds to a GenDocument message (MSG_GEN_DOCUMENT...). An application need handle only one of each pair of messages.

Each of these messages is imported in the **MetaUIMessages** range. A handler for any of these messages is allowed to destroy **ax**, **cx**, **dx**, **bp**, and the status flags, unless the handler is supposed to return a value in one of those registers.

MSG_META_DOC_OUTPUT_INITIALIZE_DOCUMENT_FILE

The GenDocumentGroup object sends this message out when a new document has been created and needs to be initialized. Applications which use VM files will allocate the map block and initialize it. If an application maintains data caches for its files, it should initialize the caches at this point.

Note that the handler for this message should not take any UI-related actions. These should be left to the handlers for



MSG_META_DOC_OUTPUT_CREATE_UI_FOR_DOCUMENT and MSG_META_DOC_OUTPUT_ATTACH_UI_FOR_DOCUMENT.

Source: The GenDocumentGroup object.

Destination: The output of GenDocumentGroup (usually the Process object).

Pass:
cx:dx	The optr of the document object which has just been created.
bp	The FileHandle of the file which has just been created or opened.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: If you are using the Procedural model, you must write a handler for this message in whatever class will be receiving it (usually the process class).

Library: **ui.def**

■ MSG_META_DOC_OUTPUT_CREATE_UI_FOR_DOCUMENT

The GenDocumentGroup object sends this message after a document has been created or opened. Before this message is sent, the GenDocumentControl object will enable those objects on its *GDCI_enableDisableList*, and the GenDocumentGroup object will copy the GenDisplay resource for the document (if one is defined).

Applications that use dynamic UI objects will commonly respond to this message by creating the objects for the newly-opened document. Applications that use static UI objects will commonly respond to this message by enabling the objects.

Source: The GenDocumentGroup object.

Destination: The output of GenDocumentGroup (usually the Process object).

Pass:
cx:dx	The optr of the appropriate document object.
bp	The FileHandle of the appropriate file.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: If you are using the Procedural model, you must write a handler for this message in whatever class will be receiving it (usually the process class).

Library: **ui.def**

Object Assembly Reference

MSG_META_DOC_OUTPUT_ATTACH_UI_TO_DOCUMENT

The GenDocumentGroup object sends this message when the UI for a newly-opened document has been created. It also sends this message when re-opening a document as part of restoring GEOS from a state file. Applications may respond to this by attaching dynamic UI objects and setting the values of static UI objects.

Source: The GenDocumentGroup object.

Destination: The output of GenDocumentGroup (usually the Process object).

Pass:

cx:dx	The optr of the document object.
bp	The FileHandle of the appropriate file.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: If you are using the Procedural model, you must write a handler for this message in whatever class will be receiving it (usually the process class).

Library: **ui.def**

MSG_META_DOC_OUTPUT_DETACH_UI_FROM_DOCUMENT

The GenDocumentGroup sends this message when a document is being closed, whether because a user closes the file or because the application is being closed. It also sends this message when GEOS is in the process of saving itself to a state file prior to shutting down. Applications generally respond to this by detaching dynamic UI objects. Note that the GenDocumentControl object will automatically disable any objects in its *GDCI_enableDisableList*.

Source: The GenDocumentGroup object.

Destination: The output of GenDocumentGroup (usually the Process object).

Pass:

cx:dx	The optr of the document object.
bp	The FileHandle of the appropriate file.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: If you are using the Procedural model, you must write a handler for this message in whatever class will be receiving it (usually the process class).

Library: **ui.def**



MSG_META_DOC_OUTPUT_DESTROY_UI_FOR_DOCUMENT

The GenDocumentGroup object sends this message out when a document is being closed, whether because a user closes the file or because the application is being closed. Applications will generally disable static display objects and delete dynamic display objects. Note that the GenDocumentControl object will automatically disable all objects in its *GDCI_enableDisableList*, and the GenDocumentGroup will delete the display block it created for a document, if any.

Source: The GenDocumentGroup object.

Destination: The output of GenDocumentGroup (usually the Process object).

Pass:

cx:dx	The optr of the appropriate document object.
bp	The FileHandle of the appropriate file.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: If you are using the Procedural model, you must write a handler for this message in whatever class will be receiving it (usually the process class).

Library: **ui.def**

MSG_META_DOC_OUTPUT_ATTACH_FAILED

When GEOS restores itself from state, the document control tries to reattach all documents which were attached when GEOS was shut down. If this is impossible (as, for example, if a document was deleted after GEOS shut down), the GenDocumentGroup object will send this message to its output.

Source: The GenDocumentGroup object.

Destination: The output of GenDocumentGroup (usually the Process object).

Pass:

cx:dx	The optr of the appropriate document object.
bp	The FileHandle of the appropriate file.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: If you are using the Procedural model, you must write a handler for this message in whatever class will be receiving it (usually the process class).

Library: **ui.def**

MSG_META_DOC_OUTPUT_UPDATE_EARLIER_COMPATIBLE_DOCUMENT

The GenDocumentGroup object sends this message to its output when the user tries to open a document with the same major protocol number as the document control and a lower minor protocol number. Applications will commonly respond to this message by changing the document's protocol number to bring it up-to-date. If the application can't use the document, it should set the carry flag and return an error code in **ax**. (File access error codes are members of the **FileError** enumerated type, defined in **file.h**.) With an error, the document control will close the document unchanged. If the application successfully updates the document, it should clear the carry flag and set **ax** to zero.

Source: The GenDocumentGroup object.

Destination: Output of GenDocumentGroup (usually the Process object).

Pass:		
	cx:dx	The optr of the appropriate document object.
	bp	The FileHandle of the appropriate file.
Returns:		
	CF	Set if error occurs.
	ax	**FileError** code (or zero if there is no error).

Destroyed: **cx, dx, bp**

Interception: You must write a handler for this message in whatever class will be receiving it (usually the process class).

Library: **ui.def**

MSG_META_DOC_OUTPUT_UPDATE_EARLIER_INCOMPATIBLE_DOCUMENT

The GenDocumentGroup object sends this message to its output when the user tries to open a document with a lower major protocol number than the document control. Applications respond to the message by making any changes to the document necessary to make it compatible with the application. The application should also change the document's protocol numbers. If the application can't use the document, it should set the carry flag and return an error code in **ax**. (File access error codes are members of the **FileError** enumerated type, defined in **file.h**.) With an error, the document control will close the document unchanged. If the application

successfully updates the document, it should clear the carry flag and set **ax** to zero.

Source: The GenDocumentGroup object.

Destination: Output of GenDocumentGroup (usually the Process object).

Pass:	**cx:dx**	The optr of the appropriate document object.
	bp	The FileHandle of the appropriate file.
Returns:	CF	Set if error occurs.
	ax	**FileError** code (or zero if there is no error).

Destroyed: **cx, dx, bp**

Interception: You must write a handler for this message in whatever class will be receiving it (usually the process class).

Library: **ui.def**

MSG_META_DOC_OUTPUT_PHYSICAL_SAVE

If you need to take special steps to save a file, you should have a handler for either this message or MSG_GEN_DOCUMENT_PHYSICAL_SAVE. The handler should write the file completely to the disk. If an error occurs, set the carry flag and write the error code in **ax**. (File access error codes are members of the **FileError** enumerated type, defined in **file.h**.) If no error occurs, clear the carry flag and **ax**.

If, for example, you copy a DOS file into a temporary VM file while you work on it, you would probably respond to this message by copying the data from the temporary file back to the DOS file.

Source: The GenDocumentGroup object.

Destination: Output of GenDocumentGroup (usually the Process object).

Pass:	**cx:dx**	The optr of the appropriate document object.
	bp	The FileHandle of the appropriate file.
Returns:	CF	Set if error occurs.
	ax	**FileError** code (or zero if there is no error).

Destroyed: **cx, dx, bp**

Interception: DOS-based applications must handle either this message or MSG_GEN_DOCUMENT_PHYSICAL_SAVE. Applications which use GEOS data files will generally not intercept this message.

Library: **ui.def**

Object Assembly Reference

MSG_META_DOC_OUTPUT_PHYSICAL_UPDATE

This message is sent when the file is auto-saved (if this is enabled), and when the document is detached as part of a GEOS shutdown. If you need to take special steps to save a file, you should have a handler for either this message or MSG_GEN_DOCUMENT_PHYSICAL_UPDATE. The handler should write the file completely to the disk. If an error occurs, set the carry flag and write the error code in **ax**. (File access error codes are members of the **FileError** enumerated type, defined in **file.h**.) If no error occurs, clear the carry flag and **ax**.

If, for example, you copy a DOS file into a temporary VM file while you work on it, you would probably respond to this message by copying the data from the temporary file back to the DOS file.

Source: The GenDocumentGroup object.

Destination: Output of GenDocumentGroup (usually the Process object).

Pass:	**cx:dx**	The optr of the appropriate document object.
	bp	The FileHandle of the appropriate file.
Returns:	CF	Set if error occurs.
	ax	**FileError** code (or zero if there is no error).

Destroyed: **cx, dx, bp**

Interception: DOS-based applications must handle either this message or MSG_GEN_DOCUMENT_PHYSICAL_UPDATE. Applications which use GEOS data files will generally not intercept this message.

Library: **ui.def**

MSG_META_DOC_OUTPUT_PHYSICAL_SAVE_AS_FILE_HANDLE

This message is sent when the Document Control is ready to "Save As" a DOS file. The Document Control will have asked the user what the new file should be, and will have created an appropriate file. The handler for this message must write the current version of the document to the new file, and restore the original file to its last-saved state. If an error occurs, set the carry flag and write the error code in **ax**. (File access error codes are members of the



FileError enumerated type, defined in **file.h**.) If no error occurs, clear the carry flag and **ax**.

Source: The GenDocumentGroup object.

Destination: Output of GenDocumentGroup (usually the Process object).

Pass:
- **cx:dx** — The optr of the appropriate document object.
- **bp** — The FileHandle of the appropriate file.

Returns:
- CF — Set if error occurs.
- **ax** — **FileError** code (or zero if there is no error).

Destroyed: **cx, dx, bp**

Interception: DOS-based applications must handle either this message or MSG_GEN_DOCUMENT_PHYSICAL_SAVE_AS_FILE_HANDLE. Applications which use GEOS data files will generally not intercept this message.

Library: **ui.def**

MSG_META_DOC_OUTPUT_PHYSICAL_REVERT

The Document Control sends this message to revert a DOS file to its last-saved state. The handler must restore the file to its condition as of the last time it was saved. If an error occurs, set the carry flag and write the error code in **ax**. (File access error codes are members of the **FileError** enumerated type, defined in **file.h**.) If no error occurs, clear the carry flag and **ax**.

Source: The GenDocumentGroup object.

Destination: Output of GenDocumentGroup (usually the Process object).

Pass:
- **cx:dx** — The optr of the appropriate document object.
- **bp** — The FileHandle of the appropriate file.

Returns:
- CF — Set if error occurs.
- **ax** — **FileError** code (or zero if there is no error).

Destroyed: **cx, dx, bp**

Interception: DOS-based applications must handle either this message or MSG_GEN_DOCUMENT_PHYSICAL_REVERT. Applications which use GEOS data files will generally not intercept this message.

Library: **ui.def**

Object Assembly Reference

MSG_META_DOC_OUTPUT_READ_CACHED_DATA_FROM_FILE

The GenDocumentGroup sends this message when the document needs to read cached data. In particular, it sends this when a document is opened, when a document is reverted to its last-saved state, and when a document is re-opened as GEOS restores from state. If the application maintains a data cache, it should read the data from the file at this point. If the document does not cache data, it can ignore this message.

Note that if the document control notices that the file has changed on disk, it will not send this message; it will, however, send a MSG_META_DOC_OUTPUT_DOCUMENT_HAS_CHANGED. The handler for that message should reread the cache or call the handler for this message.

Source: The GenDocumentGroup object.

Destination: Output of GenDocumentGroup (usually the Process object).

Pass:

cx:dx	The optr of the appropriate document object.
bp	The FileHandle of the appropriate file.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Interception: You must write a handler for this message in whatever class will be receiving it (usually the process class).

Library: **ui.def**

MSG_META_DOC_OUTPUT_WRITE_CACHED_DATA_TO_FILE

The GenDocumentGroup object sends this message when the document needs to write cached data back to the file. In particular, it sends this message just before a document is saved, auto-saved, or “Saved As,” and before the document is closed as GEOS shuts down. The document should

write its cached data back to the file. If the document does not cache data, it can ignore this message.

Source: The GenDocumentGroup object.

Destination: Output of GenDocumentGroup (usually the Process object).

Pass:
cx:dx	The optr of the appropriate document object.
bp	The FileHandle of the appropriate file.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Interception: You must write a handler for this message in whatever class will be receiving it (usually the process class).

Warning: This message will not be sent if the document is clean. Therefore, if you change the data cache without changing the file, you should send a MSG_GEN_DOCUMENT_GROUP_MARK_DIRTY_BY_FILE to the GenDocumentGroup object.

Library: **ui.def**

MSG_META_DOC_OUTPUT_SAVE_AS_COMPLETED

The GenDocumentGroup object sends this message when a "Save As" operation has been successfully completed.

Source: The GenDocumentGroup object.

Destination: Output of GenDocumentGroup (usually the Process object).

Pass:
cx:dx	The optr of the appropriate document object.
bp	The FileHandle of the appropriate file.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Interception: You must write a handler for this message in whatever class will be receiving it (usually the process class).

Library: **ui.def**

MSG_META_DOC_OUTPUT_DOCUMENT_HAS_CHANGED

If the GDGA_AUTOMATIC_CHANGE_NOTIFICATION attribute of the GenDocumentGroup object is set to on, a timer will periodically check to see if any open documents have been changed by another application. If they have, the GenDocumentGroup object will send this message out. The

application should respond by redisplaying the data on the screen and rereading any cached data from the file.

Source: The GenDocumentGroup object.

Destination: Output of GenDocumentGroup (usually the Process object).

Pass: **cx:dx** The optr of the appropriate document object.
bp The FileHandle of the appropriate file.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Interception: You must write a handler for this message in whatever class will be receiving it (usually the process class).

Library: **ui.def**

GenDocumentControlClass

Instance Data

```
GDCI_documentToken   GeodeToken <>
   UIC Syntax:  documentTokenChars
                              = byteComp : "TOKEN_CHARS_LENGTH dup (0)";
                  documentTokenID = wordComp : 0;
```

GDCI_documentToken contains the document token for every document created by this document control. (The UIC quivalent is broken into two fields, *documentTokenChars* and *documentTokenID*.

```
GDCI_selectorType    GenFileSelectorType GFST_DOCUMENTS
   UIC Syntax:  selectorType     = enumComp byte {
                     documents,
                     executables,
                     nonGeosFiles,
                     allFiles
                  } : documents;
```

selectorType (*GDCI_selectorType*) determines which files will be displayed by the File Selector generated by this object. The field has the following flags:

documents (GFST_DOCUMENTS)
Display only GEOS documents.

executables (GFST_EXECUTABLES)
Display GEOS executables.

nonGeosFiles (GFST_NON_GEOS_FILES)
Display non-GEOS files.

allFiles (GFST_ALL_FILES)
Display all files.

```
GDCI_attrs      GenDocumentControlAttrs \
                   (GDCM_SHARED_SINGLE shl offset GDCA_MODE) or \
                   mask GDCA_VM_FILE or \
                   mask GDCA_SUPPORTS_SAVE_AS_REVERT or \
                   (GDCT_NEW shl offset GDCA_CURRENT_TASK)
   UIC Syntax:  dcAttributes     = bitFieldComp word {
                      multipleOpenFiles,
                      mode:2 = {
                         viewer,
                         sharedSingle,
                         sharedMultiple
                      },
                      dosFileDenyWrite,
                      vmFile,
                      native,
                      supportsSaveAsRevert,
                      documentExists,
                      currentTask:4 = {
                         none,
                         new,
                         open,
                         useTemplate,
                         saveAs,
                         dialog
                      },
                      doNotSaveFiles,
                      forceDemandPaging
                   } : mode sharedSingle, vmFile, supportsSaveAsRevert,
                   currentTask new;
```

dcAttributes (*GDCI_attrs*) specifies certain characteristics of the file to be opened. The field has the following flags:

multipleOpenFiles (GDCA_MULTIPLE_OPEN_FILES)
Allow several documents to be open at once. If this attribute is disabled, the "New" and "Open" triggers will be disabled when a document is open. This attribute defaults to off.

mode (GDCA_MODE)
This is a two-bit field. GDCA_MODE is a mask of all the bits in this field;

the offset of this field is equal to the constant GDCA_MODE_OFFSET. The field has the following possible settings:

viewer (GDCM_VIEWER)
: All documents are opened in read-only mode; the New, Save, Save As, and Revert triggers are permanently disabled. Other applications can open the file for read/write access.

sharedSingle (GDCM_SHARED_SINGLE)
: Documents are opened for reading and writing. When a document is open, it is marked "deny-write" so other applications can open the file only for read-only access. The user can mark a document as a "public" document, in which case the default is to open a file "read-only." The default GDCA_MODE setting is GDCM_SHARED_SINGLE.

sharedMultiple (GDCM_SHARED_MULTIPLE)
: This mode is designed for documents that can have multiple writers, such as multi-user databases. Documents are ordinarily opened as in GDCM_SHARED_SINGLE mode above; however, a user can designate a file as a "multi-user" file, which means that it can be opened by several applications at once for read/write access.

Note that the GDCA_MODE attribute has a slightly different effect if the document control manages DOS files. For details, see GDCA_DOS_FILE_DENY_WRITE below.

dosFileDenyWrite (GDCA_DOS_FILE_DENY_WRITE)
: This attribute does not matter for VM files. If a DOS file is opened while the GDCA_DOS_FILE_DENY_WRITE bit is set, no other application will be able to write to that file. This is true even if the DOS file was opened for read-only access; however, if the file is a multi-user document opened in "shared-multiple" mode, other applications will be able to write to it regardless of whether the GDCA_DOS_FILE_DENY_WRITE attribute is set. By default, GDCA_DOS_FILE_DENY_WRITE is off.

vmFile (GDCA_VM_FILE)
: This attribute specifies whether the document control objects will open GEOS Virtual Memory files (if GDCA_VM_FILE is on), or DOS files (if GDCA_VM_FILE is off). The default value is on.

native (GDCA_NATIVE)
: If this bit is set and GDCA_VM_FILE is not set, documents will be stored in the format native to the file-system.

supportsSaveAsRevert (GDCA_SUPPORTS_SAVE_AS_REVERT)
: This attribute is ordinarily set only for GEOS files. If the attribute is on, the application will use the backup functionality of VM files to support "Save As" and "Revert" functionality. If the attribute is off, the file will be altered whenever it is updated to disk. The default value is on. DOS files do not normally support "Save As" and "Revert." Applications can implement "Save As" and "Revert" functionality for DOS files by defining a subclass of **GenDocumentClass** with handlers for MSG_GEN_DOCUMENT_PHYSICAL_SAVE_AS and MSG_GEN_DOCUMENT_PHYSICAL_REVERT, but this is not recommended. Ordinarily, this attribute should be off for DOS files.

documentExists (GDCA_DOCUMENT_EXISTS)
: This attribute is set and maintained at run-time by the GenDocumentControl code. The attribute is on if at least one document is open.

currentTask (GDCA_CURRENT_TASK)
: This three-bit attribute has a dual function: It determines the application's behavior at start-up, and it indicates what task the application is currently performing. The mask GDCA_CURRENT_TASK is a mask of all the bits in this field; the field's offset is equal to the constant GDCA_CURRENT_TASK_OFFSET. The possible settings are as follows:

  none (GDCT_NONE)
  : If a default file has been specified, that file will be opened; otherwise, the application will start with no file opened.

  new (GDCT_NEW)
  : A new document will be created at startup. If the GDCF_DIALOG_BOX_FOR_NEW flag is set, a dialog box will be presented at startup.

  open (GDCT_OPEN)
  : The "Open File" dialog box will be presented at startup.

  useTemplate (GDCT_USE_TEMPLATE)
  : The "Use Template" dialog box will be presented at startup.

  saveAs (GDCT_SAVE_AS)
  : This is not a valid initial setting. The GDCA_CURRENT_TASK field has this setting between when a user chooses the "Save As" command and when the document is saved.

Object Assembly Reference

GDCT_COPY_TO
(This is not a valid initial setting, and thus cannot be set in the) **.ui** file. The GDCA_CURRENT_TASK field has this setting between when the user chooses the "Copy To" command and when the command has been fully executed.

GDCT_DIALOG

GDCT_TYPE

GDCT_PASSWORD
None of these are valid initial settings.

If a document is passed in to be opened at startup (as, for example, when a user launches an application by double-clicking a file created by the application), that file will be opened, and the initial setting of GDCA_CURRENT_TASK will be ignored. The document control automatically maintains this bitfield to correspond to whatever action the document control is currently taking. The application can find out what the document control is doing by reading the attributes and checking this field.

doNotSaveFiles (GDCA_DO_NOT_SAVE_FILES)
If this bit is set, the application will not be able to save files. By setting this bit, you can turn your application into a fully-functioning demo.

forceDemandPaging (GDCA_FORCE_DEMAND_PAGING)
If this bit is set, the document will be paged with demand-paging, even if the system ordinarily forces all documents completely into memory.

```
GDCI_features  GenDocumentControlFeatures \
                  mask GDCF_READ_ONLY_SUPPORTS_SAVE_AS_REVERT or \
                  mask GDCF_SINGLE_FILE_CLEAN_CAN_NEW_OPEN or \
                  mask GDCF_SUPPORTS_TEMPLATES or \
                  mask \
                     GDCF_SUPPORTS_USER_SETTABLE_EMPTY_DOCUMENT \
                     or \
                  mask \
                  GDCF_SUPPORTS_USER_SETTABLE_DEFAULT_DOCUMENT \
                     or \
                  mask \
                     GDCF_SUPPORTS_USER_MAKING_SHARED_DOCUMENTS \
```

```
                    or \
                mask GDCF_NAME_ON_PRIMARY
UIC Syntax:  features        = bitFieldComp word {
                    readOnlySupportsSaveAsRevert,
                    singleFileCleanCanNewOpen,
                    supportsTemplates,
                    supportsUserSettableEmptyDocument,
                    supportsUserSettableDefaultDocument,
                    supportsUserMakingSharedDocuments,
                    displayNameOnPrimary;
```

features (*GDCI_features*) specifies certain extra features of the document control. The field has the following flags:

readOnlySupportsSaveAsRevert
(GDCF_READ_ONLY_SUPPORTS_SAVE_AS_REVERT)
If this feature is on, the "Save As" and "Revert" triggers are enabled when read-only documents are opened. Once the user chooses "Save As," the new file will no longer be opened as "Read Only"; the "Save" trigger will be enabled. If this feature is off, "Save As" and "Revert" triggers are disabled for read-only files. By default, this feature is on.

singleFileCleanCanNewOpen
(GDCF_SINGLE_FILE_CLEAN_CAN_NEW_OPEN)
This feature is ignored if the attribute GDCA_MULTIPLE_OPEN_FILES is on. If GDCF_SINGLE_FILE_CLEAN_CAN_NEW_OPEN is on and GDCA_MULTIPLE_OPEN_FILES is off, the "New" and "Open" triggers are enabled when the document is "clean" (i.e., the document has not been marked "dirty" since it was opened/created or saved); that is, "New" and "Open" are enabled whenever "Save" is disabled. If the user chooses "New" or "Open" when the document is "clean," the target document is closed and the new document is opened or created. If both GDCA_MULTIPLE_OPEN_FILES and GDCF_SINGLE_FILE_CLEAN_CAN_NEW_OPEN are off, the "New" and "Open" triggers are disabled whenever a document is open. By default, this attribute is on.

supportsTemplates (GDCF_SUPPORTS_TEMPLATES)
If this feature is enabled, the user can save files as templates. If the user chooses the command "Use Template," a copy of the template is opened as a "new" document, and the template is left unchanged. If this feature is disabled, the application cannot create templates. By default, this feature is on.

supportsUserSettableEmptyDocument
(GDCF_SUPPORTS_USER_SETTABLE_EMPTY_DOCUMENT)
If this feature is enabled, the user can designate a file to be the model for all new documents. When the user chooses "New," this "model" document will be copied and the copy will be opened. By default, this feature is on.

supportsUserSettableDefaultDocument
(GDCF_SUPPORTS_USER_SETTABLE_DEFAULT_DOCUMENT)
If this attribute is on, the user can choose a default document (one which will automatically be opened when the application is launched).

supportsUserMakingSharedDocuments
(GDCF_SUPPORTS_USER_MAKING_SHARED_DOCUMENTS)
If this attribute is on, the user can save a document as "shared," allowing several processes to access it at once. By default, the attribute is on.

displayNameOnPrimary (GDCF_NAME_ON_PRIMARY)
If this attribute is on, the name of the target document is displayed at the top of the Primary window. The GenDocumentControl object does this by changing the moniker of the GenPrimary object to the name of the target document. If there is no open document, the GenPrimary will display the string specified by the attribute *GDCI_noNameText*. By default, this attribute is on.

```
GDCI_docAttrs                GenDocumentAttrs
   UIC Syntax:  docAttrs        = wordComp : 0;
```

docAttrs (*GDCI_docAttrs*) specifies certain characteristics of the file to be opened.

```
GDCI_docType                 GenDocumentType
   UIC Syntax:  docType         = wordComp : 0;
```

docType (*GDCI_docType*) specifies certain extra features of the document control.

```
GDCI_docFileHandle           optr noreloc GDCI_docFileHandle
   UIC Syntax:  docFileHandle   = wordComp : 0;
```

docFileHandle (*GDCI_docFileHandle*) specifies the file handle of the current target document. This field should not be set in the **.ui** file.

```
GDCI_emptyExists             BooleanByte
   UIC Syntax:  emptyExists = byteComp : 0;
```

emptyExists (*GDCI_emptyExists*) is *on* if a model for "empty documents" exists. If so, when the user creates a "new" document, that document will be based on the model for empty documents. This field should not be set in the **.ui** file.

```
GDCI_defaultExists          BooleanByte
   UIC Syntax:  defaultExists = byteComp : 0;
```

defaultExists (*GDCI_defaultExists*) is *on* if a default document is defined. If so, when the application starts up, that document will automatically be opened. This field should not be set in the **.ui** file.

```
GDCI_enableDisableList      lptr
   UIC Syntax:  enableDisableList = nptrComp : null;
```

enableDisableList (*GDCI_enableDisableList*) contains the chunk handle of a list of optrs. All of these objects will be enabled if any documents are open, and disabled if no documents are open.

```
GDCI_openGroup              optr
   UIC Syntax:  openGroup = optrComp : null;
```

openGroup (*GDCI_openGroup*) contains the optr to the group of objects to be added to the "Open" dialog box (if any).

```
GDCI_importGroup            optr
   UIC Syntax:     importGroup     = optrComp : null;
```

importGroup (*GDCI_importGroup*) contains the optr to the group of objects to be added to the "Import" dialog box (if any).

```
GDCI_useTemplateGroup       optr
   UIC Syntax:  useTemplateGroup = optrComp : null;
```

useTemplateGroup (*GDCI_useTemplateGroup*) contains the optr to the group of objects to be added to the "Use Template" dialog box (if any).

```
GDCI_saveAsGroup     optr
   UIC Syntax:  saveAsGroup      = optrComp : null;
```

saveAsGroup (*GDCI_saveAsGroup*) contains the optr to the group of objects to be added to the "Save As" dialog box (if any).

```
GDCI_exportGroup            optr
   UIC Syntax:  exportGroup      = optrComp : null;
```

exportGroup (*GDCI_exportGroup*) contains the optr to the group of objects to be added to the "Export" dialog box (if any).

```
GDCI_dialogGroup            optr
   UIC Syntax:  dialogGroup      = optrComp : null;
```

Object Assembly Reference

dialogGroup (*GDCI_dialogGroup*) contains the optr to the group of objects to be added to the opening dialog box (if any).

```
GDCI_userLevelGroup         optr
   UIC Syntax:  userLevelDialog  = optrComp : null;
```

userLevelDialog (*GDCI_userLevelGroup*) contains the optr to the group of objects to be added to the "User Level" dialog box (if any).

```
GDCI_noNameText             lptr
   UIC Syntax:  noNameText       = nptrComp : null;
```

noNameText (*GDCI_noNameText*) is the chunk handle of the string to be displayed in the primary window if no document is open.

```
GDCI_defaultFile            lptr
   UIC Syntax:  defaultFile      = nptrComp : null;
```

defaultFile (*GDCI_defaultFile*) contains the chunk handle of the string containing the name of the default file to open at startup (if any).

```
GDCI_templateDir            lptr
   UIC Syntax:  templateDir      = nptrComp : null;
```

templateDir (*GDCI_templateDir*) contains the chunk handle of the string containing the name of the directory where this application's templates will be stored. This directory will be a subdirectory of SP_TEMPLATE.

```
GDCI_documentGroup          optr
   UIC Syntax:  appDC            = optrComp : null;
```

appDC (*GDCI_documentGroup*) is the optr to the GenDocumentGroup object.

```
GDCI_targetDocName          FileLongName
   UIC Syntax:  targetDocName = typeComp "FileLongName" : "";
```

targetDocName (*GDCI_targetDocName*) contains the name of the current target document.

```
GDCI_dialogNewText          lptr
   UIC Syntax:  dialogNewText = nptrComp : null;

GDCI_dialogTemplateText     lptr
   UIC Syntax:  dialogTemplateText = nptrComp : null;

GDCI_dialogOpenDefaultText lptr
   UIC Syntax:  dialogOpenDefaultText = nptrComp : null;

GDCI_dialogImportText       lptr
   UIC Syntax:  dialogImportText = nptrComp : null;

GDCI_dialogOpenText         lptr
   UIC Syntax:  dialogOpenText = nptrComp : null;
```

```
GDCI_dialogUserLevelText   lptr
   UIC Syntax:  dialogUserLevelText = nptrComp : null;

GDCI_dialogNewMoniker      lptr
   UIC Syntax:  dialogNewMoniker = visMonikerComp;

GDCI_dialogTemplateMoniker lptr
   UIC Syntax:  dialogTemplateMoniker = visMonikerComp;

GDCI_dialogOpenDefaultMoniker lptr
   UIC Syntax:  dialogOpenDefaultMoniker = visMonikerComp;

GDCI_dialogImportMoniker   lptr
   UIC Syntax:  dialogImportMoniker = visMonikerComp;

GDCI_dialogOpenMoniker     lptr
   UIC Syntax:  dialogOpenMoniker = visMonikerComp;

GDCI_dialogUserLevelMonikerlptr
   UIC Syntax:  dialogUserLevelMoniker = visMonikerComp;
```

The above fields override the default text and graphic monikers in the "New/Open" dialog box.

Messages

■ MSG_GEN_DOCUMENT_CONTROL_CONFIGURE_FILE_SELECTOR

Configure file selector. This message can be sub-classed to modify the behavior of the file selectors that the document control uses.

Source: Internal.

Destination: GenDocumentControl.

Pass:

cx:dx	Optr of file selector
bp	**GenDocumentControlAttrs**

Returns: Nothing.

Destroyed: Nothing.

Library: **Objects\gDocCtrl.def**

MSG_GEN_DOCUMENT_CONTROL_DISPLAY_DIALOG

This message forces the document control to display the opening "New/Open" dialog box, as if the user had selected the "New/Open" trigger on the File menu.

Source: Unrestricted.

Destination: The GenDocumentControl object.

Pass: None.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: This message is not normally intercepted.

Library: **Objects\gDocCtrl.def**

MSG_GEN_DOCUMENT_CONTROL_FILE_CHANGED

This message instructs the GenDocumentControl that the file name has changed, and it should update any UI that uses the file name.

Source: Internal.

Destination: GenDocumentControl.

Pass: `ss:bp` **DocumentFileChangedParams**

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gDocCtrl.def**

MSG_GEN_DOCUMENT_CONTROL_FILE_EXPORTED

Notification that a file has been successfully exported.

Source: Internal.

Destination: GenDocumentControl.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Library: **Objects\gDocCtrl.def**

MSG_GEN_DOCUMENT_CONTROL_GET_ATTRS

Use this message to find out what the GenDocumentControl object's *GDCA_attrs* flags are. The attributes cannot be changed by a message; they can only be read.

Source: Unrestricted.

Destination: Any GenDocumentControl object.

Pass: Nothing.

Returns: **ax** *GDCA_attrs* record.

Destroyed: Nothing.

Interception: You should not subclass this message.

Library: **Objects\gDocCtrl.def**

MSG_GEN_DOCUMENT_CONTROL_GET_CREATOR

This message gets the token for the application; this token is the "creator token" for all files created by the document control.

Source: Unrestricted.

Destination: Any GenDocumentControl object.

Pass: **cx:dx** A pointer to an empty **GeodeToken** structure.

Returns: The document **GeodeToken** is written to the variable whose address is passed.

Destroyed: **ax**

Interception: You should not subclass this message.

Library: **Objects\gDocCtrl.def**

MSG_GEN_DOCUMENT_CONTROL_GET_FEATURES

Use this message to retrieve the current *GDCI_features* flags. The flags cannot be changed at run-time.

Source: Unrestricted.

Destination: Any GenDocumentControl object.

Pass: Nothing.

Returns: **ax** The GenDocumentControl's *GDCI_features* flags.

Destroyed: Nothing.

Interception: You should not subclass this message.

Library: **Objects\gDocCtrl.def**

MSG_GEN_DOCUMENT_CONTROL_GET_TEMPLATE_DIR

Get the path-string of the template directory.

Source: Internal.

Destination: GenDocumentControl.

Pass: **cx:dx** Buffer to which to store the template path.

Returns: Path written to passed buffer.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gDocCtrl.def**

MSG_GEN_DOCUMENT_CONTROL_GET_TOKEN

This message gets the document token values for all documents created by this document control.

Source: Unrestricted.

Destination: Any GenDocumentControl object.

Pass: **cx:dx** Address of an empty **GeodeToken** structure.

Returns: The document **GeodeToken** is written to the passed address.

Destroyed: **ax**

Interception: You should not subclass this message.

Library: **Objects\gDocCtrl.def**

■ MSG_GEN_DOCUMENT_CONTROL_IMPORT_CANCELLED

Notification that an attempted import has been cancelled.

Source: Internal.

Destination: GenDocumentControl.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Library: **Objects\gDocCtrl.def**

■ MSG_GEN_DOCUMENT_CONTROL_INITIATE_COPY_TO_DOC

This message forces the document control to copy the active file to a new name (bringing up the appropriate file selector), exactly as if the user had requested it.

Source: Unrestricted.

Destination: The GenDocumentControl object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: This message is not normally intercepted.

Library: **Objects\gDocCtrl.def**

■ MSG_GEN_DOCUMENT_CONTROL_INITIATE_EXPORT_DOC

This message forces the document control to export the active file (bringing up the appropriate file selector), exactly as if the user had requested it.

Source: Unrestricted.

Destination: The GenDocumentControl object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: This message is not normally intercepted.

Library: **Objects\gDocCtrl.def**

MSG_GEN_DOCUMENT_CONTROL_INITIATE_IMPORT_DOC

This message forces the document control to import a file (bringing up the appropriate file selector), exactly as if the user had requested it.

Source: Unrestricted.

Destination: The GenDocumentControl object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: This message is not normally intercepted.

Library: **Objects\gDocCtrl.def**

MSG_GEN_DOCUMENT_CONTROL_INITIATE_MOVE_TO_DOC

This message forces the document control to move a file (bringing up the appropriate file selector), exactly as if the user had requested it.

Source: Unrestricted.

Destination: The GenDocumentControl object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: This message is not normally intercepted.

Library: **Objects\gDocCtrl.def**

MSG_GEN_DOCUMENT_CONTROL_INITIATE_NEW_DOC

This message forces the document control to create a new file, exactly as if the user had requested it.

Source: Unrestricted.

Destination: The GenDocumentControl object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: This message is not normally intercepted.

Library: **Objects\gDocCtrl.def**



MSG_GEN_DOCUMENT_CONTROL_INITIATE_OPEN_DOC

This message forces the document control to open a file (bringing up the appropriate file selector), exactly as if the user had requested it.

Source: Unrestricted.

Destination: The GenDocumentControl object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: This message is not normally intercepted.

Library: **Objects\gDocCtrl.def**

MSG_GEN_DOCUMENT_CONTROL_INITIATE_SAVE_AS_DOC

This message forces the document control to open a file (bringing up the appropriate file selector), exactly as if the user had requested it.

Source: Unrestricted.

Destination: The GenDocumentControl object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: This message is not normally intercepted.

Library: **Objects\gDocCtrl.def**

MSG_GEN_DOCUMENT_CONTROL_INITIATE_SAVE_AS_TEMPLATE_DOC

This message forces the document control to save the active file as a template (bringing up the appropriate file selector), exactly as if the user had requested it.

Source: Unrestricted.

Destination: The GenDocumentControl object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: This message is not normally intercepted.

Library: **Objects\gDocCtrl.def**

MSG_GEN_DOCUMENT_CONTROL_INITIATE_SAVE_DOC

This message forces the document control to save the active file, exactly as if the user had requested it.

Source: Unrestricted.

Destination: The GenDocumentControl object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: This message is not normally intercepted.

Library: **Objects\gDocCtrl.def**



MSG_GEN_DOCUMENT_CONTROL_INITIATE_SET_PASSWORD_DOC

This message forces the document control to set the password of the active file (bringing up the appropriate dialog box), exactly as if the user had requested it.

Source: Unrestricted.

Destination: The GenDocumentControl object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: This message is not normally intercepted.

Library: **Objects\gDocCtrl.def**

MSG_GEN_DOCUMENT_CONTROL_INITIATE_SET_TYPE_DOC

This message forces the document control to change the type (public, read-only, etc.) of the active file (bringing up the appropriate dialog box), exactly as if the user had requested it.

Source: Unrestricted.

Destination: The GenDocumentControl object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: This message is not normally intercepted.

Library: **Objects\gDocCtrl.def**

MSG_GEN_DOCUMENT_CONTROL_INITIATE_USE_TEMPLATE_DOC

This message forces the document control to create a new file from a template (bringing up an appropriate file selector), exactly as if the user had requested it.

Source: Unrestricted.

Destination: The GenDocumentControl object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: This message is not normally intercepted.

Library: **Objects\gDocCtrl.def**

MSG_GEN_DOCUMENT_CONTROL_OPEN_DEFAULT_DOC

This message instructs the document control to open the default document.

Source: Unrestricted.

Destination: The GenDocumentControl object.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Library: **Objects\gDocCtrl.def**

MSG_GEN_DOCUMENT_CONTROL_OPEN_IMPORT_SELECTED

This message is sent to the Document Control after the user selects a file to import (from the Import dialog box). The document control should create a new, unnamed document, and send MSG_GEN_DOCUMENT_IMPORT to it.

Source: Internal.

Destination: GenDocumentControl.

Pass: `ss:bp` **ImpexTranslationParams**

Returns: Nothing.

Library: **Objects\gDocCtrl.def**



MSG_GEN_DOCUMENT_CONTROL_SAVE_AS_CANCELLED

The user has cancelled a “Save As” attempt.

Source: Internal.

Destination: GenDocumentControl.

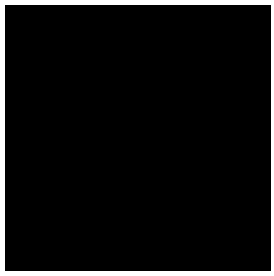
Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gDocCtrl.def

GenFileSelector



GenFileSelectorClass 441

14

GenFileSelectorClass

Instance Data

```
GFSI_selection       GenFileSelectorSelection <0>
   UIC Syntax:  selection = typeComp "GenFileSelectorSelection" : "";
```

selection (*GFSI_selection*) maintains the name of the current selection. The current full path, volume name, and disk are stored in variable data entries defined by GenClass. The application can easily, at any time, retrieve or set the current selection, path, or volume. Volume names, paths, and file names are all stored as null-terminated character strings in the GEOS character set.

The File Selector's current path is stored in a variable data field called ATTR_GEN_PATH_DATA, a variable data field defined in GenClass.

```
GFSI_fileCriteria    FileSelectorFileCriteria \
                  mask FSFC_DIRS or \
                  mask FSFC_NON_GEOS_FILES or \
                  mask FSFC_GEOS_EXECUTABLES or \
                  mask FSFC_GEOS_NON_EXECUTABLES
   UIC Syntax:  fileCriteria = bitFieldComp word {
                  dirs,
                  nonGeosFiles,
                  geosExecutables,
                  geosNonExecutables,
                  maskCaseInsensitive,
                  fileFilter,
                  filterIsC,
                  tokenNoID,
                  useMaskForDirs
                  } : dirs, nonGeosFiles, geosExecutables,
                  geosNonExecutables;
```

fileCriteria (*GFSI_fileCriteria*) is a record that indicates which of several search limiters should be considered in a directory scan. There are several possible entries in *GFSI_fileCriteria*, and any or all may be turned on at once. Each flag in the field is shown below, along with the field that it affects:

FSFC_DIRS
: This flag indicates that subdirectories should be displayed in the File Selector's directory scans. This flag is set by default.

FSFC_NON_GEOS_FILES
: This flag indicates that non-GEOS files should be displayed. This flag is set by default.

FSFC_GEOS_EXECUTABLES
: This flag indicates that GEOS executable files should be displayed. This flag is set by default.

FSFC_GEOS_NON_EXECUTABLES
: This flag indicates that GEOS non-executable files (documents and other types) should be displayed. This flag is set by default.

FSFC_MASK_CASE_INSENSITIVE
: This flag indicates that the filename mask (if any) should be case-insensitive. The mask applies only to files unless FSFC_USE_MASK_FOR_DIRS is also set (see below); the mask is set in ATTR_GEN_FILE_SELECTOR_NAME_MASK.

FSFC_FILE_FILTER
: This flag makes the File Selector use an application-defined callback routine when scanning. The callback routine is used after all other filters have been applied. The callback routine must be defined in a subclass of **GenFileSelectorClass**.

FSFC_FILTER_IS_C
: This flag indicates that the callback filter routine is written in C and follows the Pascal calling convention.

FSFC_TOKEN_NO_ID
: This flag indicates that when using a token to match files, the File Selector should ignore the token ID. In other words, it will only match the token characters.

FSFC_USE_MASK_FOR_DIRS
: This flag indicates that if a filename mask is used, the File Selector should apply the mask to directories as well as to files. By default, the mask is applied only to file names.

```
GFSI_attrs     FileSelectorAttrs \
                  (mask FSA_ALLOW_CHANGE_DIRS or \
                  mask FSA_HAS_CLOSE_DIR_BUTTON or \
                  mask FSA_HAS_OPEN_DIR_BUTTON or \
                  mask FSA_HAS_DOCUMENT_BUTTON or \
                  mask FSA_HAS_CHANGE_DIRECTORY_LIST or \
```

Object Assembly Reference

```
                mask FSA_HAS_CHANGE_DRIVE_LIST or \
                mask FSA_HAS_FILE_LIST)
UIC Syntax:  attributes = bitFieldComp word {
                allowChangeDirs,
                manualControl,
                showFixedDisksOnly,
                showFilesDisabled,
                hasCloseDirButton,
                hasOpenDirButton,
                hasDocumentButton,
                hasChangeDirectoryList,
                hasChangeDriveList,
                hasFileList,
                useVirtualRoot,
                showWritableDisksOnly
                : allowChangeDirs,hasCloseDirButton,
                hasOpenDirButton, hasDocumentButton,
                hasChangeDirectoryList, hasChangeDriveList,
                hasFileList;
```

attributes (*GFSI_attrs*) holds many flags, shown below. They are stored as a word-sized record of type **FileSelectorAttrs**; they may be retrieved with MSG_GEN_FILE_SELECTOR_GET_ATTRS and set with MSG_GEN_FILE_SELECTOR_SET_ATTRS.

FSA_ALLOW_CHANGE_DIRS
: This flag allows the user to change directories by double clicking; it is set by default.

FSA_SHOW_FIXED_DISKS_ONLY
: This flag does not allow removable volumes (e.g. floppy disks) to be shown in the File Selector volume list. By default, this flag is off.

FSA_SHOW_FILES_DISABLED
: This flag instructs the File Selector to display files as disabled. This is useful for SaveAs operations, for example, when files must be displayed but should not be selectable by the user. By default, this flag is off. Setting this flag will turn all other search criteria off, so all filenames will be displayed.

FSA_HAS_CLOSE_DIR_BUTTON
: This flag causes the File Selector to include a “Close Directory” button.

FSA_HAS_OPEN_DIR_BUTTON
: This flag causes the File Selector to include an “Open Directory” button.

FSA_HAS_DOCUMENT_BUTTON
: This flag causes the File Selector to include a "Go to Document" button that displays the DOCUMENT directory.

FSA_HAS_CHANGE_DIRECTORY_LIST
: This flag causes the File Selector to include a list that allows the user to change directories.

FSA_HAS_CHANGE_DRIVE_LIST
: This flag causes the File Selector to include a list that allows the user to change drives.

FSA_HAS_FILE_LIST
: This flag causes the File Selector to include a file list.

FSA_USE_VIRTUAL_ROOT
: This flag causes the File Selector to use the information in the ATTR_GEN_FILE_SELECTOR_VIRTUAL_ROOT attribute. This allows changing the File Selector without altering the vardata.

`GFSI_destination optr`

`GFSI_notificationMsg word`

```
UIC Syntax:  action = actionComp : null;
```

action (*GFSI_destination* and *GFSI_notificationMsg*): When defining your GenFileSelector, you must designate an output object and a message that will be sent to it when the user makes a selection. The message is stored in the *GFSI_notificationMsg* instance field, and the object's optr is stored in the *GFSI_destination* instance field. Note that instead of an optr, you may use a **TravelOption** as the output destination (TO_TARGET, TO_FOCUS, etc.); you can also use ATTR_GEN_DESTINATION_CLASS to specify the class of the output object to ensure the notification message is only delivered to a class that can handle it.

`ATTR_GEN_FILE_SELECTOR_TOKEN_MATCH vardata GeodeToken`

Since every GEOS file has a token associated with it, you can limit directory scans to include only those files with a given token. (Note that directories and volumes are not affected by this limitation.) Alternatively, because the token is made up of two items, the token chars and the token ID, you can arrange a search based on just the token characters (you can not search on only the ID).

Both the match token ID and the match token characters are stored in the File Selector's ATTR_GEN_FILE_SELECTOR_TOKEN_MATCH instance field.

`ATTR_GEN_FILE_SELECTOR_CREATOR_MATCH vardata GeodeToken`

Object Assembly Reference

Most applications will store their documents in a special type of file; each of these files will include the token of the creator geode. (This is a function of the Document Control objects.)

As a result, you may limit the File Selector's searches to only those files created by a given geode. Set the token to be matched in the File Selector's ATTR_GEN_FILE_SELECTOR_CREATOR_MATCH variable data instance field. This search is employed in the same manner as a token search (above), so it is also subject to the FSFC_TOKEN_NO_ID attribute of *GFSI_fileCriteria*.

`ATTR_GEN_FILE_SELECTOR_FILE_ATTR vardata GenFileSelectorFileAttrs`

To filter files based on the FileAttrs record, set your File Selector's variable data instance field ATTR_GEN_FILE_SELECTOR_FILE_ATTR to reflect the attributes that must be on and those that must be off.

`ATTR_GEN_FILE_SELECTOR_GEODE_ATTR vardataGenFileSelectorGeodeAttrs`

Every geode in GEOS has a structure of type GeodeAttrs stored in its file. As a result, you can limit the File Selector's searches to include only a certain set of geodes—those with the attributes set in the File Selector's ATTR_GEN_FILE_SELECTOR_GEODE_ATTR vardata instance field.

`ATTR_GEN_FILE_SELECTOR_NAME_MASK vardata GenFileSelectorMask`

By setting up a filename mask, you can limit the File Selector's displays to only those files that conform to the mask characters. For example, to show only those files with the .BAT extender, you could set the mask to *.BAT.

Only one mask may be in use at any given time. If you need to mask for two different sets of characters (e.g. *.BAT and *.EXE), you will need to subclass the File Selector and modify its behavior.

The mask is stored in the File Selector's variable data instance attribute ATTR_GEN_FILE_SELECTOR_NAME_MASK and consists of a null-terminated character string. This field is defined as type **GenFileSelectorMask**, the same as **FileLongName**. This string may contain the DOS "*" and "?" wildcard characters. The filter applied by the File Selector is the same as that applied by the **FileEnum()** routine.

`ATTR_GEN_FILE_SELECTOR_VIRTUAL_ROOT vardata GenFilePath`

This vardata field defines a "virtual root" for the file selector. The user will not be allowed to navigate above the "virtual root" except with the Change Drives popup list, if available. If the Change Drives list is used, the virtual root is afterwards ignored

`HINT_FILE_SELECTOR_NUMBER_OF_FILES_TO_SHOW vardata word`

This hint determines the number of files that are visible at once

```
HINT_FILE_SELECTOR_SCALABLE_UI_DATA \
                    vardata GenFileSelectorScalableUIEntry
```

The File Selector lets you set which features you want based on the current application features, using HINT_FILE_SELECTOR_SCALABLE_UI_DATA. This hint will query the application object to get the current application features set, and it will set the File Selector's feature set (*GFSI_attrs*) accordingly.

```
HINT_FILE_SELECTOR_FILE_LIST_WIDTH      vardata word
```

This hint defines the width of the file list. The data is a number of average-width characters, up to a maximum of 255.

Messages

MSG_GEN_FILE_SELECTOR_END_SUSPEND

This message unsuspends a File Selector that had previously been suspended with MSG_GEN_FILE_SELECTOR_SUSPEND. This message automatically causes a directory rescan. Because multiple suspension attempts are disallowed, this message only needs to be sent once to unsuspend the File Selector.

Pass: Nothing.

Returns: CF — Clear if successful (file list rescanned); otherwise set.

Destroyed: ax, cx, dx, bp.

Library: Objects\gFSelC.def

MSG_GEN_FILE_SELECTOR_GET_ACTION

This message returns the File Selector's output object and notification message.

Pass: Nothing.

Returns: ^l**cx**:**dx** — Object of action descriptor.
bp — Message of action descriptor.

Destroyed: dx.

Library: Objects\gFSelC.def

Object Assembly Reference

MSG_GEN_FILE_SELECTOR_GET_ATTRS

This message returns the current attributes set in the File Selector's *GFSI_attrs* instance data.

Pass: Nothing.

Returns: `cx` Current **FileSelectorAttrs** structure.

Destroyed: `ax, dx, bp.`

Library: **Objects\gFSelC.def**

MSG_GEN_FILE_SELECTOR_GET_CREATOR

This message returns the current creator match information set in the File Selector's ATTR_GEN_FILE_SELECTOR_CREATOR_MATCH field.

Pass: Nothing.

Returns:

`cl`	First character of creator token.
`ch`	Second character of creator token.
`dl`	Third character of creator token.
`dh`	Fourth character of creator token.
`bp`	**ManufacturerID** of creator token.

Destroyed: `ax.`

Library: **Objects\gFSelC.def**

MSG_GEN_FILE_SELECTOR_GET_DESTINATION_PATH

Returns null-terminated complete path (no drive letter— drive is determined by disk handle) for a destination-related operation like "Save As", "Copy To", etc. Note that as for **FileGetCurrentPath()**, if the object is within a standard directory, the disk handle will be a **StandardPath** constant and the path will appear to be relative. However, the **StandardPath** constant

and relative-path combination may be not be the best combination possible, so do not rely on this (use **FileParseStandardPath()** if necessary).

Pass:	**dx**:**bp**	Buffer in which to copy path string.
	cx	Size of buffer (may be zero).
Returns:	CF	Set if error (path won't fit in buffer); otherwise clear.
	ax	On error, the number of bytes required in the buffer to write out the full path string; otherwise destroyed.
	cx	Disk handle of the path.
	dx:**bp**	Preserved; on success, the buffer will be filled with the destination path string.
Destroyed:	**Nothing.**	
Library:	**Objects\gFSelC.def**	

MSG_GEN_FILE_SELECTOR_GET_FILE_ATTRS

This message returns the ATTR_GEN_FILE_SELECTOR_FILE_ATTR of the File Selector. This attribute contains two byte-sized fields which represent the match and mismatch attributes.

Pass:	Nothing.	
Returns:	**cl**	**FileAttrs** that must be set for a match.
	ch	**FileAttrs** that indicate a mismatch if set.
Destroyed:	**ax, dx, bp.**	
Library:	**Objects\gFSelC.def**	

MSG_GEN_FILE_SELECTOR_GET_FILE_CRITERIA

This message returns the File Selector's current *GFSI_fileCriteria* record.

Pass:	Nothing.	
Returns:	**cx**	**FileSelectorFileCriteria** field.
Destroyed:	**ax, dx, bp.**	
Library:	**Objects\gFSelC.def**	

MSG_GEN_FILE_SELECTOR_GET_FILTER_ROUTINE

This message returns the address of the callback routine used by the File Selector as well as the array of **FileExtAttrDesc** structures specifying the callback's parameters.

For the callback function to be used, you must set FSFC_FILE_FILTER in the File Selector's *GFSI_fileCriteria* instance data field. If your callback routine

is in C, you must also set FSFC_FILTER_IS_C. In addition, the callback routine must follow the Pascal calling convention.

Source: Sent by the File Selector object to itself before calling **FileEnum()**. Also may be used by other objects to retrieve the callback information for the File Selector.

Pass: Nothing.

Returns:
- **cx**:**ax** — Far pointer to routine to call. If routine is in movable memory, **ax** should be a virtual segment. This is normally dealt with automatically if you declare an fptr vector from which you load the address.
- **bp**:**dx** — Far pointer to **FileExtAttrDesc** array of additional attributes to fetch. **bp** is zero if no additional attributes are required. If the array is in movable memory, then **bp** must be a virtual segment.

Destroyed: ax, cx, dx.

Library: Objects\gFSelC.def

MSG_GEN_FILE_SELECTOR_GET_FULL_SELECTION_PATH

This message causes the File Selector to set its path data and its *GFSI_selection* field according to the string passed in selection. The string's format is described below. The entry may be in any directory on any volume; the volume is specified by the passed disk handle.

If the File Selector is suspended or not currently on the screen when it receives this message, it will set the path and selection but will do nothing else until it becomes unsuspended or visible. Otherwise, the message will be handled immediately, and the File Selector will navigate to and scan the proper directory, setting the selection if possible. If the path is valid but the selection is not, the File Selector will display the proper file list and set the selection to the first entry in the list. If any part of the path other than the selection is invalid, the File Selector will show the volume list.

Pass:
- **cx**:**dx** — Address to copy buffer to. Pass a **cx** of zero if selection name not desired. Buffer should be of at least size

PATH_BUFFER_SIZE + FILE_LONGNAME_BUFFER_SIZE bytes.

Returns: **cx**:**dx** Preserved, buffer filled with selection path string.
ax Disk handle of path.
bp **GenFileSelectorEntryFlags** giving information about the currently selected file. GFSEF_OPEN and GFSEF_ERROR are not used.

Library: **Objects\gFSelC.def**

MSG_GEN_FILE_SELECTOR_GET_GEODE_ATTRS

This message returns the File Selector's current geode attribute match information.

Pass: Nothing.

Returns: **cx** **GeodeAttrs** to match.
dx **GeodeAttrs** not to match.

Destroyed: **ax, bp.**

Library: **Objects\gFSelC.def**

MSG_GEN_FILE_SELECTOR_GET_MASK

This message returns the mask string in the File Selector's ATTR_GEN_FILE_SELECTOR_NAME_MASK field.

Pass: **cx**:**dx** Buffer to write mask string to.

Returns: CF Set if filemask defined.
cx:**dx** Preserved, buffer filled with mask string if defined.

Destroyed: **ax, bp.**

Library: **Objects\gFSelC.def**

Object Assembly Reference

MSG_GEN_FILE_SELECTOR_GET_SELECTION

This message returns the name, the entry number, and the entry flags of the file or directory currently selected.

Pass:	**cx:dx**	Address to write selection string to.
Returns:	**cx:dx**	Preserved, buffer filled with selection string; null if no entries selected.
	ax	Entry number of selection.
	bp	**GenFileSelectorFlags** field. GFSEF_OPEN and GFSEF_ERROR are not used.
Destroyed:	**Nothing.**	
Library:	**Objects\gFSelC.def**	

MSG_GEN_FILE_SELECTOR_GET_TOKEN

This message returns the current token match information set in the File Selector's ATTR_GEN_FILE_SELECTOR_TOKEN_MATCH field.

Pass:	Nothing.	
Returns:	**cl**	First character of token.
	ch	Second character of token.
	dl	Third character of token.
	dh	Fourth character of token.
	bp	ManufacturerID of token.
Destroyed:	**ax**	
Library:	**Objects\gFSelC.def**	

MSG_GEN_FILE_SELECTOR_OPEN_ENTRY

This message causes the File Selector to open and display the contents of the entry specified. It returns an error flag: if the entry opens successfully, the error flag is returned false; if some error occurs, the flag is returned true.

The entry specified with this message must be in the current file list. First, retrieve the entry number from the File Selector with the message MSG_GEN_FILE_SELECTOR_GET_SELECTION. If the entry can be opened (i.e. it is in the current file list and it is a volume or directory), the File Selector will open it, scan the directory, and display the new file list. If the

entry is not a directory or volume, the File Selector will do nothing. This message is only valid when the File Selector is visible on the screen.

Pass: **cx** Entry number to open.

Returns: CF Set if successful; false if unable to open entry.

Destroyed: **ax, cx, dx, bp.**

Warning: If an error is returned, the following notification method, if any, will not have the GFSEF_ERROR bit set. Use the carry flag returned here to determine that an error has occured.

Library: **Objects\gFSelC.def**

MSG_GEN_FILE_SELECTOR_RESCAN

This message causes the File Selector to rescan its current directory. The File Selector will re-build and re-display its entire file list. This message will only work when the File Selector is visible on the screen.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gFSelC.def**

MSG_GEN_FILE_SELECTOR_SET_ACTION

This message sets the File Selector's *GFSI_notificationMsg* and *GFSI_destination* fields.

Pass: ^l**cx**:**dx** Object of action descriptor.
bp Message of action descriptor.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gFSelC.def**

MSG_GEN_FILE_SELECTOR_SET_ATTRS

This message sets the File Selector's *GFSI_attrs* record to a new set of attributes. The File Selector will rescan the current directory using the new

attributes. If the File Selector is not visible when it receives this message, it will set the attributes but will not rescan the directory.

Pass: `cx` New **FileSelectorAttrs**.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\gFSelC.def**

MSG_GEN_FILE_SELECTOR_SET_CREATOR

This message sets the ATTR_GEN_FILE_SELECTOR_CREATOR_MATCH field to the two passed values. This field is a vardata instance record of type **GeodeToken**. The token set with this message will be used to filter out certain files; directories and volumes are not affected by this filter. Only files which have their creator token the same as that of the File Selector will be displayed.

If the File Selector is visible when it receives this message, it will rescan the directory using the new token as a filter. Otherwise, the File Selector will store the passed token for later use.

Pass:

`cl`	First character of new creator token.
`ch`	Second character of new creator token.
`dl`	Third character of new creator token.
`dh`	Fourth character of new creator token.
bp	**ManufacturerID** of creator token.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\gFSelC.def**

MSG_GEN_FILE_SELECTOR_SET_FILE_ATTRS

This message sets the ATTR_GEN_FILE_SELECTOR_FILE_ATTR vardata instance field of the File Selector. If the File Selector is visible on the screen when it receives this message, it will rescan the directory immediately with the new attributes.

Pass:

`cl`	**FileAttrs** that must be set for a match.
`ch`	**FileAttrs** that indicate a mismatch if set.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\gFSelC.def**

MSG_GEN_FILE_SELECTOR_SET_FILE_CRITERIA

This message sets the File Selector's *GFSI_fileCriteria* record to a new set of flags. If the File Selector is visible when it receives this message, it will rescan the current directory with the new attributes. Otherwise, it will set the *GFSI_fileCriteria* record but will not rescan.

Pass: **cx** New **FileSelectorFileCriteria**.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gFSelC.def**

MSG_GEN_FILE_SELECTOR_SET_FULL_SELECTION_PATH

This message causes the File Selector to set its path data and its *GFSI_selection* field according to the string passed in selection. The string's format is described below. The entry may be in any directory on any volume; the volume is specified by the passed disk handle.

If the File Selector is suspended or not currently on the screen when it receives this message, it will set the path and selection but will do nothing else until it becomes unsuspended or visible. Otherwise, the message will be handled immediately, and the File Selector will navigate to and scan the proper directory, setting the selection if possible.

If the path is valid but the selection is not, the File Selector will display the proper file list and set the selection to the first entry in the list. If any part of the path other than the selection is invalid, the File Selector will show the volume list.

Pass: **cx**:**dx** Buffer containing null-terminated pathname to match. This can only be a path and a file/directory name. No volume entry (i.e. drive letter) is allowed. This name is case-sensitive. If a partial path is passed, it is relative to the current path and the passed disk handle is ignored.

bp Disk handle of path (if zero, FileSelector will use its current disk handle).

Returns: CF Clear if entry exists and was successfully selected; set otherwise.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gFSelC.def**

MSG_GEN_FILE_SELECTOR_SET_GEODE_ATTRS

This message sets the File Selector's vardata instance field ATTR_GEN_FILE_SELECTOR_GEODE_ATTR to the two passed values.

If the File Selector is visible when it receives this message, it will automatically rescan the current directory, applying the new geode filter. Otherwise, the new geode match/mismatch records will be stored but no rescan will occur.

Pass: **cx** **GeodeAttrs** to match.
dx **GeodeAttrs** to mismatch.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gFSelC.def**

MSG_GEN_FILE_SELECTOR_SET_MASK

This message sets the ATTR_GEN_FILE_SELECTOR_NAME_MASK variable instance field of the File Selector. This field contains a null-terminated character string (of type **GenFileSelectorMask**); during a directory scan, the File Selector checks all filenames against this mask, and only those files that contain the mask characters are displayed. The mask string may contain DOS wildcard characters ("*" and "?").

The mask works for both DOS and GEOS files. However, unless the FSFC_USE_MASK_FOR_DIRS flag is set in the *GFSI_fileCriteria* attribute, directories will be unaffected by the mask. Volumes are unaffected in any case.

If the File Selector is visible on the screen when it receives this message, it will automatically rescan the current directory with the new mask. Otherwise, the new mask will be stored for later use.

Pass: **cx**:dx Null-terminated file mask string.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\gFSelC.def**

MSG_GEN_FILE_SELECTOR_SET_SELECTION

This message causes the File Selector to select a given file, directory, or volume from the currently-displayed file list. It sets the selection by setting the File Selector's *GFSI_selection* field to the passed string.

If the File Selector is not on the screen or is suspended when it receives this message, it will set the selection and hold it until it scans the directory. The validity of the selection is not determined until the directory is again scanned. If the directory is not scanned immediately, the message's returned CF will be set. If the File Selector is on the screen and is not suspended, it will determine the validity of the selection immediately and return a value appropriate to the file's validity.

If the selection is valid, the user will see it highlighted in the File Selector's display. The selection will not be opened by this message; it is only selected.

Pass:	**cx:dx**	Buffer containing null-terminated name to match. Depending on the current mode, this can be a volume name or a file/directory name. This is case-sensitive.
Returns:	CF	Clear if entry exists and was successfully selected; set otherwise.
Destroyed:	**ax, cx, dx, bp.**	
Library:	**Objects\gFSelC.def**	

MSG_GEN_FILE_SELECTOR_SET_TOKEN

This message sets the ATTR_GEN_FILE_SELECTOR_TOKEN_MATCH field to the two passed values. The token set with this message will be used to filter out certain files; directories and volumes are not affected by this filter. If the File Selector is visible when it receives this message, it will rescan the current directory using the new token information. Otherwise, the File Selector will store the passed token.

Pass:	**cl**	First character of token.
	ch	Second character of token.
	dl	Third character of token.
	dh	Fourth character of token.
	bp	**ManufacturerID** of token.
Returns:	Nothing.	
Destroyed:	**ax, cx, dx, bp.**	
Library:	**Objects\gFSelC.def**	

MSG_GEN_FILE_SELECTOR_SUSPEND

This message causes the File Selector to suspend directory scans until a MSG_GEN_FILE_SELECTOR_END_SUSPEND is received. This provides the ability to avoid unnecessary directory rescans when several different attributes are being changed in succession (e.g. volume, mask, and file types).

No user action is affected by this message; only the following attribute-setting messages will be affected (their normal automatic rescan will not happen):
MSG_GEN_FILE_SELECTOR_SET_SELECTION
MSG_GEN_FILE_SELECTOR_SET_FULL_SELECTION_PATH
MSG_GEN_FILE_SELECTOR_SET_MASK
MSG_GEN_FILE_SELECTOR_SET_FILE_ATTRS
MSG_GEN_FILE_SELECTOR_SET_TOKEN
MSG_GEN_FILE_SELECTOR_SET_CREATOR
MSG_GEN_FILE_SELECTOR_SET_GEODE_ATTRS
MSG_GEN_FILE_SELECTOR_SET_ATTRS
MSG_GEN_FILE_SELECTOR_SET_FILE_CRITERIA

In addition to the above messages, the GenClass message MSG_GEN_PATH_SET will also be suspended.

Pass: Nothing.

Returns: CF — Clear if successful; set if File Selector was already suspended.

Destroyed: ax, cx, dx, bp.

Library: Objects\gFSelC.def

MSG_GEN_FILE_SELECTOR_UP_DIRECTORY

This message causes the File Selector to go up one directory in the directory tree. If the File Selector is already in the root directory, it will switch to the volume list (if FSA_ALLOW_CHANGE_VOLUMES is set in GFSI_attrs). This message is only valid when the File Selector is visible on the screen.

Pass: Nothing.

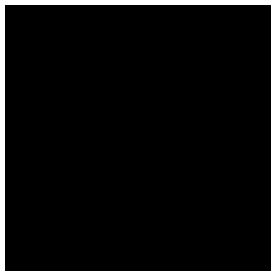
Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\gFSelC.def

Object Assembly Reference

Help Library



HelpControlClass 461

15

HelpControlClass

Instance Data

```
HCI_helpType          HelpType
   UIC Syntax:  helpType = byteComp : 0;
```

helptype (*HCI_helpType*) determines what brand of application help your geode provides. The default value is HT_SYSTEM_HELP, the normal level of help. Note that HT_SYSTEM_MODAL_HELP is used only within system-modal dialog boxes.

`HCI_curFile` `hptr`

HCI_curFile is internal.

`HCI_historyBuf` `hptr`

HCI_historyBuf is internal.

`HCI_nameArrayVM` `word`

HCI_nameArrayVM is internal.

`HCI_compressLib` `hptr`

HCI_compressLib is internal.

`ATTR_HELP_INITIAL_HELP` `vardata char`

`ATTR_HELP_INITIAL_HELP_FILE` `vardata char`

An object may use ATTR_HELP_INITIAL_HELP_FILE and ATTR_HELP_INITIAL_HELP to bring up a help context when the object from which the help is to be taken; it is only used if ATTR_HELP_INITIAL_HELP is also present. This one defines the help context to be brought up on startup.

These attributes are used normally only by help-viewer applications, as it is unlikely help will be required on startup. Normally, help is brought up by the user pressing a help trigger.

`ATTR_HELP_SUPPRESS_INITIATE` `vardata`

A new help status normally initiates the interaction; setting this attribute surpresses this behavior.

ATTR_HELP_CUSTOM_POINTER_IMAGE vardata optr

The help controller changes the pointer image when the pointer is over a link in the help text. This is so the user knows where to click and expects a link to be followed. Applications that want custom pointers can set the "link" pointer image with ATTR_HELP_CUSTOM_POINTER_IMAGE. This attribute takes an optr to a custom pointer image definition.

The pointer image is a **PointerDef** structure and must be stored in a chunk in a sharable, non-duplicated data block. Note also that to use a custom pointer image, you must explicitly declare a HelpControl object, even for Normal Help.

HINT_HELP_TEXT_FIXED_SIZE vardata

This hint is exactly the same as GenClass' HINT_FIXED_SIZE; it takes a **CompSizeHintArgs** structure as its extra data and sets the help controller's dialog box to the fixed size.

HINT_HELP_NOT_RESIZABLE vardata

Normally, help controllers set up as **GenInteractionVisibility** GIV_DIALOG are resizable. This hint will counteract that behavior and make the window not resizable.

Messages

MSG_HELP_CONTROL_FOLLOW_LINK

This message follows a hypertext link in a help file.

Pass: **cx** Token of link name.
dx Token of link file.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\helpCC.def**

Object Assembly Reference

MSG_HELP_CONTROL_GET_POINTER_IMAGE

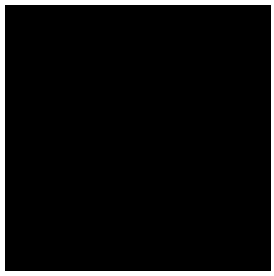
Return the pointer image to be used when the pointer is over a link in the help text. It looks for ATTR_HELP_CUSTOM_POINTER_IMAGE and returns the pointer stored there; if no custom pointer exists, the default is returned.

Pass:	**cx**	If the mouse pointer is not presently over a hyperlink "hot spot", this will be -1; any other value means the mouse pointer is over a hot spot.
Returns:	**ax**	**MouseReturnFlags** field with MRF_SET_POINTER_IMAGE set if there is a new pointer image.
	cx:**dx**	optr of pointer image, if MRF_SET_POINTER_IMAGE was set.
Destroyed:	**bp.**	
Library:	**Objects\helpCC.def**	



Object Assembly Reference

Impex Library



16

ImportControlClass 467

ExportControlClass 470

ImportControlClass

Instance Data

```
ICI_attrs              ImportControlAttrs
   UIC Syntax:  importAttrs= bitFieldComp word {
                   ignoreInput, nonDocumentImport
                   } : {};
```

importAttrs (*ICI_attrs*) is a word-length record.

```
ICI_dataClasses        ImpexDataClasses
   UIC Syntax:  importDataClasses = bitFieldComp word {
                   text, graphics, spreadSheet, font
                   } :{};
```

importDataClasses (*ICI_dataClasses*) is a word-length record specifying what kind of Metafiles your application is prepared to accept.

```
ICI_destination        optr
   UIC Syntax:  importDestination= optrComp : null;destination */
```

importDestination (*ICI_destination*) determines what object the action message will be sent to.

```
ICI_message            word
   UIC Syntax:  importMessage= wordComp : 0;
```

importMessage (*ICI_message*) is the message sent as the ImportControl's action.

Messages

MSG_IMPORT_CONTROL_GET_ACTION

This message retrieves the values of an ImportControl's *ICI_destination* and *ICI_message* fields. These fields indicate what action the ImportControl will take when it is finished preparing a file for import.

Source: Unrestricted.

Destination: Any ImportControl object.

Pass: Nothing.

Returns:

BP	The value of *ICI_message* (i.e. the message sent by the ImportControl).
cx:dx	The optr of the destination object.

Destroyed: Nothing.

Interception: This message should not be intercepted.

Library: **impex.def**

MSG_IMPORT_CONTROL_GET_ATTRS

This message retrieves the current setting of the ImportControl's *ICI_attrs* field.

Source: Unrestricted.

Destination: Any ImportControl object.

Pass: Nothing.

Returns:

cx	The ImportControl's *ICI_attrs* field.

Destroyed: Nothing.

Interception: This message should not be intercepted.

Library: **impex.def**

MSG_IMPORT_CONTROL_GET_DATA_CLASSES

This message retrieves the current setting of the ImportControl's *ICI_dataClasses* field. This tells you what kind of data can be imported.

Source: Unrestricted.

Destination: Any ImportControl object.

Pass: Nothing.

Returns: `cx` The ImportControl's *ICI_dataClasses* field.

Library: `impex.def`

MSG_IMPORT_CONTROL_SET_ACTION

This message changes the values of an ImportControl's *ICI_destination* and *ICI_message* fields. These fields indicate what action the ImportControl will take when it is finished preparing a file for import.

Source: Unrestricted.

Destination: Any ImportControl object.

Pass: `cx:dx` Set *ICI_destination* to this value.
`bp` Set *ICI_message* to this value.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: This message should not be intercepted.

Library: `impex.def`

MSG_IMPORT_CONTROL_SET_ATTRS

This message changes the current settings of an ImportControl's *ICI_attrs* field.

Source: Unrestricted.

Destination: Any ImportControl object.

Pass: `cx` The new settings for the *ICI_attrs* field.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Interception: This message should not be intercepted.

Library: `impex.def`



■ MSG_IMPORT_CONTROL_SET_DATA_CLASSES

This message changes the current settings of an ImportControl's *ICI_dataClasses* field.

Source: Unrestricted.

Destination: Any ImportControl object.

Pass: **cx** The new settings for the *ICI_dataClasses* field.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: This message should not be intercepted.

Library: **impex.def**

ExportControlClass

Instance Data

```
ECI_attrs       ExportControlAttrs
   UIC Syntax:  exportAttrs = bitFieldComp word {
                   ignoreInput
                   } : {};
```

exportAttrs (*ECI_attrs*) is a word-length record.

```
ECI_dataClasses       ImpexDataClasses
   UIC Syntax:  exportDataClasses = bitFieldComp word {
                   text, graphics, spreadSheet, font
                   } :{};
```

exportDataClasses (*ICI_attrs*) is a word-length record specifying what MetafFile formats your application is prepared to export.

```
ECI_destination       optr
   UIC Syntax:  exportDestination = optrComp : null;
```

exportDestination (*ECI_destination*) is the optr of the object which will receive the export action message.

```
ECI_message           word
   UIC Syntax:  exportMessage = wordComp : 0;
```

exportMessage (*ECI_message*) is the message of the export action.

Messages

■ MSG_EXPORT_CONTROL_GET_ACTION

This message retrieves the values of an ExportControl's *ECI_destination* and *ECI_message* fields. These fields indicate what action the ExportControl will take when it needs to have data prepared for export.

Source: Unrestricted.

Destination: Any ImportControl object.

Pass: Nothing.

Returns:

bp	The value of *ICI_message* (i.e. the message sent by the ExportControl).
cx:dx	The optr to which the message will be sent.

Destroyed: Nothing.

Interception: This message should not be intercepted.

Library: **impex.def**

■ MSG_EXPORT_CONTROL_GET_ATTRS

This message retrieves the current setting of the ExportControl's *ECI_attrs* field.

Source: Unrestricted.

Destination: Any ExportControl object.

Pass: Nothing.

Returns:

cx	The ExportControl's *ECI_attrs* field.

Destroyed: Nothing.

Interception: This message should not be intercepted.

Library: **impex.def**

■ MSG_EXPORT_CONTROL_GET_DATA_CLASSES

This message retrieves the current setting of the ExportControl's *ECI_dataClasses* field. This tells you what kind of data can be exported.

Source: Unrestricted.

Destination: Any ExportControl object.

Interception: This message should not be intercepted.

Pass: Nothing.

Returns: **cx** The ExportControl's *ECI_dataClasses* field.

Destroyed: Nothing.

Library: **impex.def**

■ MSG_EXPORT_CONTROL_SET_ACTION

This message changes the values of an ExportControl's *ECI_destination* and *ECI_message* fields. These fields indicate what action the ExportControl will take when it is finished preparing a file for export.

Source: Unrestricted.

Interception: This message should not be intercepted.

Destination: Any ExportControl object.

Pass: **cx:dx** Set *ECI_destination* to this value.
bp Set *ECI_message* to this value.

Returns: Nothing.

Library: **impex.def**

MSG_EXPORT_CONTROL_SET_ATTRS

This message changes the current settings of an ExportControl's *ECI_attrs* field.

Source: Unrestricted.

Destination: Any ExportControl object.

Pass: **cx** The new settings for the *ECI_attrs* field.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: This message should not be intercepted.

Library: **impex.def**

MSG_EXPORT_CONTROL_SET_DATA_CLASSES

This message changes the current settings of an ExportControl's *ECI_dataClasses* field.

Source: Unrestricted.

Destination: Any ExportControl object.

Pass: **cx** The new settings for the *ECI_dataClasses* field.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

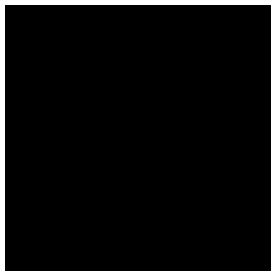
Interception: This message should not be intercepted.

Library: **impex.def**



Object Assembly Reference

Print Controller



17

PrintControlClass 477

PageSizeControlClass 495

PrintControlClass

Instance Data

```
PCI_attrs             PrintControlAttrs
   UIC Syntax:  printAttributes = bitFieldComp word {
                        unused15,
                        seeIfDocWillFit,
                        markAppBusy, /* Mark app busy w/ printing  */
                        unused15,
                        seeIfDocWillFit,
                        markAppBusy, /* Mark app busy w/ printing  */
                        verifyPrint, /* Verify before printing     */
                        showProgress, /* Display a progress dialog  */
                        progressByPercent,
                        progressByPage,
                        forceRotatedOutput,
                        copyControls, /* Use number of copies tools */
                        pageControls, /* Use the page ranges tools  */
                        qualityControls, /* Use quality tools */
                        usesDialogBox, /* Want the print dialog box */
                        graphicsOutput, /* Supports graphics output */
                        textOutput, /* Supports text-mode output  */
                        defaultQuality:2 = {
                           high,
                           medium,
                           low
                        }
                     } : copyControls, pageControls, qualityControls,
                     usesDialogBox, graphicsOutput, textOutput,
                     seeIfDocWillFit, defaultQuality high;
```

The Print Control includes several attributes which are grouped together into a record of type **PrintControlAttrs**. These represent some choices made when instantiating the Print Control.

PCA_MARK_APP_BUSY

Describing large print jobs can take a fair amount of time. If this bit is set, the application will reassure the user that it is busy (by showing the busy cursor) while spooling long print jobs.

PCA_VERIFY_PRINT

If this bit is set, the Print Control will ask the application for confirmation before printing anything. This request will come in the form of a MSG_PRINT_VERIFY_PRINT_REQUEST sent to the *PCI_output* object. The Print Control sends this message after the user has finished interacting with the Print dialog box so that the application can make sure that the user has made valid choices.

PCA_SHOW_PROGRESS, PCA_PROGRESS_PERCENT, PCA_PROGRESS_PAGE

When describing a large print job, it is considerate to let the user know when the application is making some kind of progress. If the first of these bits is set, the Print Control will display a progress dialog box when spooling a print job. The next two bits determine whether progress will be reported as a percentage, number of pages completed, both, or neither. Regardless of whether percents or page number progress is reported, the progress box can display an application-supplied string.

PCA_FORCE_ROTATION

If this bit is set, the output will automatically be printed rotated on the page. Normally, this bit is not set so the Spooler will make a choice to make the job fit on the minimum number of pages. The Banner program sets this bit to make sure that fanfold paper printers will create continuous banners.

PCA_COPY_CONTROLS

This flag determines whether the Print Control will allow the user to request that multiple copies be printed. If your application uses this option, then it is vital that the print control be updated whenever the total page count of the document changes.

PCA_PAGE_CONTROLS

If this bit is set, the user will be allowed to select specific page ranges to print. Even if it seems like users would have little reason to print short ranges, it's still nice to include this option: if the printer messes up one page, the user won't want to reprint the whole document.

PCA_QUALITY_CONTROLS

If this flag is on, the user will be allowed to select which quality

to use when printing. Note that not all printers can print at all qualities, but most support more than one.

PCA_USES_DIALOG_BOX
: This bit determines whether the Print Control will display a dialog box containing printing choices for the user. Specialty geodes and GCM applications might want to turn this flag off; it is set by default.

PCA_GRAPHICS_MODE, PCA_TEXT_MODE
: These bits determine whether the Control should support graphic mode and/or text mode output.

PCA_DEFAULT_QUALITY
: These two bits contain the default print quality to use. The enumerated type **PrintQualityEnum** defines the values available: PQT_HIGH, PQT_MEDIUM, and PQT_LOW.

Those attributes which are on by default are PCA_COPY_CONTROLS, PCA_PAGE_CONTROLS, PCA_QUALITY_CONTROLS, PCA_USES_DIALOG_BOX, PCA_GRAPHICS_OUTPUT, and PCA_TEXT_OUTPUT.

```
PCI_startPage        word

PCI_endPage          word
   UIC Syntax:     fromPage  = wordComp : 1;
                   toPage    = wordComp : 1;
```

fromPage (*PCI_startPage*) and *toPage* (*PCI_endPage*) should contain the page numbers of the first and last pages possible to print, known as the total page range.

```
PCI_startUserPage    word

PCI_endUserPage      word
   UIC Syntax:     firstPage     = wordComp : 0;
                   lastPage      = wordComp : "0x7fff";
```

firstPage (*PCI_startUserPage*) and *lastPage* (*PCI_endUserPage*) contain the beginning and ending page numbers of the range the user wants to print, often called the user page range.

```
PCI_defPrinter       word
   UIC Syntax:  defaultPrinter   = wordComp : -1;
```

defaultPrinter (*PCI_defPrinter*): When the Print Control first appears, it will normally have the system default printer selected. Using the *defaultPrinter*

option, another printer might be selected. The user may of course override the default printer and select another.

```
PCI_docSizeInfo      PageSizeReport
   UIC Syntax:     docWidth     = dwordComp : 0;
                   docHeight    = dwordComp : 0;
                   docLayout    = wordComp : 0;
                   marginLeft   = wordComp : 0;
                   marginTop    = wordComp : 0;
                   marginRight  = wordComp : 0;
                   marginBottom = wordComp : 0;
```

docWidth, *docHeight*, *docLayout*, *marginLeft*, *marginTop*, *marginRight*, and *marginBottom* (*PCI_docSizeInfo*) together specify the size of the document to be printed.

```
PCI_output           optr
   UIC Syntax:  printOutput      = optrComp : null;
```

printOutput (*PCI_output*): Some object is going to describe the print jobs to the Print Control. When instantiating the Print Control, set the chosen object's name in the *printOutput* field.

```
PCI_docNameOutput    optr
   UIC Syntax:  docNameOutput   = optrComp:  null;
```

docNameOutput (*PCI_docNameOutput*) must contain the optr of an object which can tell the Spooler the document's name. The document name is displayed on the Printer Control Panel and is how the user can figure out which job is which on the queue. If a GenDocumentGroup is set as the Print Control's *PCI_docNameOutput*, then that object will automatically do the right thing. If you use some other type of object, it must be prepared to supply document names on demand. This demand will come in the form of a MSG_PRINT_GET_DOC_NAME.

```
ATTR_PRINT_CONTROL_APP_UI  vardata optr
```

Normally the user has two ways to print up a Print dialog box: the User Print Trigger or the User Fax Trigger, both appearing in the File menu. You may specify a special trigger to be used in place of the standard trigger provided by the Print Control. Normally the default trigger is not only sufficient but preferable, if only because it will automatically display with the correct appearance for the specific UI. To define your own trigger object, create the object(s), setting the correct instance data. Then, when instantiating your Print Control, include an ATTR_GEN_CONTROL_APP_UI.

Object Assembly Reference

Imported Messages

MSG_PRINT_GET_DOC_NAME

If your *PCI_docNameOutput* object is not a GenDocumentGroup, then it must have a handler for this message.

Imported in MetaPrintMessages range from MetaClass.

Pass: **cx:dx** (*printControlOD*) The optr of the PrintControl object.

Returns: Nothing returned explicitly, but should send a MSG_PRINT_CONTROL_SET_DOC_NAME to the PrintControl.

Destroyed: **ax, cx, dx, bp.**

Library: **spool.def**

MSG_PRINT_NOTIFY_PRINT_DB

The handler for this message can do any almost anything; this message signals that the Print dialog box has just come up or gone away.

Imported in MetaPrintMessages range from MetaClass.

Pass: **cx:dx** (*printControlOD*) The optr of the PrintControl object.
bp (*pcs*) The status of the print control; either PCS_PRINT_BOX_VISIBLE or PCS_PRINT_BOX_NOT_VISIBLE.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **spool.def**

MSG_PRINT_NOTIFY_PRINT_JOB_CREATED

The PrintControl sends this message when it has sent the job to the print queue. Applications may intercept this message to retrieve the new job's ID number or for any other reason.

Imported in MetaPrintMessages range from MetaClass.

Pass: **cx:dx** (*printControlOD*) The optr of the PrintControl object.
bp (*jobID*) Print job's ID.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **spool.def**

■ MSG_PRINT_START_PRINTING

The handler for this message should call a number of graphics routines, using the passed GState. When done with graphics routines, the handler should send a MSG_PRINT_CONTROL_PRINTING_COMPLETED (or MSG_PRINT_CONTROL_PRINTING_CANCELED) to the Print Control.

Imported in MetaPrintMessages range from MetaClass.

Pass: **cx:dx** (*printControlOD*) The optr of the PrintControl object.
bp (*gstate*) The GState handle to draw to.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **spool.def**

■ MSG_PRINT_VERIFY_PRINT_REQUEST

The handler for this message should make whatever checks are necessary to make sure that the user has made valid choices with the printing UI. The handler should then send back an indication of whether the user's choices are acceptable by means of a MSG_PRINT_CONTROL_VERIFY_COMPLETED.

Imported in MetaPrintMessages range from MetaClass.

Pass: **cx:dx** (*printControlOD*) The optr of the PrintControl object.

Returns: Nothing explicitly. However, the handler for this message must send a MSG_PRINT_CONTROL_VERIFY_COMPLETED back to the print control.

Destroyed: **ax, cx, dx, bp.**

Library: **spool.def**

Object Assembly Reference

Messages

MSG_PRINT_CONTROL_CALC_DOC_DIMENSIONS

This message calculates the maximum printable area allowed by the user-selected paper size and printer-mandated margins. This message automatically resets the document size and margins to hold these values.

Note that this message correctly handles rotated pages.

Pass:	**dx:bp**	(*ptr*) Pointer to a **PageSizeReport** structure to hold return value.
Returns:	**dx:bp**	**PageSizeReport** filled, except that the width and height take into account the margin and all layout options have been applied.
Destroyed:	Nothing.	
Library:	**spool.def**	

MSG_PRINT_CONTROL_CHECK_IF_DOC_WILL_FIT

This message returns *true* in **ax** if the passed document margins will fit on the selected printer.

Pass:	**cx**	(*warning*) Pass *true* (i.e. non-zero) to display a warning if document will fit.
Returns:	**ax**	Returns *true* (i.e. non-zero) if document will fit on page; returns *false* (i.e. zero) otherwise.
Destroyed:	**ax, dx, bp.**	
Library:	**spool.def**	

MSG_PRINT_CONTROL_GET_ATTRS

Use this message to retrieve the values stored in the *PCI_attrs* structure. It will return a record of type **PrintControlAttrs** containing the desired values.

Pass:	Nothing.	
Returns:	**cx**	**PrintControlAttrs** value.
Destroyed:	Nothing.	
Library:	**spool.def**	

MSG_PRINT_CONTROL_GET_DEFAULT_PRINTER

Use this message to retrieve the number of the present application default printer. You may not assume that this printer is the one the user is printing on, since the user may have changed to another printer. A return value of -1 means that the application will use the system default printer.

Pass: Nothing.

Returns: **cx** Printer number (zero-based) or -1 to indicate we should use the system default printer.

Destroyed: Nothing.

Library: **spool.def**

MSG_PRINT_CONTROL_GET_DOC_MARGINS

Use this message to get the present values for the document margins. It returns four integers. These integers represent the left, top, right, and bottom margins, expressed in typographer's points.

Pass: **dx**:**bp** (*ptr*) Pointer to a **PCMarginParams** structure which will hold return value.

Returns: **dx**:**bp** Structure filled with margins.

Destroyed: Nothing.

Library: **spool.def**

MSG_PRINT_CONTROL_GET_DOC_NAME_OUTPUT

This message returns the optr of the Document Name Output object.

Pass: Nothing.

Returns: **cx**:**dx** The optr of the Document Name Output object.

Destroyed: Nothing.

Library: **spool.def**

MSG_PRINT_CONTROL_GET_DOC_SIZE

Use this message to retrieve the present document size. The size is returned as a width and height, each expressed in points.

Note that if the size might be a 32-bit value (which might happen if the document uses an extended graphics space), you must use

MSG_PRINT_CONTROL_GET_EXTENDED_DOC_SIZE. If either dimension is a 32-bit number, using MSG_PRINT_CONTROL_GET_DOC_SIZE will result in an error.

Remember that the document size includes the document margins.

Pass: Nothing.

Returns: **cx** Width (in points).
dx Height (in points).

Destroyed: Nothing.

Library: **spool.def**

■ MSG_PRINT_CONTROL_GET_DOC_SIZE_INFO

Use this message to set all of the information about the document size and orientation.

Pass: **dx**:**bp** (*ptr*) Pointer to an empty **PageSizeReport** structure which the message handler will fill in.

Returns: **dx**:**bp** Pointer to a filled-in **PageSizeReport** structure.

Destroyed: Nothing.

Library: **spool.def**

■ MSG_PRINT_CONTROL_GET_EXTENDED_DOC_SIZE

Use this message to retrieve the present document size. It returns two double integers representing the width and height, expressed in points. If the size is a 16 bit value, you can use MSG_PRINT_CONTROL_GET_DOC_SIZE instead.

Pass: **dx**:**bp** (*ptr*) Pointer to a **PCDocSizeParams** structure to hold document parameters.

Returns: **dx**:**bp** **PCDocSizeParams** structure filled in.

Destroyed: Nothing.

Library: **spool.def**

■ MSG_PRINT_CONTROL_GET_OUTPUT

This message returns the optr of the Print Output.

Pass:	Nothing.	
Returns:	**cx**:**dx**	The optr of the Print Output.
Destroyed:	Nothing.	
Library:	**spool.def**	

■ MSG_PRINT_CONTROL_GET_PAPER_SIZE

Use this message to retrieve the Print Control's present paper size.

Pass:	Nothing.	
Returns:	**ax**:**cx**	Paper width (in points).
	bp:**dx**	Paper height (in points).
Destroyed:	Nothing.	
Library:	**spool.def**	

■ MSG_PRINT_CONTROL_GET_PAPER_SIZE_INFO

Use this message to retrieve all of the information about the document size and orientation.

Pass:	**dx**:**bp**	(*ptr*) Pointer to **PageSizeReport** buffer.
Returns:	**dx**:**bp**	**PageSizeReport** buffer filled.
Destroyed:	Nothing.	
Library:	**spool.def**	

■ MSG_PRINT_CONTROL_GET_PRINTER_MARGINS

This message returns the margins enforced by the requested printer. If **dx** is *true*, the document margins will be set to be the same as the printer margins.

Pass:	**dx**	(*setMargin*) Pass *true* (i.e. non-zero) to set the document's margins equal to the printer's margins, *false* (i.e. zero) to leave the margins as they are.
Returns:	**ax**	Left margin.
	cx	Top margin.
	dx	Right margin.
	bp	Bottom margin.
Destroyed:	Nothing.	
Library:	**spool.def**	

■ MSG_PRINT_CONTROL_GET_PRINT_MODE

This message retrieves the current user-selected print mode, if any. Its possible return values include zero, meaning no mode has been selected; PM_GRAPHICS_LOW_RES; PM_GRAPHICS_MED_RES; PM_GRAPHICS_HI_RES; PM_TEXT_DRAFT; and PM_TEXT_NLQ.

Pass:	**cl**	**PrinterMode** (zero if no mode is yet selected).
Returns:	Nothing.	
Destroyed:	Nothing.	
Library:	**spool.def**	

■ MSG_PRINT_CONTROL_GET_SELECTED_PAGE_RANGE

This message returns the user's selected range of pages to print.

Pass:	Nothing.	
Returns:	**cx**	First page to print.
	dx	Last page to print.
Destroyed:	Nothing.	
Library:	**spool.def**	

■ MSG_PRINT_CONTROL_GET_TOTAL_PAGE_RANGE

This message returns two integers. These integers are the numbers of the first and last pages of the range of possible pages. These values aren't

necessarily the first and last pages of the range the user wants to print. Use MSG_PRINT_CONTROL_GET_SELECTED_PAGE_RANGE for that information.

Pass: Nothing.

Returns: `cx` First page of document.
`dx` Last page of document.

Destroyed: Nothing.

Library: `spool.def`

■ MSG_PRINT_CONTROL_INITIATE_OUTPUT_UI

Display the output dialog box.

Pass: `cl` (*type*) **PrinterDriverType** value specifying what sort of output device is being invoked (e.g., printer, or fax).

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: `spool.def`

■ MSG_PRINT_CONTROL_INITIATE_PRINT

This message, normally sent by the Print trigger in the File menu, is the signal that the Print Control should display the Print dialog box. Geodes with custom print triggers should make sure that those triggers send this message to the Print Control. Geodes using the Print Control's provided print trigger need not send this message.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: `spool.def`

Object Assembly Reference

MSG_PRINT_CONTROL_PRINT

This message, normally sent by the Print trigger in the Print dialog box, is the signal that the user has made his printing choices and is ready for the Print Control to verify those choices and spool the job.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Library: **spool.def**

MSG_PRINT_CONTROL_PRINTING_CANCELLED

The application may send this message to the PrintControl after receiving MSG_PRINT_START_PRINTING. Do not send both this message and MSG_PRINT_CONTROL_PRINTING_COMPLETED for a single print job.

The application may send this message to cancel a document while describing a print job, before it would send MSG_PRINT_CONTROL_PRINTING_COMPLETED.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **spool.def**

MSG_PRINT_CONTROL_PRINTING_COMPLETED

This message signals the Print Control that the application is finished describing the print job. The application must send this message (or a MSG_PRINT_CONTROL_PRINTING_CANCELLED) some time after receiving a MSG_PRINT_START_PRINTING.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax. cx, dx, bp.**

Library: **spool.def**

■ MSG_PRINT_CONTROL_REPORT_PROGRESS

This message may be sent between the time the Print Output receives a MSG_PRINT_START_PRINTING and the time it sends a MSG_PRINT_CONTROL_PRINTING_COMPLETED. Its arguments include a progress type (page number or percentage) and a number representing the progress. It returns a Boolean value, which will normally be *true* (i.e., non-zero) but will be *false* (i.e., zero) if the user wishes to cancel printing.

The application should send out this message periodically over the course of a long print job, probably about once per page. Progress may be reported as a page number, a percentage of the job completed, or as a null-terminated string.

If the return value is *false*, the application should stop drawing to the print GString immediately and send the Print Control a MSG_PRINT_CONTROL_PRINTING_CANCELLED. The return value may be safely ignored, but if the user wishes to cancel printing, it's polite to cancel immediately instead of making him wait until the MSG_PRINT_START_PRINTING has finished describing the entire job.

Pass:	**cx**	(*progress*) ProgressReportType specifying how progress is being reported.
	dx:**bp**	If progress is being reported via a text string, this is a pointer to that string.
	dx	(*pageOrPercent*) If progress is being reported via a page number or percentage, this is that page number or percentage.
Returns:	**ax**	*True* (i.e., non-zero) to continue printing; *false* (i.e., zero) to request abort printing—application should stop drawing to the GState immediately and send a MSG_PRINT_CONTROL_PRINTING_CANCELLED to the Print Control.
Destroyed:	**cx, dx, bp.**	
Library:	**spool.def**	

MSG_PRINT_CONTROL_SET_ATTRS

Use this message to change the values stored in the *PCI_attrs* structure. Pass a record of type **PrintControlAttrs** containing the desired values.

Pass: `cx` (*attributes*) The new **PrintControlAttrs**.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **spool.def**

MSG_PRINT_CONTROL_SET_DEFAULT_PRINTER

This message sets the application default printer, to be used the next time the print dialog box appears. Pass the printer number of the printer to use. Passing a value of -1 means that the system default printer should be used.

Pass: `cx` (*printerNum*) Number of the new default printer.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **spool.def**

MSG_PRINT_CONTROL_SET_DOC_MARGINS

Use this message to set new values for the document margins. It takes four arguments, the point values to use for the left, top, right, and bottom margins.

Pass: `dx:bp` (*ptr*) Pointer to a **PCMarginParams** structure with new document margins.

Returns: Nothing.

Destroyed: Nothing.

Library: **spool.def**

MSG_PRINT_CONTROL_SET_DOC_NAME

This message must be sent by the *PCI_docNameOutput* before the print job can be spooled. It is sent in response to a MSG_PRINT_GET_DOC_NAME from



the Print Control. The *string* argument should be something that will allow the user to recognize the print job in the Printer Control Panel.

Pass: **cx**:**dx** (*string*) The document's name, as a pointer to a null-terminated string. Maximum length of this string is FILE_LONGNAME_LENGTH+1.

Returns: Nothing.

Destroyed: **ax, cx, dx.**

Library: **spool.def**

■ MSG_PRINT_CONTROL_SET_DOC_NAME_OUTPUT

This message, used very rarely, specifies that a different object should become the Document Name Output object. Pass the optr of the object to use.

Pass: **cx**:**dx** (*document*) The optr of the new *PCI_docNameOutput*.

Returns: Nothing.

Destroyed: **ax. cx. dx. bp.**

Library: **spool.def**

■ MSG_PRINT_CONTROL_SET_DOC_SIZE

This message changes the values of the document size. It takes two integers, representing the new width and height for the document to use. It can only use 16-bit values, so use MSG_PRINT_CONTROL_SET_EXTENDED_DOC_SIZE when working with 32-bit extended graphics spaces.

Pass: **cx** (*width*) Width in points.
dx (*height*) Height in points.

Returns: Nothing.

Destroyed: Nothing.

Library: **spool.def**

MSG_PRINT_CONTROL_SET_DOC_SIZE_INFO

Use this message to set all of the information about the document size and orientation.

Pass: **dx:bp** (ptr) Pointer to a **PageSizeReport** structure.

Returns: Nothing.

Destroyed: Nothing.

Library: **spool.def**

MSG_PRINT_CONTROL_SET_EXTENDED_DOC_SIZE

This message changes the values of the document size using the two passed double integers as the new width and height to use. When working with normal 16 bit graphics spaces, use MSG_PRINT_CONTROL_SET_DOC_SIZE instead.

Remember that the document size includes the document margins.

Pass: **dx:bp** (*ptr*) Pointer to a **PCDocSizeParams** structure containing the document size.

Returns: Nothing.

Destroyed: Nothing.

Library: **spool.def**

MSG_PRINT_CONTROL_SET_OUTPUT

This message, used very rarely, specifies that a different object should become the Print Output. Pass the optr of the object to use.

Pass: **cx:dx** (*objectPtr*) The optr of the new *PCI_output*.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **spool.def**

MSG_PRINT_CONTROL_SET_SELECTED_PAGE_RANGE

This message takes the passed values and uses them as the first and last pages that the user wants to print.

Pass: **cx** (*firstPage*) Page number of the first page to print.
dx (*lastPage*) Page number of the last page to print.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **spool.def**

MSG_PRINT_CONTROL_SET_TOTAL_PAGE_RANGE

The application should send this message to set the first and last page numbers of the document. You might want to send this message every time the document length changes or just every time the Print dialog box is put up. The Print Control warns the *PCI_output* object of the occurrence of the latter event with a MSG_PRINT_NOTIFY_PRINT_DB.

Pass: **cx** (*firstPage*) Page number of the document's first page.
dx (*lastPage*) Page number of the document's last page.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **spool.def**

MSG_PRINT_CONTROL_VERIFY_COMPLETED

If you have decided to verify the user's Print dialog box choices, you must send this message back to the Print Control in response to MSG_PRINT_VERIFY_PRINT_REQUEST. If *true* is passed, the Print Control will continue preparing the job for printing. If the value passed is *false*, the print will be cancelled. It is up to the application to explain the problem to the user.

Pass: **cx** (*continue*) *True* (i.e., non-zero) to continue with printing; *false* (i.e., zero) to cancel printing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **spool.def**

Object Assembly Reference

PageSizeControlClass

Instance Data

```
PZCI_attrs              PageSizeControlAttrs
   UIC Syntax:      pageSizeAttrs    = wordComp : 0;
```

pageSizeAttrs (*PZCI_attrs*) fine-tunes the behavior of the object. Most important of these flags is the PZCA_ACT_LIKE_GADGET flag, which determines whether the object will act like a regular gadget or like a controller.

```
PZCI_width              dword
```

```
PZCI_height             dword
   UIC Syntax:      width        = dwordComp : 0;
                    height       = dwordComp : 0;
```

width (*PZCI_width*) and *height* (*PZCI_height*) represent the current page dimensions, measured in points. Note that these values may never go outside the bounds described by the constants MINIMUM_PAGE_WIDTH_VALUE, MINIMUM_PAGE_HEIGHT_VALUE, MAXIMUM_PAGE_WIDTH_VALUE, and MAXIMUM_PAGE_HEIGHT_VALUE.

```
PZCI_layout             PageLayout
   UIC Syntax:      layout          = wordComp : 0;
```

layout (*PZCI_layout*) contains a **PageLayout** structure describing the current page layout settings.

```
PZCI_margins            PCMarginParams
   UIC Syntax:      marginLeft = wordComp : 0;
                    marginTop       = wordComp : 0;
                    marginRight     = wordComp : 0;
                    marginBottom    = wordComp : 0;
```

marginLeft, *marginTop*, *marginRight*, and *marginBottom* (combined in *PZCI_margins*) contain the page's margins.

Imported Message

■ MSG_PRINT_REPORT_PAGE_SIZE

The page size control's output object should have a handler for this message. The control will send this message whenever the user has changed the page

size. The handler may wish to change the bounds of some appropriate object, store the new bounds somewhere, or take some other action.

Imported in MetaPrintMessages range from MetaClass.

Pass: **dx** Size of **PageSizeReport** structure.
ss:**bp** (*psr*) **PageSizeReport** structure.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **spool.def**

Messages

■ MSG_PZC_GET_PAGE_SIZE

This message returns the present page size in terms of width, height, and layout, stored in a **PageSizeReport**. Note that the PageSizeControl will be sending out a MSG_PRINT_REPORT_PAGE_SIZE every time the user applies changes to these values.

Only send this message if the PageSizeControl is operating in gadget mode.

Pass: **dx** Size of **PageSizeReport** structure.
ss:**bp** (*psr*) PageSizeReport structure.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **spool.def**

■ MSG_PZC_SET_MAXIMUM_HEIGHT

Set the maximum height for a page.

Pass: **dx**.**cx** (*height*) Height, in points.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **spool.def**

Object Assembly Reference

MSG_PZC_SET_MAXIMUM_WIDTH

Set the maximum height for a page.

Pass: **dx**.**cx** (*width*) Width, in points.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **spool.def**

MSG_PZC_SET_PAGE_SIZE

This message changes the current page size. It takes a **PageSizeReport** data structure containing the new dimensions and layout. The Apply trigger of a Page Size Control normally sends this message, passing the present user page size as arguments.

Only send this message if the PageSizeControl is operating in gadget mode.

Pass: **dx**:**bp** (*psr*) Pointer to a **PageSizeReport** structure containing page information.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **spool.def**



Object Assembly Reference

GrObj Library

18

GrObjAttributeManagerClass 501

GrObjHeadClass 501

GrObjBodyClass 503

GrObjClass 510

GroupClass 525

PointerClass 526

GrObjVisGuardianClass 526

GrObjAttributeManagerClass

Messages

■ MSG_GOAM_CREATE_ALL_ARRAYS

This message initializes the data structures in which the GOAM will store all attribute information.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

GrObjHeadClass

Instance Data

```
GH_currentTool      fptr.ClassStruct
        UIC Syntax:  currentTool = fptrComp:0;
```

currentTool (*GH_currentTool*) maintains the class of the current drawing tool. When declaring the GrObjHead, you may give a value for the *currentTool* field. For instance, by passing **RectClass** in this field, the head will automatically start out set up ready for the user to draw rectangles. This is an especially useful option if you will only allow the user to work with one sort of object, and will thus not include a tool selector control.

```
GH_initializeFloaterData    word
        UIC Syntax:  initFloaterData = wordComp:0;
```

initFloaterData (*GH_initializeFloaterData*) is internal.

```
GH_currentBody      optr
```

GH_currentBody is the optr of the current GrObjBody object.

```
GH_floater          optr
```

GH_floater is the optr of the tool object that is waiting to be placed in the current GrObjBody.

Messages

MSG_GH_GET_CURRENT_TOOL

This message returns the value of the currently active tool.

Pass: Nothing.

Returns: **cx**:**dx** Pointer to tool's class.
bp Data from MSG_GO_GROBJ_SPECIFIC_INITIALIZE.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GH_SET_CURRENT_TOOL

This message activates a tool.

Pass: **cx**:**dx** Pointer to tool's class.
bp Data for MSG_GO_GROBJ_SPECIFIC_INITIALIZE.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GH_SET_CURRENT_TOOL_WITH_DATA_BLOCK

This message activates a tool.

Pass: **cx**:**dx** Pointer to tool's class.
^h**bp** Data for MSG_GO_GROBJ_SPECIFIC_INITIALIZE.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

Object Assembly Reference

GrObjBodyClass

Instance Data

```
GBI_bounds          RectDWord <>
        UIC Syntax:  boundsLeft =     dwordComp:0;
                     boundsTop =      dwordComp:0;
                     boundsRight =    dwordComp:0;
                     boundsBottom =   dwordComp:0;
```

boundsLeft, *boundsTop*, *boundsRight*, and *boundsBottom* (*GBI_bounds*) together act as the bounds of the drawing area. Geodes using graphic bodies as part of documents may wish to make the body's bounds match the document bounds. Use MSG_GB_SET_BOUNDS to reset these bounds.

```
GBI_drawComp        CompPart <>
```

GBI_drawComp is internal.

```
GBI_reverseComp     CompPart <>
```

GBI_reverseComp is internal.

```
GBI_childCount      word
```

GBI_childCount is internal.

```
GBI_selectionArray  optr
```

GBI_selectionArray is internal.

```
GBI_targetExcl      HierarchicalGrab
                  public  GBI_targetExcl
```

GBI_targetExcl is internal.

```
GBI_focusExcl       HierarchicalGrab
```

GBI_focusExcl is internal.

```
GBI_mouseGrab       optr
                  public  GBI_mouseGrab
                  noreloc GBI_mouseGrab
```

GBI_mouseGrab is internal.

```
GBI_objBlockArray           lptr
```

GBI_objBlockArray is internal.

```
GBI_defaultOptions          GrObjFunctionsActive
         UIC Syntax:  defaultOptions = bitFieldComp word {junk}:{}

GBI_currentModifiers        GrObjFunctionsActive

GBI_currentOptions          GrObjFunctionsActive
```

defaultOptions (*GBI_defaultOptions*, *GBI_currentModifiers*, and *GBI_currentOptions*): When working with a graphic layer, the user will no doubt have certain preferences about how input should be interpreted. These preferences include the option to "stretch" objects from the center, snapping to the grid, and drawing bitmaps as plain rectangles. One set of options may be set as the default, stored in *GBI_defaultOptions*. As input events come in, the body may turn on other options, which are stored in *GBI_currentModifiers*. The *GBI_currentOptions* field contains the computed combination of these options.

```
GBI_flags                   GrObjBodyFlags
         UIC Syntax:  bodyFlags   = bitFieldComp  byte {
                      filler:5,
                      hasActionNotification,
                      defaultTarget,
                      defaultFocus
                      }: defaultTarget, defaultFocus;
```

bodyFlags (*GBI_flags*) determines some of the GrObjBody's miscellaneous behavior.

```
GBI_drawFlags               GrObjDrawFlags
         UIC Syntax:  drawFlags   = bitFieldComp  word {
                      filler:7,
                      drawQuickView,
                      drawClipOnly,
                      drawWrapTextInsideOnly,
                      drawWrapTextAroundOnly,
                      drawWithIncreasedResolution,
                      drawInstructions,
                      drawSelectedObjectsOnly,
                      drawObjectsOnly,
                      printInstructions
                      }: drawInstructions;
```

drawFlags (*GBI_drawFlags*) determines some of the GrObjBody's drawing behavior.

```
GBI_fileStatus              GrObjFileStatus
                  public  GBI_fileStatus
         UIC Syntax:  fileStatus = bitFieldComp byte {junk}:{};
```

fileStatus (*GBI_fileStatus*) is internal.

Object Assembly Reference

```
GBI_graphicsState   hptr.GState
                    noreloc GBI_graphicsState
```

GBI_graphicsState is internal.

```
GBI_head            optr
                    noreloc GBI_head
                    public  GBI_head
          UIC Syntax:  head = optrComp:null;
```

head (*GBI_head*) is the handle of the GrObjHead.

```
GBI_goam            optr
                    public  GBI_goam
          UIC Syntax:  oam = optrComp:null;
```

oam (*GBI_goam*) is the handle of the GrObjBody's attribute manager.

```
GBI_ruler           optr
                    public  GBI_ruler
          UIC Syntax:  ruler = optrComp:null;
```

ruler (*GBI_ruler*) field may hold a handle to a VisRuler. If used, this ruler will be able to coordinate with the GrObjBody to provide mouse tracking.

```
GBI_priorityList    lptr.PriorityList
```

GBI_priorityList is internal.

```
GBI_desiredHandleSize byte
                    public GBI_desiredHandleSize
          UIC Syntax:  desiredHandleSize = byteComp:\
                       DEFAULT_DESIRED_HANDLE_SIZE;
```

desiredHandleSize (*GBI_desiredHandleSize*) is the desired size, in pixels, of selection and resize handles

```
GBI_curHandleWidth  byte
                    public GBI_curHandleWidth
```

GBI_curHandleWidth is internal.

```
GBI_curHandleHeight byte
                    public GBI_curHandleHeight
```

GBI_curHandleHeight is internal.

```
GBI_curNudgeX       BBFixed
                    public GBI_curNudgeX
```

GBI_curNudgeX is internal.

```
GBI_curNudgeY       BBFixed
                    public GBI_curNudgeY
```

GBI_curNudgeY is internal.

`GBI_curScaleFactor PointWWFixed`

GBI_curScaleFactor is the current scale factor.

`GBI_interestingPoint PointDWFixed`

GBI_interestingPoint is the point to paste to, if it is within the window bounds.

`GBI_lastPtr PointDWFixed`

GBI_lastPtr is the location of the last pointer event.

`GBI_suspendCount word`

GBI_suspendCount is internal.

`GBI_unsuspendOps GrObjBodyUnsuspendOps`

GBI_unsuspendOps is internal.

`GBI_textUnsuspendOps VisTextNotificationFlags`

GBI_textUnsuspendOps is internal.

`GBI_reserved1 word`

GBI_reserved1 is internal.

`GBI_reserved2 word`

GBI_reserved2 is internal.

Messages

MSG_GB_ADD_GROBJ

This message adds a graphic object to the GrObjBody. The object will be notified by a MSG_GO_AFTER_ADDED_TO_BODY that it has been added to the body. The object won't necessarily be drawn; to force a redraw, send MSG_GB_ADD_GROBJ_THEN_DRAW instead.

Pass:	**cx**:**dx**	optr of new GrObj object.
	bp	**GrObjBodyAddGrObjFlags** value specifying where in the list of objects to add the new GrObj.
Returns:	Nothing.	
Destroyed:	**ax.**	
Library:	**grobj.def**	

Object Assembly Reference

MSG_GB_ADD_GROBJ_THEN_DRAW

This message adds a graphic object to the GrObjBody. The object will be notified by a MSG_GO_AFTER_ADDED_TO_BODY that it has been added to the body. If the object is added at the top of the draw list, it will be sent a draw message. Otherwise, it will be invalidated.

Pass: **cx**:**dx** optr of new GrObj object.
bp **GrObjBodyAddGrObjFlags** value specifying where in the list of objects to add the new GrObj.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GB_ATTACH_GOAM

This message lets the GrObjBody know where its attribute manager is. The manager's optr will be set in the instance data.

Pass: **cx**:**dx** optr of GrObjAttributeManager object.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GB_ATTACH_RULER

This message sets the *GBI_ruler* field. Note that if the GrObjBody is to interact with more than one VisRuler, they should be linked in a *VRI_slaveRuler* chain, and the ruler in the *GBI_ruler* field should be at the top of this chain.

Pass: **cx**:**dx** optr of VisRuler object.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GB_ATTACH_UI

This message lets the GrObjBody know where its head is. The head's optr will be set in the instance data. This message must be sent to the GrObjBody after it has been added to a document.

Pass: **cx**:**dx** optr of GrObjHead object.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GB_DETACH_UI

This message must be sent to the GrObjBody before it has been removed to a document, and should be in the document's MSG_GEN_DOCUMENT_DETACH_UI

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GB_DRAW

This message draws the graphic layer, allowing the caller some options. Since the GrObjBody is a subclass of VisComp, of course it is normally drawn by MSG_VIS_DRAW. MSG_GB_DRAW is normally used directly when drawing the layer to something not normally its output.

Pass: **bp** GStateHandle to draw to.
cl **DrawFlags** as would be set for MSG_VIS_DRAW.
dx **GrObjDrawFlags** that affect how to draw GrObjs.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GB_GET_BOUNDS

This message gets the current bounds of the GrObjBody.

Pass:	**ss:bp**	**RectDWord** structure to fill with bounds.
Returns:	Nothing.	
Destroyed:	**ax.**	
Library:	**grobj.def**	

MSG_GB_INSTANTIATE_GROBJ

This message instantiates a GrObj of the passed class in a block managed by the body.

Pass:	**cx:dx**	Pointer to class of new GrObj.
Returns:	**cx:dx**	optr of new GrObj.
Destroyed:	**ax.**	
Library:	**grobj.def**	

MSG_GB_SET_ACTION_NOTIFICATION_OUTPUT

This message specifies the default message and object GrObjs should send notification to when an action is performed on them. GrObjs will use this default if they don't have their own action notification.

Pass:	**cx:dx**	optr of object to notify.
	bp	Message to send.
Returns:	Nothing.	
Destroyed:	**ax.**	
Library:	**grobj.def**	

MSG_GB_SET_BOUNDS

This message sets the bounds of the GrObjBody.

Pass:	**ss:bp**	**RectDWord** structure holding new bounds.
Returns:	Nothing.	
Destroyed:	**ax.**	
Library:	**grobj.def**	

MSG_GB_SET_DESIRED_HANDLE_SIZE

This message sets the size of the selection and resize handles.

Pass: `cl` Desired handle size, in device coordinates.

Returns: Nothing.

Destroyed: `ax.`

Library: `grobj.def`

MSG_GB_SUSPEND_ACTION_NOTIFICATION

This message suspends action notification for all of a body's GrObjs. This prevents all the GrObjs from sending out any action notification.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax.`

Library: `grobj.def`

MSG_GB_UNSUSPEND_ACTION_NOTIFICATION

This message counteracts MSG_GB_SUSPEND_ACTION_NOTIFICATION. If all suspends have been balanced, the body's GrObjs will be free to send out action notification. However, action notifications that were aborted during the suspend period will not be sent out.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax.`

Library: `grobj.def`

GrObjClass

Instance Data

```
GOI_drawLink                LinkPart <>
                    public GOI_drawLink
```

GOI_drawLink is internal.

```
GOI_reverseLink     LinkPart <>
                  public  GOI_reverseLink
```

GOI_reverseLink is internal.

These fields are internal.

```
GOI_attrFlags       GrObjAttrFlags
         UIC Syntax:  attrFlags =  bitFieldComp word {
                      filler:7,
                      hasPasteInsideChildren,
                      pasteInside,
                      insertDeleteMoveAllowed,
                      insertDeleteResizeAllowed,
                      insertDeleteDeleteAllowed,
                      instruction,
                      multiplicativeRezize,
                      wrapTextType:2 = {
                       none,
                       rectangle,
                       tight,
                       inside
                      }}:{};
```

attrFlags (*GOI_attrFlags*) contains miscellaneous flags governing the GrObj's behavior.

```
GOI_optFlags        GrObjOptimizationFlags
         UIC Syntax:  optFlags =  bitFieldComp byte {
                      addedToBody,
                      inGroup,
                      grObjInvalid,
                      attributeManager,
                      floater,
                      actionNotification,
                      hasUnbalancedParentDimensions
                      }:grObjInvalid;
```

optFlags (*GOI_optFlags)* is internal.

```
GOI_msgOptFlags     GrObjMessageOptimizationFlags
         UIC Syntax:  msgOptFlags = bitFieldComp      word {
                      filler:6,
                      getDWFSelectionHandleBoundsForTrivialReject,
                      specialResize,
                      invalidateLine,
                      invalidateArea,
                      invalidate,
                      notifyAction,
```

```
                        sendUINotification,
                        drawFGArea,
                        drawFGLine,
                        drawBG}:{};
```

msgOptFlags (*GOI_msgOptFlags)* is internal.

`GOI_locks` `GrObjLocks`

GOI_locks acts to disallow performing certain operations on the object. Thus, if your application provides an object which the user should not be able to resize, then set the GOL_RESIZE lock.

`GOI_actionModes` `GrObjActionModes`

GOI_actionModes is internal.

`GOI_tempState` `GrObjTempModes`
`public GOI_tempState`

GOI_tempState is internal.

`GOI_normalTransform lptr.ObjectTransform`

GOI_normalTransform is internal.

`GOI_spriteTransform lptr.ObjectTransform`

GOI_spriteTransform is internal.

`GOI_areaAttrToken word`
`UIC Syntax: areaToken = wordComp:CA_NULL_ELEMENT;`

GOI_areaAttrToken is internal.

`GOI_lineAttrToken word`
`UIC Syntax: lineToken = wordComp:CA_NULL_ELEMENT;`

GOI_lineAttrToken is internal.

Messages

■ MSG_GO_AFTER_ADDED_TO_BODY

This message is sent to an object just after it has been added to a GrObjBody.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

Object Assembly Reference

MSG_GO_BECOME_SELECTED

This message tells an object to join the selection list, draw its handles, and otherwise take care of tasks associated with becoming selected.

Pass: `dl` **HandleUpdateMode** value.

Returns: Nothing.

Destroyed: `ax.`

Library: **grobj.def**

MSG_GO_BECOME_UNSELECTED

This message tells an object to leave the selection list, invalidate its selection handles, and otherwise take care of tasks associated with becoming unselected.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax.`

Library: **grobj.def**

MSG_GO_BEFORE_REMOVED_FROM_BODY

This message is sent to an object just before it is removed from its GrObjBody.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax.`

Library: **grobj.def**

MSG_GO_CHANGE_LOCKS

This message changes the locks on an object. The object's instance data will be changed accordingly.

Pass: `cx` **GrObjLocks** to set.
`dx` **GrObjLocks** to clear.

Returns: `cx` **GrObjLocks** before change.
`dx` **GrObjLocks** after change.

Destroyed: `ax.`

Library: **grobj.def**

MSG_GO_COMBINE_AREA_NOTIFICATION_DATA

This message combines this GrObj's attributes with the passed structure.

Pass:	^h**cx**	**GrObjNotifyAreaAttrChange** block.
Returns:	CF	Clear to continue processing; set to abort.
Destroyed:	**ax.**	
Library:	**grobj.def**	

MSG_GO_COMBINE_STYLE_NOTIFICATION_DATA

This message combines this GrObj's style attributes with the passed structure.

Pass:	^h**cx**	**NotifyStyleChange** block.
Returns:	CF	Clear to continue processing; set to abort.
Destroyed:	**ax.**	
Library:	**grobj.def**	

MSG_GO_COMBINE_AREA_NOTIFICATION_DATA

This message combines this GrObj's style sheet attributes with the passed structure.

Pass:	^h**cx**	**NotifyStyleSheetChange** block.
Returns:	CF	Clear to continue processing; set to abort.
Destroyed:	**ax.**	
Library:	**grobj.def**	

MSG_GO_CREATE_TRANSFER

Any subclass that requires more than just instance data to reconstruct itself will subclass this message to construct the **VMChain** necessary to do so.

The *GTP_curSlot* field of *params* is updated to the next slot in the tree, specified by *GTP_treeBlock*.

Pass:	**ss**:**bp**	**GrObjTransferParams** structure.
Returns:	**ss**:**bp**	*GTP_curSlot* field updated to the next slot in the tree specified by *GTP_treeBlock*.
Destroyed:	**ax.**	
Library:	**grobj.def**	

MSG_GO_FLIP_HORIZ

This message flips the GrObj about its vertical axis.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_FLIP_VERT

This message flips the GrObj about its horizontal axis.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_GET_CENTER

This message returns the coordinates of the object's center.

Pass: **ss:bp** **PointDWFixed** structure to hold return value.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_GET_POSITION

Get the position of the upper left of a GrObj. The position is in document coordinates unless the GrObj is in a group, in which case the position is relative to the upper left of the group. If the GrObj has been rotated, skewed, or otherwise transformed, then this message gets the location of the selection handle that was originally at the upper left of the GrObj.

Pass: **ss.bp** **PointDWFixed** structure to fill with position.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_GET_SIZE

Get the object's width and height in points. The dimensions are calculated by mapping the object's corners into document coordinates and calculating the distances between them. The center of the selection handles of a rectangle represent the corners mapped into document coordinates. The line thickness is not included in this calculation.

Pass: Nothing.

Returns: **dx**.**cx** **WWFixed** width in points.
bp.**ax** **WWFixed** height in points.

Destroyed: Nothing.

Library: **grobj.def**

MSG_GO_GET_WWF_OBJECT_BOUNDS

Returns the object's bounds.

Pass: **ss**:**bp** **RectWWFixed** structure to fill with return values.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_INIT_TO_DEFAULT_ATTRS

Have object initialize its attributes to the current defaults.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_INITIALIZE

This message initializes the object's size and position. It also causes the object to take on the default drawing attributes and to do any other initialization necessary before the object is added to the GrObjBody.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_MOVE

This message moves an object the passed distance.

Pass: **ss**:**bp** **PointDWFixed** structure containing amount to move object based upon its parent's coordinates.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_MOVE_CENTER_ABS

This message moves an object to an absolute position.

Pass: **ss**:**bp** **PointDWFixed** structure containing location to move object's center to in parent's coordinates.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_NOTIFY_ACTION

This message counteracts MSG_GO_SUSPEND_ACTION_NOTIFICATION. If all suspends have been balanced, the GrObj will be free to send out action notification. However, action notifications that were aborted during the suspend period will not be sent out.

Pass: **bp** **GrObjActionNotificationType** value specifying what sort of action took place.

Returns: **bp** Word of data whose meaning depends on the notification type.

Destroyed: **ax.**

Interception: If you subclass this message you must set GOMOF_NOTIFY_ACTION in the *GOI_msgOptFlags* field in the object's instance data.

Library: **grobj.def**

MSG_GO_NOTIFY_GROBJ_VALID

This message notifies the GrObj that it's ready for action. The GrObj has all attributes that it needs to draw, including a transformation. This message is

sent to the object at the end of an interactive create or after it has been created statically.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_NUDGE

This message moves an object by a number of device pixels.

Pass: **cx** Number of pixels to move object horizontally.
dx Number of pixels to move object vertically.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_READ_INSTANCE_FROM_TRANSFER

This message causes an existing GrObj to read any data needed to recreate the GrObj from a transfer item.

Pass: **ss:bp** **GrObjTransferParams** structure.

Returns: **ss:bp** *GTP_curPos* field updated to point just past the last read data; thus, the subclass could begin reading at this location.

Destroyed: **ax.**

Interception: Any subclass with data necessary to reconstruct itself will subclass this message. The subclass will first call its superclass, then read in its extra data.

Library: **grobj.def**

Object Assembly Reference

MSG_GO_REPLACE_WITH_TRANSFER

This message causes an existing GrObj to read the passed transfer item and recreate itself from that information. It is sent to an object during paste-type operations.

Pass: **ss**:**bp** **GrObjTransferParams** structure.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_ROTATE

This message rotates the GrObj about one of its handles.

Pass: **cx**.**dx** **WWFixed** degrees counterclockwise to rotate.
bp low **GrObjHandleSpecification** specifying which handle to rotate about.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_SCALE

This message scales a GrObj.

Pass: **ss**:**bp** **GrObjScaleData** structure.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_SET_ACTION_NOTIFICATION_OUTPUT

This message specifies the message and object the GrObj should send notification to when an action is performed on it.

Pass: **cx**:**dx** The optr of object to notify; pass zero in **cx** to clear the output.
bp Message to send.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**



MSG_GO_SET_AREA_ATTR

This message sets the area attributes of the object.

Pass:	**ss:bp**	**GrObjBaseAreaAttrElement** structure.
Returns:	Nothing.	
Destroyed:	**ax.**	
Library:	**grobj.def**	

MSG_GO_SET_AREA_COLOR

This message sets a GrObj's area color.

Pass:	**cl**	Red component.
	ch	Green component.
	dl	Blue component.
Returns:	Nothing.	
Destroyed:	**ax.**	
Library:	**grobj.def**	

MSG_GO_SET_AREA_DRAW_MODE

This message sets the draw mode of the object in the area structure. The draw mode is actually used for both the line and area drawing

Pass:	**cl**	**MixMode** value.
Returns:	Nothing.	
Destroyed:	**ax.**	
Library:	**grobj.def**	

MSG_GO_SET_AREA_MASK

This message sets the area mask of the object.

Pass:	**cl**	**SystemDrawMask** value for the new mask.
Returns:	Nothing.	
Destroyed:	**ax.**	
Library:	**grobj.def**	

MSG_GO_SET_LINE_ATTR

This message sets a GrObj's line attributes.

Pass: `ss:bp` **GrObjBaseLineAttrElement** structure with new line attributes.

Returns: Nothing.

Destroyed: `ax.`

Library: `grobj.def`

MSG_GO_SET_LINE_COLOR

This message sets a GrObj's area information flags so that the object may have a transparent area.

Pass: `cl` Red component.
`ch` Green component.
`dl` Blue component.

Returns: Nothing.

Destroyed: `ax.`

Library: `grobj.def`

MSG_GO_SET_LINE_END

This message sets a GrObj's area information flags so that the object may have a transparent area.

Pass: `cl` **LineEnd** value.

Returns: Nothing.

Destroyed: `ax.`

Library: `grobj.def`

MSG_GO_SET_LINE_JOIN

This message sets a GrObj's line join.

Pass: `cl` **LineJoin** value.

Returns: Nothing.

Destroyed: `ax.`

Library: `grobj.def`

MSG_GO_SET_LINE_MASK

This message sets a GrObj's line mask.

Pass: **cl** **SystemDrawMask** value of new line draw mask.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_SET_LINE_MITER_LIMIT

This message sets a GrObj's line miter limit, used when drawing corners.

Pass: **dx.cx** **WWFixed** miter limit.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_SET_LINE_STYLE

This message sets a GrObj's line style, or "dottedness."

Pass: **cl** **LineStyle** value..

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_SET_LINE_WIDTH

This message sets a GrObj's line width.

Pass: **dx.cx** **WWFixed** line width.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_SET_POSITION

This message sets the position of the original upper left of a GrObj. The position is set in document coordinates unless the GrObj is in a group, in

which case the position is in coordinates relative to the group's upper left corner.

Pass: **ss:bp** **PointDWFixed** structure containing object's desired position.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_SET_SIZE

This message sets a GrObj's size.

Pass: **ss:bp** **PointWWFixed** structure containing object's desired width and height in points.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_SET_TRANSPARENCY

This message sets a GrObj's area information flags so that the object may have a transparent area.

Pass: **cl** Pass *true* (i.e., non-zero) for transparent; otherwise *false* (i.e., zero).

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_SKEW

This message skews a GrObj.

Pass: **ss:bp** **GrObjSkewData** structure.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_SPECIAL_RESIZE_CONSTRAIN

This message suspends action notification for a GrObj. This prevents the GrObj from sending out any action notification.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_SUSPEND_ACTION_NOTIFICATION

This message prevents the receiving object from sending out any action notification.

If the GrObj has no action notification target object, then this message will be ignored.

If the GrObj is suspended, then saved and restored from state, it will reopen unsuspended.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_TOGGLE_SELECTION

This message causes a GrObj to switch its selection status.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_UNSUSPEND_ACTION_NOTIFICATION

This message counteracts MSG_GO_SUSPEND_ACTION_NOTIFICATION. If all suspends have been balanced, the GrObj will be free to send out action

notification. However, action notifications that were aborted during the suspend period will not be sent out.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax.**

Library: **grobj.def**

MSG_GO_WRITE_INSTANCE_TO_TRANSFER

This message causes the GrObj to write any data needed to create the GrObj into a transfer item.

The *GTP_curSlost* field of params is updated to point just past the last written data so that the superclass can begin writing, etc.

Pass: **ss**:**bp** **GrObjTransferParams** structure.

Returns: **ss**:**bp** *GTP_curPos* field updated to point just past the last written data; thus superclass could begin writing at this location.

Destroyed: **ax.**

Interception: Any subclass with data necessary to recreate the object will subclass this message. The subclass will first call its superclass, then write in its extra data.

Library: **grobj.def**

GroupClass

Instance Data

```
GI_drawHead          CompPart <>
```

GI_drawHead is internal.

```
GI_suspendCount      word
```

GI_suspendCount is internal.

```
GI_unsuspendOps      GroupUnsuspendOps
```

GI_unsuspendOps is internal.

Messages

MSG_GROUP_INSTANTIATE_GROBJ

This message combines this GrObj's attributes with the passed structure.

Pass:	**cx:dx**	Pointer to GrObj's class.
Returns:	**cx:dx**	optr of new object.
Destroyed:	**ax.**	
Library:	**grobj.def**	

PointerClass

Instance Data

`PTR_modes` `PointerModes`

PTR_modes is internal.

GrObjVisGuardianClass

Instance Data

`GOVGI_ward` `optr`

GOVGI_ward is the optr of the ward object.

`GOVGI_class` `fptr`

GOVGI_class is the class of the ward object.

`GOVGI_flags` `GrObjVisGuardianFlags`

GOVGI_flags is a field of miscellaneous flags.

Messages

MSG_GOVG_CREATE_VIS_WARD

This message creates a Vis object that will act as a ward.

Pass:	^h**cx**	Block in which to place ward object.
Returns:	**cx**:**dx**	optr of new ward object.
Destroyed:	**ax.**	
Library:	**grobj.def**	

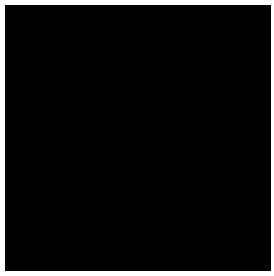
MSG_GOVG_GET_VIS_WARD

This message returns the output descriptor of this guardian's ward object.

Pass:	Nothing.	
Returns:	**cx**:**dx**	optr of ward object.
Destroyed:	**ax.**	
Library:	**grobj.def**	

Object Assembly Reference

Ruler Library



19

VisRulerClass 531

RulerViewClass 556

RulerShowControlClass 557

VisRulerClass

Instance Data

```
VRI_rulerAttrs        VisRulerAttributes
   UIC Syntax:  rulerAttrs = bitFieldComp byte {
                    ignoreOrigin,
                    showGuides,
                    showGrid,
                    showMouse,
                    horizontal,
                    isMaster
                    } : showGuides, showGrid, horizontal, isMaster;
```

rulerAttrs (*VRI_rulerAttrs*) contains flags which determine whether the ruler will support given features. This field tends to vary between applications; put some thought into which flags to turn on.

VRA_IGNORE_ORIGIN
: Using the *VRI_origin* field, it is possible to set an origin for the user to use based upon the coordinates of the graphic space within the main view. This can come in handy when the 0 point for the ruler shouldn't correspond to the very left or top of the view; GeoWrite uses this when the user wants the ruler's origin to be based upon the right edge of a text object rather than the edge of the view. The VRA_IGNORE_ORIGIN flag effectively turns off this feature.

VRA_SHOW_GUIDES
: This flag indicates that the user wants guidelines drawn over their main view.

VRA_SHOW_GRID
: This flag indicates that the user wants grid-lines drawn over the main view.

VRA_SHOW_MOUSE
: This flag indicates that the ruler should draw a tick mark to show the mouse pointer's location.

VRA_HORIZONTAL

This flag indicates that the ruler is horizontal. Horizontal rulers should have this flag turned on; vertical rulers should not.

VRA_MASTER

If a given view has only one ruler associated with it, that ruler should have this flag set. If a given view has more than one ruler (perhaps having one horizontal and one vertical), one of these rulers should be set up as the master ruler. It should have the VRA_MASTER flag set and should have the second ruler's optr in its *VRI_slave* field. If there is a third ruler, the second ruler should have its optr in its (the second ruler's) *VRI_slave* field, and so on.

```
VRI_type                VisRulerType
   UIC Syntax:  type = byteComp : "VRT_DEFAULT";

VRI_minIncrement        MinIncrementType
   UIC Syntax:          minIncrement = byteComp : "MUSM_EIGHTH_INCH"
```

type and *minIncrement* (*VRI_type* and *VRI_minIncrement*) determine the measurement type and ruler increment of the ruler. Normally, the user will work with the ruler's type via a RulerTypeControl.

```
VRI_constrainStrategy       VisRulerConstrainStrategy
   UIC Syntax:  constrainStrategy = bitFieldComp word {
                    override,
                    showMouse,
                    setReference,
                    snapX,
                    snapY,
                    relativeX,
                    relativeY,
                    verticalGuidelines,
                    horizontalGuidelines,
                    horizontalAxis,
                    verticalAxis,
                    unityAxis,
                    negativeUnityAxis,
                    vector,
                    vectorReflection,
                    internal
                    } : {};
```

Object Assembly Reference

constrainStrategy (*VRI_constrainStrategy*) contains the default mouse constraint strategy the ruler should use when constraining mouse movement.

VRCS_OVERRIDE
: This flag isn't really meant to be part of the default mouse constraint; it will be ignored if in the *VRI_constrainStrategy* field. During some events other constrain strategies may be combined with the default mouse constrain strategy; if the other strategy's VRCS_OVERRIDE flag is set, then the default mouse constraint strategy field will be ignored.

VRCS_SET_REFERENCE
: This flag signals that the location of the next mouse selection should be reference point, which is used with certain kinds of mouse constraint.

VRCS_SNAP_TO_GRID_X_ABSOLUTE
: This flag signals that the mouse position should be snapped to the grid horizontally.

VRCS_SNAP_TO_GRID_Y_ABSOLUTE
: This flag signals that the mouse position should be snapped to the grid vertically.

VRCS_SNAP_TO_GRID_X_RELATIVE
: This flag signals that the mouse position should be horizontally snapped to a grid such as would be formed if the present grid spacing was used with the reference point as the grid's origin.

VRCS_SNAP_TO_GRID_Y_RELATIVE
: This flag signals that the mouse position should be vertically snapped to a grid such as would be formed if the present grid spacing was used with the reference point as the grid's origin.

VRCS_SNAP_TO_GUIDES_X
: This flag signals that the mouse positions should be horizontally snapped to guidelines. Note that guidelines are only effective within range of their "influence" in pixels.

VRCS_SNAP_TO_GUIDES_Y
: This flag signals that the mouse positions should be vertically snapped to guidelines. Note that guidelines are only effective within range of their "influence" in pixels.

VRCS_CONSTRAIN_TO_HORIZONTAL_AXIS
: This flag signals that the mouse position should be restrained to the horizontal line passing through the reference point.

VRCS_CONSTRAIN_TO_VERTICAL_AXIS
: This flag signals that the mouse position should be restrained to the vertical line passing through the reference point.

VRCS_CONSTRAIN_TO_UNITY_SLOPE_AXIS
: This flag signals that the mouse position should be restrained to the diagonal line with slope ($y = x$) passing through the reference point.

VRCS_CONSTRAIN_TO_NEGATIVE_UNITY_SLOPE_AXIS
: This flag signals that the mouse position should be restrained to the diagonal line with slope ($y=-x$) passing through the reference point.

VRCS_CONSTRAIN_TO_VECTOR
: This flag signals that the mouse position should be constrained to the line passing through the reference and vector points.

VRCS_CONSTRAIN_TO_VECTOR_REFLECTION
: This flag signals that the mouse position should be constrained to the reflection of the line passing through the reference and vector points.

```
VRI_scale               WWFixed
   UIC Syntax:  scale = typeComp "WWFixed" : "0, 1";
```

scale (*VRI_scale*): The ruler must keep track of the main view's scale; if the main view has doubled in scale, the ruler should put its tick marks twice as far apart. This scale is stored in the *VRI_scale* field. The ruler automatically receives a MSG_VIS_RULER_VIEW_SCALE_FACTOR_CHANGED when the main view's scale changes. To set the scale independently of the main view scale, send a MSG_VIS_RULER_SET_SCALE.

```
VRI_origin              DWFixed
   UIC Syntax:  origin = typeComp "DWFixed": "0, 0";
```

origin (*VRI_origin*) allows you to set an origin for the ruler other than (0,0). The coordinates are given in terms of the main view's coordinate system.

```
VRI_reference           PointDWFixed
   UIC Syntax:      referenceX = typeComp "DWFixed": "0, 0";
                    referenceY = typeComp "DWFixed": "0, 0";
```

Object Assembly Reference

referenceX and *referenceY* (*VRI_reference*): The reference point, used for various types of mouse constraint, is stored in the *VRI_reference* field.

```
VRI_mouseMark          sdword
   UIC Syntax:  mouseMark = dwordComp : 0;
```

mouseMark (*VRI_mouseMark*) contains the coordinate at which the mouse tick is currently being drawn.

```
VRI_window             hptr.Window
                     noreloc VRI_window
   UIC Syntax:  gWin = hptrComp : null;
```

gWin (*VRI_window*), initiated automatically, holds the window handle of the main view.

```
VRI_slave              optr
   UIC Syntax:  slaveRuler = optrComp : null;
```

slaveRuler (*VRI_slave*) holds the ruler's slave ruler; see the documentation for VRA_MASTER, above, to learn about master and slave rulers.

```
VRI_grid               Grid
   UIC Syntax:      gridX = typeComp "WWFixed" : "0, 0";
                    gridY = typeComp "WWFixed" : "0, 0";
```

gridX and *gridY* (*VRI_grid*) contain the information used to maintain the grid associated with a ruler.

```
VRI_vectorSlope        WWFixed
   UIC Syntax:  vector = typeComp "WWFixed" : "0, 0";
```

vector (*VRI_vectorSlope*) defines the slope of a vector that may be used with vector mouse constraints.

```
VRI_guideArray         optr.ChunkArrayHeader
   UIC Syntax:  guideArray = dwordComp : 0;
```

guideArray (*VRI_guideArray*) field keeps track of the location where guideline data is stored; this field is initiated automatically.

```
VRI_guideInfluence   word
   UIC Syntax:  guideInfluence = wordComp : "DEFAULT_GUIDE_INFLUENCE";
```

guideInfluence (*VRI_guideInfluence*) contains the influence to use when constraining the mouse position to guidelines. Guidelines will only attract the mouse if it is within a certain distance of them—this distance is the

"influence" of the guidelines. It is measured in pixels, not points; thus, the guide influence is independent of scale.

```
VRI_desiredSize       word
   UIC Syntax:  desiredSize = wordComp : "VIS_RULER_HEIGHT";
```

desiredSize (*VRI_desiredSize*) contains the ruler's idea of its ideal thickness. By default, this is set so that the ruler will have room to draw its numbers and tick marks normally; if you will be subclassing the VisRuler to add more gadgetry, you may wish to use a different value here.

```
VRI_reserved          word
   UIC Syntax:  reserved = wordComp : 0;
```

reserved (*VRI_reserved*) is an internal field.

```
VRI_invalOD           optr
                   noreloc VRI_invalOD
   UIC Syntax:  invalOD = optrComp : null;
```

invalOD (*VRI_invalOD*) contains the object descriptor of the object which should be redrawn when the ruler needs to draw something differently to the main view; by the ruler's default behavior, this will be updated automatically via a MSG_VIS_RULER_GAINED_SELECTION.

```
VRI_transformGState  hptr.GState
                  noreloc VRI_transformGState
   UIC Syntax:  transformGState = hptrComp : null;
```

transformGState (*VRI_transformGState*) field contains the handle of a GState. This GState contains the transformation which is used to transform the coordinate system used with mouse constraints.

Messages

MSG_VIS_RULER_ADD_HORIZONTAL_GUIDE

Use this message to add a horizontal guide.

Pass: **ss:bp** **DWFixed** location of new guide.

Returns: Nothing.

Destroyed: Nothing.

Library: **ruler.def**

Object Assembly Reference

MSG_VIS_RULER_ADD_VERTICAL_GUIDE

Use this message to add a vertical guide.

Pass: **ss:bp** **DWFixed** location of new guide.

Returns: Nothing.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_CLEAR_CONSTRAIN_TRANSFORM

Use this message to remove any transformation on the coordinate system used to constrain the mouse.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax**

Library: **ruler.def**

MSG_VIS_RULER_COMBINE_GUIDE_INFO

This message allows the horizontal and vertical rulers to load their respective guide information into a single structure so that the information can be collated and passed on to a RulerGuideControl.

Pass: **^hcx** *(data)* Handle of data block.

Returns: Nothing.

Destroyed: **ax**

Library: **ruler.def**

MSG_VIS_RULER_CONSTRAIN_TO_AXES

Use this message to constrain the mouse to the axes; that is to force it to be in line either horizontally or vertically with the reference point (the point stored in *VRI_reference*).

Pass: **ss:bp** *(point)* **PointDWFixed** showing point to constrain.

Returns: **ss:bp** *(point)* **PointDWFixed** showing point constrained.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_CONSTRAIN_TO_DIAGONALS

Use this message to constrain the mouse to the diagonal axes; that is to force it to be on a line 45 degrees through the reference point (the point stored in *VRI_reference*).

Pass: **ss:bp** *(point)* **PointDWFixed** showing point to constrain.

Returns: **ss:bp** *(point)* **PointDWFixed** showing point constrained.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_CONSTRAIN_TO_HORIZONTAL_AXIS

Use this message to constrain the mouse, forcing it to be in line horizontally with the reference point (the point stored in *VRI_reference*).

Pass: **ss:bp** *(point)* **PointDWFixed** showing point to constrain.

Returns: **ss:bp** *(point)* **PointDWFixed** showing point constrained.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_CONSTRAIN_TO_VECTOR

This message computes the point resulting from constraining the passed point using vector constraint. This means that the mouse's constrained location will end up at a certain angle from the reference angle. This angle is determined by means of a point set via MSG_VIS_RULER_SET_VECTOR.

The point will be constrained either to the vector line or to that line's reflection.

Pass: **ss:bp** *(point)* **PointDWFixed** showing point to constrain.

Returns: **ss:bp** *(point)* **PointDWFixed** showing point constrained.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_CONSTRAIN_TO_VERTICAL_AXIS

Use this message to constrain the mouse, forcing it to be in line vertically with the reference point (the point stored in *VRI_reference*).

Pass: **ss:bp** *(point)* **PointDWFixed** showing point to constrain.

Returns: **ss:bp** *(point)* **PointDWFixed** showing point constrained.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_CONSTRAIN_X_TO_NEGATIVE_SLOPE_AXIS

Use this message to determine the coordinates resulting from horizontally snapping a given point to the Southeast diagonal line passing through the reference point.

Pass: **ss:bp** *(point)* **PointDWFixed** showing point to constrain.

Returns: **ss:bp** *(point)* **PointDWFixed** showing point constrained.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_CONSTRAIN_X_TO_UNITY_SLOPE_AXIS

Use this message to determine the coordinates resulting from horizontally snapping a given point to the Northeast diagonal line passing through the reference point.

Pass: **ss:bp** *(point)* **PointDWFixed** showing point to constrain.

Returns: **ss:bp** *(point)* **PointDWFixed** showing point constrained.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_CONSTRAIN_X_TO_VECTOR

This message computes the point resulting from horizontally constraining the passed point to the line going through the reference point with the present vector slope.

Pass: **ss:bp** *(point)* **PointDWFixed** showing point to constrain.

Returns: **ss:bp** *(point)* **PointDWFixed** showing point constrained.

Destroyed: Nothing.

Library: **ruler.def**

■ MSG_VIS_RULER_CONSTRAIN_X_TO_VECTOR_REFLECTION

This message computes the point resulting from horizontally constraining the passed point to the line going through the reference point with the negative of the present vector slope.

Pass: **ss:bp** *(point)* **PointDWFixed** showing point to constrain.

Returns: **ss:bp** *(point)* **PointDWFixed** showing point constrained.

Destroyed: Nothing.

Library: **ruler.def**

■ MSG_VIS_RULER_CONSTRAIN_Y_TO_NEGATIVE_SLOPE_AXIS

Use this message to determine the coordinates resulting from vertically snapping a given point to the Southeast diagonal line passing through the reference point.

Pass: **ss:bp** *(point)* **PointDWFixed** showing point to constrain.

Returns: **ss:bp** *(point)* **PointDWFixed** showing point constrained.

Destroyed: Nothing.

Library: **ruler.def**

■ MSG_VIS_RULER_CONSTRAIN_Y_TO_UNITY_SLOPE_AXIS

Use this message to determine the coordinates resulting from vertically snapping a given point to the Northeast diagonal line passing through the reference point.

Pass: **ss:bp** *(point)* **PointDWFixed** showing point to constrain.

Returns: **ss:bp** *(point)* **PointDWFixed** showing point constrained.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_CONSTRAIN_Y_TO_VECTOR

This message computes the point resulting from vertically constraining the passed point to the line going through the reference point with the present vector slope.

Pass: **ss:bp** *(point)* **PointDWFixed** showing point to constrain.

Returns: **ss:bp** *(point)* **PointDWFixed** showing point constrained.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_CONSTRAIN_Y_TO_VECTOR_REFLECTION

This message computes the point resulting from vertically constraining the passed point to the line going through the reference point with the negative of the present vector slope.

Pass: **ss:bp** *(point)* **PointDWFixed** showing point to constrain.

Returns: **ss:bp** *(point)* **PointDWFixed** showing point constrained.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_CREATE_GUIDE_ARRAY

By default, the VisRuler stores its guides in a chunk array, known as a guide array. If the ruler has no guides defined, it doesn't bother keeping this data structure around. When the first guide is created, the VisRuler will send itself this message, to which it will respond by setting up the data region.

Subclasses might intercept this message if they have data structures which need initializing when the first guide is created.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax**

Library: **ruler.def**

MSG_VIS_RULER_DELETE_HORIZONTAL_GUIDE

Use this message to delete the horizontal guide at the passed coordinate, if any.

Pass: **ss**:**bp** The location of the guide to delete as a **DWFixed** number.

Returns: Nothing.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_DELETE_VERTICAL_GUIDE

Use this message to delete the vertical guide at the passed coordinate, if any.

Pass: **ss**:**bp** The location of the guide to delete as a **DWFixed** number.

Returns: Nothing.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_DESELECT_ALL_HORIZONTAL_GUIDES

Use this message to deselect all of the ruler's horizontal guides.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_DESELECT_ALL_VERTICAL_GUIDES

Use this message to deselect all of the ruler's vertical guides.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_DRAW_GRID

This message will draw the grid to the passed GState if the VRA_SHOW_GRID_FLAG is set.

Pass: **^hbp** *(gstate)* GState to which the grid should be drawn.

Returns: Nothing.

Destroyed: **ax**

Library: **ruler.def**

MSG_VIS_RULER_DRAW_GUIDES

Use this message to draw guidelines to a document. The default VisRuler handler for this message will draw the lines at the appropriate coordinates to the passed GState.

Pass: **^hbp** *(gstate)* GState to which the grid should be drawn.

Returns: Nothing.

Destroyed: **ax**

Library: **ruler.def**

MSG_VIS_RULER_DRAW_GUIDE_INDICATORS

This message asks the VisRuler to draw tick-marks on itself to show the positions of guides. The default VisRuler has no handler for this message; subclasses are welcome to intersect it and draw indicators appropriately.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_DRAW_MOUSE_TICK

This message indicates that the ruler should draw a line indicating the mouse position and informs the ruler of the mouse's new position.

Pass: ss:bp **PointDWFixed** location of mouse.

Returns: Nothing.

Destroyed: **ax**

Library: **ruler.def**

MSG_VIS_RULER_GAINED_SELECTION

This message notifies the ruler that the ruled object (the object in the main view) is selected and the ruler should update its UI to reflect its own attributes.

Pass: `^lcx:dx` *(data)* Handle of block containing **VisRulerNotifyGuideChangeBlockHeader** structure.

Returns: Nothing.

Destroyed: `ax`

Library: **ruler.def**

MSG_VIS_RULER_GET_CONSTRAIN_STRATEGY

This message gets the current default mouse constrain strategy for the ruler.

Pass: Nothing.

Returns: `cx` **VisRulerConstrainStrategy** showing the ruler's default mouse constrain strategy.

Destroyed: `ax`

Library: **ruler.def**

MSG_VIS_RULER_GET_DESIRED_SIZE

This message returns how much space the ruler thinks it needs to draw the numbers and hatch marks.

This method should be subclassed if the subclassed ruler's size needs to vary with respect to scale factor, whether the ruler is horizontal or vertical, or whatever size seems appropriate.

Pass: Nothing.

Returns: `cx` Desired size, in points.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_GET_GRID_SPACING

This message returns the current grid spacing.

Pass: Nothing.

Returns:

dx.cx	**WWFixed** horizontal grid spacing
bp:ax	**WWFixed** vertical grid spacing.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_GET_GUIDE_INFLUENCE

Guidelines will only affect the mouse if the mouse pointer lies within the guide's area of influence. This message returns the present influence value.

Pass: Nothing.

Returns:

cx	Influence, in pixels.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_GET_ORIGIN

Use this message to get the ruler's origin point.

Pass: Nothing.

Returns:

dxcx.bp	The ruler's origin, expressed as a **DWFixed** value.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_GET_STRATEGIC_GRID_SPACING

This message returns a grid spacing that will look nice on the screen at the current scale factor.

Pass: Nothing.

Returns:

dx.cx	**WWFixed** horizontal grid spacing
bp:ax	**WWFixed** vertical grid spacing.

Destroyed: Nothing.

Library: **ruler.def**



MSG_VIS_RULER_GET_TYPE

This message returns the value in the *VRI_type* field.

Pass: Nothing.

Returns: `cl` Ruler type as **VisRulerType**.

Destroyed: `ax`

Library: **ruler.def**

MSG_VIS_RULER_HIDE_GRID

Use this message to request that the grid not be drawn to the main view, clearing the VRA_SHOW_GRID flag.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_HIDE_MOUSE

Sets a flag such that the ruler won't draw the mouse tick on pointer events.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax`

Library: **ruler.def**

MSG_VIS_RULER_INVALIDATE_WITH_SLAVES

The ruler's default behavior will result in it redrawing itself at appropriate times. Depending on what use you put your ruler to, you may have to force it to redraw itself. If so, you should probably use this message.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax`

Library: **ruler.def**

Object Assembly Reference

MSG_VIS_RULER_LOST_SELECTION

This message notifies the ruler that the ruled object is no longer selected.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax

Library: ruler.def

MSG_VIS_RULER_RULE_LARGE_PTR

This message takes a mouse pointer location, constrains it using all active current mouse constraints known of by the rulers, and returns the resulting constrained mouse location.

Pass: ss:bp **PointDWFixed** with current mouse location.
cx **VisRulerConstrainStrategy** with mouse constraints to combine with default mouse constraints. Note that if you set the VRCS_OVERRIDE flag, then the ruler's default mouse constraints will be ignored.

Returns: ss:bp **PointDWFixed** structure with constrained mouse location.

Destroyed: ax

Library: ruler.def

MSG_VIS_RULER_SCALE_FACTOR_CHANGED

The ruler will receive this message when the main view's scale factor changes. Specifically, when the RulerContent's scale is changing (which will happen because of the RulerView's link to the main view), its handler for MSG_META_CONTENT_VIEW_SCALE_FACTOR_CHANGED sends this message to the VisRuler.

Pass: ss:bp **ScaleChangedParams** structure.

Returns: Nothing.

Destroyed: Nothing.

Library: ruler.def

MSG_VIS_RULER_SELECT_HORIZONTAL_GUIDE

Use this message to select a horizontal guide at the passed coordinate, if any. Note that only one horizontal guide may be selected at any time.

Pass: **ss:bp** Location of guide as a **DWFixed** number.

Returns: Nothing.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_SELECT_VERTICAL_GUIDE

Use this message to select a vertical guide at the passed coordinate, if any. Note that only one horizontal guide may be selected at any time.

Pass: **ss:bp** Location of guide as a **DWFixed** number.

Returns: Nothing.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_SET_CONSTRAIN_STRATEGY

This message sets the default mouse constrain strategy for the ruler.

Pass: **cx** (*setflags*) A **VisRulerConstrainStrategy** with the constraint flags to turn on.

dx (*clearflags*) A **VisRulerConstrainStrategy** with the constraint flags to turn off.

Returns: Nothing.

Destroyed: **ax**

Library: **ruler.def**

MSG_VIS_RULER_SET_CONSTRAIN_TRANSFORM

Use this message to impose a transformation on the coordinate system used to constrain the mouse. You can see this at work in GeoDraw when resizing an object which has been rotated or skewed.

Pass: **ss:bp** (*tm*) A **TransMatrix** value holding the new matrix to use.

Returns: Nothing.

Destroyed: **ax**

Library: **ruler.def**

MSG_VIS_RULER_SET_GRID_SPACING

Set the grid's horizontal and vertical spacing, working with the *VRI_grid* field.

Pass: **dx.cx** (*spacing*) A **WWFixed** value specifying the spacing to use, as measured in points.

Returns: Nothing.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_SET_GUIDE_INFLUENCE

Guidelines will only affect the mouse if the mouse pointer lies within the guide's area of influence. This message sets the influence distance, measured in pixels.

Pass: **cx** (*influence*) The new influence to use, measured in pixels.

Returns: Nothing.

Destroyed: **ax**

Library: **ruler.def**

MSG_VIS_RULER_SET_HORIZONTAL_GRID_SPACING

Set the horizontal grid spacing, leaving the vertical alone.

Pass: **dx.cx** (*spacing*) A **WWFixed** value specifying the spacing to use, as measured in points.

Returns: Nothing.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_SET_IGNORE_ORIGIN

This message sets the "ignore origin" state.

Pass: **cx** (*ignore*) This should be non-zero to ignore the origin, zero otherwise.

Returns: Nothing.

Destroyed: **ax**

Library: **ruler.def**



MSG_VIS_RULER_SET_MIN_INCREMENT

This message sets the *VRI_minIncrement* field to the passed value. It must be passed the proper MinIncrementType value.

Pass: `cl` (*min*) A **MinIncrementType** value.

Returns: Nothing.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_SET_ORIGIN

Use this message to set the ruler's origin point.

Pass: `dxcx.bp` A **DWFixed** value to set the ruler's origin point.

Returns: Nothing.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_SET_REFERENCE

Set the ruler's reference point.

Pass: `ss:bp` A **PointDWFixed** structure containing the new reference point.

Returns: Nothing.

Destroyed: `cx`

Library: **ruler.def**

MSG_VIS_RULER_SET_SCALE

This message sets the *VRI_scale* field.

Pass: `dx.cx` (*scale*) A **WWFixed** value showing the scale factor to use.

Returns: Nothing.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_SET_TYPE

This message sets the *VRI_type* field.

Pass: `cl` (*type*) A **VisRulerType** with the ruler type to use.

Returns: Nothing.

Destroyed: `ax`

Library: **ruler.def**

MSG_VIS_RULER_SET_VECTOR

Set the point to use to define the "vector"-together with the reference point, this will form a slope which may be used for mouse constraints.

Pass: `ss:bp` Point which, together with the reference point, defines the vector.

Returns: Nothing.

Destroyed: `ax`

Library: **ruler.def**

MSG_VIS_RULER_SET_VERTICAL_GRID_SPACING

Set the vertical grid spacing, leaving the horizontal alone.

Pass: `dx.cx` (*spacing*) A **WWFixed** value specifying the spacing to use, as measured in points.

Returns: Nothing.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_SHOW_GRID

Use this message to request that the grid be drawn to the main view, setting the VRA_SHOW_GRID flag.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_SHOW_MOUSE

Sets a flag such that the ruler will draw the mouse tick on pointer events.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax**

Library: **ruler.def**

MSG_VIS_RULER_SNAP_RELATIVE_TO_REFERENCE

Returns the point closest to the passed point that is an integral number of grid spacings from the reference point.

Pass: **ss**:**bp** (*point*) **PointDWFixed** specifying point to snapped.

Returns: **ss**:**bp** (*point*) **PointDWFixed** with snapped point.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_SNAP_RELATIVE_TO_REFERENCE_X

Returns the point horizontally closest to the passed point that is an integral number of grid spacings from the reference point.

Pass: **ss**:**bp** (*point*) **PointDWFixed** specifying point to snapped.

Returns: **ss**:**bp** (*point*) **PointDWFixed** with snapped point.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_SNAP_RELATIVE_TO_REFERENCE_Y

Returns the point vertically closest to the passed point that is an integral number of grid spacings from the reference point.

Pass: **ss**:**bp** (*point*) **PointDWFixed** specifying point to snapped.

Returns: **ss**:**bp** (*point*) **PointDWFixed** with snapped point.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_SNAP_TO_GRID

This message snaps the passed coordinate to the grid.

Pass:	**ss:bp**	A **PointDWFixed** structure, the point to be snapped.
Returns:	**ss:bp**	A **PointDWFixed** structure, the point snapped.
Destroyed:	**ax**	
Library:	**ruler.def**	

MSG_VIS_RULER_SNAP_TO_GRID_X

This message horizontally snaps the passed coordinate to the grid.

Pass:	**ss:bp**	A **PointDWFixed** structure, the point to be snapped.
Returns:	**ss:bp**	A **PointDWFixed** structure, the point snapped.
Destroyed:	**ax**	
Library:	**ruler.def**	

MSG_VIS_RULER_SNAP_TO_GRID_Y

This message vertically snaps the passed coordinate to the grid.

Pass:	**ss:bp**	A **PointDWFixed** structure, the point to be snapped.
Returns:	**ss:bp**	A **PointDWFixed** structure, the point snapped.
Destroyed:	**ax**	
Library:	**ruler.def**	

MSG_VIS_RULER_SNAP_TO_GUIDES

Use this message to snap the passed point to the guides.

Pass:	**ss:bp**	A **PointDWFixed** structure, the point to be snapped.
Returns:	**ss:bp**	A **PointDWFixed** structure, the point snapped.
Destroyed:	**ax**	
Library:	**ruler.def**	



MSG_VIS_RULER_SNAP_TO_GUIDES_X

Use this message to horizontally snap the passed point to the guides.

Pass: **ss:bp** A **PointDWFixed** structure, the point to be snapped.

Returns: **ss:bp** A **PointDWFixed** structure, the point snapped.

Destroyed: **ax**

Library: **ruler.def**

MSG_VIS_RULER_SNAP_TO_GUIDES_Y

Use this message to vertically snap the passed point to the guides.

Pass: **ss:bp** A **PointDWFixed** structure, the point to be snapped.

Returns: **ss:bp** A **PointDWFixed** structure, the point snapped.

Destroyed: **ax**

Library: **ruler.def**

MSG_VIS_RULER_TURN_GRID_SNAPPING_OFF

Set the default **VisRulerConstrainStrategy** to not include grid snapping, turning off the VRCS_SNAP_TO_GRID_X_ABSOLUTE and VRCS_SNAP_TO_GRID_Y_ABSOLUTE flags.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_TURN_GRID_SNAPPING_ON

Set the default **VisRulerConstrainStrategy** to support grid snapping, setting the VRCS_SNAP_TO_GRID_X_ABSOLUTE and VRCS_SNAP_TO_GRID_Y_ABSOLUTE flags.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_TURN_GUIDES_SNAPPING_OFF

Set the default **VisRulerConstrainStrategy** to not include grid snapping, turning off the VRCS_SNAP_TO_GUIDES_X and VRCS_SNAP_TO_GUIDES_Y flags.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_TURN_GUIDES_SNAPPING_ON

Set the default **VisRulerConstrainStrategy** to include guides snapping, setting the VRCS_SNAP_TO_GUIDES_X and VRCS_SNAP_TO_GUIDES_Y flags.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Library: **ruler.def**

MSG_VIS_RULER_UPDATE_CONTROLLERS

This message signals that the ruler should update its associated controllers.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax**

Library: **ruler.def**

MSG_VIS_RULER_UPDATE_GRID_CONTROLLER

This message signals that the ruler should update its grid controller, if any.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax**

Library: **ruler.def**



MSG_VIS_RULER_UPDATE_GUIDE_CONTROLLER

This message signals that the ruler should update its guide controller, if any.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax

Library: ruler.def

MSG_VIS_RULER_UPDATE_TYPE_CONTROLLER

This message signals that the ruler should update its type controller, if any.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax

Library: ruler.def

RulerViewClass

Instance Data

```
RVI_attrs               RulerViewAttributes
   UIC Syntax:  rulerAttrs = bitFieldComp byte {
                   horizontal
                   } : horizontal;
```

rulerAttrs (*RVI_attrs*) contains one flag, which determines whether the ruler is horizontal, as opposed to vertical.

RulerShowControlClass

Instance Data

```
RSCI_attrs              RulerShowControlAttributes
   UIC Syntax:  rulerAttrs = bitFieldComp word {
                   showVertical,
                   showHorizontal
                   } : showVertical, showHorizontal;
```

rulerAttrs (*RSCI_attrs*) determines the attributes of the controller.

```
RSCI_gcnList            GCNListType
   UIC Syntax:      gcnManuf = wordComp : “MANUFACTURER_ID_GEOWORKS”;
                    gcnType = wordComp :
                      “GAGCNLT_DISPLAY_OBJECTS_WITH_RULERS”;
```

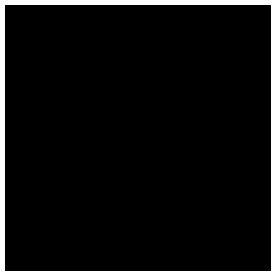
gcnManuf and *gcnType* (*RSCI_gcnList*) keep track of which GCN list should receive notification when the rulers call for an update.

```
RSCI_message            word
   UIC Syntax:  message = wordComp : 0
```

message (*RSCI_message*): This message will receive a **RulerShowControlAttributes** value in the **cx** register.

Object Assembly Reference

Spreadsheet Library



20

SpreadsheetClass 561

SpreadsheetRulerClass 587

SpreadsheetClass

Instance Data

```
SSI_cellParams        CellFunctionParameters
   UIC Syntax:  cellParams    = typeComp "CellFunctionParameters" : "<>";
```

cellParams (*SSI_cellParams*) is a cached **CellFunctionParameters** structure used to work with DB structures.

```
SSI_chunk             word
```

SSI_chunk is the chunk handle of the spreadsheet object. This is set and maintained by the object.

```
SSI_chartBody         optr
                    noreloc SSI_chartBody
   UIC Syntax:  chartBody        = optrComp : null;
```

chartBody (*SSI_chartBody*) is an internal field.

```
SSI_mapBlock          word
```

SSI_mapBlock is the VM handle of the attached file's spreadsheet map block. The spreadsheet sets this when a spreadsheet file is initialized; applications must provide this when they instruct a spreadsheet object to open a file.

```
SSI_styleArray        word

SSI_rowArray          word

SSI_formatArray       word

SSI_nameArray         word
```

SSI_styleArray, *SSI_nameArray*, *SSI_formatArray*, and *SSI_nameArray* store the handles of VM blocks within the spreadsheet file. These blocks contain information about the file. The handles are also stored in the spreadsheet map block. The spreadsheet object sets and maintains these fields automatically.

```
SSI_maxRow            word
   UIC Syntax:  maxRow = wordComp : “0”;
```

```
SSI_maxCol              word
   UIC Syntax:   maxCol = wordComp : "0";
```

maxRow (*SSI_maxRow*) and *maxCol* (*SSI_maxCol*) store the indices of the leftmost row and bottommost column allowed in the spreadsheet. The application may set these; they must be no larger than the spreadsheet library constants MAX_ROW (8191) and MAX_COLUMN (255).

```
SSI_offset              PointDWord
```

SSI_offset is the offset to the visible portion of the spreadsheet. This field is maintained automatically by the spreadsheet object.

```
SSI_visible              CellRange
```

SSI_offset stores the row and column indices of the currently visible portion of the spreadsheet. This field is maintained automatically by the spreadsheet object.

```
SSI_active              CellReference
```

SSI_active contains a reference to the active cell. This field is automatically set and maintained by the spreadsheet object.

```
SSI_quickSource         CellRange
```

SSI_quickSource is used for bookkeeping during a quick move/copy. During the move, the row/column indices of the source area are stored here.

```
SSI_selected             CellRange
```

SSI_selected contains the row/column indices of the currently selected region. This field is automatically set and maintained by the spreadsheet object.

```
SSI_curAttrs            word
```

SSI_curAttrs contains the token for the current style attributes. All text entered will have this style. This field is automatically set and maintained by the spreadsheet object.

```
SSI_gstate              hptr.GState
                     noreloc SSI_gstate
```

SSI_gstate is used to cache the gstate handle used for drawing to the screen. This field is set and maintained by the spreadsheet object.

```
SSI_gsRefCount          SSRefCountAndFlags
```

SSI_gsRefCount is used to keep track of the number of references to the gstate; this lets the spreadsheet object know when to free the gstate. This field is set and maintained by the spreadsheet object.

Object Assembly Reference

```
SSI_flags               SpreadsheetFlags
   UIC Syntax:   flags = wordComp : 0;
```

flags (*SSI_flags*) keeps track of details about the Spreadsheet's current state. Some of these fields are saved to the Spreadsheet map block, and some are not. There are only a two fields in this record that are of interest to the application:

SF_MANUAL_RECALC
: If this flag is clear, whenever a cell's value is changed, the spreadsheet will automatically recalculate the cell's value (and the values of all cells which depend on it). If the flag is set, the application must force a recalculation by sending MSG_SPREADSHEET_RECALC to the spreadsheet object. By default, the flag is clear.

SF_ALLOW_ITERATION
: If this flag is clear, the spreadsheet will not allow circular cell references (e.g. A1 = 10 + B1, B1 = 1/A1); all cells in a circular reference will evaluate to the error PSEE_CIRCULAR_REF, and all cells which depend on these cells will evaluate to PSEE_CIRCULAR_DEP. If the flag is set, the spreadsheet will follow circular references for a set number of circuits (specified in *SSI_circCount*) or until values converge within a specified tolerance (specified in *SSI_converge*).

Both of these flags are saved in the spreadsheet map block; thus, when a spreadsheet file is opened, these flags will be set to their values as of the last time the file was saved.

```
SSI_drawFlags           SpreadsheetDrawFlags
   UIC Syntax:   drawFlags = wordComp : 0;
```

drawFlags (*SSI_drawFlags*) is a word-sized record of **SpreadsheetDrawFlags**. These flags specify how the spreadsheet will draw itself on-screen. The application can change some of these at run-time.

SDF_DRAW_GRAPHICS
: Draw graphics associated with the spreadsheet.

SDF_DRAW_NOTE_BUTTON
: Add glyphs in cells which contain notes, as well as in header or footer cells.

SDF_DRAW_GRID
: Draw grid lines between cells.

SDF_SIDEWAYS
: Draw the spreadsheet sideways. This is relevant only for printing.

SDF_SCALE_TO_FIT
: Scale the selected range to fill a page. This is significant only for printing.

SDF_DRAW_ROW_COLUMN_TITLES
: Draw the row and column titles.

SDF_SKIP_DRAW
: Do not draw anything to the screen. Set this if you want to find out how the pages will lie, without actually drawing or printing them.

SDF_CENTER_VERTICALLY
: Center each page's contents vertically. This is relevant only for printing.

SDF_CENTER_HORIZONTALLY
: Center each page's contents vertically. This is relevant only for printing.

SDF_CONTINUOUS
: Print the spreadsheet continuously; that is, don't leave space between pages or at the edges (or leave as little margin as the printer allows). This is relevant only for printing.

SDF_DRAW_HEADER
: Print the header cells at the top of each page. This is relevant only for printing.

SDF_DRAW_FOOTER
: Print the footer cells at the bottom of each page. This is relevant only for printing.

SDF_PRINT_DOCUMENT
: This flag is never set or checked by the spreadsheet object. It is provided for the convenience of objects which use the spreadsheet.

SDF_PRINT_NOTES
: This flag is never set or checked by the spreadsheet object. It is provided for the convenience of objects which use the spreadsheet.

Object Assembly Reference

```
SSI_attributes        SpreadsheetAttributes
   UIC Syntax:  attributes =  bitFieldComp byte {
                   targetable,
                   engineMode,
                   isLayer
                   } : targetable;
```

To run a spreadsheet in engine mode, set the SA_ENGINE_MODE bit in the field *SSI_attributes*. This is the only field in *attributes* (*SSI_attributes*) with which applications need concern themselves.

```
SSI_header            CellRange

SSI_footer            CellRange
```

SSI_header and *SSI_footer* specify "header" and "footer" ranges of cells. These cells will automatically be printed at the top and bottom of each page (when the spreadsheet is printed). Users will generally set these with the Header and Footer controllers

```
SSI_circCount         word
   UIC Syntax:  circCount = wordComp : 1;
```

circCount (*SSI_circCount*) specifies how many times the Spreadsheet should follow circular references to approximate the cells' values. This is ignored if the SF_ALLOW_ITERATION bit is not set. If the count is set to zero, circular references will be flagged as errors.

```
SSI_converge          FloatNum <>
   UIC Syntax:  converge = typeComp "FloatNum" : "0, 0, 0, 0, 0";
```

converge (*SSI_converge*) specifies how close an approximation we want with circular references. If each of the cells in a circular reference changes its value by less than the *SSI_converge* value, the spreadsheet will stop following the loop. If the converge value is left at zero, the spreadsheet will make the full *SSI_circCount* circuits.

```
SSI_ancestorList      hptr

SSI_childList         hptr

SSI_finalList         hptr
```

SSI_ancestorList, *SSI_childList*, and *SSI_finalList* are cached in the instance data for efficiency. Applications should ignore these fields.

```
SSI_ruler             optr
   UIC Syntax:  ruler = optrComp : null;
```

The spreadsheet object needs to be linked to the SpreadsheetRuler objects. To set up the link, put an optr to one of the rulers in *ruler* (*SSI_ruler*); have the other ruler be a slave of the first.

```
SSI_bounds          RectDWord
```

SSI_bounds holds the bounds of the portion of the spreadsheet which contains data. This field is set and maintained by the spreadsheet.

Messages

■ MSG_SPREADSHEET_ADD_NAME

This message adds a name to the spreadsheet's name list.

This message cannot go across threads, since it's arguments are on the stack; to send an add-name message across threads, use MSG_SPREADSHEET_ADD_NAME_WITH_PARAM_BLOCK.

Pass:	**dx**:**bp**	**SpreadsheetNameParameters** structure with the following fields filled in: *SNP_textLength*, *SNP_text*, *SNP_defLength*, *SNP_definition*. This structure should be on the stack, with **dx** = **ss**.
Returns:	**cx**	If name added successfully, entry number in the defined name list; if name could not be added, then **cx** will return -1.
	dx	If name added successfully, number of defined names in the name list; if name could not be added, then **dl** will be a **ParserScannerEvaluatorError**.
	bp	If name added successfully, number of undefined names in the name list.
Destroyed:	**ax**	
Library:	**ssheet.def**	

■ MSG_SPREADSHEET_ALTER_DRAW_FLAGS

This message changes the settings of the spreadsheet object's *SSI_drawFlags* field. If the same bit is set in **cx** and **dx**, the results are undefined.

Pass:	**cx**	**SpreadsheetDrawFlasg** bits to set.
	dx	**SpreadsheetDrawFlags** bits to clear.
Returns:	Nothing.	
Destroyed:	**ax, cx, dx, bp.**	
Library:	**ssheet.def**	

Object Assembly Reference

MSG_SPREADSHEET_ATTACH_FILE

This message instructs a spreadsheet object to attach itself to a spreadsheet file. The spreadsheet object will copy the appropriate information from the specified map block into its own instance data. Applications must send this when they open any spreadsheet file, whether a newly-initialized one or a pre-existing one.

Pass: **dx** VM file handle.
cx VM handle of spreadsheet map block.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **ssheet.def**

MSG_SPREADSHEET_ATTACH_UI

This message attaches UI objects to the spreadsheet object.

Pass: **dx** VM file handle.
cx Handle of **SpreadsheetSetupData** block. The only field in this block, *SSD_chartBody*, should be NULL.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **ssheet.def**

MSG_SPREADSHEET_CHANGE_COLUMN_WIDTH

This message expands or contracts the width of a column (or all selected columns).

Pass: **cx** Signed word; amount by which to expand column widths.
dx Column number or SPREADSHEET_ADDRESS_USE_SELECTION.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **ssheet.def**

MSG_SPREADSHEET_CHANGE_NAME

This message changes a name in the spreadsheet name list.

This message cannot be sent across threads, due to the way the arguments are passed on the stack.

Pass:	**dx**:**bp**	**SpreadsheetNameParameters** structure with the following fields filled in: *SNP_flags*, *SNP_listEntry*, *SNP_textLength*, *SNP_text*, *SNP_defLength*, *SNP_definition*. This structure should be on the stack, with **dx** = **ss**.
Returns:	**cx**	If entry changed successfully, entry number in the defined name list; otherwise -1.
	dx	If entry changed successfully, number of defined names in the name list; otherwise **dl** is a **ParserScannerEvaluatorError**.
	bp	If entry changed successfully, number of undefined names in the name list.
Destroyed:	**ax.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_CHANGE_RECALC_PARAMS

This message changes the current values of the spreadsheet object's *SSI_flags*, *SSI_circCount*, and *SSI_converge* fields. The spreadsheet object copies the SpreadsheetRecalcParams structure's *SRP_circCount*, and *SRP_converge* fields into its own *SSI_circCount* and *SSI_converge* fields. It also sets its SF_MANUAL_RECALC and SF_ALLOW_ITERATION flags to correspond to the settings in *SRP_flags*; it does not change any of the other flags in *SSI_flags*.

Pass:	**ss**:**bp**	**SpreadsheetRecalcParams** structure.
	dx	sizeof **SpreadsheetRecalcParams**.
Returns:	Nothing.	
Destroyed:	**ax, cx, dx, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_CHANGE_ROW_HEIGHT

This message expands or contracts the height of a row (or all selected rows).

Pass:	**cx**	Signed word; amount by which to expand row heights.
	dx	Row number or SPREADSHEET_ADDRESS_USE_SELECTION.
Returns:	Nothing.	
Destroyed:	**ax, cx, dx, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_CHECK_INSERT_SPACE

This message checks to see whether an insert/delete would delete any data cells.

Pass: `cx` **SpreadsheetInsertFlags** field.

Returns: `al` **SpreadsheetInsertError** value.

Destroyed: `ah, cx, dx, bp.`

Library: **ssheet.def**

MSG_SPREADSHEET_CLEAR_SELECTED

Clear the selected cells.

Pass: `cx` **SpreadsheetClearFlags** field.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **ssheet.def**

MSG_SPREADSHEET_COMPLETE_REDRAW

This message forces the entire spreadsheet to redraw. The spreadsheet will also tell its rulers to redraw.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **ssheet.def**

MSG_SPREADSHEET_DELETE_NAME

This message deletes a name from the spreadsheet name list.



This message cannot be sent across threads, due to the way the arguments are passed on the stack.

Pass:	**dx**:**bp**	**SpreadsheetNameParameters** structure with the *SNP_listEntry* field filled in. This structure should be on the stack, with **dx** = **ss**.
Returns:	**cx**	Token of the deleted name.
	dx	Number of defined names on the list.
	bp	Number of undefined names on the list.
Destroyed:	**ax.**	
Library:	**ssheet.def**	

■ MSG_SPREADSHEET_DISPLAY_NOTE

The spreadsheet should display the note associated with the current active cell. The spreadsheet sends this message to itself when the user double-clicks on a cell that contains a note.

By default, the spreadsheet object will do nothing in response to this message; a subclass could intercept it, however.

Pass:	**dl**	**SpreadsheetDoubleClickFlags** field.
	dh	**CellType** (if cell has data yet).
Returns:	Nothing.	
Destroyed:	**ax, cx, bp.**	
Library:	**ssheet.def**	

■ MSG_SPREADSHEET_DRAW_RANGE

This message draws a range of cells. It is useful when printing.

Pass:	**ss**:**bp**	**SpreadsheetDrawParams** structure.
	dx	sizeof **SpreadsheetDrawParams**.
Returns:	Nothing.	
Destroyed:	**ax, cx, dx, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_ENTER_DATA

Enter data into the active cell. The spreadsheet will grab the focus if the data is entered correctly.

Pass:	^h**dx**	Handle of text block.
	cx	Length of text string (not including trailing NULL).
Returns:	CF	Set on error (and data was thus not entered); clear otherwise.
Destroyed:	**ax, cx, dx, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_ENTER_DATA_WITH_EVENT

Enter data into the active cell. The spreadsheet will grab the focus if the data is entered correctly. If the data is valid, the spreadsheet will then send itself the event passed in **bp**. If the data is not valid, the spreadsheet will free the event.

Pass:	^h**dx**	Handle of text block.
	cx	Length of text string (not including trailing NULL).
	^h**bp**	Event handle, or zero if none.
Returns:	CF	Set on error (and data was thus not entered); clear otherwise.
Destroyed:	**ax, cx, dx, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_ERROR

The spreadsheet sends itself this message when it encounters some error. Intercept it to tell the user about the error.

Pass:	**dl**	**ParserScannerEvalutatorError** value.
Returns:	Nothing.	
Destroyed:	**ax, cx, dx, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_EVAL_EXPRESSION

This assembly-only message asks the spreadsheet to evaluate an expression.

Pass:	**ss**:**bp**	**SpreadsheetEvalParameters** with *SEP_expression* field filled in.
	cl	**EvalFlags** field.
	dx	sizeof **SpreadsheetFormatParameters**.
Returns:	**al**	**ParserScannerEvaluatorError** value if an error encountered; -1 otherwise.
Destroyed:	**Nothing.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_EVAL_FUNCTION

Evaluate an application-defined function. By default, there is no handler for this message. If a spreadsheet object has its SF_APPLICATION_FUNCTIONS bit set, then its class must handle this message.

Pass:	**ss**:**bp**	**SpreadsheetEvalFuncParameters** structure.
Returns:	CF	Set on error; clear otherwise.
	al	Error code on error.
Destroyed:	**ah, cx, dx, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_EXTEND_CONTRACT_SELECTION

Extend or contract the selected range to include the specified range.

Pass:	**ss**:**bp**	**SpreadsheetRangeParams** structure (*SRP_active* field ignored).
	dx	sizeof **SpreadsheetRangeParams**.
Returns:	Nothing.	
Destroyed:	**ax, cx, dx, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_FILL_RANGE

Fill a range with values copied from the first row/column.

Pass: `cl` **SpreadsheetSeriesFillFlags** field.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **ssheet.def**

MSG_SPREADSHEET_FILL_SERIES

This message asks the spreadsheet to fill a range with a series of values, either by linear step or geometric step.

Pass: `ss:bp` **SpreadsheetSeriesFillParams** structure.
`dx` sizeof **SpreadsheetSeriesFillParams**.

Returns: `al` **SpreadsheetFillError** value.

Destroyed: `ah, cx, dx, bp.`

Library: **ssheet.def**

MSG_SPREADSHEET_FORMAT_EXPRESSION

Format an expression of parser tokens.

Pass: `ss:bp` **SpreadsheetFormatParameters** with *SFP_expression*, *SFP_text*, and *SFP_length* fields filled in.
`dx` sizeof **SpreadsheetFormatParameters**.

Returns: In **SpreadsheetFormatParameters** structure, **SFP_text* will hold the formatted text (null terminated), and *SFP_length* will hold the length of the formatted text.

Destroyed: `ax, cx, dx, bp.`

Library: **ssheet.def**

MSG_SPREADSHEET_FORMAT_FUNCTION

This message returns the string used to name an application-defined function (e.g., “SUM”). By default, there is no handler for this message. If a

spreadsheet object has its SF_APPLICATION_FUNCTIONS bit set, then its class must handle this message.

Pass:	**ss:bp**	Pointer to a 256 character (512 bytes on a DBCS system) buffer.
	cx	Function ID.
Returns:	**ss:bp**	String buffer filled in with data.
	cx	Number of characters written.
Destroyed:	**ax, dx, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_FORMAT_RANGE_REFERENCE

Format a range reference, taking a cell range value and constructing a string to describe it.

Pass:	**ss:bp**	**SpreadsheetFormatParseRangeParams** structure. *SFPRP_range* should be the range to use.
	dx	sizeof **SpreadsheetFormatParseRangeParams**.
Returns:	**ss:bp**	Same structure, with *SFPRP_text* field filled in.
Destroyed:	**ax, cx, dx.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_FUNCTION_TO_CELL

This message converts an application-defined function into a cell so that dependencies can be added to the function. By default, there is no handler for this message. If a spreadsheet object has its SF_APPLICATION_FUNCTIONS bit set, then its class must handle this message.

Pass:	**cx**	**FunctionID** value.
	ss:bp	**EvalParameters** structure.
Returns:	**dx**	Row (zero means no dependency required).
	cx	Column.
	CF	Set on error; clear otherwise.
	al	On error, error code.
Destroyed:	**ah, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_FUNCTION_TO_TOKEN

This message converts the name of an application-defined function into a token that can be used to identify the function later. By default, there is no

handler for this message. If a spreadsheet object has its SF_APPLICATION_FUNCTIONS bit set, then its class must handle this message.

Pass:	**dx**:**bp**	Pointer to text of the identifier.
	cx	Length of the string.
Returns:	CF	Set if the string is a function.
	di	**FunctionID**.
Destroyed:	**ax, cx, dx, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_GET_COLUMN_AT_POSITION

Get the column at the specified position.

Pass:	**dx**:**cx**	32-bit *x* position.
Returns:	**ax**	Column number.
	cx	Distance to column right edge, a non-positive number.
	dx	Distance to column left edge, a non-negative number.
Destroyed:	**bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_GET_COLUMN_WIDTH

Get the width of a column.

Pass:	**cx**	Column number.
Returns:	**dx**	Column width, or -1 if there is no such column.
Destroyed:	**ax, cx, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_GET_DRAW_FLAGS

This message returns the spreadsheet object's *SSI_drawFlags* instance data field.

Pass:	Nothing.	
Returns:	**dx**	**SpreadsheetDrawFlags** field.
Destroyed:	**ax, cx, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_GET_EXTENT

This message gets the extent of the spreadsheet.

Pass:	**cx**	**SpreadsheetExtentType** value.
Returns:	**ax**, **cx**	Row, column of first cell in extent; **ax** is -1 if there is no data in the spreadsheet..
	dx, **bp**	Row, column of last cell in extent.
Destroyed:	**Nothing.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_GET_FILE

This message returns the **VMFileHandle** of the file attached to the recipient Spreadsheet object.

Pass:	Nothing.	
Returns:	**cx**	File handle.
Destroyed:	**ax, dx, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_GET_FOOTER_RANGE

Get the range that is the footer.

Pass:	Nothing.	
Returns:	**ax**, **cx**	Row, column of first cell in footer range; if there is no footer, **ax** will be -1.
	dx, **bp**	Row, column of last cell in footer range.
Destroyed:	**Nothing.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_GET_HEADER_RANGE

Get the range that is the header.

Pass:	Nothing.	
Returns:	**ax**, **cx**	Row, column of first cell in header range; if there is no header, **ax** will be -1.
	dx, **bp**	Row, column of last cell in header range.
Destroyed:	**Nothing.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_GET_NAME_COUNT

This message returns the number of names in the spreadsheet name list.

Pass: Nothing.

Returns:

dx	Number of defined names in the name list.
bp	Number of undefined names in the name list.

Destroyed: **ax, cx.**

Library: **ssheet.def**

MSG_SPREADSHEET_GET_NAME_INFO

This message returns information about a name in the name list.

This message, because of the way it passes arguments on the stack, must not be called across threads.

Pass:

dx:**bp**	**SpreadsheetNameParameters** structure, with the following fields filled in: *SNP_flags*, *SNP_listEntry*. This structure is on the stack, with **dx**=**ss**.

Returns: All fields of **SpreadsheetNameParameters** structure filled in.

Destroyed: **Nothing.**

Library: **ssheet.def**

MSG_SPREADSHEET_GET_NOTE

This message retrieves the note data for a given cell.

Pass:

dx, **bp**	Row, column of the cell whose note we want.

Returns:

cx	Size of the note (without NULL terminator), or zero if there is no text.
^h**dx**	Block of the block containing the note text.

Destroyed: **ax, bp.**

Library: **ssheet.def**

MSG_SPREADSHEET_GET_NOTE_FOR_ACTIVE_CELL

This message retrieves the note data for the active cell.

Pass:	**Nothing.**	
Returns:	**cx**	Size of the note (without NULL terminator), or zero if there is no text.
	^h**dx**	Block of the block containing the note text.
Destroyed:	**ax, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_GET_RANGE_BOUNDS

Get the bounds around a given range.

Pass:	**dx:bp**	Pointer to **CellRange** structure.
	dx:**cx**	Pointer to **RectDWord** structure to fill in.
Returns:	Nothing.	
Destroyed:	**Nothing.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_GET_RECALC_PARAMS

This message gets the current values of the Spreadsheet object's *SSI_flags*, *SSI_circCount*, and *SSI_converge* fields. The fields are copied into the corresponding fields of a **SpreadsheetRecalcParams** structure.

Pass:	**ss**:**bp**	**SpreadsheetRecalcParams** structure to hold return value.
	dx	sizeof **SpreadsheetRecalcParams**.
Returns:	**SpreadsheetRecalcParams** structure filled in.	
	bp	Preserved.
Destroyed:	**ax, cx, dx.**	
Library:	**ssheet.def**	

Object Assembly Reference

MSG_SPREADSHEET_GET_ROW_AT_POSITION

This message computes the row at the specified coordinate position.

Pass:	**dx**:**cx**	**DWord** *y* position.
Returns:	**ax**	Row number.
	cx	Distance to row bottom edge, as a negative number.
	dx	Distance to row top edge, as a positive number.
Destroyed:	**bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_GET_ROW_HEIGHT

This message gets the height of a row.

Pass:	**cx**	Row number.
Returns:	**dx**	Row height, or -1 if there is no such row.
Destroyed:	**ax, cx, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_GET_SELECTION

This message returns the range of cells the user has selected.

Pass:	**ss**:**bp**	**SpreadsheetRangeParams** structure.
	dx	sizeof **SpreadsheetRangeParams**.
Returns:	**ss**:**bp**	**SpreadsheetRangeParams** structure filled in. *SRP_selection* will hold the selected range, and *SRP_active* is the active cell.
Destroyed:	**ax, cx, dx.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_GOTO_CELL

This message parses the text in the passed block, selects the range of cells named in that block, and makes sure that the selected cells are then visible.

Pass:	**dx**	Handle of text block.
	cx	Length of text (not counting NULL terminator).
Returns:	Nothing.	
Destroyed:	**ax, cx, dx, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_HANDLE_SPECIAL_FUNCTION

This message handles one of the special functions from the parser. There is no default handler for this message; subclasses must intercept this message to handle these special functions.

Pass: `cx` **SpecialFunction** value.
`ss:dx` Pointer to buffer to fill with file if SF_FILENAME passed.

Returns: `cx` If the passed value was SF_FILENAME, this will be the length of the text in the buffer; if SF_PAGE, then the current page number; if SF_PAGES, then the total page count.

Destroyed: Nothing.

Library: **ssheet.def**

MSG_SPREADSHEET_INSERT_SPACE

Insert or delete space in a spreadsheet.

Pass: `cx` **SpreadsheetInsertFlags** field.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Warning: In engine mode, this message will not recalculate the spreadsheet, on the assumption that; the caller may want to delete several different ranges before doing a recalculation.

Library: **ssheet.def**

MSG_SPREADSHEET_MAKE_FOCUS

Make the spreadsheet the current focus object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **ssheet.def**

MSG_SPREADSHEET_MOVE_ACTIVE_CELL

This message instructs the spreadsheet to change the active cell. Applications should not usually need to send this message; they can rely on the spreadsheet's user interface for this.

Pass: **bp**, **cx** Cell to move to (row, column).

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **ssheet.def**

MSG_SPREADSHEET_NAME_UPDATE_NAME

Given the optr of a text object in which the name string resides and a list entry, this message extracts the text from the text object and makes it the moniker of the list entry.

Pass: ^l**cx**:**dx** Text object whose text is the name string.
bp List entry number.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **ssheet.def**

MSG_SPREADSHEET_NOTES_ENUM

This message calls a callback routine for each note in the spreadsheet.

The caller must be run in the same thread as the spreadsheet object.

Pass: **cx**:**dx** Callback routine.
ss:**bp** Parameters to pass to the callback.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Callback Routine Specifications:

Passed:	**ax**, **cx**	Row, column of cell.
	ss:**bp**	Stack frame passed to MSG_SPREADSHEET_NOTES_ENUM.
	***es**:**di**	Pointer to notes text.
	^l**dx**:**si**	Spreadsheet object.
Return:	CF	Set to abort enumeration; clear to continue.
May Destroy:	Nothing.	

Library: **ssheet.def**

MSG_SPREADSHEET_PARSE_EXPRESSION

This message parses an expression. All names appearing in the expression must be recognized; do not introduce any new ones.

Pass:	**ss:bp**	**SpreadsheetParserParameters** structure with the following fields filled in: *SPP_text*, *SPP_expression*, *SPP_exprLength*, and *SPP_parserParams.PP_flags*.
	dx	sizeof **SpreadsheetParserParameters**.
Returns:	**al**	On success, -1 and the **SpreadsheetParserParameters** structure's *SPP_expression* field will have been filled; on error, **ParserScannerEvaluatorError** value.
Destroyed:	**ah, cx, dx.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_PARSE_RANGE_REFERENCE

Parse a cell reference into a range, if possible.

Pass:	**ss:bp**	**SpreadsheetFormatParseRangeParams** structure, with the *SFPRP_text* field containing the string describing the range reference.
Returns:	CF	Set on error; clear on success, and *SFPRP_range* will be filled with the appropriate **CellRange** value.
Destroyed:	**ax, cx, dx.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_READ_CACHED_DATA

This message instructs a spreadsheet object to read its cached data from the specified file and block. The handles passed supersede the handles in the Spreadsheet object's instance data. Applications which use the Document Control should send this message in their handlers for MSG_GEN_DOCUMENT_READ_CACHED_DATA_FROM_FILE.

Pass:	**dx**	VM file handle.
	cx	VM handle of spreadsheet map block.
Returns:	Nothing.	
Destroyed:	**ax, cx, dx, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_RECALC

This message instructs the spreadsheet to recalculate the values of all of its cells. This message is needed if the spreadsheet has had automatic recalculation disabled; otherwise, the spreadsheet will automatically recalculate cells as needed.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: ssheet.def

MSG_SPREADSHEET_SET_CELL_BORDER_COLOR

This message sets the color for borders for the current selection.

Pass:

ch	**ColorFlag** field.
cl	Red color value (or color index).
dl	Green color value.
dh	Blue color value.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: ssheet.def

MSG_SPREADSHEET_SET_CELL_BORDER_GRAY_SCREEN

This message sets the draw mask for borders for the current selection.

Pass: cl **SysDrawMask** value.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: ssheet.def

MSG_SPREADSHEET_SET_CELL_BORDER_PATTERN

This message sets the graphics pattern for borders for the current selection.

Pass: cx **GraphicPattern** value.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: ssheet.def

MSG_SPREADSHEET_SET_CELL_BORDERS

This message specifies which of the selection's borders should be drawn.

Pass:	`cl`	**CellBorderInfo** field.
Returns:	Nothing.	
Destroyed:	Nothing.	
Library:	**ssheet.def**	

MSG_SPREADSHEET_SET_COLUMN_WIDTH

This message sets the column width of one column or all selected columns.

Pass:	`cx`	Width of columns. You may bitwise-OR this value with COLUMN_WIDTH_BEST_FIT to try for the best fit.
	`dx`	Column number or SPREADSHEET_ADDRESS_USE_SELECTION.
Returns:	Nothing.	
Destroyed:	`ax, cx, dx, bp.`	
Library:	**ssheet.def**	

MSG_SPREADSHEET_SET_FOOTER_RANGE

Set the range that is the footer.

Pass:	`cx`	Zero to use the current selection; non-zero to remove the header.
Returns:	Nothing.	
Destroyed:	`ax, cx, dx, bp.`	
Library:	**ssheet.def**	

MSG_SPREADSHEET_SET_HEADER_RANGE

Set the range that is the header.

Pass:	`cx`	Zero to use the current selection; non-zero to remove the header.
Returns:	Nothing.	
Destroyed:	`ax, cx, dx, bp.`	
Library:	**ssheet.def**	

MSG_SPREADSHEET_SET_NOTE

This message adds or modifies a note for a given cell.

Pass:	^h**cx**	Handle of the block containing the null-terminated text of the note, or zero if there is to be no note text.
	dx, **bp**	Row, column of cell to which the note belongs.
Returns:	Nothing.	
Destroyed:	**ax, cx, dx, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_SET_NOTE_FOR_ACTIVE_CELL

This message adds or modifies a note for the active cell.

Pass:	^h**cx**	Handle of the block containing the null-terminated text of the note, or zero if there is to be no note text. The block will be freed before the message returns.
	dx, **bp**	Row, column of cell to which the note belongs.
Returns:	Nothing.	
Destroyed:	**ax, cx, dx, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_SET_NUM_FORMAT

Set the number format for the current selection.

Pass:	**cx**	**FormatIDType**
Returns:	Nothing.	
Destroyed:	**ax, cx, dx, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_SET_ROW_HEIGHT

Set the height of a row or of all selected rows.

Pass:	**cx**	Height of rows; bitwise-OR with ROW_HEIGHT_AUTOMATIC for automatic height.
	dx	Row number or SPREADSHEET_ADDRESS_USE_SELECTION
Returns:	Nothing.	
Destroyed:	Nothing.	
Library:	**ssheet.def**	

MSG_SPREADSHEET_SET_SELECTION

This message sets the selected range for the spreadsheet.

Pass:	**ss:bp**	**SpreadsheetRangeParams** structure.
Returns:	Nothing.	
Destroyed:	**ax, cx, dx, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_SORT_RANGE

This message sorts a range.

Pass:	**cl**	**RangeSortFlags** field.
Returns:	**cx**	Zero if we sorted successfully; **RangeSortError** otherwise.
Destroyed:	Nothing.	
Library:	**ssheet.def**	

MSG_SPREADSHEET_VALIDATE_NAME

This message ensures that a given name can be added to the spreadsheet name list.

Pass:	**dx:bp**	**SpreadsheetNameParameters** structure with following fields filled in: *SNP_textLength* and *SNP_text*. This structure is on the stack, with **dx** = **ss**.
Returns:	**cx**	If there was an error, -1; otherwise, some other value.
	dl	If there was an error, this will be a **ParserScannerEvaluatorError**.
Destroyed:	**ax, dh, bp.**	
Library:	**ssheet.def**	

MSG_SPREADSHEET_WRITE_CACHED_DATA

This message instructs the spreadsheet object to write any cached data to the spreadsheet file. The file handle passed supersedes any file handle in the spreadsheet's instance data. Applications which use the Document Control

should send this message in their handlers for MSG_GEN_DOCUMENT_READ_CACHED_DATA_FROM_FILE.

Pass: **dx** VM file handle.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **ssheet.def**

SpreadsheetRulerClass

Instance Data

```
SRI_flags               SpreadsheetRulerFlags
   UIC Syntax:  flags     = bytecomp : 0;
```

flags (*SRI_flags*) determines some of the ruler's behavior. Only one of these flags is not internal: SRF_NO_INTERACTIVE_RESIZE, which specifies that the ruler doesn't allow interactive resizing of columns or rows. If you wish your ruler to support this feature, then you must write the support for it yourself.

To set this flag in your **.ui** file, use a line like the following:

flags = "mask SRF_NO_INTERACTIVE_RESIZE";

```
SRI_spreadsheet         optr
SRI_resizeRC            word
SRI_startRCPos          dword
```

These fields are internal.

■ MSG_SPREADSHEET_RULER_DRAW_RANGE

This message draws a range of the spreadsheet ruler.

Pass: **ss:bp** **SpreadsheetDrawParams** structure.

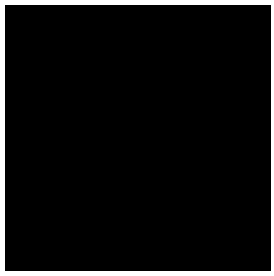
Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **ssheet.def**

Object Assembly Reference

Pen Library



21

InkClass 591

InkControlClass 595

InkClass

Instance Data

```
II_flags        InkFlags
   UIC Syntax:  flags      = bitFieldComp word {
                        reserved1,
                        reserved2,
                        reserved3,
                        reserved4,
                        dirty,
                        onlyChildOfContent,
                        controlled,
                        invalErasures,
                        hasUndo
                     } : hasUndo;
```

flags (*II_flags*) holds flags governing the object's behavior.

```
II_tool         InkTool
   UIC Syntax:  tool       = wordComp: 0;
```

tool (*II_tool*) keeps track of how the user is interacting with the Ink. There are three possible tools: IT_PENCIL, IT_SELECTOR, and IT_ERASER.

```
II_penColor           Color
   UIC Syntax:  color      = byteComp: 0;
```

color (*II_penColor*) specifies the color to use when drawing ink.

```
II_segments           lptr
   UIC Syntax:  segments   = nptrComp : null;
```

(Do not set this field explicitly.) *segments* (*II_segments*) is a handle to the chunk array containing the pen segments. The segments are stored as an array of **Point** structures. The stored coordinates are all positive; any *x* coordinate with its sign bit set belongs to the last point in a gesture.

```
II_dirtyOutput        optr

II_dirtyMsg           word
   UIC Syntax:  dirtyAD    = actionComp : null;
```

Together, *II_dirtyOutput* and *II_dirtyMsg* form an Action Descriptor (*dirtyAD*). When the Ink processes a point of pen information, erases

anything, or handles an undo event the IF_DIRTY flag will be set. If the flag was not set already, then the Ink will send the AD's message to the AD's object. The handler for this message should probably clear the IF_DIRTY bit.

```
II_selectBounds      Rectangle <>
II_cachedGState      hptr.GState
II_antTimer          hptr
II_antTimerID        word
II_antMask           byte
```

These fields are internal to the Pen library.

```
ATTR_INK_STROKE_SIZE vardata InkStrokeSize
```

ATTR_INK_STROKE_SIZE sets the stroke size for the ink object.

```
ATTR_INK_MAX_POINTS  vardata word
```

ATTR_INK_MAX_POINTS is the maximum number of points of ink data the object is prepared to store.

Messages

MSG_INK_DELETE_LAST_GESTURE

Removes the last gesture in the ink; if there is no ink, then nothing happens.

Pass: Nothing.

Returns: **ax** Zero if no ink deleted.

Destroyed: **cx, dx, bp.**

Library: **pen.def**

MSG_INK_GET_FLAGS

This message gets the value of the *II_flags* field.

Pass: Nothing.

Returns: cx **InkFlags** value with current flags.

Destroyed: **Nothing.**

Library: **pen.def**

Object Assembly Reference

MSG_INK_GET_TOOL

This message returns the Ink's present tool, as stored in *II_tool*.

Pass: Nothing.

Returns: **cx** InkTool value representing current tool.

Destroyed: **Nothing.**

Library: **pen.def**

MSG_INK_LOAD_FROM_DB_ITEM

This message loads the compressed data into the Ink from the passed DB item. If a NULL handle is passed, then the Ink is cleared. This message marks the Ink as clean.

Pass: **ss:bp** (*ptr*) **InkDBFrame** structure.

Returns: Nothing.

Destroyed: **Nothing.**

Library: **pen.def**

MSG_INK_SAVE_TO_DB_ITEM

This message saves the Ink's pen data into the passed DB item. The pen data will be stored compressed. Calling this message sets the object not dirty.

Pass: **ss:bp** **InkDBFrame** structure

Returns: ax:bp DB group and item written to (within the passed **VMChain**).

Destroyed: **Nothing.**

Library: **pen.def**

MSG_INK_SET_DIRTY_AD

Sets the action descriptor to be activated when the user dirties the object.

Pass: **bp** (*method*) Message.
cx:**dx** (*object*) The optr of the object.

Returns: Nothing.

Destroyed: **Nothing.**

Library: **pen.def**

■ MSG_INK_SET_FLAGS

This message changes the value of the *II_flags* field. Note that something which sets the IF_DIRTY bit should probably also perform the action stored in the *II_dirtyMsg* and *II_dirtyOutput* fields.

Pass: `cx` Flags to set.
`dx` Flags to clear.

Returns: Nothing.

Destroyed: `Nothing.`

Library: `pen.def`

■ MSG_INK_SET_MAX_POINTS

Set the maximum number of points of ink data the object is prepared to store.

Pass: `cx` Maximum number of points.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: `pen.def`

■ MSG_INK_SET_PEN_COLOR

This message changes the color used to draw the ink, changing the value in *II_penColor*.

Pass: `cl` (*clr*) **Color** value.

Returns: Nothing.

Destroyed: `Nothing.`

Library: `pen.def`

■ MSG_INK_SET_STROKE_SIZE

Set the brush size to use when drawing ink.

Pass: `cl` (*width*) Stroke width in pixels.
`ch` (*height*) Stroke height in pixels.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: `pen.def`

Object Assembly Reference

MSG_INK_SET_TOOL

This message allows the Ink to switch between pencil and eraser tools, changing the *II_tool* field.

Pass: **cx** (*tool*) InkTool value representing tool to use.

Returns: Nothing.

Destroyed: **Nothing.**

Library: **pen.def**

InkControlClass

Features

```
InkControlFeatures record
                  ICF_PENCIL_TOOL:1
                  ICF_ERASER_TOOL:1
                  ICF_SELECTION_TOOL:1

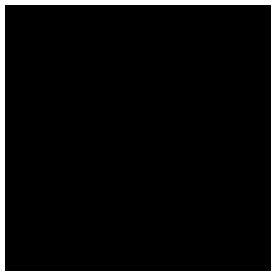
InkControlFeatures      end

InkControlToolboxFeatures       record
                  ICTF_PENCIL_TOOL:1
                  ICTF_ERASER_TOOL:1
                  ICTF_SELECTION_TOOL:1

InkControlToolboxFeatures end
```

Object Assembly Reference

Config Library



22

PrefClass 599

PrefValueClass 603

PrefItemGroupClass 604

PrefStringItemClass 606

PrefBooleanGroupClass 607

PrefDynamicListClass 608

TitledGlyphClass 609

PrefInteractionClass 609

PrefDialogClass 610

PrefTextClass 610

PrefControlClass 611

PrefTimeDateControlClass 611

PrefTriggerClass 611

PrefTocListClass 612

PrefClass

Instance Data

```
PI_attrs        PrefAttributes
   UIC Syntax:  prefAttrs = bitFieldComp byte {
                    rebootIfChanged,
                    loadIfUsable,
                    saveIfUsable,
                    saveIfEnabled,
                    saveIfChanged
                    } : loadIfUsable, saveIfUsable;
```

prefAttrs (*PI_attrs*) contains the following flags used to customize the behavior of Pref objects:

PA_REBOOT_IF_CHANGED
: This bit signals that changes in the state of this object require a system reboot to take effect.

PA_LOAD_IF_USABLE
: Load options only if this object is usable (this is on by default).

PA_SAVE_IF_USABLE
: Save options only if this object is usable (this is on by default).

PA_SAVE_IF_ENABLED
: Save options only if this object is enabled.

PA_SAVE_IF_CHANGED
: Save options only if this object has changed.

```
ATTR_PREF_REBOOT_STRING     vardata     optr
```

This is the string passed to MSG_PREF_MGR_DO_REBOOT_DIALOG— the string must be in a shareable LMem resource and is incorporated into the middle of another sentence. In English, the sentence is “The system software will shut down and restart to change the *reboot_string*. Do you wish to proceed?”

If an object has its PA_REBOOT_IF_CHANGED attribute set, it will scan for this attribute, first in itself, then up the generic tree, until it finds one.

ATTR_PREF_REQUIRED_FEATURES vardata PrefMgrFeatures

These are features bits that must be set for this object to be usable.

ATTR_PREF_PROHIBITED_FEATURES vardata PrefMgrFeatures

These are features bits that must not be set for this object to be usable.

ATTR_PREF_MIN_LEVEL vardata UIInterfaceLevel

This is the interface level below which this object will not be usable.

ATTR_PREF_MAX_LEVEL vardata UIInterfaceLevel

This is the interface level above which this object will not be usable.

Messages

MSG_PREF_GET_REBOOT_INFO

This message asks whether the system will need to reboot as a result of changes. If a reboot is necessary, the dialog will then notify the user of the reboot, and give the user a chance to abort (since the system reset may be a lengthy process on some systems).

Pass: **Nothing.**

Returns: ^l**cx**:**dx** OD of string to put up in ConfirmDialog if a reboot needed; if no reboot is needed, then **cx** will be zero.

Destroyed: **ax, bp**

Library: **config.def**

MSG_PREF_GET_REBOOT_STRING

This message should return the optr of a string in a sharable local memory resource. This string will display in the dialog box which asks the user if it is all right to shut down. If a given object doesn't know of a good string to supply, it should forward the message on to its parent in the generic tree. The default **PrefClass** handler returns the value stored in ATTR_PREF_REBOOT_STRING.

Pass: **Nothing.**

Returns: **cx**:**dx** Optr of string (cx = 0 if not found)

Destroyed: **ax**

Library: **config.def**

MSG_PREF_HAS_STATE_CHANGED

This message determines whether the object or any of its children have changed since the last time options were loaded. This is used in conjunction with the PA_REBOOT_IF_CHANGED flag that can be set for an object—this flag signals that, if the object changes state, then GEOS should be restarted for the change to go into effect.

Pass: `Nothing`

Returns: CF Set if something has changed.

Destroyed: `ax, cx, dx, bp.`

Library: `config.def`

MSG_PREF_INIT

This message initializes the object. The object should perform any required set-up to appear on-screen. The PrefDialog object will send this object to all of its children when the dialog is first initiated.

Default behavior is to determine whether to make the object usable based on UI level and features (or will be, when implemented). Subclass should call superclass before handling, as subclass behavior may be different if the object is not usable.

Pass: `cx` (*features*) This **PrefMgrFeatures** tells you a bit what sorts of changes the user prepared to make.

`dx` (*level*) A **UIInterfaceLevel** representing the user's sophistication.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: `config.def`

MSG_PREF_NOTIFY_DIALOG_CHANGE

This message goes out via PDGCNLT_DIALOG_CHANGE list bound to PrefDialog object telling interested parties of a change in the box's state. It is sent when the dialog box is being opened, closed, or destroyed.

This message has no default handler; it must be intercepted to do anything useful.

Pass: `cx` (*type*) **PrefDialogChangeType** value containing new state of the dialog.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: `config.def`

MSG_PREF_SET_INIT_FILE_CATEGORY

Pref objects automatically respond to this message by changing the value of the ATTR_GEN_INIT_FILE_CATEGORY variable data field. This is a utility message; similar functionality is available via MSG_META_ADD_VAR_DATA.

Pass: `cx:dx` (category) Far pointer to a null-terminatedstring containing the category name. (This is a DBCS string in DBCS.)

Returns: Nothing.

Destroyed: `ax`

Library: `config.def`

MSG_PREF_SET_ORIGINAL_STATE

This message copies the object's current state to the "original" value field, which will be used on a MSG_GEN_RESET.

Subclasses should respond to this message by copying their current value to whatever instance field represents the "original" state of the object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `Nothing.`

Library: `config.def`

Object Assembly Reference

PrefValueClass

Instance Data

```
PVI_originalValue           word
   UIC Syntax:  originalValue = wordComp : 0;
```

originalValue (*PVI_originalValue*) In case the user wants to revert the value, we store the original value here.

```
ATTR_PREF_VALUE_WRAP        vardata     word
```

If the value is incremented beyond its top value, it will "wrap around" to its bottom value, and vice versa.

Messages

MSG_PREF_VALUE_SET_ORGINAL_VALUE

This message sets the "original" value of the object—it also sets the value stored by the superclass.

Pass: **cx** Original value.
bp Non-zero if indeterminate.

Returns: Nothing.

Destroyed: **ax.**

Library: **config.def**

PrefItemGroupClass

Instance Data

```
PIGI_initFileFlags          PrefInitFileFlags
   UIC Syntax:  initFileFlags = bitFieldComp byte {
                    useItemStrings,
                    useItemMonikers,
                    appendToKey,
                    absentKeyOverridesDefaults,
                    suspendOnLoadOptions
                    } : {};
```

initFileFlags (*PIGI_initFileFlags*): These flags fine-tune the object's treatment of the .INI file.

```
PIGI_originalSelection      word
   UIC Syntax:  originalSelection = wordComp : 0;
```

originalSelection (*PIGI_originalSelection*): In case the user wants to revert, this piece of instance data keeps track of the original selection.

```
PIGI_originalNumSelections word
   UIC Syntax:  originalNumSelections = wordComp : 0;
```

originalNumSelections (*PIGI_originalNumSelections*):In case the user wants to revert, this instance data field keeps track of the original number of selections.

```
PIGI_suspendCount           byte
   UIC Syntax:  suspendCount = byteComp : 0;
```

suspendCount (*PIGI_suspendCount*): When the suspend count is nonzero, the item group won't update text objects or enable/disable objects each time the selection changes.

Messages

MSG_PREF_ITEM_GROUP_GET_ITEM_MONIKER

This message retrieves the text in the moniker of one of the items of a PrefItemGroup. This works for dynamic list subclasses as well. The message returns the number of characters in the returned text.

Pass:	`ss:bp`	**GetItemMonikerParams** structure.
Returns:	`bp`	Number of characters returned.
Destroyed:	`Nothing.`	
Library:	`config.def`	

MSG_PREF_ITEM_GROUP_GET_SELECTED_ITEM_TEXT

This message returns the text in the moniker of the selected item. The value returned is the number of characters in the string. If nothing is selected, the value returned is zero.

Pass:	`cx:dx`	(*buffer*) Buffer to hold text.
	`bp`	(*bufsize*) Number of characters in buffer.
Returns:	`bp`	Zero if no item selected; otherwise, number of characters written to buffer.
Destroyed:	`ax`	
Library:	`config.def`	

MSG_PREF_ITEM_GROUP_SET_ORIGINAL_SELECTION

Set the "originalSelection" field of the item group. Also, sends MSG_GEN_ITEM_GROUP_SET_SINGLE_SELECTION to superclass. This message can only be used with exclusive item groups—for other types, use the GenItemGroup messages, and then send the object MSG_PREF_SET_ORIGINAL_STATE.

Pass:	`cx`	(*selection*) Which item to select.
Returns:	Nothing.	
Destroyed:	`ax, cx, dx, bp.`	
Library:	`config.def`	

MSG_PREF_ITEM_GROUP_UPDATE_TEXT

This message updates the specified text object with text of the currently selected item. The PrefItemGroup sends this to itself in conjunction with either ATTR_PREF_ITEM_GROUP_TEXT_DISPLAY or ATTR_PREF_ITEM_GROUP_STATUS_TEXT. By default, this message will also cause the text object to send out its status message.

Pass: **cx** (*textObject*) ChunkHandle of the text object to update.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **config.def**

PrefStringItemClass

Instance Data

```
PSII_initFileString         lptr
   UIC Syntax:  initFileString = nptrComp : null;
```

initFileString (*PSII_initFileString*) is the string associated with this item in the .INI file.

Messages

MSG_PREF_STRING_ITEM_CHECK_IF_IN_INIT_FILE_KEY

This message checks the passed string to see if the string bound to this item is in one of its pieces. A PrefItemGroup will send this object when loading options if it has the PIFF_USE_ITEM_STRINGS flag set.

May be intercepted if the subclass needs to check other things to decide whether it should be selected. If you also wish to have the default behavior,

you may call the superclass either before or after you've made your own decision, as appropriate.

Pass: `ss:bp` (*vars*) **PrefItemGroupStringVars** value.

Returns: `CF` Set if bound string is within the passed string; clear otherwise.

Destroyed: `Nothing.`

Library: **config.def**

MSG_PREF_STRING_ITEM_SET_INIT_FILE_STRING

This message sets the string that is read and written in the .INI file.

Interception is unlikely but possible. Default behavior sets the string and then checks again with the .INI file to make sure item should be selected.

Pass: `cx:dx` (*str*) Far pointer to null-terminated ASCII string.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **config.def**

PrefBooleanGroupClass

Instance Data

```
PBGI_initFileFlags          PrefInitFileFlags
   UIC Syntax:  initFileFlags = bitFieldComp byte {
                    useItemStrings, useItemMonikers,
                    dontCreateCategory
                    } : {};
```

initFileFlags (*PBGI_initFileFlags*) contains flags which fine-tune the way the object will work with the .INI file.

```
PBGI_originalState          word
   UIC Syntax:  originalState = wordComp : 0;
```

originalState (*PBGI_originalState*) Original state of the user's selection, useful if the user wants to revert.

Messages

MSG_PREF_BOOLEAN_GROUP_SET_ORIGINAL_STATE

This message sets the "original" state of the Boolean group.

Pass:	`cx`	(*state*) Flags to use as the original state.
Returns:	Nothing.	
Destroyed:	`Nothing.`	
Library:	`config.def`	

PrefDynamicListClass

Messages

MSG_PREF_DYNAMIC_LIST_BUILD_ARRAY

The PrefDynamicList sends itself this message in response to MSG_PREF_INIT.

Pass:	Nothing.
Returns:	Nothing.
Destroyed:	`Nothing.`
Library:	`config.def`

MSG_PREF_DYNAMIC_LIST_FIND_ITEM

This message finds an item given a moniker. It is normally used in the context of loading options.

Pass:	**cx**:**dx**	(str) String to search for.
	bp	(ignoreCase) Nonzero to find best fit (ignore case and match up to the length of the passed string); zero to find an exact match.
Returns:	CF	Clear if item found; set otherwise.
	ax	Item number if item found; otherwise first item following requested item.
Destroyed:	**Nothing.**	
Library:	**config.def**	

TitledGlyphClass

No instance data fields or messages have been defined for this class.

PrefInteractionClass

Instance Data

```
PII_attrs       PrefInteractionAttrs
   UIC Syntax:  prefInteractionAttrs = bitFieldComp byte {
                    loadOptionsOnInitiate, saveOptionsOnApply
                    } : {};
```

prefInteractionAttrs (*PII_attrs*): These flags determine how and when the PrefInteraction loads and saves its options.

Messages

No messages have been defined for this class.

PrefDialogClass

Instance Data

```
ATTR_PREF_DIALOG_SYS_SHUTDOWN_TYPE      vardata word
```

Specifies a SysShutdownType to use instead of SST_RESTART.

Messages

MSG_PREF_DIALOG_CONFIRM_REBOOT

By default, the PrefDialog will respond to this message by putting up a dialog box asking the user if he wants to restart the system.

Pass: ^l**cx**:**dx** (*string*) The optr of string which will be inserted in a sentence of the form "The system software will shut down and restart to change the *string*. Do you wish to proceed?"

Returns: CF Set to confirm; clear otherwise.

Destroyed: **ax, cx, dx, bp.**

Library: **config.def**

MSG_PREF_DIALOG_REBOOT

Dismiss the interaction and cause GEOS to restart.

Pass: Nothing.

Returns: Doesn't.

Destroyed: **Not applicable.**

Library: **config.def**

PrefTextClass

Instance Data

```
PTI_originalText      lptr.char
   UIC Syntax:  originalText = nptrComp : null;
```

originalText (*PTI_originalText*) contains the text which should replace the object's text if the user wants to revert.

Messages

No messages have been defined for this class.

PrefControlClass

Instance Data

No instance data has been defined for this class.

Messages

No messages have been defined for this class.

PrefTimeDateControlClass

This class' instance data and messages are internal.

PrefTriggerClass

Instance Data

`ATTR_PREF_TRIGGER_ACTION  vardata  PrefTriggerAction`

Specifies an additional action to be performed. There may be more than one of these— they'll be processed in order.

Messages

No messages have been defined for this class.

PrefTocListClass

Instance Data

```
PTLI_flags      TocUpdateCategoryFlags
   UIC Syntax:  tocFlags = bitFieldComp word {
                   extendedDeviceDrivers
                   } : {};
```

tocFlags (*PTLI_flags*) Flags customizing the list's behavior.

```
PTLI_tocInfo    TocCategoryStruct
   UIC Syntax:  tokenChars = typeComp "TokenChars" : "<>";
                   static dwordComp : 0;            /* TCS_files */
                   static dwordComp : 0;            /* TCS_devices */
```

PTLI_tocInfo determines what sorts of drivers the list has been set up to work with.

```
ATTR_PREF_TOC_LIST_EXTRA_ENTRY_1        vardata PrefTocExtraEntry
```

Information used to add extra item to list.

```
ATTR_PREF_TOC_LIST_EXTRA_ENTRY_2        vardata PrefTocExtraEntry
```

Information used to add yet another extra item to list.

```
ATTR_PREF_TOC_LIST_INFO_KEY       vardata char
```

This key is used to write the "info" word for device lists. This string is always SBCS, even under DBCS systems.

Messages

MSG_PREF_TOC_LIST_CHECK_DEVICE_AVAILABLE

This message determines if the device selected by the list is available in the machine. This message may only be called for PrefTocList objects that have the TCF_EXTENDED_DEVICE_DRIVERS flag set.

Pass: Nothing.

Returns:

CF	Set if device available; clear otherwise.
ax	If device is available and is a video device , this is the DisplayType; if device is available and is not a video device, then this is undefined; if device is unavailable, this will be

zero of the device is not present and the GeodeLoadError plus one if the library couldn't load the driver.

Destroyed: cx, dx.

Library: config.def

MSG_PREF_TOC_LIST_GET_SELECTED_DRIVER_NAME

This message returns the name of the driver for the selected device. This message may only be used if TCF_EXTENDED_DEVICE_DRIVERS is set.

Pass:	cx:dx	(*buf*) Pointer to buffer to fill with the driver name.
	bp	(*bufSize*) Size of buffer.
Returns:	bp	Zero if buffer was not big enough to hold driver name; number of characters returned otherwise.

Destroyed: ax

Library: config.def

MSG_PREF_TOC_LIST_GET_SELECTED_ITEM_INFO

This message returns the info word about the currently selected item. This message can only be used with PrefTocList objects that have the TCF_EXTENDED_DEVICE_DRIVERS flag set.

Pass:	Nothing.	
Returns:	CF	Clear if found; set otherwise.
	ax	(*infoPtr*) Information word for selected item.

Destroyed: Nothing.

Library: config.def

MSG_PREF_TOC_LIST_GET_SELECTED_ITEM_PATH

This message returns the absolute path of the selected driver or file.

Pass:	Nothing.	
Returns:	cx:dx	Pointer to string containing full path.
	ax	Handle of block holding path (locked).
	bp	Disk handle.

Destroyed: Nothing.

Library: config.def

MSG_PREF_TOC_LIST_SET_TOKEN_CHARS

Set the PTLI_tocInfo.TCS_tokenChars field.

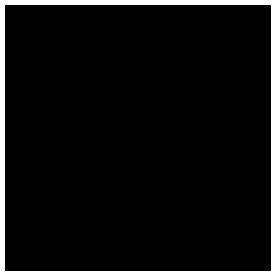
Pass: `ss:bp` Pointer to **TokenChars** structure.

Returns: Nothing.

Destroyed: `Nothing.`

Library: **config.def**

Vis



23

VisClass 617

VisClass

Instance Data

```
VI_bounds       Rectangle <0, 0, 0, 0>
   UIC Syntax:      left   = wordComp : 0;
                    top    = wordComp : 0;
                    right  = wordComp : 0;
                    bottom = wordComp : 0;
```

left, *top*, *right*, and *bottom* (*VI_bounds*) form the bounds of visible object. They can be set vis MSG_VIS_SET_POSITION and MSG_VIS_SET_SIZE. Bounds can be obtained via a MSG_VIS_GET_BOUNDS

```
VI_typeFlags    VisTypeFlags    0
   UIC Syntax:      typeFlags = bitFieldComp byte {
                    composite,
                    window,
                    portal,
                    winGroup,
                    content,
                    inputNode } : {};
```

typeFlags (*VI_typeFlags*) contains visible type flagsspecifying whether the object has a window, is a composite, win group, content object, etc. Set via MSG_VIS_SET_TYPE_FLAGS, retrieved via MSG_VIS_GET_TYPE_FLAGS.

```
VI_attrs        VisAttrs (
                    mask VA_MANAGED or \
                    mask VA_DRAWABLE or \
                    mask VA_DETERACTABLE or \
                    mask VA_FULLY_ENABLED)
   UIC Syntax:  attr = bitFieldComp byte { visible,
                    fullyEnabled,
                    managed,
                    drawable,
                    detectable,
                    branchNotMinimizable,
                    notUsed2,
                    realized
                    } : managed, drawable, detectable, fullyEnabled;
```

attr (*VI_attrs*) contains various state flags for the visible object for whether it's managed, drawable, enabled, visible, detectable, realized, etc. Set via MSG_VIS_SET_ATTRS; obtained via MSG_VIS_GET_ATTRS.

```
VI_optFlags    VisOptFlags (
                  mask VOF_GEOMETRY_INVALID or \
                  mask VOF_GEO_UPDATE_PATH or \
                  mask VOF_IMAGE_INVALID or \
                  mask VOF_WINDOW_INVALID or \
                  mask VOF_WINDOW_UPDATE_PATH or \
                  mask VOF_IMAGE_INVALID or \
                  mask VOF_IMAGE_UPDATE_PATH)
   UIC Syntax:  optFlags = bitFieldComp byte {
                  geometryInvalid,
                  updatePath,
                  imageInvalid,
                  imageUpdatePath,
                  windowInvalid,
                  windowUpdatePath,
                  updatePending,
                  inComposite
                  } : geometryInvalid, updatePath,
                  windowInvalid,windowUpdatePath, imageInvalid,
                  imageUpdatePath;
```

optFlags (*VI_optFlags*) contains various flags for keeping track of the invalid, drawn, realized, or geometry state of the object. Cannot be set directly; one must use MSG_VIS_MARK_INVALID to set an object invalid in some way.

```
VI_geoAttrs    VisGeoAttrs 0
   UIC Syntax:  visGeoAttrs = bitFieldComp byte {
                  geometryCalculated,
                  noSizeHints,
                  notifyGeometryValid,
                  dontCenter,
                  useVisSetPosition,
                  useVisCenter,
                  onlyRecalcSizeWhenInvalid,
                  alwaysRecalcSize} : {};
```

visGeoAttrs (*VI_geoAttrs*) maintains vis level geometry manager flags, keeping track of various optimizations to speed up geometry handling. It can be set via MSG_VIS_SET_GEO_ATTRS; obtained via MSG_VIS_GET_GEO_ATTRS.

Object Assembly Reference

```
VI_specAttrs   SpecAttrs 0
   UIC Syntax:  visSpecAttr = bitFieldComp byte {
                   attached,
                   realizable,
                   branchMinimized,
                   usesDualBuild,
                   customVisParent,
                   simpleGenObj } : {};
```

visSpecAttr (*VI_specAttr*) contains specific UI attributes, for generic objects only. It can be set via MSG_SPEC_SET_ATTRS; obtained via MSG_SPEC_GET_ATTRS.

```
VI_link        LinkPart <>
   UIC Syntax:  visLink = linkComp;
```

visLink (*VI_link*) holds the visual link to its next sibling. This field is not accessible directly; use MSG_VIS_ADD_CHILD, MSG_VIS_MOVE_CHILD, MSG_VIS_REMOVE_CHILD to manipulate an object's place in the visual tree. Use MSG_VIS_GET_PARENT to return the visual parent of the object.

```
ATTR_VIS_GEOMETRY_DATA     vardata VarGeoData
```

This attribute is added and removed by the geometry manager as needed. Keeps track of some internal variables for some of the more complex features in the geometry manager.

Messages

■ MSG_VIS_ADD_BUTTON_POST_PASSIVE

A visible object sends itself this message (with MF_CALL) when it wants to gain the post-passive mouse grab. The object will receive all subsequent mouse events in the form of MSG_META_POST_PASSIVE_- after the event's destination object receives them; this message will carry the same input information as passed with the input event. The object should handle the appropriate MSG_META_POST_PASSIVE_- to properly receive the events. The post-passive grab will be released when the object receives a MSG_VIS_REMOVE_BUTTON_POST_PASSIVE.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_ADD_BUTTON_PRE_PASSIVE

A visible object sends itself this message (with MF_CALL) when it wants to gain a pre-passive mouse grab. The object will receive all subsequent mouse events in the form of MSG_META_PRE_PASSIVE_- before the event's destination object receives them; this message will carry the same input information as passed with the input event. The object should handle MSG_META_PRE_PASSIVE_- and, if the event is not to be sent normally to its destination, should return the flag MRF_PREVENT_PASS_THROUGH from that handler. The pre-passive grab will be released when the object sends itself a MSG_VIS_REMOVE_BUTTON_PRE_PASSIVE.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_ADD_CHILD

This message attaches the passed object as a child of the composite handling the message. If the parent is already opened and on the screen, you must invalidate the child with MSG_VIS_MARK_INVALID, passing the invalidation VOF_WINDOW_INVALID; the composite and child will be updated appropriately according to the **VisUpdateMode** passed with the invalidation. If, however, the parent is not opened, the child will automatically be opened when the parent is opened.

Pass: ^l**cx:dx** Optr of child object to add
bp **CompChildFlags** describing how to add child.

Returns: **cx**, **dx** Preserved.

Destroyed: **ax, bp.**

Library: **Objects\visC.def**

MSG_VIS_ADD_NON_DISCARDABLE_VM_CHILD

This message performs exactly the same function as MSG_VIS_ADD_CHILD except that it also increments the object's in-use count; it will never be discarded.

This is used on top objects in visible sub-trees that are saved to document files: The object is added as a non-discardable file when the document is opened, and it is removed as a non-discardable child (with

MSG_VIS_REMOVE_NON_DISCARDABLE_VM_CHILD or MSG_VIS_REMOVE_NON_DISCARDABLE) when the document is closed.

Pass: ^l**cx**:**dx** Optr of child object to add.
bp **CompChildFlags** describing how to add child; CCF_MARK_DIRTY flag is ignored.

Returns: **cx**, **dx** Preserved.

Destroyed: **ax, bx.**

Library: **Objects\visC.def**

MSG_VIS_ADD_RECT_TO_UPDATE_REGION

This message adds the passed rectangular region to the window group's list of regions that require visual update. The handler for this message will ensure that the window group object for this branch of the tree marks the region as invalid so it will be included in the next visual update. .

Pass: **ss**:**bp** **VisAddRectParams** structure.
dx sizeof **VisAddRectParams**.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_BOUNDS_CHANGED

The geometry manager sends this message when a geometry update changes an object's bounds.

Source: Unrestricted—sent by the geometry manager when a visible object's bounds have changed as the result of a geometry change.

Pass: **ss**:**bp** **Rectangle** structure containing the *old* bounds.
dx sizeof **Rectangle**.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_CALL_PARENT

This message simply delivers a method to the visual parent of the object which receives it.

Pass:	^h**cx**	**ClassedEvent** to relay to parent.
Returns:	CF	Clear if no parent; otherwise, as returned by message handler.
Destroyed:	**ax, cx, dx, bp.**	
Library:	**Objects\visC.def**	

MSG_VIS_CALL_ROUTINE

Used to do a direct call to a far routine, but under the disguise of being a **VisClass** message handler—all registers are set up as if the far routine *were* a message handler for **VisClass**.

Pass:	**cx**	Data to pass in ax
	dx	Number of bytes on stack
	bp	Pointer to stack frame

Pass on stack:

Word value to pass in **cx**.
Word value to pass in **dx**.
Word value to pass in **bp**.
Segment, then offset of routine to call (which must be in locked or fixed memory).

Returns:	**ax**, **cx**, **dx**, **bp**	Whatever is returned by the routine.
Include:	**Objects\visC.def**	

MSG_VIS_CLOSE

This message is part of the visual update mechanism and is sent to objects that are being taken off the screen. The system will send this message to any objects in a visual branch whose VTF_IS_WIN_GROUP object is set not visible (~VA_VISIBLE). This message closes appropriate windows and propagates to all children to the bottom leaf objects of the visible branch. This message will not cross window group boundaries; if a lower VTF_IS_WIN_GROUP object is encountered during the message propagation, an error will likely occur.

Therefore, lower window groups must be closed before higher window groups may be closed.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\visC.def

MSG_VIS_CLOSE_WIN

This message is sent to a window or portal object that has its graphics window currently on the screen. The window and its visible branch will be closed (taken off the screen).

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\visC.def

MSG_VIS_COUNT_CHILDREN

This message returns the number of children the recipient object has. The count is not zero-based; if a composite has five children, this message will return five.

Pass: Nothing.

Returns: dx Number of children.

Destroyed: ax, cx, bp.

Library: Objects\visC.def

MSG_VIS_CREATE_CACHED_GSTATES

This message may be used to create and cache a GState, typically to avoid having a complex Vis object update several times for several simple operations (such as pointer events).

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\visC.def

MSG_VIS_CREATE_VIS_MONIKER

This message creates a new chunk for a visual moniker within the recipient's object block. The new moniker can be created from an already existing visual moniker, a visual moniker list, a text string, a GString, or a token from the token database. The source may be defined by a far pointer, a global memory handle, or an optr. If the source is a text string or GString, a visual moniker structure will be created for the string. The newly-created chunk is marked dirty if the CVMF_DIRTY flag is passed.

If a moniker list is passed, the entire list will be copied into the destination object block. You must make sure that all the monikers in the list will still exist when the list is used.

Pass: `ss:bp` **CreateVisMonkerFrame** structure.
`dx` Size of **CreateVisMonikerFrame**.

Returns: `ax` Chunk handle of new visual moniker.

Destroyed: `cx, dx, bp.`

Library: **Objects\visC.def**

MSG_VIS_DESTROY

This message is the high level way of destroying branches of visible trees.

This message will close and destroy the entire branch, unlinking any visible objects in the branch under the recipient object. The parent of the removed branch will be marked invalid for later visual update. This message may be subclassed to replace the geometry update handling, but this is not easily done and is rare.

Pass: `dl` **VisUpdateMode** value.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\visC.def**

MSG_VIS_DESTROY_CACHED_GSTATES

This message may be used to destroy any cached GStates the visible object may have

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Interception: Must be intercepted to have any effect; there is no default behavior.

Library: Objects\visC.def

MSG_VIS_DRAW

This message causes the visible object to draw itself. The default behavior of this message for composite objects is to pass the message on to all children. There is no default behavior for **VisClass** for this message.

Pass:

cl	**DrawFlags** giving some context information.
^hbp	Handle of GState to draw to. This GState will be the default GState, except that the pen position and color palette may have changed. For most objects, the default text font and size will be set to the system default; for specific UI objects, the font and size will be the UI default, and the GState's private data area will hold the specific UI's **DisplayScheme**. (To access the private data area, use **GrGetInfo()**).

Returns: GState must be returned unchanged, except that the pen position and colors may be destroyed.

Destroyed: ax, cx, dx, bp.

Library: Objects\visC.def

MSG_VIS_DRAW_MONIKER

This message draws a visual moniker for the object. This message may be called by an object on itself in its MSG_VIS_DRAW handler.

Pass:

ss:bp	**MonikerMessageParams** structure.
dx	sizeof **MonikerMessageParams**.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\visC.def

MSG_VIS_FIND_CHILD

This message returns the zero-based position of the specified child among the parent's children.

Pass:	^l**cx**:**dx**	Optr of child object.
Returns:	CF	Set of child was not found; clear otherwise.
	bp	Child position (Zero means it was the first child; -1 means object was not found).
	cx, **dx**	Preserved.
Destroyed:	**ax.**	
Library:	**Objects\visC.def**	

MSG_VIS_FIND_CHILD_AT_POSITION

This message returns the optr of the child occupying the position specified among the recipient's children.

Pass:	**cx**	Zero-based ordinal position of child object.
Returns:	CF	Set if child was not found; clear otherwise.
	^l**cx**:**d**x	Optr of child, or null if no child was at that position.
Destroyed:	**ax, bp.**	
Library:	**Objects\visC.def**	

MSG_VIS_FIND_MONIKER

This message locates the given visual moniker list and returns the optr of the moniker most appropriate for the passed display scheme.

Pass:	**ss**:**bp**	**FindMonikerParams** structure.
	dx	sizeof **FindMonikerParams** structure.
Returns:	^l**cx**:**dx**	VisMoniker (null if none).
Destroyed:	**ax, bp.**	
Library:	**Objects\visC.def**	

MSG_VIS_FIND_PARENT

This message returns the optr of the recipient's parent composite.

Pass: Nothing.

Returns: ^l**cx**:**dx** Parent object; null if none.

Destroyed: **ax, bp.**

Library: **Objects\visC.def**

MSG_VIS_FORCE_GRAB_LARGE_MOUSE

A visible object sends itself this message (with MF_CALL) when it wants to acquire the mouse grab and when it wants 32-bit coordinates. If the object wants 16-bit coordinates, it should use MSG_VIS_GRAB_MOUSE, above. The object must relinquish the mouse grab with MSG_VIS_RELEASE_MOUSE.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_FORCE_GRAB_MOUSE

A visible object sends itself this message (with MF_CALL) when it wants to forcibly grab the mouse input. The object will acquire the mouse grab even if another object currently has it; the other object will receive a MSG_VIS_LOST_GADGET_EXCL when it loses the mouse by a forced grab. The object must release the grab later with MSG_VIS_RELEASE_MOUSE.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**



MSG_VIS_FUP_QUERY_FOCUS_EXCL

This message queries the focus hierarchy to see which object has the current focus. The current focus object does not have to be in the caller's visible tree.

Pass: Nothing.

Returns:

^l**cx**:**dx**	Object with current focus.
bp	**HierarchicalGrabFlags** field.

Destroyed: **ax.**

Interception: This message is not deliverd to all visible objects, rather only those marked as VTF_WIN_GROUP or VTF_IS_INPUT_NODE. If you are implementing a new focus node, and need this message, and are not a window group, you will need to set the VTF_IS_INPUT_NODE bit in your init routine.

Library: **Objects\visC.def**

MSG_VIS_GET_ATTRS

This message returns a visible object's current *VI_attrs* field.

Pass: Nothing.

Returns:

cl	**VisAttrs** field

Destroyed: **ax, ch, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_GET_BOUNDS

This message returns the current rectangle structure stored in the object's *VI_bounds* field. This rectangle represents both the position and the size of the object by giving its left, top, right, and bottom bounds.

Pass: Nothing.

Returns:

ax	Left
bp	Top
cx	Right
dx	Bottom

Destroyed: **Nothing.**

Library: **Objects\visC.def**

MSG_VIS_GET_CENTER

This message returns the amount of space needed by the object in each direction from its center. It does not, as the name suggests, return the coordinates of the center of the object. Instead, the object must determine the

amount of space it requires from its center to each edge for geometry management and return them in the structure passed.

Pass: Nothing.

Returns:

cx	Minimum space needed to the left of center.
dx	Minimum space needed to the right of center.
ax	Minimum space needed above center.
bp	Minimum space needed below center.

Destroyed: **Nothing.**

Interception: Any object that wants to effect a different centering than the default should subclass this method. For example, if a set of objects can be overlapped, or if they should have extra space between them, they should subclass this message and alter the returned values appropriately.

Library: **Objects\visC.def**

MSG_VIS_GET_GEO_ATTRS

This message returns a visible object's current *VI_geoAttrs* field.

Pass: Nothing.

Returns:

cl	**VisGeoAttrs** field.

Destroyed: **ax, ch, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_GET_MONIKER_POS

This message returns the position at which the moniker would appear if it were drawn with MSG_VIS_DRAW_MONIKER. The moniker is not actually drawn by this message.

Pass:

ss:**bp**	**MonikerMessageParams** structure.
dx	sizeof **MonikerMessageParams**.

Returns:

cx, **dx**	*x* and *y* origin for moniker (zeroes if no moniker).

Destroyed: **ax, bp.**

Library: **Objects\visC.def**

MSG_VIS_GET_MONIKER_SIZE

This message returns the size of the moniker specified by the parameters. The moniker is not drawn.

Pass:	**ss**:**bp**	**MonikerMessageParams** structure.
	dx	sizeof **MonikerMessageParams**.
Returns:	**cx**, **dx**	x and y dimensions of moniker.
Destroyed:	**ax, bp.**	
Library:	**Objects\visC.def**	

MSG_VIS_GET_OPT_FLAGS

This message returns the current *VI_optFlags* field of the object.

Pass:	Nothing.	
Returns:	**cl**	**VisOptFlags** field.
Destroyed:	**ax, ch, dx, bp.**	
Library:	**Objects\visC.def**	

MSG_VIS_GET_POSITION

This message returns the current origin of the object. The origin returned consists of the left and top object bounds (*VI_bounds.R_left* and *VI_bounds.R_top*, respectively).

Pass:	Nothing.	
Returns:	**cx**	Left bound of object.
	dx	Top bound of object.
Destroyed:	**ax, bp.**	
Library:	**Objects\visC.def**	

MSG_VIS_GET_SIZE

This message returns the current size (width and height) of the object.

Pass:	Nothing.	
Returns:	**cx**	Current width.
	dx	Current height
Destroyed:	**ax, bp.**	
Interception:	This should never be subclassed.	
Library:	**Objects\visC.def**	

MSG_VIS_GET_TYPE_FLAGS

This message returns a visible object's current *VI_typeFlags* field.

Pass: Nothing.

Returns: `cl` **VisTypeFlags** field.

Destroyed: `ax, ch, dx, bp.`

Library: **Objects\visC.def**

MSG_VIS_GRAB_LARGE_MOUSE

A visible object sends itself this message (with MF_CALLl) when it wants to acquire the mouse grab and when it wants 32-bit coordinates. If the object wants 16-bit coordinates, it should use MSG_VIS_GRAB_MOUSE, above. The object must relinquish the mouse grab with MSG_VIS_RELEASE_MOUSE.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\visC.def**

MSG_VIS_GRAB_MOUSE

A visible object sends itself this message (with MF_CALL) when it wants to acquire the normal mouse grab. The message MSG_VIS_RELEASE_MOUSE must be used to relinquish the grab.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\visC.def**

MSG_VIS_INVAL_ALL_GEOMETRY

This is a brute-force, desperation message used to invalidate all geometry in the recipient's tree; that is, all geometry of the recipient and all objects under



it in the tree will be invalidated. You should only use this message when the whole tree should be recalculated.

Pass: `dl` **VisUpdateMode** indicating when the visual update of the recalculation should occur.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\visC.def**

MSG_VIS_INVALIDATE

This message invalidates the entire region within the object's bounds. The message will cause the window system to generate a MSG_META_EXPOSED for the area covered by the bounds, causing a visual update to occur.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\visC.def**

MSG_VIS_INVAL_TREE

This message has the effect of MSG_VIS_INVALIDATE on an entire branch of the visible tree. The window system will generate MSG_META_EXPOSED for each affected window.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\visC.def**

MSG_VIS_LAYER_GET_DOC_BOUNDS

This message returns the 32-bit document bounds of a particular large visible object. Typically, a VisContent will query its large children with this message when it needs to know their bounds.

Pass: `cx:dx` Buffer of at least size **RectDWord** in which to store the bounds.

Returns: `cx:dx` Pointer to **RectDWord** of object's bounds.

Destroyed: `ax, bp.`

Library: **Objects\visC.def**

Object Assembly Reference

MSG_VIS_LAYER_INSERT_OR_DELETE_SPACE

This message should be sent by one layer to another when it changes sizes. An example would be a spreadsheet application with a graphic layer; when a column in the spreadsheet resizes, the graphic layer must resize accordingly.

Pass: **ss**:**bp** **InsertDeleteSpaceParams** structure.
dx sizeof **InsertDeleteSpaceParams**.

Returns: Nothing.

Destroyed: **ax.**

Interception: This message must be intercepted; it has no default behavior.

Library: **Objects\visC.def**

MSG_VIS_LAYER_SET_DOC_BOUNDS

This message sets the 32-bit document bounds for a particular layer object. This message is typically sent by a VisContent object to its children, which are assumed to be layer objects.

Pass: **ss**:**bp** **RectDWord** structure with new visible bounds.
dx sizeof **RectDWord** structure.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_LOST_GADGET_EXCL

This message, when received, indicates that the recipient has lost its hold on the gadget exclusive. When an object receives this, it should release the active mouse grab, if appropriate.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_MARK_INVALID

This message allows the caller to set the recipient's *VI_optFlags* record so the object will get updated properly during the next visual update. The flags that



can be set with this message are VOF_GEOMETRY_INVALID, VOF_WINDOW_INVALID, and VOF_IMAGE_INVALID.

Pass: `cl` **VisOptFlags** to set: one or more of VOF_GEOMETRY_INVALID, VOF_WINDOW_INVALID, and VOF_IMAGE_INVALID.
`dl` **VisUpdateMode** value specifying when to update.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Interception: This message should not be intercepted.

Library: **Objects\visC.def**

■ MSG_VIS_MOVE_CHILD

This message moves a child of the recipient to another location among its siblings. It essentially removes the child from the branch and then re-adds it in the same manner as MSG_VIS_ADD_CHILD. This message does not move the child from one parent to another; you must use a combination of MSG_VIS_REMOVE and MSG_VIS_ADD_CHILD to achieve that.

Pass: ^l**cx**:**dx** Child object to move.
bp **CompChildFlags** describing where to put child.

Returns: **cx**, **dx** Unchanged.

Destroyed: `ax, bp.`

Library: **Objects\visC.def**

■ MSG_VIS_MOVE_RESIZE_WIN

This message causes the window object to move and/or resize itself based on the bounds of its content object. This message is used only by window or portal objects and is rarely, if ever, used by objects in your applications.

Source: Part of the visual update mechanism-typically called by the window object on itself in its MSG_VIS_UPDATE_WINDOWS_AND_IMAGE handler.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\visC.def**

MSG_VIS_NOTIFY_GEOMETRY_INVALID

This message is sent by the geometry manager to objects that have VGA_NOTIFY_GEOMETRY_VALID set. The message is sent only after all geometry messages have been handled for the object. It has no default behavior.

Destination: Any visible object—typically a child of the object sending the message.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_OPEN

This message is part of the visual update mechanism and will be sent by the system when an object must be visually updated to appear on the screen. It will be sent to any VOF_WINDOW_INVALID objects in a branch whose VTF_IS_WIN_GROUP object has become VA_VISIBLE. Any object added into a visible tree having VOF_WINDOW_INVALID set will receive a MSG_VIS_OPEN at the next visual update. This message propagates down the visible tree and does not cross over boundaries of window groups. This message is often subclassed by objects that want to initialize some information before being drawn on the screen. If an object subclasses this message, it should be sure to call its superclass somewhere in the handler.

Pass: **bp** Zero if opening top window; otherwise this is the window handle.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**



MSG_VIS_OPEN_WIN

This message is sent to a window or portal object to open its window. The graphics window will be opened and drawn on the screen, and a visual update will propagate down the displayed visible tree.

Pass: ^h**bp** Handle of parent window.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Interception: The window group or portal must intercept in order to open the new graphics window and set initial parameters (*VCI_window*).

Library: **Objects\visC.def**

MSG_VIS_POSITION_AND_INVAL_IF_NEEDED

This message is an optimized version of MSG_VIS_POSITION_BRANCH. It repositions the branch only if necessary and will mark the image invalid only if the position was changed. It returns a flag indicating whether or not the object was repositioned.

Pass: **cx** **RecalcSizeArgs** suggesting a width.
dx **RecalcSizeArgs** suggesting a height.

Returns: **cx** Width returned by object.
dx Height returned by object.

Destroyed: **ax, bp.**

Library: **Objects\visC.def**

MSG_VIS_POSITION_BRANCH

This message repositions an entire visible branch. It propagates down the branch, causing each visible object to reposition itself based on the new origins passed. Composites must pass the appropriate altered positions to their children.

Pass: **cx** Left coordinate.
dx Top coordinate.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_QUERY_WINDOW

This message returns the window handle of the window the object currently resides in. In many cases, this will be the window handle of a GenView window.

Pass: Nothing.

Returns: ^h**cx** Window.

Destroyed: **ax, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_RECALC_SIZE

This message takes the suggested height and width of the object and recalculates the object's desired size based on them. Composite objects are expected to pass this message to their children and then calculate their sizes based on those of their children. The default behavior of the handler for this is to return the current size no matter what the suggested size is.

Pass: **cx** **RecalcSizeArgs** with suggested width for object.
dx **RecaclSizeArgs** with suggested height for object.

Returns: **cx** Width returned by object.
dx Height returned by object.

Destroyed: **ax, bp.**

Library: **Objects\visC.def**

MSG_VIS_RECALC_SIZE_AND_INVAL_IF_NEEDED

This message is the same as MSG_VIS_CALC_RESIZE except that it is somewhat optimized. It will recalculate only if certain flags are set for the object, and it will mark the geometry invalid only if recalculation is done.

Pass: **cx** **RecalcSizeArgs** with suggested width.
dx **RecalcSizeArgs** with suggested height.

Returns: **cx** Width returned by object.
dx Height returned by object.

Destroyed: **ax, bp.**

Library: **Objects\visC.def**

MSG_VIS_RECREATE_CACHED_GSTATES

This message may be used to have the Vis object destroy and recreate any cached GStates it has. For example, cached GStates of a complex visible

object that gets moved or resized will no longer map to the proper place in the object's document. After the move or resize, this message may be used to update those GStates.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_REDRAW_ENTIRE_OBJECT

This message causes the object to send itself a MSG_VIS_DRAW, creating and destroying a GState for itself.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_RELEASE_GADGET_EXCL

This message causes the passed object to release the gadget exclusive. The object specified by child will then receive a MSG_VIS_LOST_GADGET_EXCL.

Pass: ^l**cx**:**dx** Child object which is releasing exclusive.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_RELEASE_MOUSE

A visible object sends itself this message if it has the mouse grab and wants to release it. Typically, this message will be called in the object's MSG_META_END_SELECT handler. It works for both large and small grabs.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_REMOVE

This is the high-level message that closes (if necessary) and removes a visible branch from the object tree. The parent of the branch is marked invalid for visual update according to the passed **VisUpdateMode**.

Unlike MSG_VIS_DESTROY, this message does not destroy the visible branch but only unlinks it from the tree. This message may not be subclassed by any object. This message is useful for hiding visible branches which may be added again later with MSG_VIS_ADD_CHILD.

Pass: `dl` **VisUpdateMode** specifying when to update.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Interception: Do not subclass this message.

Library: **Objects\visC.def**

MSG_VIS_REMOVE_BUTTON_POST_PASSIVE

This message removes any post-passive mouse grab from the object.

Pass: Nothing..

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\visC.def**

MSG_VIS_REMOVE_BUTTON_PRE_PASSIVE

This message removes the pre-passive grab from a visible object.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\visC.def**

MSG_VIS_REMOVE_CHILD

This message removes the specified child from the object tree. This message should be rarely used and used with care; it does not close the visible branch but simply removes it. Consider using the higher-level MSG_VIS_REMOVE

instead. Note that all grabs (focus, gadget, mouse, etc.) must be released by the child and all its children before the branch can be removed safely.

Pass:	^l**cx**:**dx**	Child object to remove.
	bp	mask CCF_MARK_DIRTY to mark links dirty.
Returns:	**cx**, **dx**	Unchanged.
Destroyed:	**ax, bp.**	
Library:	**Objects\visC.def**	

■ MSG_VIS_REMOVE_NON_DISCARDABLE

This message performs exactly the same function as MSG_VIS_REMOVE except that it decrements the object's in-use count before removing it. This message should therefore be used in conjunction with the adding message MSG_VIS_ADD_NON_DISCARDABLE_VM_CHILD and should not be used with MSG_VIS_ADD_CHILD. Likewise, MSG_VIS_REMOVE should not be used with this MSG_VIS_ADD_NON_DISCARDABLE_VM_CHILD.

Pass:	**dl**	**VisUpdateMode** specifying when to update.
Returns:	Nothing.	
Destroyed:	**ax, cx, dx, bp.**	
Library:	**Objects\visC.def**	

■ MSG_VIS_REMOVE_NON_DISCARDABLE_VM_CHILD

This message performs exactly the same function as MSG_VIS_REMOVE_CHILD except that it is used with MSG_VIS_ADD_NON_DISCARDABLE_VM_CHILD rather than with MSG_VIS_ADD_CHILD. This message decrements the in-use count before removing the child, thereby undoing the extra increment performed by MSG_VIS_ADD_NON_DISCARDABLE_VM_CHILD.

Pass:	^l**cx**:**dx**	Child object to remove.
Returns:	**cx**, **dx**	Unchanged.
Destroyed:	**ax, bp.**	
Library:	**Objects\visC.def**	

MSG_VIS_RESET_TO_INITIAL_SIZE

This message propagates down an entire visual branch, causing the objects in it to reset their sizes to their original width and height. First it invalidates the object's geometry, then it recalculates the geometry starting at the top.

Pass: `dl` **VisUpdateMode** specifying when to update.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\visC.def**

MSG_VIS_SEND_TO_CHILDREN

This message propagates down an entire visual branch, causing the objects in it to reset their sizes to their original width and height. First it invalidates the object's geometry, then it recalculates the geometry starting at the top. (In Goc, you should use **@send @visChildren** instead.)

Pass: ^h`cx` **ClassedEvent** handle.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp; ClassedEvent handle freed.`

Library: **Objects\visC.def**

MSG_VIS_SEND_TO_PARENT

This message delivers the passed recorded event to the parent of the recipient object. It acts as a send, allowing the caller to continue execution without waiting for the event to be handled. (In Goc, you should use **@send @visParent** instead.)

Pass: ^h`cx` **ClassedEvent** handle.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp; ClassedEvent handle freed.`

Library: **Objects\visC.def**

MSG_VIS_SET_ATTRS

This message sets the object's *VI_attrs* field to the values passed. The value of `cl` will be OR-ed with *VI_attrs*, and the value of `ch` will be logically inverted

and then AND-ed with the field. The clear operation will occur after the set, and therefore a flag set in both parameters will be cleared.

Pass: **cl** **VisAttrs** field containing flags to set.
ch **VisAttrs** field containing flags to clear.
dl **VisUpdateMode** specifying when update should occur.

Returns: **cx**, **dx**, **bp** Unchanged.

Destroyed: **ax.**

Library: **Objects\visC.def**

MSG_VIS_SET_GEO_ATTRS

This message sets the object's *VI_geoAttrs* field to the values passed. The value of **cl** will be OR-ed with *VI_geoAttrs*, and the value of **ch** will be logically inverted and then AND-ed with the field. The clear operation will occur after the set, and therefore a flag set in both parameters will be cleared. Many of the attributes in this record are used only by Specific UI libraries and should not be altered by applications.

Pass: **cl** **VisGeoAttrs** field containing flags to set.
ch **VisGeoAttrs** field containing flags to clear.
dl **VisUpdateMode** specifying when update should occur.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_SET_POSITION

This message causes the object to set its *VI_bounds* to the new location specified, retaining its current size. The left bound (*VI_bounds.R_left*) is set to the value of **cx** , and the top bound (*VI_bounds.R_top*) is set to that of **dx**. The right and bottom bounds are set to match the new origin and the width and height of the object.

Pass: **cx** Left edge.
dx Top edge.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_SET_SIZE

This message sets the object's height and width to the passed values, retaining the object's current position. The object's position will not be recalculated as a result of the size change.

Pass: **cx** New width.
dx New height.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_SET_TYPE_FLAGS

This message sets the object's *VI_typeFlags* field to the values passed. The value of **cl** will be OR-ed with *VI_typeFlags*, and the value of **ch** will be logically inverted and then AND-ed with the field. The clear operation will occur after the set, and therefore a flag set in both parameters will be cleared.

Destination: Any visible object; object must not be realized when it receives the message.

Pass: **cl** **VisTypeFlags** to set.
ch **VisTypeFlags** to clear.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_TAKE_GADGET_EXCL

This message causes the passed object to be given the gadget exclusive. Any object currently having the gadget exclusive will subsequently receive a MSG_VIS_LOST_GADGET_EXCL. This is used primarily by the Specific UI.

Pass: ^l**cx**:**dx** Child object releasing exclusive.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_UPDATE_GEOMETRY

This message is sent during a visual update to cause objects to recalculate their geometry. Applications can use this message to ensure that geometry

gets updated for a visual branch even if the branch is not currently drawn on the screen.

Pass: Nothing.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\visC.def**

MSG_VIS_UPDATE_WINDOWS_AND_IMAGE

This message is called by a window group during a visual update. It should not be sent or handled by anything other than a window group. It causes both the geometry and image of the affected branch of the visible tree to be updated, and it is used on branches which are already visually realized on the screen.

Source: A window group object.

Destination: A window group object.

Pass: `cl` **VisUpdateImageFlags** field.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Interception: This message may not be intercepted.

Library: **Objects\visC.def**

MSG_VIS_UPDATE_WIN_GROUP

This message may only be sent by a window group object to itself. It causes the window group to actually go through a visual update.

Pass: `dl` **VisUpdateMode** specifying when update should occur.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp.`

Library: **Objects\visC.def**

MSG_VIS_VUP_ALLOW_GLOBAL_TRANSFER

This message is sent by a visible object to itself when a quick-transfer operation is underway and the pointer has to be allowed to leave the bounds of the object's view window. This message will travel up to the content object,

which will indicate to the GenView that the pointer events must be allowed to go to other windows in the system.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\visC.def

MSG_VIS_VUP_ALTER_INPUT_FLOW

This message is the primitive employed by the grab/release messages shown above. Objects will rarely use this unless they are large objects (using 32-bit coordinates). Large objects and composites should subclass this message. In the case of a mouse grab, they should add in their 32-bit offset amounts to the translation structure before passing the event on to their superclasses. This will result in large mouse events being properly translated for the duration of the mouse grab.

Pass: ss:bp — **VupAlterInputFlowData** structure.
dx — sizeof **VupAlterInputFlowData**.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\visC.def

MSG_VIS_VUP_BUMP_MOUSE

The UI can control the mouse image and events as they occur. Because the UI is based on the visible classes, the messages that control the mouse status are also available to you, though they will probably be of very little utility. These messages are given below.

Pass: cx — Signed distance to bump mouse in *x* direction.
dx — Signed distance to bump mouse in *y* direction.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\visC.def

MSG_VIS_VUP_CALL_OBJECT_OF_CLASS

This message searches up the visible tree until it encounters an object of the proper class for the recorded event. The class is specified within the passed

event. When the first such object is found, it will be called with the classed event as if an the object had been sent directly with the MF_CALL flag set.

Pass: ^h**cx** **ClassedEvent** containing recorded event.

Returns: CF, **ax**, **cx**, **dx**, **bp**
As returned by recorded event.

Destroyed: **(As destroyed by recorded event.)**

Library: **Objects\visC.def**

MSG_VIS_VUP_CALL_WIN_GROUP

VisClass offers two specific messages for contacting window group objects in a visible tree. These are similar to MSG_VIS_VUP_CALL_OBJECT_OF_CLASS and MSG_VIS_VUP_SEND_TO_OBJECT_OF_CLASS, except they find the window group of the recipient and deliver the event to the window object. These two messages are detailed below.

Pass: ^h**cx** **ClassedEvent** containing recorded event.

Returns: CF, **ax**, **cx**, **dx**, **bp**
As returned by recorded event.

Destroyed: **(As destroyed by recorded event.)**

Library: **Objects\visC.def**

MSG_VIS_VUP_CREATE_GSTATE

This message travels up the visible object tree until it reaches either the root object or a window group object. It then creates a graphic state associated with that object; this graphic state is then used for the subsequent visual update. You may wish to subclass this message in order to alter the GState created or to set it up to take 32-bit coordinates. If you subclass it, however, be sure to call the superclass somewhere in the handler.

Pass: **Nothing.**

Returns:

CF — Default Vis behavior sets this flag to indicate that the message has been handled.

^h**bp** — **GStateHandle** of created GState. The GState's associated window will be that of the window object, if any; otherwise the GState's associated window will be null. To destroy the GState later, call **GrDestroyState()**.
The GState will be in the default state, except that the pen position and palette may have been altered. If the object handling the message is in a 32-bit coordinate visible tree, the GState's transformation matrix will have an offset in

32-bit coordinate space as set by the large composite object. Non-UI objects will have their font and font size set to the system default. Specific UI objects will have their font and font size set to the UI default; the GState's private data will be set to the specific UI's **DisplayScheme**; the line end will be set to LE_BUTTCAP and the line join will be set to LJ_MITERED.

Destroyed: **ax, cx, dx.**

Library: **Objects\visC.def**

MSG_VIS_VUP_EC_ENSURE_OBJ_BLOCK_NOT_REFERENCED

This message travels up the visible object tree, ensuring that the memory handle (the handle of an object block) passed is not stored anywhere that it might cause trouble. This message may be used just before the object block's handle is freed to ensure that no stale references will remain.

Pass: ^h**cx** Object block being destroyed.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_VUP_EC_ENSURE_OD_NOT_REFERENCED

This message travels up the visible object tree, ensuring that the optr passed is not stored by any objects in the tree. This message may be sent when the object is being destroyed; any objects methods encountering a match between a stored optr and the passed optr should force a fatal error.

Pass: ^l**cx**:**dx** Object being destroyed.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_VUP_EC_ENSURE_WINDOW_NOT_REFERENCED

This message travels up the visible object tree, ensuring that the window handle passed is not stored anywhere that it might cause trouble. This

message may be used when a window is being destroyed and its handle is about to be freed.

Pass: ^h**cx** Windown being destroyed.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

■ MSG_VIS_VUP_FIND_OBJECT_OF_CLASS

This message searches up the visible object tree until it encounters an object of the specified class. It then returns the optr of that object.

Pass: **cx**:**dx** Class of object we're looking for.

Returns: ^l**cx**:**dx** Object, if found; otherwise null.
CF Set if object found; otherwise clear.

Destroyed: **ax, bp.**

Library: **Objects\visC.def**

■ MSG_VIS_VUP_GET_MOUSE_STATUS

This message is rarely used by any objects outside the UI library. It returns the information passed with the last mouse event. The word return value contains two byte-sized sets of flags. The high byte is a list of the active UI functions, and the low byte represents the current button information.

Pass: Nothing.

Returns: **bp** low byte **ButtonInfo** field showing which buttons are down.
bp high byte **UIFunctionsActive** field showing what sort of operation is in progress.

Destroyed: **ax, cx, dx.**

Library: **Objects\visC.def**

■ MSG_VIS_VUP_QUERY

This message simply queries up the tree until it is handled. It is extensible so visible objects can implement their own upward queries without adding handlers to every class in between it and the query handler.

Object Assembly Reference

It is extremely rare that any object would send or handle MSG_VIS_VUP_QUERY on its own; many classes, however, may alias this message and create their own versions of the upward query.

Pass: **cx** **VisUpwardQueryType** or **SpecVisUpwardQueryType**.

Returns: CF Set if query has been responded to.
ax, **cx**, **dx**, **bp** Depends on query type; null if query not responded to.

Destroyed: **Depends on query type.**

Library: **Objects\visC.def**

MSG_VIS_VUP_QUERY_FOCUS_EXCL

This message queries up the visible hierarchy to find the object having the current focus in the current window. The current window is taken to be the window in which the object receiving the message resides.

Pass: Nothing.

Returns: ^l**cx**:**dx** Object with the focus exclusive.
bp **HierarchicalGrabFlags** field.

Destroyed: **ax.**

Library: **Objects\visC.def**

MSG_VIS_VUP_SEND_TO_OBJECT_OF_CLASS

This message searches up the visible tree until it encounters an object of the proper class for the recorded event. The class is specified within the passed event. When the first such object is found, the message will be delivered to that object as if the event had been sent directly.

Pass: ^h**cx** **ClassedEvent** containing recorded event.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_VUP_SEND_TO_WIN_GROUP

This message searches up the visible object tree until it encounters a window group object. When the first window group is found, the passed event will be delivered to that object as if sent directly.

Pass: ^h**cx** **ClassedEvent** containing recorded event.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_VUP_SET_MOUSE_INTERACTION_BOUNDS

A visible object requesting the mouse grab sends this message up the visible tree. This message will be handled by a content object. It indicates a new set of draggable bounds in case the user initiates a drag-scrolling operation. This message is sent automatically when the visible object requests the grab.

Pass: **ss**:**bp** **Rectangle** structure with visible bounds.
dx sizeof **Rectangle**.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_VUP_TERMINATE_ACTIVE_MOUSE_FUNCTION

This message is sent by a visible object to itself to terminate any active mouse function, forcing it to be a function of type "other." This message is used only by the Specific UI library in cases where input synchronization problems occur in specific places. Applications should generally not use this message.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

Object Assembly Reference

MSG_VIS_VUP_TEST_FOR_OBJECT_OF_CLASS

This message searches up the visible tree and determines whether an object of the given class is in the tree.

Pass: **cx**:**dx** Class of object to search for.

Returns: CF Set if object found; clear otherwise.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_VUP_UPDATE_WIN_GROUP

This message travels up the visible tree from the recipient object to the first window group encountered; it will then cause that window group to go through a visible update by sending the window group object a MSG_VIS_UPDATE_WIN_GROUP, below.

Pass: **dl** **VisUpdateMode** specifying when update should occur.

Returns: CF Set if message handled.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

MSG_VIS_WIN_ABOUT_TO_BE_CLOSED

This message notifies the recipient that the window it's displayed in is about to be closed. The default handler for this will remove the visible branch of the window off the screen so no redraws will occur before the window closes.

Pass: Nothing.

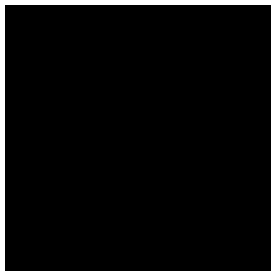
Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\visC.def**

Object Assembly Reference

VisComp



VisCompClass 655

24

VisCompClass

Instance Data

```
VCI_comp                CompPart <>
   UIC Syntax:  children = compositeComp visLink;
```

children (*VCI_comp*) contains the link to the composite object's first child.

```
VCI_gadgetExcl          optr <>
   UIC Syntax:  gadgetExcl = optrComp : null;
```

gadgetExcl (*VCI_gadgetExcl*) is an optr to the object that currently has the gadget exclusive. This field is rarely used directly by applications.

```
VCI_window              hptr.Window <>
   UIC Syntax:  gWin = hptrComp : null;
```

gWin (*VCI_window*) contains the window handle of the graphics window associated with this object. This field is rarely accessed directly by applications; it is set by the visual update mechanism.

```
VCI_geoAttrs            VisCompGeoAttrs
   UIC Syntax:  visCompGeoAttrs = bitFieldComp byte {
                   orientChildrenVertically,
                   includeEndsInChildSpacing,
                   allowChildrenToWrap,
                   onePassOptimization,
                   customManageChildren,
                   hasMinimumSize,
                   wrapAfterChildCount,
                   onlyDrawsInMargins
                   } : {};
```

visCompGeoAttrs (*VCI_geoAttrs*) is a record that defines some of the geometry management information for the composite. None of its values are set by default.

```
VCI_geoDimensionAttrs VisCompGeoDimensionAttrs
   UIC Syntax:  justFlags = bitFieldComp byte {
                   widthJustification:2 = {
                     leftJustify,
                     rightJustify,
                     centerHorizontally,
```

```
    fullJustifyHorizontally
  },
  expandWidthToFitParent,
  canTruncateWidthToFitParent,
  heightJustification:2 = {
    topJustify,
    bottomJustify,
    centerVertically,
    fullJustifyVertically
  },
  expandHeightToFitParent,
  canTruncateHeightToFitParent
  } : {}; /* Justification flags */
```

justFlags (*VCI_geoDimensionAttrs*) is a record that contains additional information about the composite's geometry. This field contains two two-bit fields among its other flags.

Messages

MSG_VIS_COMP_GET_CHILD_SPACING

This message returns the child spacing used by the composite.

Pass: Nothing.

Returns:

cx	Spacing between children.
dx	Spacing between lines of wrapping children.

Destroyed: **ax, bp.**

Include: **Objects\vCompC.def**

MSG_VIS_COMP_GET_GEO_ATTRS

This message retrieves the flags set in the object's *VCI_geoAttrs* and *VCI_geoDimensionAttrs* fields. This message does not invalidate or update the object's geometry.

Pass: Nothing.

Returns:

cl	**VisCompGeoAttrs** field with the value of *VCI_geoAttrs*.
ch	**VisCompGeoDimensionAttrs** field with the value of *VCI_geoDimensionAttrs*.

Destroyed: **ax, dx, bp.**

Include: **Objects\vCompC.def**

Object Assembly Reference

MSG_VIS_COMP_GET_MARGINS

This message returns the margins the composite should use when recalculating its child spacing. If you want a special left, top, right, or bottom margin around the composite's children, intercept this message and return the margin(s) in the appropriate register(s).

Pass: Nothing.

Returns:

ax	Left margin.
bp	Top margin.
cx	Right margin.
dx	Bottom margin.

Destroyed: **Nothing.**

Include: **Objects\vCompC.def**

MSG_VIS_COMP_GET_MINIMUM_SIZE

This message returns the minimum size of the composite. It is used by the geometry manager if the composite has VCGDA_HAS_MINIMUM_SIZE set. This message does not invalidate or update the object's geometry.

Pass: Nothing.

Returns:

cx	Minimum width for composite.
dx	Minimum height for composite.

Destroyed: **ax, bp.**

Include: **Objects\vCompC.def**

MSG_VIS_COMP_GET_WRAP_COUNT

This message returns the number of children to be counted before wrapping if the composite has VCGA_WRAP_AFTER_CHILD_COUNT set.

Pass: Nothing.

Returns:

cx	Child count.

Destroyed: **ax, dx, bp.**

Include: **Objects\vCompC.def**

MSG_VIS_COMP_SET_GEO_ATTRS

MSG_VIS_COMP_GET_GEO_ATTRS returns the current geometry and justification flags for the composite.

Pass: **cx** (*attrsToSet*) A word of attributes that should be set for the object. The high byte is a record of **VisCompGeoDimensionAttrs**, and the low byte is a record of **VisCompGeoAttrs**. The attributes set in this parameter will be set for the object.

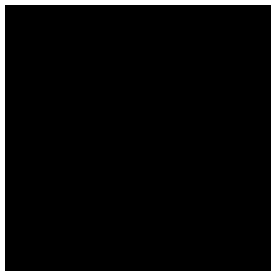
dx (*attrsToClear*) A word of attributes to be cleared from the object's instance data. It has the same form as *attrsToSet*, above. Any attribute set in this parameter will be cleared in the instance fields.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Include: **Objects\vCompC.def**

VisContent



25

VisContentClass 661

VisContentClass

Instance Data

```
VCNI_view            optr <>
   UIC Syntax:  view = optrComp : null;
```

view (*VCNI_view*) contains the optr of the GenView object. The content should not ever actually use this field for the view's optr; if you need to contact the view, you should instead use the messages MSG_VIS_VUP_SEND_TO_OBJECT_OF_CLASS and MSG_VIS_VUP_CALL_OBJECT_OF_CLASS. The field will be set automatically by the GenView when the view is opened; the view will send a MSG_META_CONTENT_VIEW_OPENING.

```
VCNI_window          hptr.Window <>
   UIC Syntax:  portGWin = hptrComp : null;
```

portGWin (*VCNI_window*) contains the window handle of the view's window. This field is set by the view object and should not be accessed by the application. If an object under the content needs to get the window handle, it should use MSG_VIS_QUERY_WINDOW. This field will be set automatically by the GenView when the view's window is first opened. The view will send the messages MSG_META_CONTENT_VIEW_WIN_OPENED and MSG_META_CONTENT_VIEW_OPENING to set the window handle. When the view window closes, the view will send MSG_META_CONTENT_VIEW_CLOSING and MSG_META_CONTENT_VIEW_WIN_CLOSED.

```
VCNI_viewHeight      word 0
   UIC Syntax:  viewHeight = wordComp : 0;
```

viewHeight (*VCNI_viewHeight*) contains the height, in document coordinates, of the view window. This will be set by the view object and should not be altered by the application; it can be retrieved with the message MSG_VIS_CONTENT_GET_WIN_SIZE (below). The view automatically notifies the content each time the window size changes with MSG_VIS_CONTENT_VIEW_SIZE_CHANGED.

```
VCNI_viewWidth       word 0       ; width of window
   UIC Syntax:  viewWidth = wordComp : 0;
```

viewWidth (*VCNI_viewWidth*) contains the width, in document coordinates, of the view window. This will be set by the view object and should not be altered by the application; it can be retrieved with the message MSG_VIS_CONTENT_GET_WIN_SIZE (below). The view automatically notifies the content on each window resize with MSG_META_CONTENT_VIEW_SIZE_CHANGED.

```
VCNI_attrs            VisContentAttrs 0
   UIC Syntax:  contentAttrs = bitFieldComp byte {
                    sameWidthAsSubview,
                    sameHeightAsSubview,
                    largeDocumentModel,
                    windowCoordinateMouseEvents,
                    activeMouseGrabRequiresLargeEvents,
                    viewDocBoundsSetManually,
                    viewDoesNotWinScroll } : {};
```

contentAttrs (*VCNI_attrs*) is a record of **VisContentAttrs** that contains several attributes which affect how the content object interacts with the view and with the visible object tree. You can set these attributes with MSG_VIS_CONTENT_SET_ATTRS dynamically, and you can retrieve them with MSG_VIS_CONTENT_GET_ATTRS.

```
VCNI_docOrigin        PointDWord <<0,0>, <0,0>>
```

VCNI_docOrigin contains the current origin of the view window. The origin is the location of the view's upper left corner in the document.

```
VCNI_scaleFactor      PointWWFixed <<0,1>, <0,1>>
```

VCNI_scaleFactor contains the current scale factor the view is displaying. Scaling is implemented almost entirely in the GenView object; some content objects, however, will want to react in a special way when the scale factor is changed.

```
VCNI_prePassiveMouseGrabList      lptr.ChunkArrayHeader <>
   UIC Syntax:  prePassiveMouseGrabList = nptrComp : null
```

prePassiveMouseGrabList (*VCNI_prePassiveMouseGrabList*) is a pointer to a chunk array containing the list of objects that currently have the prepassive mouse grab.

```
VCNI_impliedMouseGrab VisMouseGrab <      \
                  <>, <>, <0, 0>, <0, 0>, \
                  (mask VIFGF_MOUSE or mask VIFGF_PTR), 0 >

noreloc VCNI_impliedMouseGrab.VMG_object
```

Object Assembly Reference

```
noreloc VCNI_impliedMouseGrab.VMG_gWin
   UIC Syntax:      impMouseGrab =             optrComp : null;
                    impMouseGrabData =         wordComp : 0;
                    impMouseTranslationX =     dwordComp : 0;
                    impMouseTranslationY =     dwordComp : 0;
                    impMouseGrabFlags =        byteComp : \
                          "mask VIFGF_MOUSE or mask VIFGF_PTR";
```

impMouseGrab, *impMouseGrabData*, *impMouseTranslationX*, *impMouseTranslationY*, and *impMouseGrabFlags* (*VCNI_impliedMouseGrab*) form a **VisMouseGrab** structure that contains details about the object which has the implied mouse grab.

```
VCNI_activeMouseGrab   VisMouseGrab < \
                    <>, <>, <0, 0>, <0, 0>, 0, 0 >

noreloc         VCNI_activeMouseGrab.VMG_object

noreloc         VCNI_activeMouseGrab.VMG_gWin
   UIC Syntax:      mouseGrab =         optrComp : null;
                    mouseGrabData =     wordComp : 0;
                    mouseTranslationX = dwordComp : 0;
                    mouseTranslationY = dwordComp : 0;
                    mouseGrabFlags =    byteComp : 0;
```

mouseGrab, *mouseGrabData*, *mouseTranslationX*, *mouseTranslationY*, and *mouseGrabFlags* (*VCNI_activeMouseGrab*) form a **VisMouseGrab** structure that contains details about the object which has the active mouse grab.

```
VCNI_postPassiveMouseGrabList     lptr.ChunkArrayHeader
   UIC Syntax:  postPassiveMouseGrabList = nptrComp : null;
```

postPassiveMouseGrabList (*VCNI_postPasiveMouseGrabList*) is a pointer to a chunk array containing the list of objects that currently have the postpassive mouse grab.

```
VCNI_kbdGrab          KbdGrab < <>, 0, >
   UIC Syntax:      kbdGrab =        optrComp : null;
                    kbdGrabData =    wordComp : 0;
```

kbdGrab (*VCNI_kbdGrab*) is a **KbdGrab** structure that contains details about the object which currently has the keyboard grab.

```
VCNI_focusExcl        FTVMCGrab < <>, (mask MAEF_FOCUS), >
                    noreloc VCNI_focusExcl.FTVMC_OD
   UIC Syntax:      focusExcl =      optrComp : null;
                    focusExclData =  wordComp : "mask MAEF_FOCUS"
```

focusExcl and *focusExclData* (*VCNI_focusExcl*) contain information about the object with the focus exclusive.

```
VCNI_targetExcl       FTVMCGrab < <>, (mask MAEF_TARGET), >
                      noreloc VCNI_targetExcl.FTVMC_OD
   UIC Syntax:        targetExcl =     optrComp : null;
                      targetExclData = wordComp : "mask MAEF_TARGET";
```

targetExcl and *targetExclData* (*VCNI_targetExcl*) contain information about the object with the target exclusive.

```
VCNI_holdUpInputQueue      hptr <>
```

VCNI_holdUpInputQueue contains the queue handle of the queue where held-up input will be temporarily stored. Input events will go into this event queue until they are allowed to be handled again; then they will be sent to their proper recipients.

```
VCNI_holdUpInputCount      word    0
```

VCNI_holdUpInputCount contains a count of the number of objects that have requested that input be held up. If this count is positive, input will be held up and input events will go into the hold-up queue.

```
VCNI_holdUpInputFlags      HoldUpInputFlags 0
```

VCNI_holdUpInputFlags contains a record of flags which determine the state of the hold-up mechanism. The following two flags are allowed:

HUIF_FLUSHING_QUEUE
: This flag indicates that the hold-up queue is currently being flushed.

HUIF_HOLD_UP_MODE_DISABLED
: This flag forces input events to flow normally. It is used primarily by GEOS to ensure that the user can interact with a system-modal dialog box.

Messages

MSG_VIS_CONTENT_DISABLE_HOLD_UP

This message sets the HUIF_HOLD_UP_MODE_DISABLED flag, forcing all input events to flow normally until the flag is cleared. In essence, it turns off the hold-up mechanism.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\vCntC.def**

MSG_VIS_CONTENT_ENABLE_HOLD_UP

This message clears the HUIF_HOLD_UP_MODE_DISABLED flag, allowing input events to be held up in the hold-up event queue.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\vCntC.def

MSG_VIS_CONTENT_GET_ATTRS

This message returns the current contents of the object's *VCNI_attrs* field, a record of **VisContentAttrs**.

Pass: Nothing.

Returns: cl — Content attributes.

Destroyed: ax, ch, dx, bp.

Library: Objects\vCntC.def

MSG_VIS_CONTENT_GET_WIN_SIZE

This message returns the size of the content object's associated window in terms of width and height.

Pass: Nothing.

Returns: cx, dx — Width and height oc content window.

Destroyed: ax, bp.

Library: Objects\vCntC.def

MSG_VIS_CONTENT_HOLD_UP_INPUT_FLOW

This message increments the count in *VCNI_holdUpInputCount*. If this count is non-zero and HUIF_HOLD_UP_MODE_DISABLED is clear, subsequent input events will be sent into the hold-up queue until either the flag is set or the count once more drops to zero.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\vCntC.def

MSG_VIS_CONTENT_NOTIFY_ACTIVE_MOUSE_GRAB_WIN_CHANGED

The content will send itself as part of its MSG_VIS_VUP_ALTER_INPUT_FLOW handler in the case that its *VCNI_activeMouseGrab.VMG_gWin* changes. Used by the specific UI's implementation of **GenApplicationClass** to keep track of the currently active window within the application.

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp.

Library: Objects\vCntC.def

MSG_VIS_CONTENT_RECALC_SIZE_BASED_ON_VIEW

This message causes the content to recalculate its size based on the view's geometry. It will try to set its width if it has VCNA_SAME_WIDTH_AS_VIEW set, and it will try to set its height if it has VCNA_SAME_HEIGHT_AS_VIEW set.

Pass: Nothing.

Returns:

cx	Width.
dx	Height.

Destroyed: ax, bp.

Library: Objects\vCntC.def

MSG_VIS_CONTENT_REMOTE_OBJ_MESSAGE_VIA_HOLD_UP_QUEUE

This message is basically the same as MSG_META_REMOTE_OBJ_MESSAGE, but instead of being relative to the flow object, is relative to the input flow -- i.e. if the input flow is being held up, then so will the MSG_META_REMOTE_OBJ_MESSAGE. Since the caller may wish to just queue this up with other input, or alternatively put this at the front of the input queue for processing, the message flags for how to put this method into the hold-up queue may be passed.

This is an Assembly-only message.

Pass:

cx	**MessageFlags** to be used for putting this onto the hold-up; input queue, if one exists. Should be one of mask MF_FORCE_QUEUE or mask MF_ON_STACK or mask MF_FORCE_QUEUE or

mask MF_INSERT_AT_FRONT or
mask MF_ON_STACK

dx	Size of **ObjMessageParams**.
ss:bp	Pointer to **ObjMessageParams**.

Returns: Nothing.

Destroyed: Nothing.

Library: **Objects\vCntC.def**

MSG_VIS_CONTENT_RESUME_INPUT_FLOW

This message decrements the count in *VCNI_holdUpInputCount*. If the count becomes zero with this call, the hold-up event queue is flushed and all the events in it are "played back." If the count goes below zero, GEOS will give an error. Therefore, do not use this message unless it is preceded with a MSG_VIS_CONTENT_HOLD_UP_INPUT_FLOW.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\vCntC.def**

MSG_VIS_CONTENT_SET_ATTRS

This message sets the *VCNI_attrs* field of the content object according to the passed values.

Pass:

cl	(*attrsToSet*) **VisContentAttrs** bits to set.
ch	(*attrsToClear*) **VisContentAttrs** bits to clear.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\vCntC.def**

MSG_VIS_CONTENT_SET_DOC_BOUNDS

This message is used to set the document bounds of a content's large document. The content will send MSG_GEN_VIEW_SET_DOC_BOUNDS to its view and MSG_VIS_LAYER_SET_DOC_BOUNDS to each of its children with

the new bounds. The recipient content *must* have the flag VCNA_LARGE_DOCUMENT_MODEL set in *VCNI_attrs*.

Pass: **dx** Size of **RectDWord** structure.
ss:**bp** Pointer to **RectDWord** containing new document bounds.

Returns: Nothing.

Destroyed: Nothing.

Library: **Objects\vCntC.def**

MSG_VIS_CONTENT_TEST_IF_ACTIVE_OR_IMPLIED_WIN

This message checks to see if the passed window handle is the same as the window of the object having either the implied or active mouse grab. This is typically used by objects in a Specific UI library to determine if the mouse event was actually within the window or directly on the window's border.

Pass: **bp** (*window*) Window handle to test against.

Returns: CF Clear if match.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\vCntC.def**

MSG_VIS_CONTENT_UNWANTED_KBD_EVENT

This message will be received by the content if a keyboard event was sent and there was no keyboard grab set up. The default action of the content is to beep (on presses only, not releases) and return as if the event had been processed.

Pass: **cx, dx, bp** As passed to MSG_META_KBD_CHAR.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\vCntC.def**

MSG_VIS_CONTENT_UNWANTED_MOUSE_EVENT

This message is received by the content if a mouse event was received and there was no active or implied grab. This is most frequently encountered when the user presses a mouse button outside a modal dialog box. The

default action of this handler is to beep (on presses only, not releases) and get rid of the event as if it had been handled.

Pass: **cx**:**dx** (mouseGrab) Pointer to **VisMouseGrab**.
bp Same as passed to MSG_META_PTR.

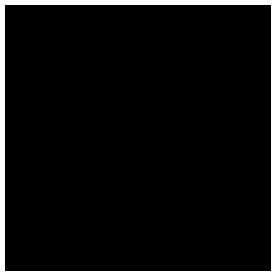
Returns: Nothing.

Destroyed: **ax, cx, dx, bp.**

Library: **Objects\vCntC.def**

Object Assembly Reference

System Objects



26

GenSystemClass 673

GenFieldClass 677

GenScreenClass 682

GenSystemClass

Instance Data

```
GSYI_specificUI      hptr
   UIC Syntax:  NONE
```

GSYI_specificUI stores the handle of the specific UI in use by this system object. By default, this specific UI will also be the same specific UI used by all applications underneath this system object.

```
GSYI_defaultUI       hptr
   UIC Syntax:  NONE
```

GSYI_defaultUI stores the handle of the default specific UI to use for the next loaded application. This may be overridden by a GenField (though this is rare). For all intents and purposes, *GSYI_defaultUI* will be the same as *GSYI_specificUI*.

```
GSYI_defaultScreen   optr
   UIC Syntax:  NONE
```

GSYI_defaultScreen stores an optr of a GenScreen. By default, new GenField objects will be made visible children of this GenScreen.

```
GSYI_defaultField    optr
   UIC Syntax:  NONE
```

GSYI_defaultField stores the optr of a GenField. By default, new application objects will be made generic children of this GenField. This may be overridden with MSG_GEN_APP_ADD_TO_PARENT, though normal applications will not do this.

```
GSYI_screenComp      CompPart <>
   UIC Syntax:  NONE
```

GSYI_screenComp stores the optr of the first GenScreen object child. The GenSystem is the head of two object trees. One is a visible tree containing all GenScreen objects; the other is a generic tree containing all application

objects (and their generic children). The generic tree is specified by *GI_comp*, as usual.

Messages

MSG_GEN_SYSTEM_BRING_GEODE_TO_TOP

This message raises a geode's window layer to the top and gives that geode the focus and target (if the geode if focusable/targetable). This message is called from within the UI to implement "autoraise," the automatic raising of a geode and the transferring of the focus and target when clicked upon. This message is also called by the GenApplication's handler for MSG_GEN_BRING_TO_TOP.

Source: Usually the specific UI, in response to an autoraise, or in response to MSG_GEN_BRING_TO_TOP.

Destination: The GenSystem object.

Pass: **cx** GeodeHandle of the application.

Returns: Nothing

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gSysC.def**

MSG_GEN_SYSTEM_GET_DEFAULT_FIELD

This message returns the optr of the default GenField. By default, new applications will be made generic children of this GenField.

Source: Unrestricted.

Destination: The GenSystem object.

Pass: Nothing.

Returns: **cx:dx** optr to default field

Destroyed: **ax, bp**

Library: **Objects\gSysC.def**

Object Assembly Reference

MSG_GEN_SYSTEM_GET_DEFAULT_SCREEN

This message returns the optr of the default GenScreen. By default, any GenFields will be made visible children of this GenScreen.

Source: Unrestricted.

Destination: The GenSystem object.

Pass: Nothing.

Returns: `cx:dx` optr to default screen

Destroyed: `ax, bp`

Library: **Objects\gSysC.def**

MSG_GEN_SYSTEM_LOWER_GEODE_TO_BOTTOM

This message lowers a geode's window layer to the bottom, releases any focus and target exclusives, and assigns new focus and target exclusives to the most suitable remaining geode. This message is called by the GenApplication's handler for MSG_GEN_LOWER_TO_BOTTOM.

Source: Usually in response to MSG_GEN_LOWER_TO_BOTTOM.

Destination: The GenSystem object.

Pass:
- `cx` **GeodeHandle** of the application.
- `dx` **LayerID** of the window.
- `bp` Handle of the parent window to lower to the bottom.

Returns: Nothing.

Library: **Objects\gSysC.def**

MSG_GEN_SYSTEM_MARK_BUSY

This message is called by the GenField or GenApplication object while an application is being launched but is not yet on screen. While marked busy, the UI will continue to allow mouse events through. Each message sent to the system object needs a MSG_GEN_SYSTEM_MARK_NOT_BUSY to undo its busy state. Therefore, if multiple MSG_GEN_SYSTEM_MARK_BUSY messages



are sent, an equal number of the MSG_GEN_SYSTEM_MARK_NOT_BUSY messages need to be sent to take down the busy cursor.

Source: The UI. You should not send this message yourself.

Destination: The GenSystem object.

Interception: Not intercepted.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Library: **Objects\gSysC.def**

■ MSG_GEN_SYSTEM_MARK_NOT_BUSY

This message is called by the GenField or GenApplication object when an application no longer needs to mark an application busy that has been brought on-screen.

Source: The UI. You should not send this message yourself.

Destination: The GenSystem object.

Interception: Not intercepted.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gSysC.def**

■ MSG_GEN_SYSTEM_NOTIFY_SYS_MODAL_WIN_CHANGE

This message is sent to the system object by the UI when it needs to check the status of any system modal windows.

Source: The UI. You should not send this message yourself.

Destination: The GenSystem object.

Interception: Not intercepted.

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Library: **Objects\gSysC.def**

Object Assembly Reference

■ MSG_GEN_SYSTEM_SET_DEFAULT_FIELD

This message changes the default GenField for the GenSystem. By default, new applications will be made generic children of this GenField.

Source: Unrestricted.

Destination: The GenSystem object.

Pass: `*ds:si` The instance data of the new default GenField object.
`cx:dx` The optr of the new default GenField object.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gSysC.def**

■ MSG_GEN_SYSTEM_SET_PTR_IMAGE

This message alters the system-wide pointer image.

Source: Unrestricted.

Destination: The GenSystem object.

Pass: `cx:dx` handle:offset of a **PointerDef** structure.
`bp` The **PtrImageLevel** to use.

Returns: Nothing.

Destroyed: `ax, cx, dx, bp`

Library: **Objects\gSysC.def**

GenFieldClass

Instance Data

```
GFI_flags                           GenFieldFlags
   UIC Syntax:  internalFlags    = bitFieldComp byte {
                   detaching,
                   loadBitmap,
                   restoringApps,
```

```
                needsWorkspaceMenu,
                hasDefaultLauncher,
                needDefaultLauncher
                } : {};
```

internalFlags (GFI_flags) stores the **GenFieldFlags** of the GenField object. These flags are for internal bookkeeping purposes.

```
GFI_visParent                   optr
   UIC Syntax:  static       optrComp : null;
```

GFI_visParent stores the optr of the GenScreen object that acts as this GenField's visible parent. This is typically set just before setting the GenField GS_USABLE (although it may be left to the specific UI to fill in.) During MSG_META_DETACH, this field is cleared; it may not be saved to a state file.

```
GFI_numDetachedApps              byte
   UIC Syntax:  static       byteComp : 0;
```

GFI_numDetachedApps stores the number of applications that were detached by the system upon MSG_META_DETACH. These applications and their associated state data will be saved to the *GFI_apps* chunk so that they will be brought up in their previous state.

```
GFI_numAttachingApps            byte
   UIC Syntax:  static       byteComp : 0;
```

GFI_numAttachingApps stores the number of applications currently trying to attach to the system. The system keeps track of these applications in case the system must shutdown during an application's attach cycle.

```
GFI_apps                        lptr.ChunkArrayHeader
   UIC Syntax:  static       wordComp : 0;
```

GFI_apps stores a chunk array of **AppInstanceReference** structures. This data is used on MSG_META_ATTACH to bring up applications to their previous state.

```
GFI_processes                   lptr.hptr
   UIC Syntax:  static       wordComp : 0;
```

GFI_processes similarly stores a chunk array of processes in progress under this GenField. This chunk array is saved to state so that these processes may continue when GEOS is restarted.

Object Assembly Reference

```
GFI_genApplications                lptr.optr
   UIC Syntax:   static       wordComp : 0;
```

GFI_genApplications stores the number of GenApplications currently launched within the GenField. This data is not saved to state.

```
GFI_numAppsToCheck                 byte
   UIC Syntax:   static       byteComp : 0;
```

GFI_numAppsToCheck stores the number of applications the UI must check with before shutting down.

```
GFI_notificationDestination      optr
   UIC Syntax:   notificationDestination    = optrComp : null;
```

notificationDestination (GFI_notificationDestination) stores the object that should receive the Field's notification messages (MSG_META_FIELD_NOTIFY_DETACH, MSG_META_FIELD_NOTIFY_NO_FOCUS, MSG_META_FIELD_NOTIFY_START_LANUCHER_ERROR).

```
   UIC Syntax:   default genStates      = default -usable;
   UIC Syntax:   default genAttributes  = default +targetable;
```

Messages

MSG_GEN_FIELD_ABOUT_TO_CLOSE

This message is sent by an environment application to inform a GenField that it is about to be closed. The GenField then has the option of quitting any applications or waiting until the applications themselves are closed by the

user. If *quitOnClose* is set the GenField's .INI file category, then the GenField will quit all open applications. Otherwise, it will ignore the request to quit.

Source: Unrestricted, though usually an environment application resident on top of a GenField object.

Destination: The GenField object.

Interception: May be intercepted if you have a custom GenField.

Pass: Nothing.

Returns: CF — Set if GenField has *quitOnClose* set, and field should not be closed until all applications have exited. Clear if GenField doesn't have *quitOnClose* set, and can be closed immediately.

Destroyed: ax, cx, dx, bp

Library: Objects\gFieldC.def

MSG_GEN_FIELD_EXIT_TO_DOS

This message requests the GenField to exit the system and return to DOS. The message is sent to the GenField so that custom fields can intercept the message and react accordingly by initiating shutdown procedures.

Source: Usually the UI.

Destination: GenField object.

Interception: May be intercepted if you have a custom GenField object that needs to perform certain shutdown procedures before exiting to DOS (such as shutting down applications).

Pass: Nothing.

Returns: Nothing.

Destroyed: ax, cx, dx, bp

Library: Objects\gFieldC.def

Object Assembly Reference

MSG_GEN_FIELD_GET_LAUNCH_MODEL

This message returns the GenField's **UILaunchModel** in use.

Source: Unrestricted.

Destination: GenField object.

Pass: Nothing.

Returns: **ax** **UILaunchModel** in use by the GenField.

Destroyed: Nothing.

Library: **Objects\gFieldC.def**

MSG_GEN_FIELD_GET_TOP_GEN_APPLICATION

This message returns the GenField's top application object.

Source: Unrestricted.

Destination: GenField object.

Pass: Nothing.

Returns: **cx:dx** optr of top visual GenApplication object.

Library: **Objects\gFieldC.def**

MSG_GEN_FIELD_NO_APPS_RESTORED

This message serves as notification that no processes have been restarted from the state file.

Source: The kernel

Destination: GenField object

Pass: Nothing.

Returns: Nothing.

Destroyed: **ax, cx, dx, bp**

Interception: May be intercepted to support custom behavior for a custom GenField object.

Library: **Objects\gFieldC.def**

MSG_GEN_FIELD_OPEN_WINDOW_LIST

This message may be sent by any object that wishes to bring up a GenField's window list dialog (if available).

Source: Unrestricted.

Destination: GenField object under a specific UI that supports window list dialogs.

Pass: Nothing.

Returns: Nothing.

Destroyed: Nothing.

Library: **Objects\gFieldC.def**

GenScreenClass

Instance Data

```
GSCI_videoDriver     hptr
```

Messages

There are no messages defined for this class.